IC Compiler™ II
Design Planning User Guide

中文学习译本

原版版本：V-2023.12，December 2023

Synopsys, Inc.

原文版权归 Synopsys, Inc. 所有（IC Compiler II Design Planning User Guide, Version V-2023.12, December 2023）。
本文件为非官方个人学习译本，仅供学习参考，不得用于商业传播。
命令名、选项名、文件名等技术标识符保持英文原文不译。

基于官方 PDF 的结构化翻译，插图由原书截图嵌入

目录

关于本用户指南

About This User Guide

Synopsys IC Compiler II 提供从网表到 GDSII 的完整设计方案，在整个设计流程中结合了设计规划（design planning）、物理综合（physical synthesis）、时钟树综合（clock tree synthesis）与布线（routing），用于逻辑与物理实现。

本指南说明 IC Compiler II 的设计规划（design planning）流程。有关工具的更多信息，请参阅下列配套文档：

- *Library Manager User Guide*
- *IC Compiler II Implementation User Guide*
- *IC Compiler II Design Planning User Guide*
- *IC Compiler II Data Model User Guide*
- *IC Compiler II Timing Analysis User Guide*

本用户指南面向使用 IC Compiler II 实现设计的设计工程师。使用该工具需要具备物理设计与综合的技能，并熟悉：

- 物理设计原理
- Linux 或 UNIX 操作系统
- 工具命令语言（Tcl）

本版本新增内容

关于新功能、增强与变更、已知限制，以及已解决的 Synopsys Technical Action Requests（STAR）信息，请参阅 SolvNetPlus 站点上的 *IC Compiler II Release Notes*。

相关产品、文档与商标

有关 IC Compiler II 工具的更多信息，请参阅 Synopsys SolvNetPlus 支持站点上的文档：`https://solvnetplus.synopsys.com`

您可能还希望参阅以下相关 Synopsys 产品文档：

- Design Compiler®
- IC Validator

- PrimeTime Suite®
- StarRC™

排版约定

Synopsys 文档使用下列约定。

表 **1:** 文档排版约定

约定	说明
Courier	表示语法，例如 `write_file`
Courier italic	表示语法中的用户自定义值，例如 `write_file design_list`
Courier bold	表示示例中用户原样输入的内容，例如 `prompt> write_file top`
紫色高亮	• 在示例中表示特别关注的信息； • 在命令语法中表示默认值，例如 `include_enclosing = true | false`
[]	表示语法中的可选参数，例如 `write_file [-format fmt]`
...	表示参数可按需重复，例如 `pin1 pin2 ... pinN`
|	表示在多个备选中选择其一，例如 `low | medium | high`
****	表示命令行续行
/	表示目录层次
粗体	表示带有关联操作的图形用户界面（GUI）元素
Edit > Copy	表示菜单命令路径，例如打开 Edit 菜单并选择 Copy
Ctrl+C	表示组合键，例如按住 Ctrl 再按 C

客户支持

客户支持通过 SolvNetPlus 提供。

访问 **SolvNetPlus**

SolvNetPlus 站点包含技术文章知识库以及对 Synopsys 工具常见问题的解答，并可访问软件下载、文档与技术支持等在线服务。

访问地址：`https://solvnetplus.synopsys.com`

如提示，请输入用户名与密码。若尚无 Synopsys 账号，请按页面说明注册。若使用 SolvNetPlus 时需要帮助，请点击右上角菜单栏中的 **REGISTRATION HELP**。

联系客户支持

请访问 `https://solvnetplus.synopsys.com`。

Chapter 1

设计规划简介

Introduction to Design Planning

IC Compiler II 面向超大规模设计的高效设计规划（design planning）、布局、布线与分析。可以利用设计规划与芯片级分析能力，完成初始布图规划（floorplan）与可行性分析（feasibility analysis）。

本章包括以下主题：

- 设计规划概述
- 层次化设计规划流程
- 多层物理层次上的设计规划
- 设计规划期间的设计验证

1.1 设计规划概述

Design Planning Overview

设计规划是从寄存器传输级（RTL）到 GDSII 设计过程中不可缺少的一环。在设计规划阶段，可以在设计流程早期评估不同实现策略的可行性。对于大型设计，设计规划帮助你「分而治之」：把设计划分成更小、更易管理的部分，从而提高处理效率。

布图规划是把逻辑模块划分成物理模块（block）、确定模块尺寸并摆放模块、在布图规划层次上布局宏单元与标准单元，以及创建电源方案（power plan）的过程。布图规划的目标是提高后续物理设计步骤的效率，从而得到稳健、优化的设计。为了给物理模块生成准确的预算估计，需要生成时序估计（timing estimate），用以指导模块与设计中顶层单元之间的时序预算（timing budget）分配。布图规划也可以是迭代过程：重新塑形（shaping）模块、创建新的单元布局、重新分配时序预算，并重新检查顶层时序，直到得到优化的布图规划。

有效的布图规划把模块摆放到能缩短关键路径、降低布线拥塞（routing congestion）的位置，从而帮助实现时序收敛（timing closure）。难点在于：布图规划既要有良好的面积效率和较小的占用面积（footprint），又要为布线留出足够面积。

同样，设计规划支持基于图案的电源规划（pattern-based power planning），包括可用于多电压（multivoltage）设计的低功耗（low-power）技术。使用基于图案的电源规划，可以在设计中创建不同

的电压区域（voltage area），并为每个电压区域定义电源方案策略。工具根据你的规格创建电源地网格（power and ground mesh），并把宏单元与标准单元接到该网格上。你可以快速迭代不同的电源方案策略，考察不同实现并选出最优电源方案。

IC Compiler II 对通道式（channeled）与紧贴式（abutted）两种版图风格都支持完整的层次化设计规划。详见后文「支持的布图规划类型（Supported Types of Floorplans）」。

IC Compiler II 中的设计规划为层次化设计与平坦设计（flat design）提供下列能力，例如：

- 多层物理层次布图规划（Multi-Level Physical Hierarchy Floorplanning）
- 面向大型设计处理的高级抽象（abstraction）管理
- 借助图形化 **Task Manager** 的引导式布图规划
- 用于宏单元管理与布局辅助的数据流分析（data flow analysis）
- 宏单元与标准单元的快速布局
- 基于图案的电源规划
- 使用统一功耗格式（Unified Power Format，UPF）描述电源网络
- 虚拟原位时序优化（virtual in-place timing optimization）
- 引脚布局与总线成束（bus bundling）
- 时序预算

1.2 层次化设计规划流程

Hierarchical Design Planning Flow

层次化设计规划流程为管理大型设计提供了高效方法。把设计分成多个模块后，不同设计团队可以从 RTL 一直到物理实现，并行处理不同模块。处理较小的模块并使用多次例化模块（multiply instantiated block，MIB）可以缩短整体运行时间。

在下列情形下应考虑采用层次化方法：

- 设计规模大、复杂度高，以平坦形式处理会消耗过多计算资源。
- 你预计某些模块的交付可能推迟，并可能导致进度滑移。稳健的层次化方法能够容纳个别模块的后期设计变更，同时把对设计进度的干扰降到最低。
- 设计含有 RAM 等硬知识产权宏（hard intellectual property macro，hard IP），或者设计先前已经实现、可以转换后复用。

在 Design Compiler 拓扑模式（topographical mode）生成初始设计网表（netlist）之后，就可以在 IC Compiler II 中使用层次化方法进行设计规划。设计规划在层次化流程的第一阶段进行：把设计划分成模块，生成层次化物理设计约束，并把顶层时序预算分配给较低层的物理模块。

实现层次化设计规划的流程如图 **??** 所示。

图中的流程步骤如下：

1. 读入设计数据（`read_verilog_outline`）
2. 初始化设计（`initialize_floorplan`）

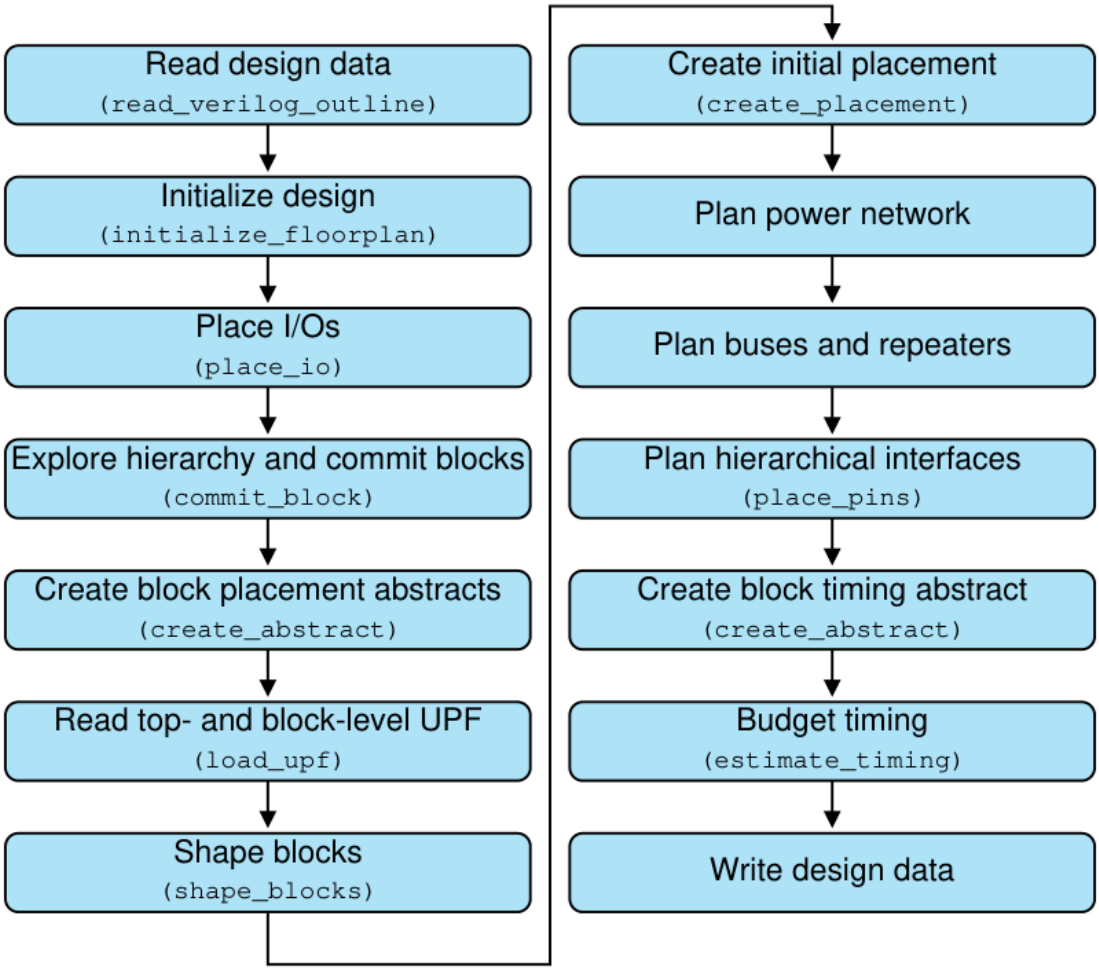


图 **1.1:** 层次化设计规划流程

3. 摆放 I/O（`place_io`）
4. 探索层次并提交模块（`commit_block`）
5. 创建模块布局抽象（`create_abstract`）
6. 读入顶层与模块级 UPF（`load_upf`）
7. 塑形模块（`shape_blocks`）
8. 创建初始布局（`create_placement`）
9. 规划电源网络
10. 规划总线与中继器（repeater）
11. 规划层次接口（`place_pins`）
12. 创建模块时序抽象（`create_abstract`）
13. 做时序预算（`estimate_timing`）
14. 写出设计数据

参见

设计划分；确定物理划分。

1.2.1 设计划分

Design Partitioning

读入网表并初始化布图规划之后，就可以为层次化设计项目确定物理划分。确定物理划分时考虑下列因素：

- 规模
  用规模相近的模块来划分设计。适当的时候，应把小模块组合起来，把大模块拆开。
- 功能
  按功能单元划分设计，以便验证与仿真。同时考虑顶层连接关系，并尽量减少模块引脚数量，以避免拥塞与时序问题。
- 布图规划风格
  不同的布图规划风格需要不同的物理层次来支持。紧贴式布图规划没有顶层逻辑；通道式布图规划则含有少量或大量顶层逻辑。
- 与 **Design Compiler** 拓扑模式一致的层次
  要在模块级与顶层交换 SCANDEF 信息，Design Compiler 拓扑模式中使用的物理层次也必须在 IC Compiler II 中使用。

1.2.2 确定物理划分

Deciding on the Physical Partitions

IC Compiler II 提供下列功能，帮助你确定物理划分。

使用层次浏览器

Using the Hierarchy Browser

可以使用层次浏览器（Hierarchy Browser）浏览设计层次、查看逻辑设计层次，并显示设计中层次单元与逻辑模块的信息。可以在版图视图或原理图视图中选择层次单元、叶单元或其他对象。查看器提供树形浏览器，帮助理解设计层次，并给出设计中各模块的信息，例如利用率（utilization）、标准单元数量等。

详见后文「探索设计层次（Exploring the Design Hierarchy）」。

提交模块

Committing Blocks

确定初始划分之后，就可以提交模块。模块在布图规划流程早期提交，并使用抽象视图生成设计的初始布图规划与时序预算。

关于提交模块的更多信息，见后文「提交设计模块（Committing Design Blocks）」。

1.3 多层物理层次上的设计规划

Design Planning at Multiple Levels of Physical Hierarchy

大型、复杂的片上系统（system-on-chip，SoC）设计需要能够同时管理多层物理层次的层次化版图方法。许多传统设计工具——包括物理规划、布局布线以及其他工具——仅限于两层物理层次：顶层与模块。IC Compiler II 为具有多层物理层次的设计提供全面支持，从而缩短出结果时间、改善结果质量（quality of results，QoR），并提高物理设计团队的生产率。使用 `set_editability` 命令启用或禁用特定模块以及用于规划的设计层次级别。

IC Compiler II 在若干方面支持具有多层物理层次的设计。

1.3.1 数据模型

Data Model

IC Compiler II 的数据模型内建对多层物理层次的支持。原生物理层次支持为多层物理层次的规划与实现带来显著优势。在执行模块塑形、布局、布线、时序以及其他步骤时，工具可以快速访问完成该功能所需的、与物理层次相关的特定数据。

1.3.2 模块塑形

Block Shaping

在具有多层物理层次的复杂设计中，模块塑形器需要知道每个子芯片（sub-chip）的目标面积、硬宏子对象所要求的宽长比约束，以及存在于同级到同级、父到子、子到父接口上的任何互连。对于多电压设计，模块塑形器还需要电压区域的目标位置。这些要求为塑形器增加了额外约束。对于多层物理层次规划，较低层子芯片上的模块塑形约束必须传播到顶层；这些约束表现为父级子芯片上的模块塑形约束。为提高性能，塑形器不需要每个子芯片或模块内部的完整网表内容。

IC Compiler II 数据模型为模块塑形提供完成这些目标所需的特定数据。对于多电压设计，工具读取 UPF，并在子芯片级保存功耗意图（power intent）。工具从数据模型中取出数据，按自然设计利用率计算目标，或取出指定设计目标的用户定义属性。

1.3.3 单元与宏单元布局

Cell and Macro Placement

模块塑形之后，单元与宏单元布局功能可以看到物理层次边界处互连路径与数据流的全局视图，以及到宏单元的连接关系。凭借这些信息，工具在每一层层次上为每个子芯片摆放宏单元。因为工具理解所有层次上边界处互连路径的相对位置要求，所以会在相邻子芯片边缘预留足够资源来容纳互连路径。布局器会预见层次引脚布局的需要，并把宏单元摆放到互连路径无需大量缓冲即可把信号驱动跨越宏单元的位置。

布局器通过考虑子芯片形状、位置以及全局宏单元布局，对子级与父级子芯片边界处的外部环境建模。利用这些信息，布局器在每一层层次上为每个子芯片创建单元布局作业。通过把子芯片布局分派到多个进程，工具在尽量利用计算资源的同时缩短周转时间。

1.3.4 电源规划

Power Planning

对于电源规划，IC Compiler II 提供了基于图案的方法。描述构造规则——形成环与网格所需的宽度、层和节距（pitch）——的图案，被应用到布图规划的不同区域，例如电压区域、宏单元组等。策略把单个图案或多个图案与区域关联起来。给定这些策略定义后，IC Compiler II 对电源方案进行特征化，并自动为所有层次上的子芯片生成策略定义。完整的电源方案以分布式方式生成。由于特征化后的策略是用每一子芯片级的对象来书写的，因此可以很容易地重新创建电源方案，以适应任意层次上的布图规划变更。

1.3.5 引脚布局

Pin Placement

模块形状形成、宏单元摆放且电源完成布线之后，引脚布局从所有层次取出接口数据，并调用全局布线器来确定摆放层次引脚的最优位置。全局布线器识别所有层次上的物理边界，以保证层次引脚接口处的资源得到有效利用。可能时，引脚会在多层之间对齐。与 IC Compiler II 的所有操作一样，全局布线器理解多次例化模块，并创建符合每一次 MIB 例化的走线。要为 MIB 摆放引脚，引脚布局算法会确定对所有实例都适用的最佳引脚位置，并保证每个实例上的引脚位置相同。此外，引脚布局会在整个层次中为所有子芯片（包括 MIB）创建穿越（feedthrough）。全局布线器跨 MIB 创建穿越，确定穿越的复用，并按需要把未使用的穿越接到电源或地。

1.3.6 时序预算

Timing Budgeting

IC Compiler II 估计层次接口处的时序，并为子芯片创建时序预算。IC Compiler II 中的时序预算器为全芯片内的全部子级接口引脚、中间层子芯片的父级与子级接口，以及最低层子芯片的初级引脚创建时序约束。整个设计可以并行并以分布式方式继续进行布局与优化。

要在版图中检查关键时序路径或执行其他设计规划任务，可以在全芯片上下文中交互地查看、分析并手工编辑设计的任意层次。可以选择只查看顶层，或查看多层层次。查看多层时，交互布线按设计是平坦的方式进行。完成后，走线被推入子级，并自动添加层次引脚。

1.4 设计规划期间的设计验证

Design Validation During Design Planning

在设计规划流程中推进时，可以使用 `check_design` 命令验证设计，并确认它已为下一设计阶段做好准备。该命令根据你指定的参数，对设计执行一项或多项检查。要用 `check_design` 检查设计，指定该命令以及 `-checks` 选项，后跟一个或多个检查关键字。下列示例执行 `dp_pre_floorplan` 检查：

```
icc2_shell> check_design -checks {dp_pre_floorplan}
```

`-checks` 选项中与设计规划相关的关键字如下。

- `dp_pre_floorplan`
  - 检查工艺文件（technology file）信息是否正确
  - 检查是否已设置层方向（layer direction）
  - 检查设计是否同时包含水平层与垂直层
- `dp_pre_create_placement_abstract`
  - 检查约束映射文件（constraint mapping file）是否为所有模块指定了 UPF 文件
  - 检查各模块轮廓视图（outline view）所用的 Verilog 文件是否已就位
- `dp_pre_block_shaping`
  - 检查是否所有黑盒（black box）都存在目标利用率或面积
  - 检查是否至少有一个模块或电压区域可供塑形
  - 检查核心区域（core area）与模块边界是否有效
  - 若设计包含多次例化模块（MIB），检查是否已设置模块网格（block grid）
  - 执行与 `shape_blocks` 命令相关的多电压检查
- `dp_pre_macro_placement`
  - 检查模块形状是否互不重叠
  - 检查模块与电压区域是否位于父级边界之内
  - 执行基本布局约束检查，例如相对位置约束重叠或越界、宏单元宽度与高度是否为 FinFET 网格（FinFET grid）的偶数倍，以及宏单元方向
  - 检查每个模块是否具有逻辑层次
  - 检查模块、电压区域、独占移动边界（exclusive movebound）与硬移动边界（hard movebound）的利用率

  - 执行与 create_placement 命令相关的多电压检查

- dp_pre_power_insertion
  - 检查是否已为布线层设置首选布线方向
  - 检查布线层是否存在布线轨道（track）
- dp_pre_pin_placement
  - 执行引脚约束检查，包括拓扑约束、个别约束、从个别引脚传播来的约束、成束引脚约束、模块引脚约束、与尺寸相关的约束、引脚引导（pin guide）与引脚阻挡（pin blockage）约束、布线引导（routing guide）约束以及布线阻挡约束
  - 检查拓扑约束中每一层的布线方向是否与模块边缘约束一致
  - 检查拓扑约束对中的边缘方向是否一致
  - 检查属于穿越约束的偏出边缘引脚（off-edge pin）
  - 检查是否可能与模块内部已有对象重叠，例如布线阻挡、引脚阻挡、固定引脚、预布线（pre-route）等
  - 运行 check_mib_for_pin_placement 命令所执行的检查
- dp_pre_push_down
  - 运行 check_mib_alignment 命令所执行的检查
- dp_pre_create_timing_abstract
  - 检查约束映射文件是否为所有模块指定了 Synopsys 设计约束（Synopsys Design Constraints，SDC）文件
- dp_pre_timing_estimation（时序抽象检查）
  - 检查顶层与模块级是否已定义模式（mode）与工艺角（corner）
  - 检查顶层模式与工艺角是否有对应的模块级模式与工艺角
  - 检查顶层是否至少包含一个时钟、至少一个使用非估计工艺角（nonestimated corner）的场景（scenario），并且该工艺角具有寄生参数（parasitic parameters）
- dp_pre_timing_estimation（布局抽象检查）
  - 执行 pre_create_timing_abstract 检查
  - 检查布局抽象是否存在
- dp_pre_budgeting
  - 检查 estimated_corner 是否可用
  - 执行与 dp_pre_timing_estimation 相同的检查
- dp_floorplan_rules
  - 检查线段奇偶规则（segment parity rule）

- 检查宏单元间距规则（macro spacing rule）
- 运行 `check_finfet_grid` 命令所执行的检查

Chapter 2

在设计规划模式下工作

Working in Design Planning Mode

IC Compiler II 支持设计规划模式（design planning mode）。该模式只启用与设计规划任务相关的功能、命令和操作，并限制工具在其他方面的使用。

本章包括以下主题：

- 用户界面
- 输入 `icc2_shell` 命令
- 使用变量
- 使用应用选项
- 查看 man 页
- 使用 Tcl 脚本
- 使用 setup 文件
- 使用命令日志文件
- 并行运行任务
- 监视分布式任务

2.1 用户界面

User Interfaces

IC Compiler II 在 UNIX 或 Linux 的 X Window 环境中运行。它提供灵活的工作环境，同时具有 shell 命令行界面和图形用户界面（graphical user interface，GUI）。shell 命令行界面 `icc2_shell` 在 IC Compiler II 会话期间始终可用。可以在 `icc2_shell` 或 GUI 中启动或退出会话，也可以在会话期间随时打开或关闭 GUI。

IC Compiler II 使用工具命令语言（tool command language，Tcl）。Tcl 被 EDA 行业的许多应用采用。使用 Tcl，可以通过编写可复用的过程与脚本扩展 `icc2_shell` 命令语言（见 *Using Tcl With Synopsys Tools* 手册）。

下列主题说明如何使用命令行界面启动与退出工具：

- 启动命令行界面
- 退出 IC Compiler II 工具

关于 GUI 的用法，见 *IC Compiler II Graphical User Interface User Guide*。

2.1.1 启动命令行界面

Starting the Command-Line Interface

`icc2_shell` 命令行界面是纯文本环境，在命令行提示符下输入命令。它通常用于脚本、批处理模式和一键操作（push-button operation）。

要在设计规划模式下启动 `icc2_shell`：

1. 确保 `bin` 目录的路径已包含在 `PATH` 变量中。
2. 在 UNIX 或 Linux shell 中输入 `icc2_shell` 命令，并指定 `-dp_mode` 选项。

```
% icc2_shell -dp_mode
```

启动 `icc2_shell` 时，可以在命令行上加入各种选项。例如可以使用：

- `-f script_file_name`：执行脚本
- `-x command`：执行一条 `icc2_shell` 命令
- `-output_log_file file_name`：创建会话的日志文件
- `-h`：显示可用选项列表（不启动 `icc2_shell`）

启动时，`icc2_shell` 执行下列任务：

1. 创建命令日志文件。
2. 读取并执行 setup 文件。
3. 执行命令行上分别由 `-f` 与 `-x` 选项指定的任何脚本文件或命令。
4. 在启动 `icc2_shell` 的 shell 中显示程序头和 `icc2_shell>` 提示符。

参见

使用命令日志文件；使用 setup 文件；使用 Tcl 脚本。

2.1.2 退出 IC Compiler II 工具

Exiting the IC Compiler II Tool

可以随时结束会话并退出 IC Compiler II。要退出工具，使用 `exit` 或 `quit` 命令。

注意

从命令行退出工具时，工具退出且不保存已打开的模块。

2.2 输入 icc2_shell 命令

Entering icc2_shell Commands

通过 `icc2_shell` 命令与 IC Compiler II 交互。这些命令基于 Tcl，并包含实现特定 IC Compiler II 功能所需的命令扩展。IC Compiler II 命令语言提供与 UNIX 命令 shell 类似的能力，包括变量、命令的条件执行以及控制流命令。你可以：

- 在 `icc2_shell` 的 `icc2_shell>` 提示符下交互输入单条命令
- 在 GUI 的控制台命令行上交互输入单条命令
- 运行一个或多个 Tcl 脚本，即包含 `icc2_shell` 命令的文本文件（见「使用 Tcl 脚本」）

输入命令、选项或文件名时，可以在已经键入足够字符、能够唯一确定名称之后按 Tab 键，以减少键入。IC Compiler II 会补全其余字符。若所键入的字符可用于多个名称，工具会列出符合条件的名称，可以用方向键和 Enter 键从中选择。

若需要复用命令行界面输出中的某条命令，可以选中该部分，把指针移到 `icc2_shell` 命令行，并用鼠标中键单击以粘贴。

运行命令时，IC Compiler II 会在 `icc2_shell` 中回显命令输出（包括处理消息以及任何警告或错误消息）；若 GUI 已打开，也会在控制台日志视图中回显。默认情况下工具不使用分页模式（page mode），因此输出可能会滚动。要启用分页模式，把 `sh_enable_page_mode` 变量设置为 `true`。

2.2.1 中断或终止命令处理

Interrupting or Terminating Command Processing

若为命令输入了错误的选项，或输入了错误的命令，可以中断命令处理并仍留在 `icc2_shell` 中。要中断或终止命令，按 **Ctrl+C**。

某些命令和进程不能被中断。要停止这些命令或进程，必须在系统级终止 `icc2_shell`。终止进程或 shell 时，不会保存任何数据。

使用 **Ctrl+C** 时请记住下列几点：

- 若正在处理脚本文件，并且中断了其中一条命令，则脚本处理被中断，后续脚本命令不再被处理。
- 若在命令对中断作出响应之前按了三次 **Ctrl+C**，`icc2_shell` 会被中断，并以下列消息退出：

```
Information: Process terminated by interrupt.
```

2.2.2 获取命令信息

Getting Information About Commands

使用 IC Compiler II 时，可获得下列在线信息资源：

- 命令帮助，即关于某条 icc2_shell 命令的信息
- man 页

参见

查看 man 页。

2.2.3 显示命令帮助

Displaying Command Help

命令帮助可以是 icc2_shell 命令的简要说明，也可以是该 icc2_shell 命令所支持的选项与参数列表。

- 要显示某条 icc2_shell 命令的简要说明，输入 help 命令并后跟命令名。例如，要显示 report_timing 命令的简要说明，使用下列命令：

```
icc2_shell> help report_timing
```

- 要显示某条 icc2_shell 命令所支持的选项，在 icc2_shell 命令行上输入命令名以及 -help 选项。例如，要查看 report_timing 命令所支持的选项，使用下列命令：

```
icc2_shell> report_timing -help
```

2.3 使用变量

Using Variables

一般而言，IC Compiler II 通过应用选项（application option）而不是应用变量来修改默认行为。工具确实支持用户定义的 Tcl 变量，以及少量应用变量，例如 search_path 变量。

使用 printvar 命令列出可用的应用变量。

示例：

```
printvar *
printvar search_path
```

2.4 使用应用选项

Using Application Options

IC Compiler II 使用应用选项控制工具行为。应用选项采用下列命名约定：

```
category[.subcategory].option_name
```

其中 category 是受该应用选项影响的引擎名称。某些应用选项类别还有子类别，以进一步细化应用选项所影响的范围。

应用选项具有全局作用域或模块作用域。

- 模块作用域的应用选项只作用于设置它们的那个模块。它们保存在设计库中，并在工具会话之间保持。
- 全局作用域的应用选项作用于所有模块，但仅在当前会话内有效。它们不保存在设计库中；必须在每个 `icc2_shell` 会话中指定。可以考虑把全局设置加入 `.synopsys_icc2.setup` 文件。

要得到可用应用选项的列表，使用 `get_app_options` 命令。默认情况下，该命令列出所有应用选项。要限制所报告的应用选项，为该命令提供一个模式字符串作为参数。

例如，要列出所有可用的应用选项，使用下列命令：

```
icc2_shell> get_app_options
```

要列出所有可用的 timer 应用选项，使用下列命令：

```
icc2_shell> get_app_options timer.*
```

要生成应用选项报告，使用 `report_app_options` 命令。

关于应用选项的详细信息，见 *IC Compiler II Data Model User Guide* 中的“Application Options”。

参见

使用 setup 文件。

2.5 查看 man 页

Viewing Man Pages

要显示某条 `icc2_shell` 命令或某个应用变量的 man 页，输入 `man` 命令并后跟命令名或变量名。例如，要查看 `report_timing` 命令的 man 页，使用下列命令：

```
icc2_shell> man report_timing
```

要显示某个 `icc2_shell` 应用选项的 man 页，输入 `man` 命令并后跟选项名。也可以查看汇总下列各类应用选项的 man 页：

- 类别

  输入 `man` 命令并后跟 `category_options`。例如，要查看汇总全部 floorplan 应用选项的 man 页，使用下列命令：

  ```
  icc2_shell> man plan_options
  ```

- 子类别

  输入 `man` 命令并后跟 `category.subcategory_options`。例如，要查看汇总全部 floorplan 布局应用选项的 man 页，使用下列命令：

  ```
  icc2_shell> man plan.place_options
  ```

- 命令

  输入 `man` 命令并后跟 `command_options`。例如，要查看汇总影响 `report_timing` 命令的全部应用选项的 man 页，使用下列命令：

  ```
  icc2_shell> man report_timing_options
  ```

注意

若在 `icc2_shell` 命令行上输入 `man` 命令，man 页显示在 IC Compiler II shell 中；若 GUI 已打开，也显示在控制台日志视图中。若在 GUI 的控制台命令行上输入该命令，man 页显示在 GUI 的 man 页查看器中。

2.6 使用 Tcl 脚本

Using Tcl Scripts

可以使用 Tcl 脚本完成例行、重复或复杂的任务。把一系列 `icc2_shell` 命令放入文本文件，即可创建命令脚本文件。任何 `icc2_shell` 命令都可以在脚本文件中执行。

在 Tcl 中，行首的井号（`#`）表示注释。例如：

```
# This is a comment
```

关于编写脚本和脚本文件的更多信息，见 *Using Tcl With Synopsys Tools* 手册。

使用下列方法之一运行 Tcl 脚本：

- 启动 IC Compiler II 时，在 `icc2_shell` 命令上使用 `-f` 选项。
- 在 `icc2_shell` 命令行上使用 `source` 命令。
- 在 GUI 中选择 **File > Execute Script**。

运行命令出错时，IC Compiler II 会引发 `TCL_ERROR` 条件，从而立即停止脚本执行。要容忍错误并允许脚本继续执行：

- 对可能产生错误的命令，用 Tcl 的 `catch` 命令检查 `TCL_ERROR` 错误条件；或者
- 在脚本文件中把 `sh_continue_on_error` 变量设置为 `true`。

参见

启动命令行界面。

2.6.1 从电子表格数据生成 Tcl 脚本

Generating a Tcl Script From Spreadsheet Data

可以使用 `generate_pg_script -input` 命令，从电子表格中的数据快速生成电源地（PG）脚本。

电子表格中的数据必须采用特定格式，并且电子表格应为 CSV 文件。电子表格格式见本主题末尾的示例。使用这些示例构建 CSV 文件时，务必不要改动以黄色高亮的任何信息。以任何方式改动这些文本，例如换成其他文本或不遵守大小写，都可能导致无法生成 PG 脚本。

根据电子表格数据写入脚本文件的命令如下。不支持其他 PG 命令。

- `create_pg_composite_pattern`
- `create_pg_special_pattern`
- `create_pg_wire_pattern`
- `set_pg_strategy`

- set_pg_strategy_via_rule

在下列示例中，generate_pg_script 命令从 special_pattern_spreedsheet.csv 文件取得数据，并输出带有 PG 命令的 test.tcl 文件。

```
icc2_shell> generate_pg_script -input special_pattern_spreedsheet.csv \
   -output test.tcl
```

表 **2.1:** 示例：输入电子表格与输出 Tcl 文件

CSV 数据示例	为该 **CSV** 数据生成的 **Tcl** 脚本
按本主题末尾的电子表格模板填写。黄色高亮单元格必须原样保留，包括大小写。本例对应 create_pg_special_pattern。 create_pg_special_pattern channelPattern1 -insert_channel_straps { {layer: M7} {direction: vertical} {width 1} {channel_between_objects: macro placement_blockage voltage_area} } create_pg_special_pattern channelPattern2 -insert_channel_straps { {layer: M1Z} {direction: vertical} {width 2} {spacing: 2} {channel_threshold: 15} {channel_between_objects: macro placement_blockage} {check_one_layer: TRUE} }	# Generate PG create_pg_special_pattern script from spreadsheet

同一 Tcl 脚本如下，便于复制：

```
# Generate PG create_pg_special_pattern script from spreadsheet
create_pg_special_pattern channelPattern1 -insert_channel_straps { {layer: M7} {direction: vertical
    } {width 1} {channel_between_objects: macro placement_blockage voltage_area} }
create_pg_special_pattern channelPattern2 -insert_channel_straps { {layer: M1Z} {direction:
    vertical} {width 2} {spacing: 2} {channel_threshold: 15} {channel_between_objects: macro
    placement_blockage} {check_one_layer: TRUE} }
```

代码 **2.1:** 由表 2-1 的 CSV 数据生成的 Tcl 脚本

示例 **1**

Example 1

`pg_strategy_via_rule_command` 的电子表格模板（Spreadsheet Template）。

示例 **2**

Example 2

`composite_pattern_command` 的电子表格模板。

示例 **3**

Example 3

`special_pattern_command`（`insert_channel_straps`）的电子表格模板。

示例 **4**

Example 4

`special_pattern_command`（`insert_power_switch_alignment_straps`）的电子表格模板。

示例 **5**

Example 5

`special_pattern_command`（`insert_physical_cell_alignment_straps`）的电子表格模板。

示例 **6**

Example 6

`special_pattern_command`（`insert_terminal_alignment_straps`）的电子表格模板。

示例 **7**

Example 7

`special_pattern_command`（`honor_max_stdcell_strap_distance`）的电子表格模板。

示例 **8**

Example 8

`strategy_command` 的电子表格模板。

2.7 使用 setup 文件

Using Setup Files

启动 IC Compiler II 时，它自动执行 `.synopsys_icc2.setup` 文件中的命令。

工具在主目录以及项目目录（启动 IC Compiler II 时的当前工作目录）中查找该文件。文件按下列顺序读取：

1. 主目录中的 `.synopsys_icc2.setup` 文件
2. 项目目录中的 `.synopsys_icc2.setup` 文件

setup 文件可以包含执行基本任务的命令，例如初始化应用选项和设置 GUI 选项。可以把命令和 Tcl 过程加入主目录与项目目录中的 setup 文件。例如：

- 要设置定义 IC Compiler II 工作环境的应用选项，在主目录中创建 setup 文件。
- 要设置影响某模块处理的、项目专用或模块专用的应用选项，在设计目录中创建 setup 文件。

参见

用户界面；使用应用选项；使用 Tcl 脚本。

2.8 使用命令日志文件

Using the Command Log File

命令日志文件记录 IC Compiler II 所处理的 `icc2_shell` 命令，包括 setup 文件命令和应用选项设置。默认情况下，IC Compiler II 把命令日志写到调用 `icc2_shell` 时所在目录中名为 `icc2_command.log` 的文件。

可以在 `.synopsys_icc2.setup` 文件中设置 `sh_command_log_file` 变量，以更改命令日志文件的名称。对该变量的任何更改都应在启动 IC Compiler II 之前完成。若用户定义的或项目专用的 setup 文件不含该变量，工具会自动创建 `icc2_command.log` 文件。

每个 IC Compiler II 会话都会覆盖命令日志文件。要保存命令日志文件，请移动或重命名它。可以使用命令日志文件来：

- 为特定实现策略生成脚本
- 记录物理实现过程
- 记录所遇到的任何问题

2.9 并行运行任务

Running Tasks in Parallel

要在设计的若干模块上高效地运行同一任务，可以使用 `run_block_script` 命令启用分布式处理并并行执行这些任务。`run_block_script` 命令接受一个 Tcl 脚本，并把脚本中的命令应用到所指定的模块。更多信息见 *IC Compiler II Implementation User Guide* 中的“Enabling Multicore Processing”。

2.10 监视分布式任务

Monitoring Distributed Tasks

IC Compiler II 支持分布式处理管理器（Distributed Processing Manager），帮助你在分布式任务运行时监视其进度。分布式处理管理器与 `create_abstract` 等分布式命令通信，以提供最新状态。任务完成时，GUI 会更新以显示该任务已结束。

要使用分布式处理管理器监视正在运行的作业：

1. 用 `run_monitor_gui` 命令启动分布式处理管理器。

```
icc2_shell> run_monitor_gui
10948
```

该命令返回由此命令创建的 dpmanager 任务的进程号。

2. 启动分布式任务。

```
icc2_shell> create_abstract -placement \
   -host_options block_script -all_blocks
Submitting job for block BLENDER_0 ...
Submitting job for block BLENDER_1 ...
Submitting job for block BLENDER_2 ...
...
```

3. 在分布式处理管理器窗口中监视正在运行的任务，如图 **??** 所示。

当所有任务完成时，分布式处理管理器把全部作业/Task ID 的 **Status** 列更新为 `COMPLETED`，并把 **Job History** 窗口中的状态从 not done 更新为 done。

最多可以打开两个分布式处理管理器窗口。要打开第二个分布式处理管理器窗口，再次使用 `run_monitor_gui` 命令。要关闭任何已打开的分布式处理管理器窗口并重新启动单个窗口，使用 `run_monitor_gui -kill` 命令。

若下列命令以分布式模式运行，分布式处理管理器会监视它们的进度：

- `create_abstract -all_blocks | -blocks`
- `create_placement -floorplan`
- `estimate_timing`
- `merge_abstract`
- `route_global -floorplan -virtual_flat`
- `shape_blocks`

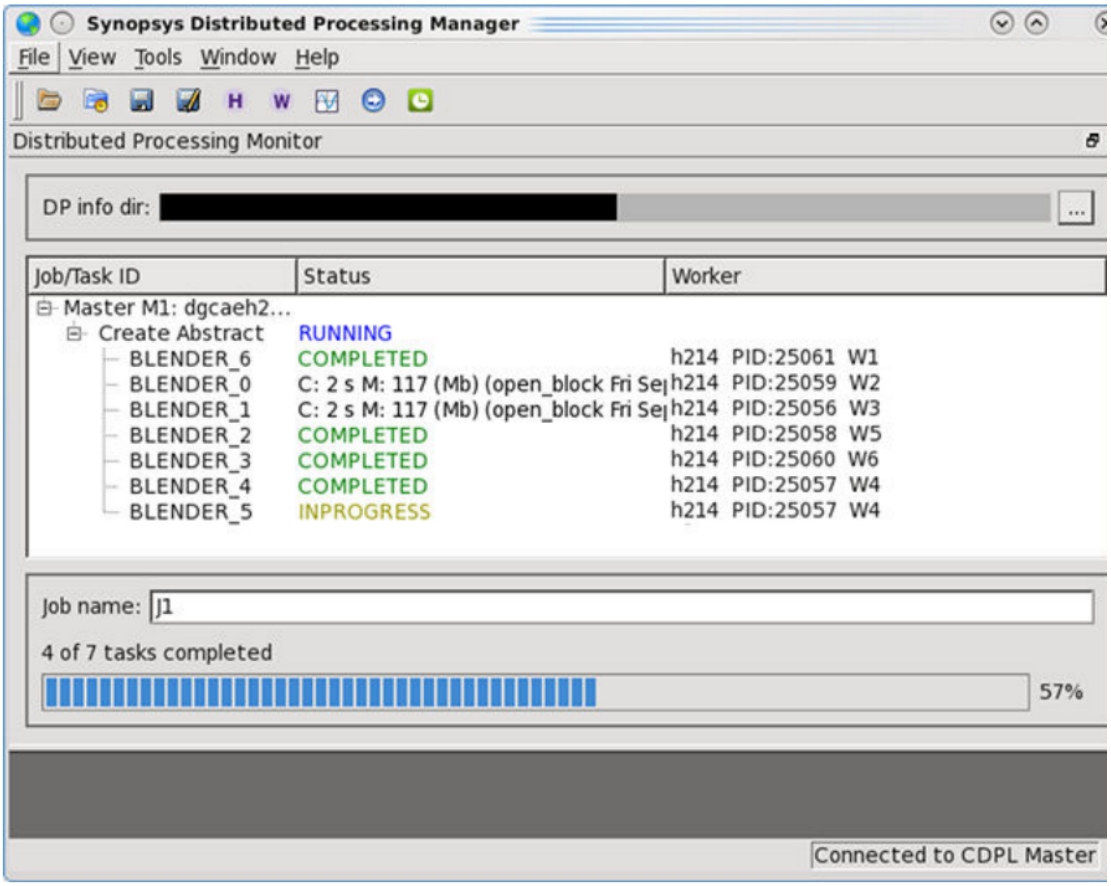


图 **2.1:** 分布式处理管理器

Chapter 3

拆分约束

Splitting Constraints

要生成时序预算，必须把芯片级 Synopsys 设计约束（Synopsys Design Constraints，SDC）与统一功耗格式（Unified Power Format，UPF）文件划分成顶层文件和模块级文件。当使用模块的抽象视图时，IC Compiler II 使用顶层约束来约束顶层逻辑。

本章包括以下主题：

- 拆分约束流程
- 拆分约束的输出文件
- 拆分约束示例 1
- 拆分约束示例 2

3.1 拆分约束流程

Split Constraints Flow

若设计只有芯片级约束，可以使用 `split_constraints` 命令划分全芯片 SDC 与 UPF 约束，并创建单独的顶层与模块级约束文件。运行 `split_constraints` 之后，顶层约束文件包含顶层 UPF，以及全部顶层时序和顶层到模块边界的时序，而不包含模块级内部约束。模块级约束文件在模块级优化期间使用。

`split_constraints` 命令生成的模块级约束文件包含该模块的内部约束，并且只应施加一次。在施加时序预算之前，必须施加手工生成的约束，或者施加由 `split_constraints` 命令生成的约束。

若设计已经具有模块级 SDC 与 UPF 约束，可以忽略本节，不必运行 `split_constraints` 命令而继续。

要拆分芯片级 SDC 与 UPF 约束：

1. 读入完整的 Verilog 网表。

```
icc2_shell> read_verilog -top ORCA.v
```

拆分约束流程要求用 read_verilog 命令读入完整的 Verilog 网表。此步骤不能使用 read_verilog_outline 命令。

2. 为整个设计加载并提交 UPF 约束文件。

```
icc2_shell> load_upf ORCA.upf
icc2_shell> commit_upf
```

load_upf 命令读入 UPF 约束，commit_upf 命令把 UPF 约束应用到设计上。commit_upf 还会尝试解决 UPF 文件中指定的功耗意图与设计中实际电源、地网络之间的任何冲突。

3. 为设计建立时序环境。
对于具有多个模式（mode）与工艺角（corner）的设计，必须使用下列命令中的一个或多个：

   - create_corner
   - create_mode
   - read_sdc
   - set_corner_status
   - set_parasitic_parameters
   - set_process_number
   - set_temperature
   - set_voltage

若设计使用 UPF 描述电源网络，必须对 UPF 文件中定义的所有供电网络组使用 set_voltage 命令。

4. 复位先前的任何预算约束，并指定要写出约束的模块。

```
icc2_shell> set_budget_options -reset
icc2_shell> set_budget_options -add_blocks {I_ORCA_TOP/I_BLENDER_1 \
   I_ORCA_TOP/I_BLENDER_2 I_ORCA_TOP/I_BLENDER_3 \
   I_ORCA_TOP/I_BLENDER_4 I_ORCA_TOP/I_BLENDER_5 \
   I_ORCA_TOP/I_BLENDER_6 I_ORCA_TOP/I_BLENDER_7}
```

若设计使用 UPF 描述电源网络，必须对 UPF 文件中定义的所有供电网络组使用 set_voltage 命令。

5. 把时序约束和 UPF 文件拆分为每个模块各自的文件。

```
icc2_shell> split_constraints
```

若设计包含多层物理层次，用 -design_subblocks 选项指定中间层次。当嵌套模块在其父模块的上下文中使用时，这些嵌套模块不会被抽象。

```
icc2_shell> split_constraints -design_subblocks {block_1 block_2}
```

3.1.1 拆分约束的选项

Split Constraints Options

使用 `split_constraints` 命令的选项控制约束如何拆分。

- 用 `-sdc_only` 选项或 `-upf_only` 选项只拆分 SDC 文件或只拆分 UPF 文件。

```
icc2_shell> split_constraints -sdc_only
icc2_shell> split_constraints -upf_only
```

- 在使用层次抽象（hierarchical abstract）时，用 `-hier_abstract_subblocks block_names` 选项保留子模块的内部约束。使用该选项表示指定的模块以层次抽象表示。
- 用 `-modes` 选项只为某些模式写出约束文件。

```
icc2_shell> split_constraints -modes {s1.mode}
```

- 用 `-corners` 选项只为某些工艺角写出约束文件。

```
icc2_shell> split_constraints -corners {s1.corner}
```

- 用 `-compress gzip` 选项压缩输出的约束文件。

```
icc2_shell> split_constraints -compress gzip
```

- 用 `-output` 选项指定用于输出的目录。

```
icc2_shell> split_constraints -output split2
```

3.2 拆分约束的输出文件

Split Constraints Output Files

图 **??** 给出 `split_constraints` 命令为 ORCA 设计创建的目录结构。



图 **3.1:** `split_constraints` 之后的约束文件

图中的目录与文件如下：

```
split/
ORCA.cut/
top.tcl
top.upf
design.tcl
clocknets.tcl
mode_s1.tcl
corner_s1.tcl
```

```
scenario_s1.tcl
BLENDER_0/
top.tcl
top.upf
design.tcl
clocknets.tcl
mode_s1.tcl
corner_s1.tcl
scenario_s1.tcl
.../
top.tcl
top.upf
design.tcl
clocknets.tcl
mode_s1.tcl
corner_s1.tcl
scenario_s1.tcl
BLENDER_6/
top.tcl
top.upf
design.tcl
clocknets.tcl
mode_s1.tcl
corner_s1.tcl
scenario_s1.tcl
mapfile
```

代码 **3.1:** split_constraints 创建的目录结构

在本例中，ORCA 是顶层设计，`BLENDER_0` 到 `BLENDER_6` 是模块。`split_constraints` 命令为顶层设计以及每一个用 `set_budget_options -add_blocks` 命令指定的模块创建下列文件：

- `top.tcl`：提供工艺角与模式定义，并按正确顺序 source 所有其他文件。只需 source 这一个文件，即可为指定模块施加拆分后的约束。
- `top.upf`：以 UPF 格式提供功耗约束。
- `design.tcl`：提供带有模式与工艺角关联的、全设计范围的约束。其中包含 `set_corner_status` 命令，用于为模式与工艺角的组合配置分析设置。
- `clocknets.tcl`：提供时钟网络的约束。
- `mode_modename.tcl`：为时钟、例外、延迟等提供模式专用约束。工具为每个已定义的模式创建一个 Tcl 文件。
- `corner_cornername.tcl`：为降额因子（derating factor）、寄生定义以及工艺、电压与温度（process, voltage, and temperature，PVT）值提供工艺角专用约束。工具为每个已定义的工艺角创建一个 Tcl 文件。
- `scenario_scenarioname.tcl`：为时钟延迟（clock latency）、时钟不确定性（clock uncertainty）以及其他因素提供场景专用约束。工具为每个已定义的场景创建一个 Tcl 文件。

split_constraints 命令还会创建约束映射文件 mapfile，其中列出每个模块的 SDC 与 UPF 约束文件。本例中 mapfile 的内容如下：

```
BLENDER_0 CLKNET BLENDER_0/clocknets.tcl
BLENDER_0 SDC BLENDER_0/top.tcl
BLENDER_0 UPF BLENDER_0/top.upf
...
BLENDER_6 CLKNET BLENDER_6/clocknets.tcl
BLENDER_6 SDC BLENDER_6/top.tcl
BLENDER_6 UPF BLENDER_0/top.upf
ORCA CLKNET ORCA/clocknets.tcl
ORCA SDC ORCA/top.tcl
ORCA UPF TOP/top.upf
```

3.3 拆分约束示例 1

Split Constraints Example 1

示例 1 的设计如图 **??** 所示。

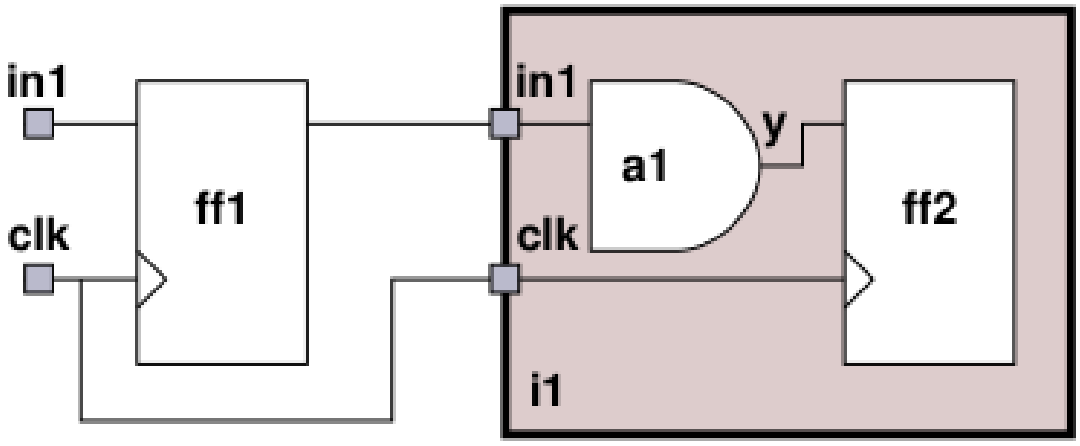


图 **3.2:** 示例 1 的设计

该设计示例使用下列芯片级 SDC 约束文件。

```
create_clock -period 1 [get_ports clk]
set_input_delay 0.5 [get_ports in1]
set_multicycle_path 2 -from ff1/clk -through i1/a1/y
```

代码 **3.2:** 芯片级 SDC 约束文件

split_constraints 命令把芯片级 SDC 约束文件划分成下列顶层与模块级 mode_modename.tcl 约束文件。

```
# Top-level constraints
create_clock -period 1 [get_ports clk]
set_input_delay 0.5 [get_ports in1]
set_multicycle_path 2 -from ff1/ck -through i1/a1/y
```

代码 **3.3:** split_constraints 之后的顶层约束文件

```
# Block-level constraints
create_clock -add -name "clk" -period 1 [get_ports clk]
create_clock -name virtual_clk -period 1
set_multicycle_path 2 -from virtual_clk -through [get_ports in] \
   -through a1/y
```

代码 **3.4:** split_constraints 之后的 i1 约束文件

3.4 拆分约束示例 2

Split Constraints Example 2

示例 2 的设计如图 **??** 所示。

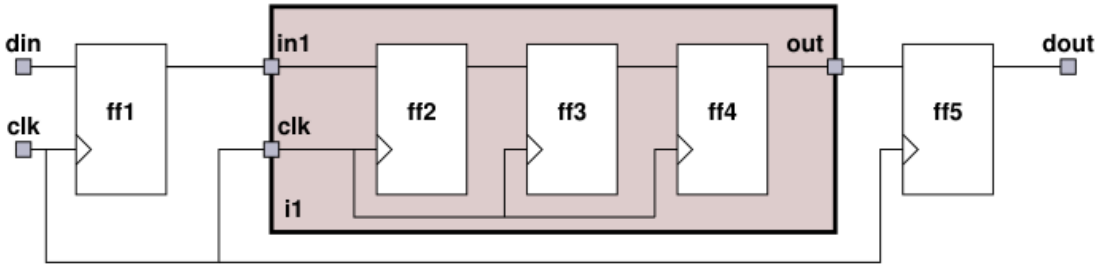


图 **3.3:** 示例 2 的设计

该设计示例使用下列顶层 SDC 约束文件。

```
create_clock -period 1 [get_ports clk]
set_input_delay -clock [get_clocks clk] 0.5 [get_ports din]
set_output_delay -clock [get_clocks clk] 0.5 [get_ports dout]
set_false_path -from [get_pins i1/ff2/CK] -to [get_pins i1/ff3/D]
```

代码 **3.5:** 芯片级 SDC 约束文件

`split_constraints` 命令把芯片级 SDC 约束文件划分成下列顶层与模块级 `mode_modename.tcl` 约束文件。

```
# Top-level constraints
create_clock -name clk -period 1 [get_ports {clk}]
set_input_delay -clock [get_clocks {clk}] 0.5 [get_ports {din}]
set_output_delay -clock [get_clocks {clk}] 0.5 [get_ports {dout}]
```

代码 **3.6:** split_constraints 之后的顶层约束文件

```
# Block-level constraints
create_clock -name "clk" -period 1 [get_ports {clk}]
set_false_path -from [get_pins {ff2/CK}] -to [get_pins {ff3/D}]
```

代码 **3.7:** split_constraints 之后的模块 i1 约束文件

Chapter 4

创建布图规划

Creating a Floorplan

本章说明如何根据已有网表（netlist）或布图描述创建并细化布图规划（floorplan）。布图规划描述内核（core）的尺寸，标准单元（standard cell）行与布线通道（routing channel）的形状及摆放，标准单元布局（placement）约束，以及外围输入/输出（I/O）、电源（power）、地（ground）、拐角（corner）与填充焊盘（filler pad）单元的摆放。

更多信息见下列主题：

- 支持的布图类型
- 读入 Verilog 网表
- 为大型设计创建密集与稀疏轮廓视图
- 创建初始布图
- 创建 L、T 或 U 形布图
- 创建复杂直角多边形布图
- 基于物理规则文件创建布图
- 调整布图宽高比
- 定义布线轨道
- 翻转布图中的第一行
- 在不扰动指定对象类型的情况下更新布图
- 验证 FinFET 网格
- 缩放芯片面积
- 报告布图信息
- 读入、写出与复制布图信息
- 为先进工艺节点创建并验证布图规则

4.1 支持的布图类型

Supported Types of Floorplans

IC Compiler II 工具支持多种布图风格，以适应设计要求。通道式、紧贴式与窄通道布图风格在下列各节说明。

- 通道式布图
- 紧贴式布图
- 窄通道布图

4.1.1 通道式布图

Channeled Floorplans

通道式布图（channeled floorplan）在块（block）之间留出间距，用于摆放顶层宏单元（macro cell），如图 **??** 所示。这些间距使工具能够在块之间放置标准单元。

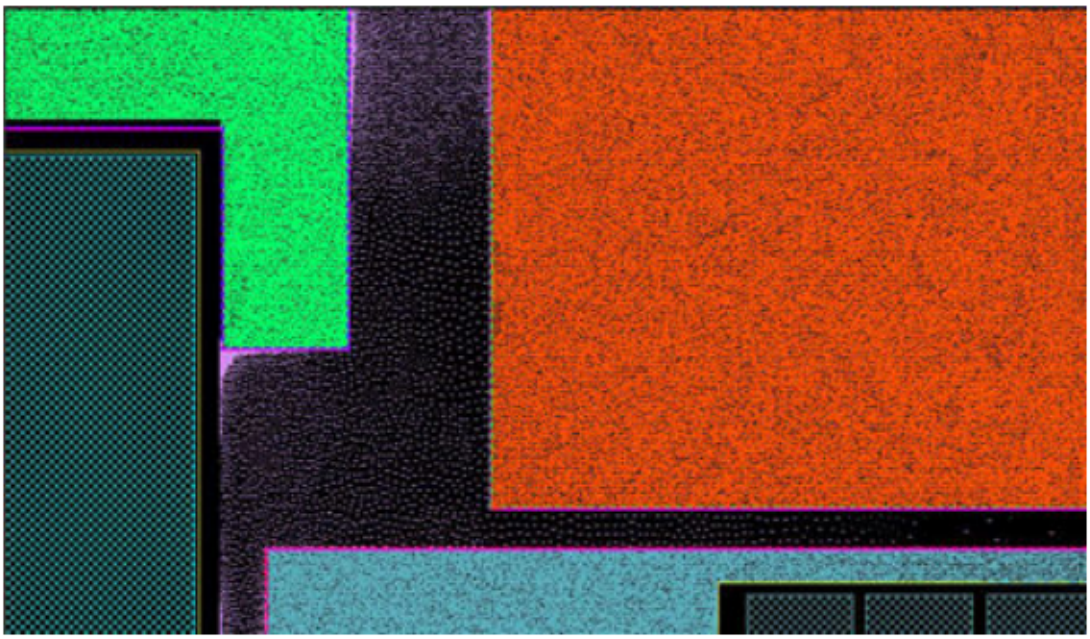


图 **4.1:** 通道式布图

4.1.2 紧贴式布图

Abutted Floorplans

在紧贴式布图（abutted floorplan）中，块彼此相接，工具不在块之间为宏单元布局分配空间。采用紧贴式布图的设计不需要顶层优化（top-level optimization），因为全部逻辑都已下推到块中。不过，紧贴式布图可能需要跨块布线（over-the-block routing）才能满足设计要求。这种布图风格也更需要关注时钟规划（clock planning）与馈通（feedthrough）管理。对紧贴式布图设计，布线拥塞（routing

congestion）也可能成为问题。由于紧贴式布图实现难度较大，常常改用窄通道布图风格，以尽量减小块之间的间距。

紧贴式布图的示例如图 **??** 所示。

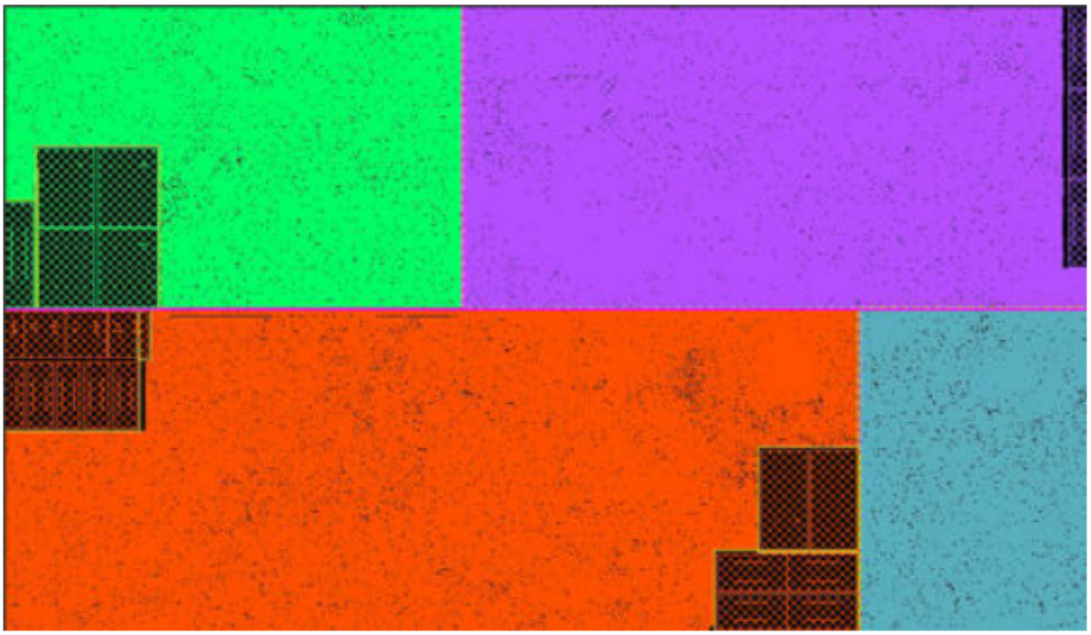


图 **4.2:** 紧贴式布图

4.1.3 窄通道布图

Narrow-Channel Floorplans

窄通道布图（narrow-channel floorplan）在通道式布图与紧贴式布图之间取得平衡。在不需要顶层单元的位置，可以使某些块紧贴；在其他区域，可以在某些块之间预留通道，用于顶层时钟布线或其他特殊用途单元。窄通道布图的示例如图 **??** 所示。

4.2 读入 Verilog 网表

Reading the Verilog Netlist

要读入 Verilog 网表并开始布图规划，按下列步骤进行。

1. 用 `create_lib` 命令创建设计库（design library）以容纳设计。若库已存在，用 `open_lib` 命令打开该库。

```
icc2_shell> set libs [glob -directory ${lib_dir} *.ndm]
icc2_shell> create_lib -technology technology_file.tf \
   -ref_libs $libs lib/design
```

2. 用 `read_verilog_outline` 命令读入 Verilog 网表，并指定适当的选项。

```
icc2_shell> read_verilog_outline -top ORCA ORCA.v
```

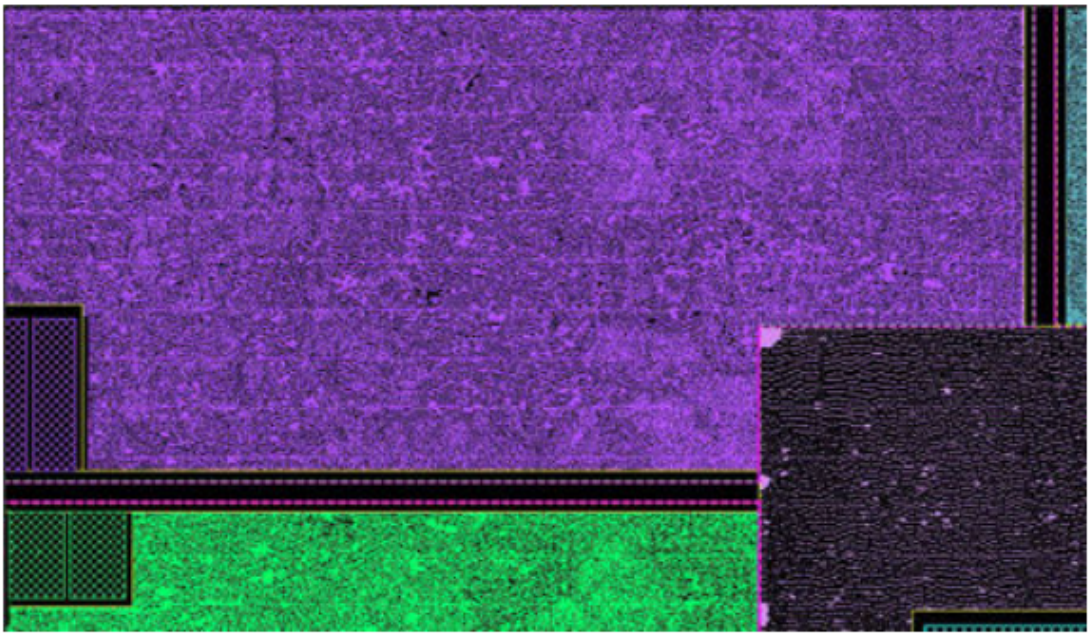


图 **4.3:** 窄通道布图

IC Compiler II 工具中的 Verilog 网表轮廓阅读器可以高效处理非常大的设计。为节省资源并缩短周转时间，网表阅读器为设计层次各层中的块创建轮廓视图（outline view）。轮廓设计包含层次，但不包含叶单元（leaf cell）或网线。

`read_verilog_outline` 命令并不划分设计。要划分设计，用 `commit_block` 命令提交设计块，并用 `expand_outline` 命令把块展开为完整网表表示。关于提交块的更多信息，见「提交设计块（Committing Design Blocks）」。

也可以用扁平网表阅读器按完整网表流程处理设计。关于用扁平网表阅读器读入设计的信息，见 `read_verilog` 命令。

4.3 为大型设计创建密集与稀疏轮廓视图

Creating Dense and Sparse Outline Views for Large Designs

`read_verilog_outline` 命令读入大型设计的 Verilog 网表，并准备设计数据库，以便高效处理设计数据。在早期布图规划期间，可以只保存硬宏（hard macro）、接口逻辑（interface logic）、标准单元面积与标准单元数量，从而降低工具的磁盘与内存需求。这组精简信息存放在轮廓视图中；在布图级布局与引脚分配（pin assignment）之前细化块外形时，该视图已经足够。

为了在设计规划期间尽量提高工具性能，可以进一步调整设计中各块轮廓视图的内容。对非常大的设计做设计规划时，块应只包含完成有意义分析所必需的最少信息。为支持这种分析，工具可以把 Verilog 模块分成两类内容：稀疏模块（sparse module）与密集模块（dense module）。密集模块保留其大部分内容（缓冲器与反相器除外），并支持全面的数据流分析（data flow analysis）。稀疏模块的内容大幅减少，只包含宏单元，并且只支持连接关系分析所需的尺寸与接口信息。

为帮助定义轮廓视图的内容，工具支持分配方法（allocation method）与划分方法（partition method），用以决定哪些模块被标为密集或稀疏。在 `read_verilog_outline` 命令中加入 `-partition` 或 `-allocation` 选项来选择所用方法。也可以设置应用选项（application option）

plan.outline.partition 来指定方法。默认使用划分方法。

做数据流分析时，顶层模块需要密集的连接关系。用应用选项 plan.outline.dense_module_depth 指定应标为密集的模块层次深度。默认深度为 1，只有顶层模块被标为密集模块。也可以用 -depth 选项覆盖默认设置。取值为 -1 时，把所有非稀疏模块都标为密集。层次深度会覆盖由 -target_cell_count 选项指定的目标单元数。

4.3.1 划分方法

Partition Method

在划分方法中，工具检查层次中的模块，并根据目标块规模以及块中的叶单元数量确定模块密度。模块层次中的顶层模块被标为密集，直到该模块之下的层次单元数小于目标块规模。缓冲器（buffer）与反相器（inverter）计入层次单元数。这种做法在层次的上层保持连接关系，以供数据流分析。目标块规模由应用选项 plan.outline.target_block_size 指定；默认目标块规模为 1,000,000 个叶单元。可以用 -target_block_size 选项覆盖默认值。

指定划分块之后，每个划分内部的小型模块层次会被指定为密集模块，以保持数据流分析所需的连接关系。分配给这些模块层次的目标叶单元数由应用选项 plan.outline.glue_cell_count 指定；默认数量为 100,000 个叶单元。指定 -1 则避免把任何小型模块标为密集模块。缓冲器与反相器不出现在轮廓设计中，也不计入目标胶合单元（glue cell）数量。可以用 -glue_cell_count 选项覆盖默认设置。

4.3.2 分配方法

Allocation Method

分配方法是指定密集模块与稀疏模块的默认方法，用于在叶单元实例数量上限所对应的内存预算内，尽可能多地探索设计层次。把应用选项 plan.outline.partition 设为 false，或对 read_verilog_outline 命令指定 -allocation 选项，即可指定分配方法。

在分配方法中，叶单元数超过阈值的模块被视为大型模块，并在轮廓视图中作为稀疏模块处理。缓冲器与反相器不出现在轮廓设计中，也不计入叶单元数。叶单元数阈值由应用选项 plan.outline.large_threshold 指定；默认叶单元数为 10,000。也可以用 -large_threshold 选项覆盖默认设置。

4.4 创建初始布图

Creating an Initial Floorplan

读入 Verilog 网表之后，使用 initialize_floorplan 命令（或在 GUI 的任务助手（Task Assistant）中选择 **Floorplan Preparation > Floorplan Initialization**），根据设计准则定义布图边界（boundary）。可以指定布图各边的长度比、内核或芯片（die）各边的长度、宽高比（aspect ratio），或精确的宽度与高度来创建布图。这一步使用顶层设计的轮廓视图。

创建布图的步骤如下。

1. 用带有适当选项的 initialize_floorplan 命令创建初始布图。

```
icc2_shell> initialize_floorplan -core_offset 100
```

也可以在 GUI 中从布图任务助手（Floorplan Task Assistant）选择 **Floorplan Initialization** 面板。

2. 如有需要，用不同的利用率（utilization） 、边长、内核到边界间距及其他设置再次运行 `initialize_floorplan` 命令，以细化布图。

`initialize_floorplan` 命令按照所指定的选项创建布图。该布图如图 **??** 所示。

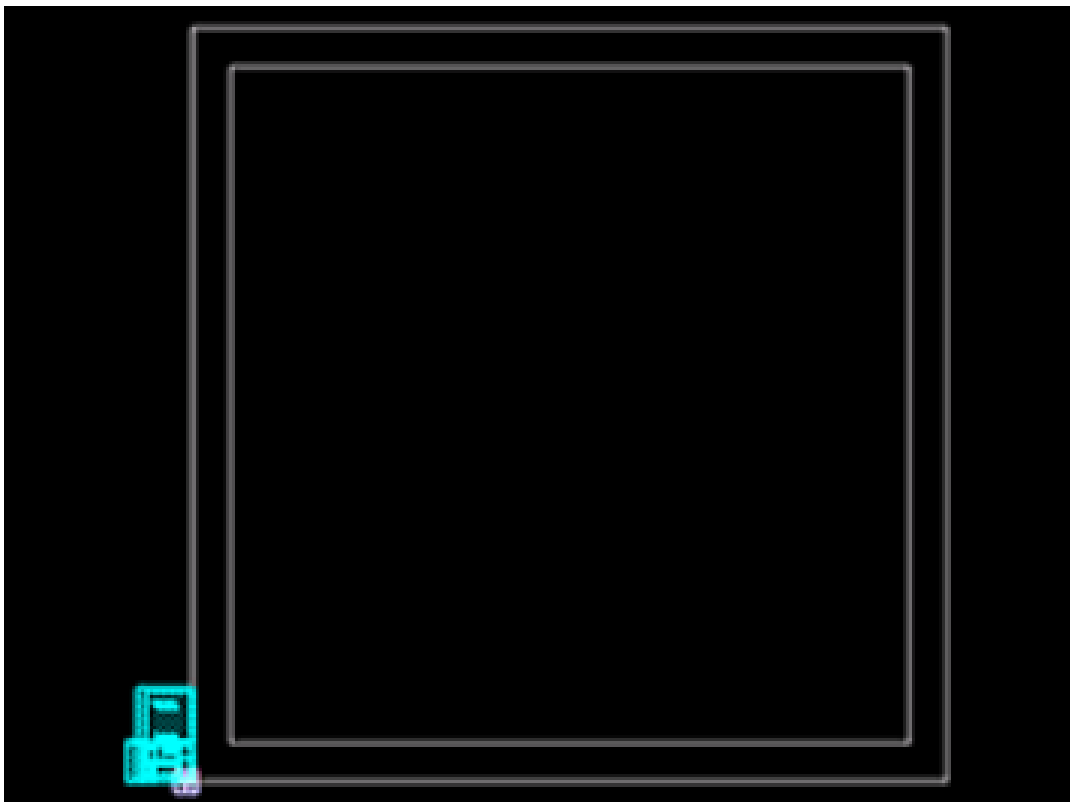


图 **4.4:** 内核偏移为 100 的布图

4.5 创建 L、T 或 U 形布图

Creating an L-, T-, or U-Shaped Floorplan

要创建 L 形、T 形或 U 形布图，按下列步骤进行。

1. 用带 `-shape shape` 选项的 `initialize_floorplan` 命令创建布图。支持的形状为 R（矩形）、L、T 与 U。下例创建如图 **??** 所示的 L 形布图。

```
icc2_shell> initialize_floorplan -core_utilization 0.7 -shape L \
   -orientation N -side_ratio {1 1 1 1} -core_offset {100.0} \
   -flip_first_row true -coincident_boundary true
```

2. 如需调整各边长度，修改 `-side_ratio` 或 `-side_length` 选项的参数，并重新发出 `initialize_floorplan` 命令。

要进一步改变布图形状，必须通过指定 `-side_ratio {side_a side_b ...}` 选项或 `-side_length {side_a side_b ...}` 选项来设置布图边界的尺寸。参数中数字的位置与布图形

状上各边的对应关系见图 **??**。任务助手中也提供该信息。



图 **4.5:** L 形布图

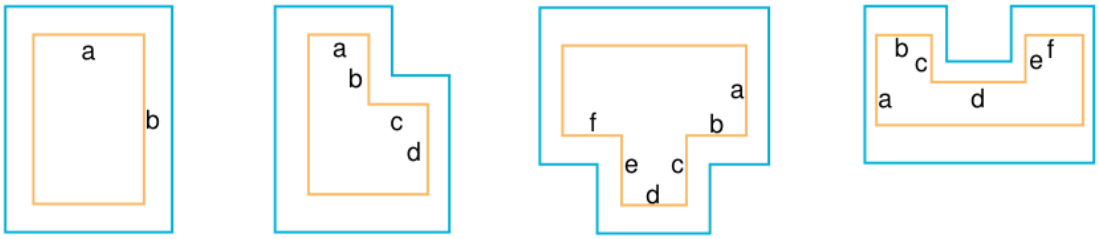


图 **4.6:** 布图形状的边标签对照

请注意，边长比默认作用于内核边界。要把比值作用于芯片边界，指定 `-control_type die` 选项。

下例创建 U 形布图。在 `-side_ratio` 的第二个位置（b）指定 1、在第六个位置（f）指定 2，使右臂宽度为左臂的两倍。

```
icc2_shell> initialize_floorplan -core_utilization 0.7 -shape U \
   -orientation N -side_ratio {2 1 1 1 1 2} -core_offset 100 \
   -coincident_boundary true
```

4.6 创建复杂直角多边形布图

Creating a Complex Rectilinear Floorplan

用带 `-boundary` 选项的 `initialize_floorplan` 命令创建复杂的布图形状。在 `-boundary` 选项上指定 `-control_type die` 或 `-control_type core`，以给出芯片或内核的坐标。

下例通过设置布图芯片的坐标来创建 H 形布图。

```
icc2_shell> initialize_floorplan -control_type die \
   -core_offset 100 \
   -boundary { {0 0} {0 3000} {1000 3000} {1000 2000} \
   {2000 2000} {2000 3000} {3000 3000} {3000 0} \
   {2000 0} {2000 1000} {1000 1000} {1000 0} }
```

图 **??** 是用上一条命令创建的布图。

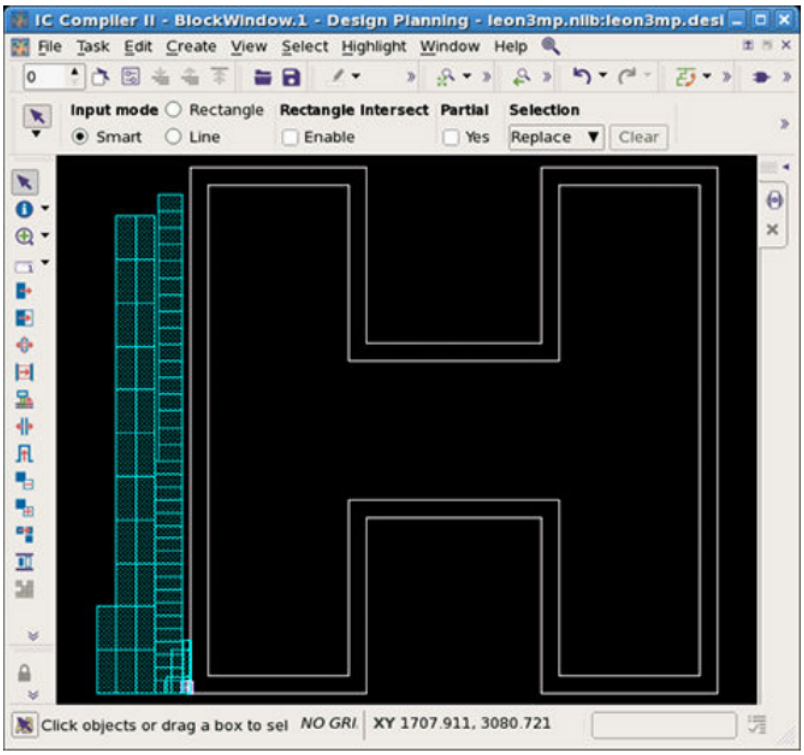


图 **4.7:** 复杂直角多边形布图

4.7 基于物理规则文件创建布图

Creating a Floorplan Based on a Physical Rules File

按下列示例流程，根据物理规则文件（physical rules file）中列出的规格创建布图。

注意

在物理规则文件中，可以把布局规则约束、物理单元与引脚属性以及 stream-in 指令放在统一的文件格式里。

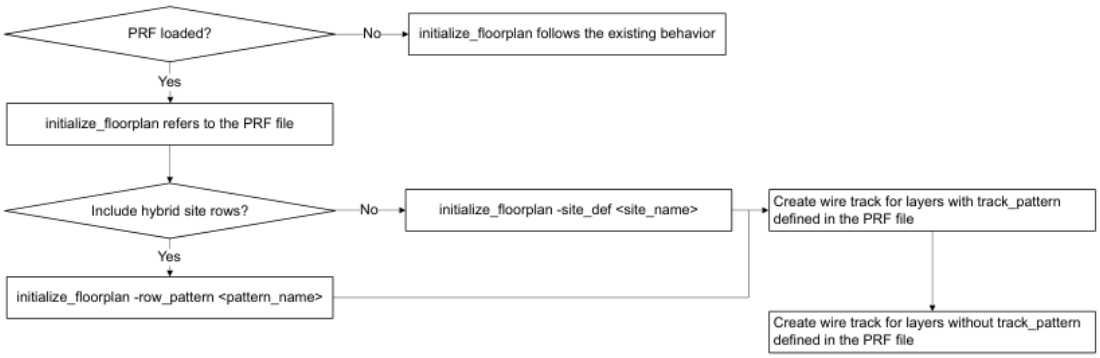


图 **4.8:** 示例：基于物理规则文件的布图创建流程

1. 按要求设置物理规则文件。

   关于物理规则文件的更多信息，见 SolvNetPlus。

2. 加载物理规则文件。

   可以把物理规则文件的内容放在工艺文件（technology file）中，也可以放在单独文件中，再用 `read_physical_rules` 命令读入。

   ```
   icc2_shell> read_physical_rules in.prf
   ```

3. 用必要的选项运行 `initialize_floorplan` 命令。

   例如，若已在物理规则文件中定义混合行（hybrid row），可以运行 `initialize_floorplan -row_pattern` 命令。工具随后使用该文件中的信息创建布图。不过，当混合设计中的 site 行高度不同时，需要平衡这些单元，才能得到合适的内核面积。将应用选项 `plan.floorplan.enable_adjust_core_area_for_hybrid_design` 设为启用，即可完成这种平衡。

注意

为确保工具使用物理规则文件中的信息来计算内核面积：

1. 设置 `initialize_floorplan -row_pattern`。
2. 不要在 `initialize_floorplan` 命令中设置 `-boundary`、`-keep_boundary` 或 `-side_length` 选项。

4.8 调整布图宽高比

Adjusting the Floorplan Aspect Ratio

要用不同的边长创建布图，用 `-side_ratio` 选项指定宽高比。设置布图宽高比的步骤如下。

1. 确定布图宽度与高度之比。
2. 用带 `-side_ratio` 选项的 `initialize_floorplan` 命令，按指定的宽高比创建布图。

```
icc2_shell> initialize_floorplan -core_utilization 0.7 -shape R \
   -orientation N -side_ratio {3.0 1.0} -core_offset {100.0} \
   -flip_first_row true  -coincident_boundary true
```

请注意，比值默认作用于内核边界。要把比值作用于芯片边界，指定 `-control_type die` 选项。

图 **??** 是用不同的 `-side_ratio` 选项设置得到不同宽高比的两份布图。左侧布图由 `-side_ratio {3.0 1.0}` 创建，右侧布图由 `-side_ratio {1.0 3.0}` 创建。



图 **4.9:** 宽高比分别为 3.0（左）与 0.33（右）的布图

要用 GUI 中的布图任务助手创建这些布图，按下列步骤进行。

1. 在 GUI 中选择 **Task > Task Assistant** 打开布图任务助手，并在 **Floorplan Preparation** 下选择 **Floorplan Initialization** 面板。
2. 选择 **Core side ratio**，并在 **Aspect ratio** 框中输入宽/高值。
3. 单击 **Preview**，显示布图的线框预览。
4. 单击 **Apply** 创建布图。

要创建图 **??** 所示的布图，在 **Aspect ratio** 框中输入 3.0 以创建左侧布图，或输入 0.33 以创建右侧布图。

4.9 定义布线轨道

Defining Routing Tracks

要在布图中创建布线轨道（routing track），使用 `create_track` 命令。可以指定层、轨道数量、轨道之间的节距（pitch）等。

可以用两种方式指定轨道位置：

- 要指定第一条轨道的位置，使用 `-coord` 选项。工具使用由 `-dir` 选项设置的 x 坐标与 y 坐标；`-dir` 指定放置轨道时的步进方向。默认情况下，第一条轨道位于芯片区域内部、距边界为间距一半的位置。

- 要使其中一条轨道线与内核区域包围盒（bounding box）或块包围盒的下边界对齐，使用 `-relative_to` 选项。即使块边界大于内核边界，轨道也会完全创建在块边界之内。

还可以用 `-end_grid_relative_to` 选项，指定后续网格点相对于内核区域包围盒或块包围盒的左下角创建。

也可以在 GUI 中选择菜单 **Create > Track** 来创建布线轨道。

图 **??** 是用 `create_track` 命令创建的布图。本例中，`remove_tracks` 命令先移除全部已有布线轨道，然后 `create_track` 命令在 METAL3 上沿优先方向（preferred direction）以 0.5 的节距创建布线轨道。`report_tracks` 命令报告设计中每一组布线轨道的方向、起点、轨道数量与节距。

```
icc2_shell> remove_tracks -all
1
icc2_shell> create_track -layer METAL3 -space 0.5
{TRACK_0}
icc2_shell> report_tracks
****************************************
Report report_tracks
Design : TEST
...
****************************************
Layer          Direction     Start         Tracks    Pitch          Attr
--------------------------------------------------------------------------------
METAL3             Y         0.250          4023     0.500       default
1
```

也可以用 `create_track` 命令基于非默认规则（nondefault rule, NDR）创建轨道。关于用非默认规则创建轨道的信息，见 SolvNet 文章 3001923，*How Can I Create Routing Tracks Based on Nondefault Rules (NDRs)?*。

4.10 翻转布图中的第一行

Flipping the First Row in the Floorplan

要使布图中的第一行不被翻转，按下列示例指定 `-flip_first_row false` 选项。

```
icc2_shell> initialize_floorplan -flip_first_row false
```

在图 **??** 中，左侧版图是用 `-flip_first_row true` 选项创建的，右侧布图是用 `-flip_first_row false` 选项创建的。默认情况下，`initialize_floorplan` 命令会翻转第一行。

要在边界与内核之间创建偏移，按下列示例对 `initialize_floorplan` 命令指定 `-core_offset` 选项。

```
icc2_shell> initialize_floorplan -core_utilization 0.8 \
   -core_offset {50 100}
```

图 **??** 是用带 `-core_offset` 选项的 `initialize_floorplan` 命令创建的布图的局部放大。该选项设置布图内核与布图边界之间的距离。`-core_offset` 选项接受一组偏移，每个偏移对应布图的一条边。参数位置与布图各边的对应关系见 `initialize_floorplan` 的 man 页。

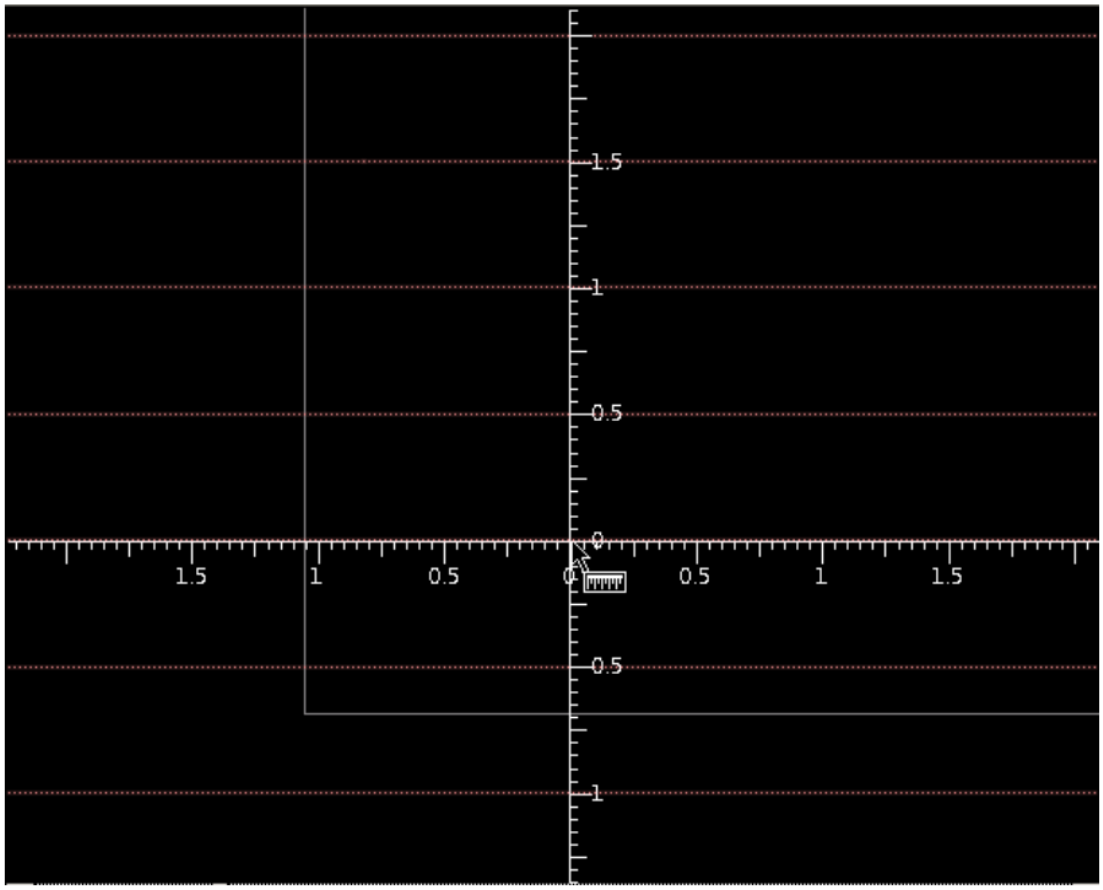


图 **4.10:** 插入布线轨道之后的布图

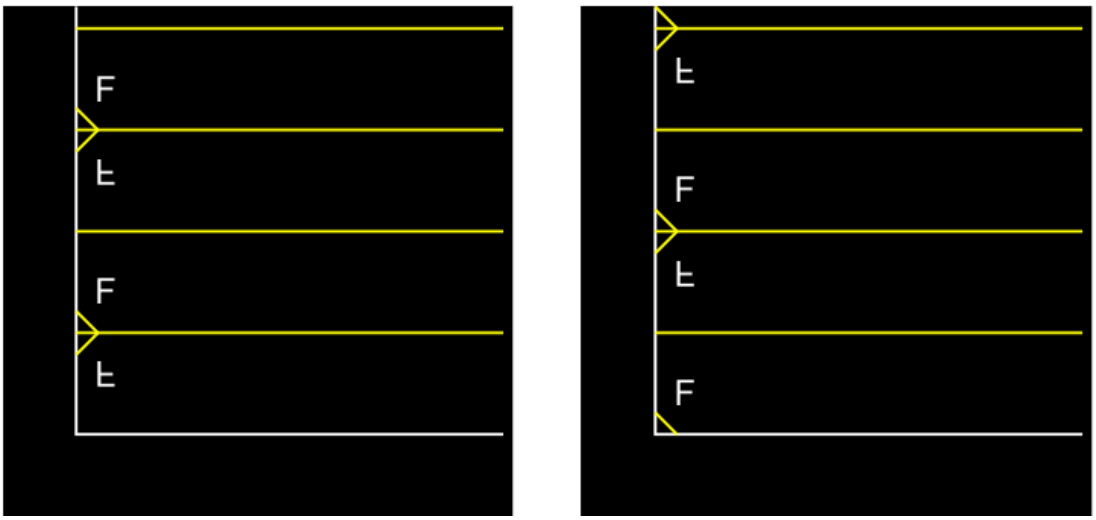


图 **4.11:** 第一行翻转与不翻转的布图

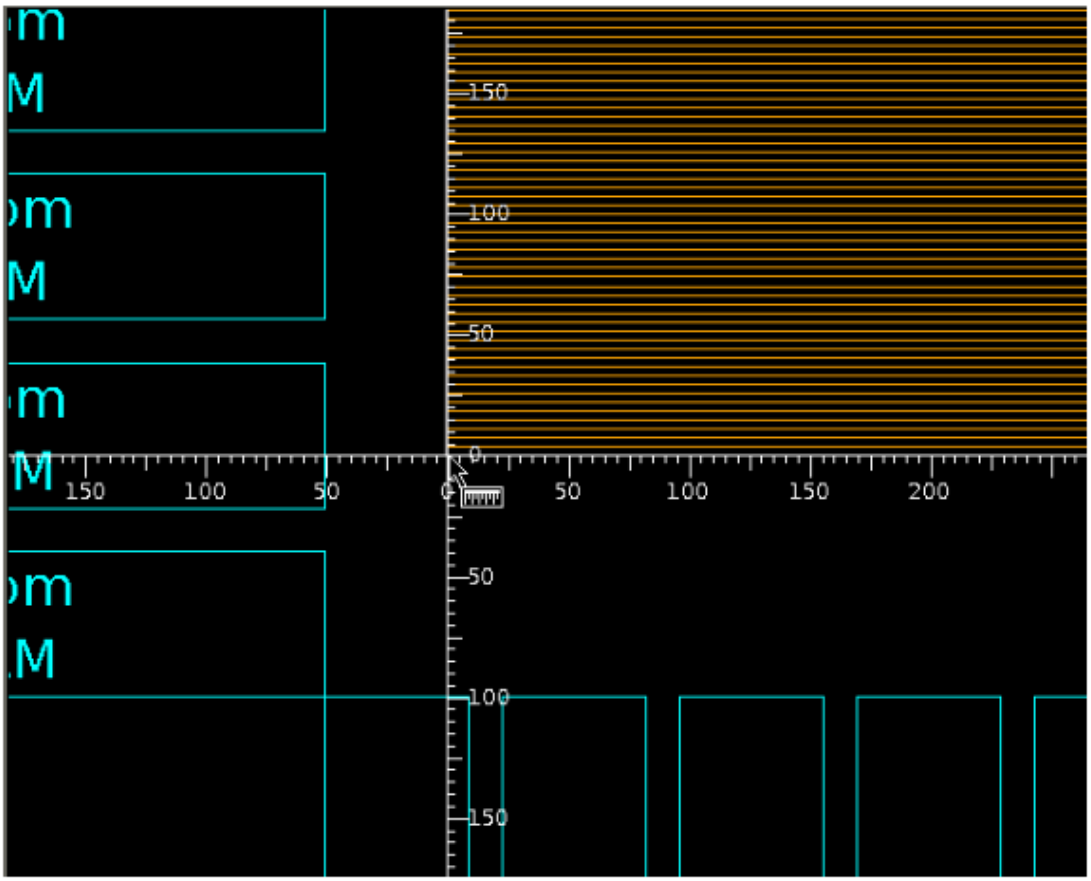


图 **4.12:** 带边界到内核间距的布图

4.11 在不扰动指定对象类型的情况下更新布图

Updating the Floorplanning Without Disturbing Specified Object Types

用 `initialize_floorplan` 命令创建初始布图之后，可以用不同选项重复该命令，以试验不同的布图形状与特性。更改布图时，若要保留指定对象类型的布局或形状，使用一个或多个 `-keep_*` 选项。

- `-keep_boundary`：更新布图时保留布图边界。
- `-keep_pg_route`：更新布图时保留电源与地布线。
- `-keep_detail_route`：更新布图时保留除电源与地布线以外的全部布线。
- `-keep_placement {block}`：更新布图时保留块的布局。
- `-keep_placement {io}`：更新布图时保留端子与焊盘的布局。
- `-keep_placement {macro}`：更新布图时保留宏的布局。
- `-keep_placement {std_cell}`：更新布图时保留标准单元布局。
- `-keep_placement {physical_only}`：更新布图时保留仅物理单元（physical-only cell）的布局；可用下列命令识别这些单元：

```
icc2_shell> get_cells -filter "design_type==physical_only"
icc2_shell> get_cells -filter "design_type==fill"
```

- `-keep_placement {all}`：更新布图选项时保留端子、焊盘、宏、块、标准单元以及仅物理对象。

4.12 验证 FinFET 网格

Validating the FinFET Grid

IC Compiler II 工具支持 FinFET 网格（FinFET grid），用以引导含 FinFET 器件的库单元的布局。FinFET 网格的 x 方向与 y 方向间距及偏移值在工艺文件中指定。

要报告 FinFET 网格的信息，按下列方式使用 `report_grids -type finfet` 命令。

```
icc2_shell> report_grids -type finfet
FinFET Grid             Description
-------------------------------------------------
Top Design              orca_hier_lib:ORCA.design
FinFET Grid defined     true
FinFET Grid X Pitch     0.001
FinFET Grid Y Pitch     0.048
FinFET Grid X Offset    0
FinFET Grid Y Offset    0
1
```

要检查相对于 FinFET 网格的布局或边界违例，按下列方式使用 `check_finfet_grid` 命令。

```
icc2_shell> check_finfet_grid
...
Error: boundary point (0 101.97) of hard macro
'I_ORCA_TOP/I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_7'
   is not on FinFET grid. (DPCHK-002)
Error: boundary point (0 101.97) of hard macro
'I_ORCA_TOP/I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_6'
   is not on FinFET grid. (DPCHK-002)
...
```

要检查特定对象类型，只把这些对象指定给 check_finfet_grid 命令，如下所示。

```
icc2_shell> check_finfet_grid -objects [get_cells -hierarchical \
   -filter "design_type==pad"]
...
Reporting violations in block ORCA ...
Error: boundary point (0 345.565) of io pad 'test_mode_iopad'
   is not on FinFET grid. (DPCHK-002)
Error: boundary point (0 345.565) of io pad 'sdr_clk_iopad'
   is not on FinFET grid. (DPCHK-002)
```

工具中下列命令能够识别 FinFET 网格：

- create_blackbox
- create_grid
- create_io_guide
- create_io_ring
- create_macro_array
- create_placement
- create_power_switch_array
- create_power_switch_ring
- create_site_array
- create_site_row
- initialize_floorplan
- place_io
- report_grids
- set_block_grid_references
- set_grid
- set_signal_io_constraints
- shape_blocks

4.13 缩放芯片面积

Scaling the Die Area

可以用 `modify_die_area` 命令缩小或扩大芯片面积（die area），或改变设计的尺寸。

- 缩小设计尺寸可能带来一些潜在好处，例如芯片越小，每片晶圆上的芯片数量越多。不过，应只做小幅改动。大幅改动可能引起问题，包括利用率过高，或宏无法放入设计。当缩小后的设计仍然可布线、满足工作速度要求，并且也满足允许的功耗要求时，可以认为这一操作成功。
- 扩大设计有助于缓解拥塞。

表 **??** 列出 `modify_die_area` 命令中控制芯片面积缩放量的选项。

表 **4.1:** `modify_die_area` 命令中控制芯片面积缩放量的选项

要完成的操作	使用此选项

控制内核面积及其高度、宽度的缩放。有效值范围为 0.1 到 10	`-area_scaling_factor`
	`-height_scaling_factor`
	`-width_scaling_factor`

指定内核面积所需的利用率	`-core_utilization`
列出希望长度保持不变的边	`-fixed_edges`
定义宽高比约束	`-sizing_type`
指定布图创建时使用的行图案	`-row_pattern`
计算并打印估计布图版图的摘要，并基于这些估计创建新布图	`-compile`
在芯片缩放之后运行自定义脚本，以重新创建轨道。默认情况下，该命令从输入布图导出轨道信息并加以复用	`-run_track_creation_script`

下例把内核面积缩小到原来尺寸的 85%。

```
icc2_shell> modify_die_area -area_scaling_factor 0.85
```

下例把内核高度缩小到芯片原始高度的 80%，即降低 20%。

```
icc2_shell> modify_die_area -height_scaling_factor 0.8
```

下例缩小内核，使其利用率为 75%。

```
icc2_shell> modify_die_area -core_utilization 0.75
```

下例把内核高度增加到内核原始高度的 110%，即增加 10%。

```
icc2_shell> modify_die_area -height_scaling_factor 1.1
```

下例把内核面积缩小到原来尺寸的 85%，但水平尺寸保持固定。

```
icc2_shell> modify_die_area -area_scaling_factor 0.85 -sizing_type fixed_width \
   -compile
```

```
--------------------------------------------------------------------------------
Die Size Exploration Summary
--------------------------------------------------------------------------------
Original Floorplan Layout
        Utilization Ratio: 0.3934
        Core Area: 3150962.2170
        Core Width: 1775.3000
        Core Height: 1774.8900
        Boundary Co-ordinates: {350.0000 350.0000} {350.0000 2124.8900}
 {2125.3000 2124.8900} {2125.3000 350.0000}

Estimated Floorplan Layout
        Utilization Ratio: 0.4628
        Core Area: 2678317.8845
        Core Width: 1775.3000
        Core Height: 1508.6565
        Boundary Co-ordinates: {350.0000 350.0000} {350.0000 1858.6565}
 {2125.3000 1858.6565} {2125.3000 350.0000}
```

4.14 报告布图信息

Reporting Floorplan Information

用 `report_design -floorplan` 或 `report_design -all` 命令报告布图的下列信息：

- 面积（area）：内核面积与芯片面积。
- site 行：每个 site 行的名称、宽度、高度、行数、tile 数量与面积。
- 阻挡（blockage）：每种阻挡类型的数量与面积，包括硬布局阻挡、软布局阻挡、硬宏阻挡、部分布局阻挡、布线阻挡、类别阻挡、预留布局组阻挡、整形阻挡、设计规则布线阻挡、只允许缓冲器的布局阻挡，以及每种阻挡类型的禁止寄存器阻挡。
- 电源域（power domain）：每个电源域的电源域名称、电压区域（voltage area）名称，以及主电源网与主地网。
- 电压区域：每个电压区域的名称、形状数量、面积、目标利用率与包围盒坐标。
- 组界限（group bound）：无尺寸组界限的数量。
- 移动界限（move bound）：每个硬、软与互斥移动界限的名称、面积、利用率与包围盒。
- 布线引导（route guide）：下列各项各自的数量与面积：额外绕行区域、片内覆盖（in-chip overlay）单元层上的布线引导、河流布线（river routing）引导、布线引导中的全部双通孔、访问偏好布线引导、默认布线引导，以及仅切换方向的布线引导。
- 多比特寄存器（multibit register）：多比特寄存器的数量。
- 预留布局组（reserved placement group）：每个预留布局组的顶层预留布局组数量、预留布局单元数量、预留布局单元面积，以及预留布局单元面积百分比。
- 层：每层的层名、方向、节距、默认宽度、最小宽度、最小间距，以及同一网线之间的最小间距。

4.15 读入、写出与复制布图信息

Reading, Writing, and Copying Floorplan Information

创建布图之后，可以用 write_floorplan 命令保存描述该布图的 Tcl 脚本，或用 write_def 命令保存 DEF 文件。

- 读入布图信息
- 读入 DEF 文件
- 写出布图文件
- 在块之间复制布图对象

4.15.1 读入布图信息

Reading in Floorplan Information

对扁平块，要加载由 write_floorplan 命令写出的布图信息，用 source 命令读入设计的 fp.tcl 文件，如下所示。

```
icc2_shell> source floorplan/fp.tcl
```

对可能混有设计表示与抽象（abstract）块表示的层次化设计，按下列方式使用 set_constraint_mapping_file 与 load_block_constraints 命令。

```
icc2_shell> set_constraint_mapping_file floorplan/mapfile
icc2_shell> load_block_constraints -type FLOORPLAN -all_blocks
```

load_block_constraints -type FLOORPLAN 命令及选项会 source 映射文件中包含的每个 fp.tcl 文件，从而为每个块重新创建布图。由 write_floorplan 命令写出的约束映射文件（constraint mapping file）包含块名列表、FLOORPLAN 关键字，以及每个块的 fp.tcl 文件位置，如下列约束映射文件所示。

```
BLENDER_0    FLOORPLAN  floorplan/BLENDER_0/fp.tcl
BLENDER_1    FLOORPLAN  floorplan/BLENDER_1/fp.tcl
BLENDER_2    FLOORPLAN  floorplan/BLENDER_2/fp.tcl
CONTEXT_MEM  FLOORPLAN  floorplan/CONTEXT_MEM/fp.tcl
ORCA         FLOORPLAN  floorplan/fp.tcl
PCI_TOP      FLOORPLAN  floorplan/PCI_TOP/fp.tcl
RISC_CORE    FLOORPLAN  floorplan/RISC_CORE/fp.tcl
SDRAM_TOP    FLOORPLAN  floorplan/SDRAM_TOP/fp.tcl
```

4.15.2 读入 DEF 文件

Reading DEF Files

要读入 DEF 文件，使用 read_def 命令。下列命令读入 orca.def 这一 DEF 文件。

```
icc2_shell> read_def orca.def
```

要在把对象标注到设计上之前先分析输入 DEF 文件的语法，指定 -syntax_only 选项。

```
icc2_shell> read_def -syntax_only orca.def
```

4.15.3 写出布图文件

Writing Out Floorplan Files

要写出当前布图，使用 write_floorplan 命令。write_floorplan 命令创建名为 floorplan 的目录，并把 floorplan.def、floorplan.tcl 与 fp.tcl 文件写到该目录。floorplan.def 文件包含设计的物理版图、连接关系与设计约束信息。floorplan.tcl 文件包含无法在 DEF 文件中描述的附加设计意图，例如硬宏禁布边距（keepout margin）。fp.tcl 文件包含用于移除设计约束的命令，例如电压区域、界限、布线引导等。可以用命令选项控制写入这些文件的信息。write_floorplan 命令对块同时支持设计视图（design view）与抽象视图（abstract view）。

write_floorplan 命令还会写出约束映射文件，把每个块映射到对应的 fp.tcl 文件。读入布图数据时，用 set_constraint_mapping_file 命令指定该映射文件，并由 load_block_constraints 命令使用。

write_floorplan 命令可以包含或排除一种或多种对象类型。默认写出全部受支持的对象类型。用 -include object_type_list 或 -exclude object_type_list 选项指定要包含或排除的对象类型。合法的对象类型如下：

- blockages（阻挡）
- bounds（界限）
- cells（单元）
- corridors（走廊）
- die_area（芯片区域）
- edit_groups（编辑组）
- io_guides（I/O 引导）
- macros（宏）
- module_boundaries（模块边界）
- nets（网线）
- pin_guides（引脚引导）
- pins（引脚）
- route_guides（布线引导）
- rows（行）
- tracks（轨道）
- vias（通孔）
- scan_chains（扫描链）

- routing_directions（布线方向）
- voltage_areas（电压区域）
- fills（填充）
- pg_metal_fills（电源地金属填充）
- routing_rules（布线规则）
- pg_regions（电源地区域）
- placement_attractions（布局吸引）
- bump_regions（凸点区域）
- anchors（锚点）

下列示例针对不同设计要求写出布图文件：

- 写出宏，并设置原点、方向、状态及其他属性。

```
icc2_shell> write_floorplan -output "macros" -objects [get_cells \
   -physical_context -filter is_hard_macro==true]
```

- 写出电源地区域；若 pg 目录中的文件已存在，则覆盖它们。

```
icc2_shell> write_floorplan -force -objects [get_pg_regions] \
   -output pg
```

- 写出物理状态为 fixed 或 unplaced 的焊盘单元与宏单元。

```
icc2_shell> write_floorplan -force -include cells \
   -include_physical_status {fixed unplaced} -cell_types {pad macro}
```

- 把时钟、电源与地网写到 DEF 文件；信号网被排除。

```
icc2_shell> write_floorplan -force -output nets \
   -net_types {clock power ground}
```

- 在输出的 fp.tcl 布图文件里，为 read_def 命令加入 -add_def_only_objects {cells} 与 -no_incremental 选项。

```
icc2_shell> write_floorplan -output fp -nosplit -def_version 5.8 \
   -read_def_options {-add_def_only_objects {cells} -no_incremental}
```

4.15.4 在块之间复制布图对象

Copying Floorplan Objects From Block to Block

要把布图信息从一个块复制到另一个块，使用 copy_floorplan 命令。该命令把布图数据从源块的设计视图复制到已存在的目标块的设计视图。这些设计可以在同一个库中，也可以在不同的库中。可以在块名之外再指定设计标签（design label）。

复制时，copy_floorplan 命令可以包含或排除一种或多种对象类型。默认复制全部对象类型。用 -include object_type_list 或 -exclude object_type_list 选项指定要包含或排除的对象类型。合法的对象类型如下：

- blockages
- bounds
- cells
- corridors
- die_area
- edit_groups
- io_guides
- macros
- module_boundaries
- pin_guides
- pins
- route_guides
- rows
- tracks
- vias
- scan_chains
- routing_directions
- voltage_areas
- fills
- pg_metal_fills
- routing_rules
- pg_regions
- port
- supernet
- bundle
- topology_plan
- net_estimation_rule

copy_floorplan 命令还可以把下列对象类型从源块复制到目标块：

- block（块）

- design（设计）
- routing_blockage（布线阻挡）
- shaping_blockage（整形阻挡）
- placement_blockage（布局阻挡）
- bound（界限）
- cell（单元）
- routing_corridor（布线走廊）
- core_area（内核区域）
- edit_group（编辑组）
- fill_cell（填充单元）
- io_guide（I/O 引导）
- via（通孔）
- shape（形状）
- net（网线）
- pin_guide（引脚引导）
- pg_region（电源地区域）
- routing_guide（布线引导）
- site_row（site 行）
- site_array（site 阵列）
- track（轨道）
- voltage_area（电压区域）
- routing_rule（布线规则）
- port（端口）
- supernet（超网）
- bundle（线束）
- topology_plan（拓扑规划）

用 -objects 选项指定对象集合。用该命令的其他选项指定源布图、目标布图，以及复制哪些布图对象。

- 把 test 库中带 placed 标签的 mem 设计的全部布图信息，复制到当前打开的块与库。

```
icc2_shell> copy_floorplan -from_block test:mem/placed
```

若使用 -force 选项，即使对象已经存在于现有布图中，也会以新名称复制到该布图。

- 用 -to_block 选项，把 test 库中带 placed 标签的 mem 设计的全部布图信息连同电源开关（power switch）逻辑连接信息，复制到 test 库中带 placed 标签的 mem_copy 块。

```
icc2_shell> copy_floorplan -from_block test:mem/placed \
   -to_block test:mem_copy/placed
```

注意

-force 选项对电源开关单元及其连接没有作用。与其他对象不同，该选项不会重命名这些单元。因此，若单元已经存在而逻辑连接尚不存在，再次带 -force 选项运行 copy_floorplan 命令只会创建逻辑连接。

- 用 -include 选项只复制阻挡与界限的布图信息。

```
icc2_shell> copy_floorplan -verbose -from_block test:mem/placed \
   -to_block test:mem_copy/placed -include {blockages bounds}
```

- 用 -exclude 选项复制除阻挡与界限以外的全部布图信息。

```
icc2_shell> copy_floorplan -verbose -from_block test:mem/placed \
   -to_block test:mem_copy/placed -exclude {blockages bounds}
```

- 用 -objects 选项，只把通孔 VIA_S_0 与 VIA_C_0 从源设计复制到目标设计。

```
icc2_shell> copy_floorplan -verbose -from_block test:mem/placed \
   -to_block test:mem_copy/placed \
   -objects [get_vias {VIA_S_0 VIA_C_0}]
```

- 用 -hierarchical 选项按层次复制布图对象。

```
icc2_shell> copy_floorplan -verbose -from_block test:mem/placed \
   -to_block test:mem_copy/placed \
   -hierarchical
```

表 ?? 给出 copy_floorplan 对部分对象的工作方式示例。

表 **4.2:** copy_floorplan 命令的工作方式

对象	copy_floorplan 默认	copy_floorplan -objects	copy_floorplan -include	copy_floorplan -exclude	copy_floorplan -force
被复制对象在目标设计中不存在时					
supernet	复制到目标设计	复制到目标设计	复制到目标设计	不复制到目标设计	复制到目标设计
bundle	复制到目标设计	复制到目标设计	复制到目标设计	不复制到目标设计	复制到目标设计
net_estimation_rule	复制到目标设计	不能与 -objects 选项同时指定	复制到目标设计	不复制到目标设计	复制到目标设计
topology_plan	复制到目标设计	复制到目标设计	复制到目标设计	不复制到目标设计	复制到目标设计

表 **4.2:** copy_floorplan 命令的工作方式（续）

对象	copy_floorplan 默认	copy_floorplan -objects	copy_floorplan -include	copy_floorplan -exclude	copy_floorplan -force
topology_edge	若关联的 topology_plan 也被复制，则复制到目标设计	不能与 -objects 选项同时指定	不能与 -include 选项同时指定	不能与 -exclude 选项同时指定	不能与 -force 选项同时指定
topology_node	若关联的 topology_plan 也被复制，则复制到目标设计	不能与 -objects 选项同时指定	不能与 -include 选项同时指定	不能与 -exclude 选项同时指定	不能与 -force 选项同时指定
topology_repeater	若关联的 topology_plan 也被复制，则复制到目标设计	不能与 -objects 选项同时指定	不能与 -include 选项同时指定	不能与 -exclude 选项同时指定	不能与 -force 选项同时指定
被复制对象在目标设计中已经存在时					
supernet	不复制到目标设计	不复制到目标设计	不复制到目标设计	不能与 -exclude 选项同时指定	不复制到目标设计
bundle	不复制到目标设计	不复制到目标设计	不复制到目标设计	不复制到目标设计	复制到目标设计
net_estimation_rule	不复制到目标设计	不能与 -objects 选项同时指定	不复制到目标设计	不复制到目标设计	复制到目标设计
topology_plan	不复制到目标设计	不复制到目标设计	不复制到目标设计	不复制到目标设计	复制到目标设计
topology_edge	不复制到目标设计	不能与 -objects 选项同时指定	不能与 -include 选项同时指定	不能与 -exclude 选项同时指定	不能与 -force 选项同时指定
topology_node	不复制到目标设计	不能与 -objects 选项同时指定	不能与 -include 选项同时指定	不能与 -exclude 选项同时指定	不能与 -force 选项同时指定
topology_repeater	不复制到目标设计	不能与 -objects 选项同时指定	不能与 -include 选项同时指定	不能与 -exclude 选项同时指定	不能与 -force 选项同时指定

4.16 为先进工艺节点创建并验证布图规则

Creating and Validating Floorplan Rules for Advanced Technology Nodes

IC Compiler II 工具为面向先进工艺节点（advanced technology node）开发的布图提供附加的规则创建与检查功能。这些布图对设计中标准单元、宏及其他对象的布局与间距有额外要求。

4.16.1 创建附加布图规则

Creating Additional Floorplan Rules

对有额外要求的布图，用下表所列命令创建布图规则。各命令的详细说明见该表之后的内容。

表 **4.3:** 创建附加布图规则的命令

命令	说明
set_floorplan_area_rules	定义最小面积、最大面积、合法或非法面积的列表，或合法或非法面积的范围。
set_floorplan_enclosure_rules	定义包围规则，用于检查被包围对象与其包围对象之间的合法间距。
set_floorplan_halo_rules	定义光晕规则，用于检查被包围对象与其包围对象之间的合法间距。
set_floorplan_spacing_rules	定义间距规则，用于检查对象之间的合法间距。
set_floorplan_width_rules	定义宽度规则，用于检查对象之间的合法间距。
set_floorplan_density_rules	定义用于检查布图密度的规则。

布图面积规则

Floorplan Area Rules

要为一组库单元或设计对象定义最小面积、最大面积、合法或非法面积的列表，或合法或非法面积的范围，使用 set_floorplan_area_rules 命令。合法对象为：

- block_boundary
- core_area
- hard_macro
- placement_blockage
- routing_blockage
- shape
- soft_macro
- va_boundary
- unplaceable_area
- std_cell_area

下例创建名为 area_rule_1 的布图面积规则。该规则指定标准单元的最小面积为 100。

```
icc2_shell> set_floorplan_area_rules -name area_rule_1 \
   -object_types std_cell_area -min 100
```

布图包围规则

Floorplan Enclosure Rules

要定义包围规则,以检查被包围对象与其包围对象之间的合法间距,使用 set_floorplan_enclosure_rules 命令。用选项指定：

- 对象类型
- 对象的边
- 对象的角
- 允许的对象旋转
- 合法或非法的间距范围与取值

下例创建名为 enclosure_rule_1 的布图包围规则。该规则指定：当布线阻挡包围其中一个指定库单元时，布线阻挡必须在所有边上完全包围该单元，并且布线阻挡与库单元之间的间距必须为 5.0。

```
icc2_shell> set_floorplan_enclosure_rules -name enclosure_rule_1 \
   -from_object_types routing_blockage -layers METAL \
   -to_lib_cells $lib_cells -follow_rotations -sides all \
   -must_enclose -min 5.0
```

布图光晕规则

Floorplan Halo Rules

要定义光晕（halo）规则，以检查被包围对象与其包围对象之间的合法间距，使用 set_floorplan_halo_rules 命令。用选项指定：

- 对象类型
- 对象的边
- 对象的角
- 允许的对象旋转
- 合法或非法的间距范围与取值

下例创建名为 halo_rule_1 的布图光晕规则。该规则指定：当布线阻挡包围一个宏时，布线阻挡必须在所有边上完全包围该宏，并且布线阻挡与库单元之间的间距必须为 3.0。

```
icc2_shell> set_floorplan_halo_rules -name halo_rule_1 \
   -from_object_types routing_blockage -layers METAL2 \
   -to_object_types hard_macro -sides all -type outer \
   -must_enclose -min 3.0
```

布图间距规则

Floorplan Spacing Rules

要定义间距规则，以检查对象之间的合法间距，使用 set_floorplan_spacing_rules 命令。用选项指定：

- 对象类型
- 对象的边
- 对象的层

- 允许的对象旋转
- 最小平行走线长度
- 重叠
- 合法或非法的间距范围与取值

下例创建名为 spacing_rule_1 的布图间距规则。该规则指定所给库单元与标准单元区域之间的间距。

```
icc2_shell> set_floorplan_spacing_rules -name spacing_rule_1 \
   -from_lib_cells $lib_cells -follow_rotations \
   -to_object_types std_cell_area -directions vertical \
   -min_parallel_run_length -1.0 -step 2.0 -offset 13.0
```

布图宽度规则

Floorplan Width Rules

要定义宽度规则，以检查对象之间的合法间距，使用 set_floorplan_width_rules 命令。用选项指定对象类型、检查的宽度类型、边或方向、对象的层、允许的对象旋转、最大与最小宽度，以及合法或非法的宽度范围与取值。合法对象为：

- block_boundary
- core_area
- hard_macro
- placement_blockage
- routing_blockage
- shape
- soft_macro
- unplaceable_area
- boundary_cell_region
- io_pad
- cover_bump
- va_boundary
- std_cell_area

默认情况下，工具分别测量每个对象的宽度。不过，若要测量全部对象合并后的形状，可以把 set_floorplan_width_rules -merge_object 设为 true。

下例创建名为 width_rule_1 的布图宽度规则。该规则指定标准单元区域的水平宽度必须是 8.0 的偶数倍，再加上 40.0 的偏移。

```
icc2_shell> set_floorplan_width_rules -name width_rule_1 \
   -object_types std_cell_area -type simple -direction horizontal \
   -step 8.0 -offset 40.0
```

布图密度规则

Floorplan Density Rules

要定义布图密度规则，使用 set_floorplan_density_rules 命令。布图密度规则的一个例子是芯片的最大凸点密度。用选项指定：

- 对象类型
- 库单元
- 布线层
- 密度计算方法
- 切割方法
- 应计算密度的设计区域
- 窗口尺寸
- 窗口步进
- 对象之间必须保持的距离

下例创建名为 ds1 的布图密度规则。

```
icc2_shell> set_floorplan_density_rules -name ds1 \
   -from_object_types shape -to_object_types io_pad \
   -numerator amount -cut_method drop -density_method local_window \
   -window_size {1 2} -window_step {3 4} -max 10
```

4.16.2 验证布图规则并修复错误

Validating Floorplan Rules and Repairing Errors

用「创建附加布图规则」中列出的命令创建先进节点布图规则之后，使用 check_floorplan_rules 命令，按这些规则检查布图。

要在错误浏览器中显示违例，在 GUI 的 check_floorplan_rules 对话框中选择 **Display errors in layout window** 选项。

按下列方式使用 check_floorplan_rules 命令的选项与参数，以控制如何执行布图规则检查。

- 在命令名之后列出要检查的布图规则。用 check_floorplan_rules 命令列出当前已指定的布图规则。

```
icc2_shell> check_floorplan_rules {floorplan_rule_1 floorplan_rule_2}
...
Error: found region {{30 115} {47 262}} from {hard_macro: MACRO1} to
{hard_macro: MACRO2} violates rule floorplan_rule_1 on vertical
```

```
 direction:
not on grid. (DPCHK-010)
```

- 用 -object_types 选项指定要检查的对象类型（hard_macro、std_cell_area、soft_macro、block_boundary 或 core_area）。

```
icc2_shell> check_floorplan_rules -object_types {std_cell hard_macro}
...
Error: found region {{340 1} {431 58}} from
{block_boundary} to {hard_macro: MACRO5} violates rule rule_1
on horizontal direction: not on grid. (DPCHK-010)
```

- 用 -lib_cells 选项指定要检查的库单元。

```
icc2_shell> check_floorplan_rules -lib_cells MACRO4
Error: found region {{262 58} {299 58}} from {cell: U3/MACRO4} to
{cell: U0/MACRO4} violate rule rule_2 on vertical direction: less than
offset, not on grid. (DPCHK-010)
```

用 set_floorplan_area_rules、set_floorplan_enclosure_rules、set_floorplan_halo_rules、set_floorplan_spacing_rules、set_floorplan_width_rules 与 check_floorplan_rules 命令创建并检查布图规则之后，使用 fix_floorplan_rules 命令根据这些规则修改设计。该命令对设计做下列更改：

- 移动宏，以满足全部网格类间距规则，以及来自块边界的包围规则。
- 在硬宏周围创建硬禁布边距（hard keepout margin），以满足间距规则与光晕规则。
- 在宏布局之后创建布线阻挡，以满足全部布线阻挡到宏的包围规则与光晕规则。
- 在宏布局之后创建布局阻挡，以满足全部宽度规则、到宏的间距规则、来自块边界的包围规则，以及标准单元区域的面积规则。
- 调整内核区域与块边界，以缓解块边界到内核区域的包围规则违例，以及内核区域的宽度规则违例。
- （可选）写出一个 Tcl 文件，其中包含用于实现 fix_floorplan_rules 命令所做设计更改的命令。

下例检查布图规则，用 fix_floorplan_rules 命令调整硬宏的位置，然后再次检查布图规则。第二次检查表明，fix_floorplan_rules 命令已修复全部布图规则。

```
icc2_shell> check_floorplan_rules
INFO: found 4 error(s) violate rule: rule_1
INFO: found 4 error(s) violate rule: rule_2
INFO: found 50 error(s) violate rule: rule_3
INFO: found 8 error(s) violate rule: rule_4
INFO: checking floorplan rules finished.

icc2_shell> fix_floorplan_rules -include hard_macro
Fixing hard macro floorplan spacing rules.

icc2_shell> check_floorplan_rules
INFO: found 0 error(s) violate rule: rule_1
```

```
INFO: found 0 error(s) violate rule: rule_2
INFO: found 0 error(s) violate rule: rule_3
INFO: found 0 error(s) violate rule: rule_4
```

Chapter 5

处理黑盒

Handling Black Boxes

IC Compiler II 工具在设计规划（design planning）流程中支持模块的黑盒（black box）。黑盒是没有网表（netlist）、只有部分网表，或未绑定到更高层模块的模块。使用黑盒可以在不要求所有模块都具备完整网表的情况下继续设计规划。该工具在物理与时序两方面、以及设计规划的各个阶段都支持黑盒。请注意，工具中只有物理层次可以作为黑盒。

黑盒可用的设计规划操作如下：

- 多次例化（multiply-instantiated）黑盒支持
- 塑形（shaping）
- 宏单元与标准单元布局（针对部分网表黑盒）
- 电源地（PG）布线
- 引脚分配与穿越（feedthrough）创建
- 下推（push-down）与上弹（pop-up）
- 穿越缓冲
- 抽象（abstract）创建与合并
- 时序模型创建
- 时序预算（budgeting）

创建黑盒物理模型与时序模型的流程如图 **??** 所示。

更多细节见下列主题：

- 识别潜在黑盒
- 创建黑盒参考
- 创建黑盒时序模型
- 黑盒时序示例
- 黑盒设计的 UPF

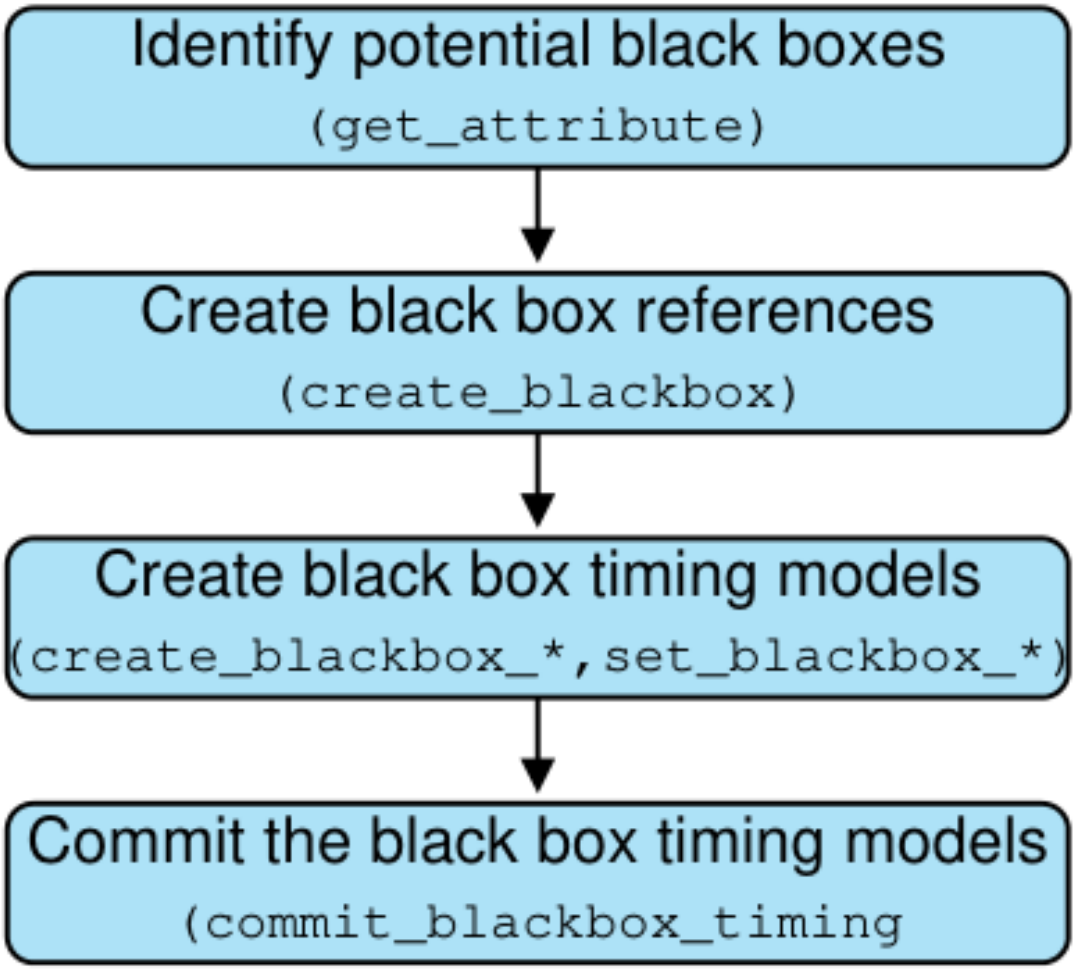

Identify potential black boxes
(get_attribute)
Create black box references
(create_blackbox)
Create black box timing models
(create_blackbox_*,set_blackbox_*)
Commit the black box timing models
(commit_blackbox_timing

图 **5.1:** 黑盒流程

5.1 识别潜在黑盒

Identifying Potential Black Boxes

可通过检索 black_box 属性设置为 true 的单元来识别黑盒。黑盒还可进一步分为缺失/未绑定、空、部分、穿越以及绑值（tie-off）。此外，可通过检查块上的下列属性来识别穿越模块与绑值模块：

- has_timing_model
- hierarchy_hard_macro_count
- hierarchy_has_shadow_netlist
- hierarchy_has_shadow_netlist_only
- hierarchy_pad_cell_count
- hierarchy_physical_only_cell_count
- hierarchy_std_cell_count
- hierarchy_switch_cell_count
- is_unbound
- target_boundary_area

请按下列准则识别不同类型的黑盒。

5.1.1 缺失模块

Missing modules

缺失模块在 Verilog 网表中没有模块定义（没有 module 语句），并且 is_unbound 属性设置为 true。

```
icc2_shell> get_attribute -objects $cells -name is_unbound
true
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_std_cell_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist
false
```

5.1.2 空模块

Empty modules

空模块有模块定义，但没有内容。使用标准单元与硬宏（hard macro）的层次计数属性来识别这类模块。

```
icc2_shell> get_attribute -objects $cells -name is_unbound
false
```

```
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_std_cell_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_hard_macro_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist
false
```

5.1.3 部分模块

Partial modules

只有部分网表的模块包含不完整的单元集合。使用层次计数属性来识别这类模块。

```
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_hard_macro_count
3
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_std_cell_count
119
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist
true
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist_only
false
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_pad_cell_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_physical_only_cell_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_switch_cell_count
0
```

5.1.4 穿越模块

Feedthrough modules

穿越模块包含输入端口与输出端口之间的网络连接，但不包含单元。

```
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_std_cell_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_hard_macro_count
0
```

```
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist
true
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist_only
true
```

5.1.5 绑值模块

Tie-off modules

绑值模块只有被绑到逻辑值的输出端口。

```
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_std_cell_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_hard_macro_count
0
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist
false
icc2_shell> get_attribute -objects $cells \
   -name hierarchy_has_shadow_netlist_only
false
```

5.2 创建黑盒参考

Creating Black Box References

识别出要作为黑盒处理的模块之后，使用 `create_blackbox` 命令为该单元创建黑盒物理层次。该命令还会更新指定单元的参考（reference）。请注意，读入网表之后，或提交块之后，可以把空模块、部分模块或缺失模块转换为黑盒。

下列示例在当前库中为实例 `u0_3` 创建黑盒，并把目标边界面积设置为 2000000。创建黑盒之后的版图窗口如图 **??** 所示。

```
icc2_shell> create_blackbox -target_boundary_area 2000000 u0_3
0
```

要查找设计中由 `create_blackbox` 命令创建的黑盒，或检查某个模块是否为黑盒，可按如下方式检查该模块的 `design_type` 属性：

```
icc2_shell> get_cells -hierarchical \
   -filter ref_block.design_type==black_box
{I_ORCA_TOP/I_BLENDER_0}

icc2_shell> get_attribute [get_cells I_ORCA_TOP/I_BLENDER_0] design_type
black_box
```

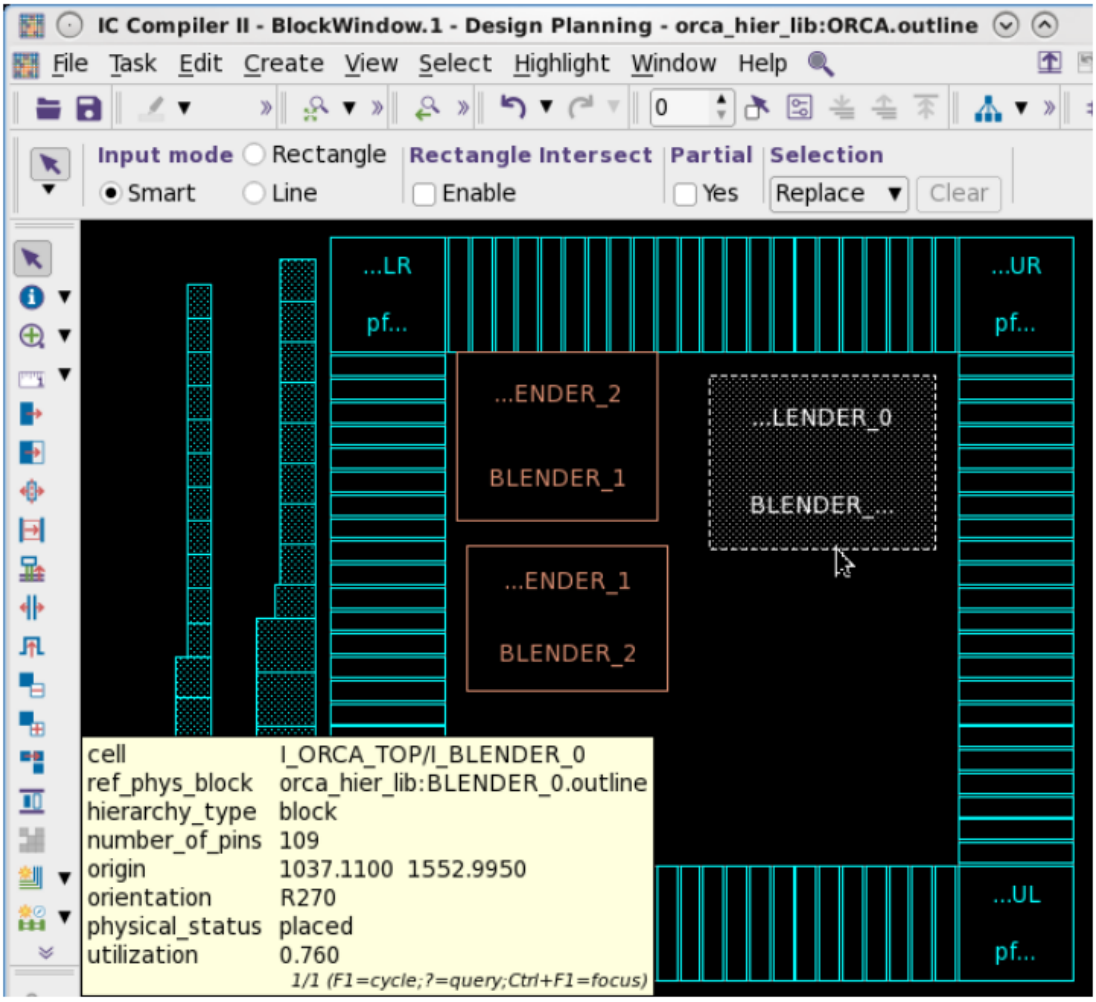


图 **5.2:** 图形界面中的黑盒

图形用户界面在层次浏览器（Hierarchy Browser）与属性编辑器（Properties Editor）中提供标注，以帮助识别黑盒，如图 **??** 所示。

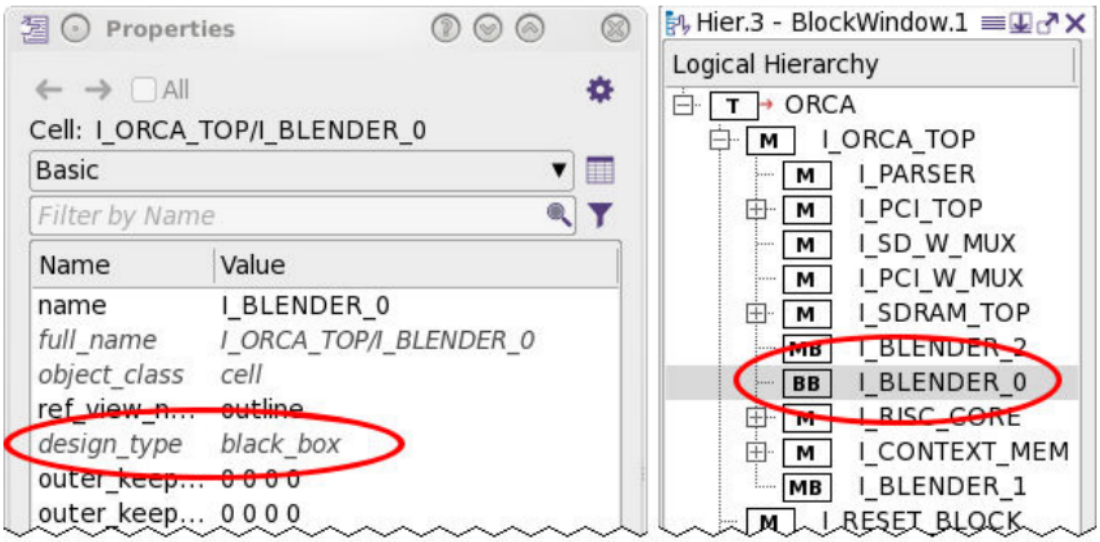


图 **5.3:** 层次浏览器与属性编辑器中的黑盒显示

要修改由 `create_blackbox -target_boundary_area` *area* 命令设置的面积，可用下列命令直接在黑盒单元上设置 `target_boundary_area` 属性：

```
icc2_shell> set_attribute -objects [get_cells I_ORCA_TOP/I_BLENDER_0] \
   -name target_boundary_area -value 1500000
{u0_3}
```

对于由部分 Verilog 网表创建的黑盒，可用下列命令指定目标利用率（target utilization）：

```
icc2_shell> set_attribute -objects [get_cells I_ORCA_TOP/I_BLENDER_0] \
   -name target_utilization -value 0.5
{u0_3}
```

块塑形期间，`shape_blocks` 命令会同时遵守 `target_boundary_area` 与 `target_utilization` 属性。

5.3 创建黑盒时序模型

Creating a Black Box Timing Model

黑盒默认不含时序信息。任何始于或终于黑盒的路径都是无约束的。IC Compiler II 工具提供创建黑盒时序模型的命令，使该模块可用于时序预算与优化。

创建黑盒时序模型的步骤如下：

1. 定义时钟端口、驱动与负载。

```
icc2_shell> set_blackbox_clock_port ...
icc2_shell> set_blackbox_port_drive ...
icc2_shell> set_blackbox_port_load ...
```

2. 设置时钟网络延时、引脚到引脚延时，以及建立（setup）与保持（hold）约束。

```
icc2_shell> create_blackbox_clock_network_delay ...
icc2_shell> create_blackbox_constraint ...
icc2_shell> create_blackbox_delay ...
icc2_shell> create_blackbox_drive_type ...
icc2_shell> create_blackbox_load_type ...
```

3. 把时序模型提交到黑盒。

```
icc2_shell> commit_blackbox_timing
1
```

通过指定模型端口、输入上的建立与保持约束、时钟到输出路径延时以及输入到输出路径延时，为黑盒创建时序模型。还可以指定输入端口上的负载以及输出端口的驱动强度。

黑盒时序信息存放在设计视图中。从黑盒创建抽象时，会在抽象视图内即时创建带特殊标记的单元、网络与连接。这种方法使空网表或部分网表也能进行时序分析，并允许新创建的穿越立即参与时序，而无需重新创建模型。

对于部分黑盒，只有未连接的端口可以用黑盒时序约束来约束。部分黑盒上的其他端口应使用 Synopsys 设计约束（Synopsys Design Constraints，SDC）文件来约束。

生成时序预算时，工具会保持你用黑盒时序命令指定的时序。若允许预算器调整黑盒时序预算，按如下方式设置 `plan.budget.bbt_fixed_delay` 应用选项（application option）：

```
icc2_shell> set_app_options -name plan.budget.bbt_fixed_delay \
   -value false
plan.budget.bbt_fixed_delay false
```

5.4 黑盒时序示例

Black Box Timing Example

下列示例给出创建黑盒时序模型所用的命令。得到的模型如图 **??** 所示。

```
set_blackbox_clock_port [get_ports {clk1_in clk1_out clk2_in}]
create_blackbox_load_type -lib_cell buf1 lt1
set_blackbox_port_load -type lt1 -factor 3 [get_ports in1]
create_blackbox_clock_network_delay -value 3.5 clk_1
create_blackbox_delay -rise_from in1 -fall_to out1 -value 5.0
create_blackbox_delay -fall_from in1 -rise_to out1 -value 5.2
create_blackbox_delay -from in2 -to out2 -clock main_clock \
   -value 0.4
create_blackbox_constraint -setup -edge rise -from clk2_in \
   -to in3 -value 1.2
create_blackbox_constraint -hold -edge rise -from clk2_in \
   -to in3 -value 0.8
create_blackbox_delay -from clk1_in -to clk1_out -value 1.1
create_blackbox_constraint -edge rise -from clk1_in -to in3 \
   -clock main_clock -value 0.5
create_blackbox_constraint -edge rise -from clk1_in -to in2 \
```

```
   -value 1.3
create_blackbox_delay -rise_from clk1_in -to out2 -value 2.2
create_blackbox_delay -fall_from clk1_in -to out3 -value 1.7
commit_blackbox_timing
```

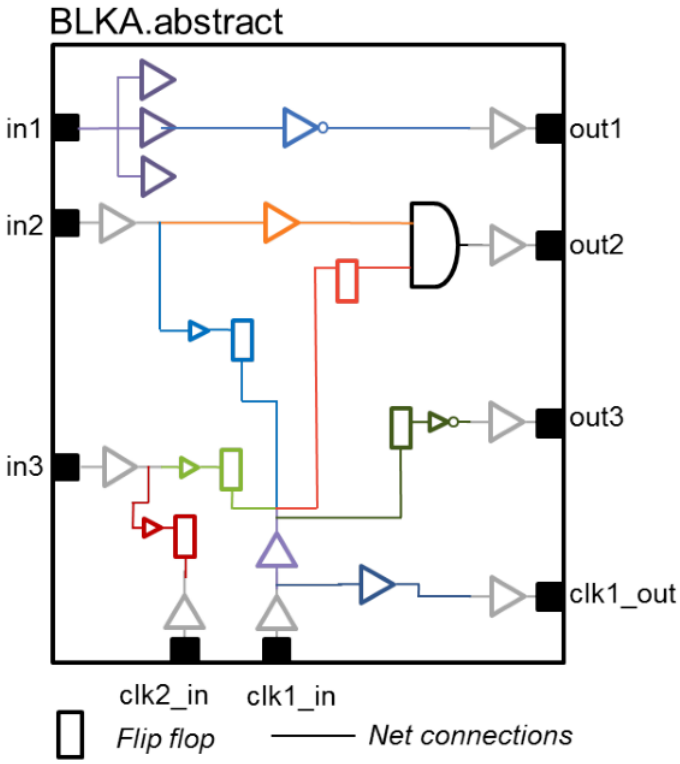


图 **5.4:** 黑盒时序模型示例

5.5 黑盒设计的 UPF

UPF for Black Box Designs

黑盒的统一功耗格式（Unified Power Format，UPF）约束与普通块的 UPF 约束指定方式不同，因为黑盒的行为如同叶单元，黑盒 UPF 约束从顶层角度描述上下文。黑盒 UPF 还为块输入端口定义接收端供电信息。

黑盒的 UPF 可以专门为该黑盒编写，也可以用 `save_upf -for_empty_blackbox` 命令与选项从已有设计中提取，如下例所示。

```
create_blackbox I_ORCA_TOP/I_BLENDER
current_design BLENDER
load_upf inputs/BLENDER.upf
commit_upf
save_upf -for_empty_blackbox ./split/BLENDER/top.upf
```

通常不会运行 `split_constraints` 命令来为黑盒或空模块生成 UPF 约束。若某条网络没有真实驱动，则使用该域的主供电。黑盒是唯一可以使用 `set_port_attributes` 命令或

set_related_supply_net（SRSN）命令来覆盖供电信息的层次单元。

应为设计中的黑盒创建抽象，因为这会创建黑盒时序单元。除了为输出与输入提供驱动与负载信息之外，这些单元还在网络上提供驱动端与接收端，并使 check_mv_design 等多电压相关命令按预期工作。

Chapter 6

规划 I/O 与倒装芯片凸点

Planning I/Os and Flip-Chip Bumps

创建布图规划（floorplan）之后，可以为设计例化输入/输出驱动器（I/O driver）与凸点单元（bump cell）。IC Compiler II 工具支持对 I/O 驱动器与倒装芯片（flip-chip）凸点单元做无约束布局，也支持基于约束的布局。可以为每个 I/O 创建布局约束，并指定顺序、xy 坐标以及布局边（placement side）。工具同时支持封装驱动（package-driven）与芯片驱动（die-driven）两种 I/O 布局流程，并可针对不同的芯片封装场景做适配。工具支持不同的 I/O 驱动器样式，例如带集成驱动器的焊盘（pad）。

芯片 I/O 可以沿芯片周边（periphery）放置，也可以放在芯片内核（core）内部的岛（island）中。约束包括信号约束（signal constraint）与电源约束（power constraint），用来规定 I/O 相邻关系（adjacency）要求与电源要求。可以使用 Tcl 命令和图形用户界面（GUI）执行 I/O 布局。

例化 I/O 驱动器与焊盘的流程如图 **??** 所示。

参见

更多细节见下列主题：

- 校验库要求
- 创建 I/O 环或 I/O 引导
- 创建凸点单元阵列
- 创建凸点单元的行或列
- 在预定位置放置凸点单元
- 创建电源 I/O 布局约束
- 创建信号 I/O 布局约束
- 把 I/O 焊盘分配到凸点
- 放置 I/O 并写出约束
- 添加并放置仅物理 I/O 单元
- 检查 I/O 布局

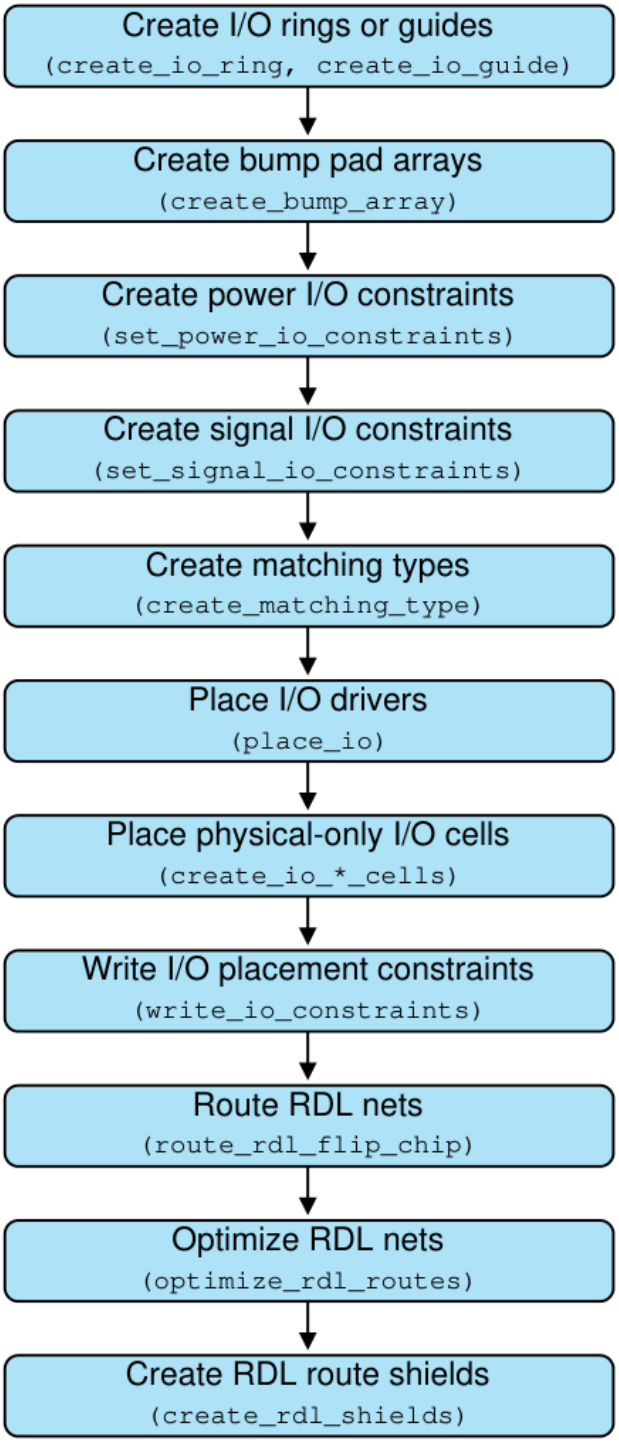

Create I/O rings or guides
(create_io_ring, create_io_guide)
Create bump pad arrays
(create_bump_array)
Create power I/O constraints
(set_power_io_constraints)
Create signal I/O constraints
(set_signal_io_constraints)
Create matching types
(create_matching_type)
Place I/O drivers
(place_io)
Place physical-only I/O cells
(create_io_*_cells)
Write I/O placement constraints
(write_io_constraints)
Route RDL nets
(route_rdl_flip_chip)
Optimize RDL nets
(optimize_rdl_routes)
Create RDL route shields
(create_rdl_shields)

图 **6.1:** I/O 规划流程

- 布线再分布层网线
- 优化再分布层布线
- 创建再分布层网线屏蔽
- 创建具有多层层次的倒装芯片设计

6.1 校验库要求

Validating Library Requirements

开始倒装芯片设计之前，必须确认包含倒装芯片元件单元的参考库（reference library）已按工具要求正确配置。具体而言，应使用 IC Compiler II Library Manager 或 IC Compiler II 工具，检查下列倒装芯片参考单元的 `design_type` 属性：

- 凸点单元：`design_type` 属性必须设为 `flip_chip_pad`。

  在凸点单元的库单元引脚上，`is_pad` 属性必须设为 `true`。必须设置该属性，工具才能在凸点被连接或分配到端口（port）时，自动为顶层端口创建端子（terminal）。

- 驱动单元（driver cell）：`design_type` 属性必须设为 `flip_chip_driver` 或 `pad`。

  若已定义倒装芯片匹配类型（matching type），也可以把 `design_type` 属性设为 `macro`，并把该驱动单元用于 I/O 布局。

要连接到再分布层（redistribution layer，RDL）布线的驱动单元，其端子或引脚形状的 `class` 属性应设为 `bump`。若没有带此设置的端子，RDL 布线器会选择该端口最上层的端子。

- 填充单元（filler cell）与间断单元（break cell）：`design_type` 属性必须设为 `pad_spacer`。
- 拐角单元（corner cell）：`design_type` 属性必须设为 `corner`。

可以在 IC Compiler II Library Manager 中查看倒装芯片库单元框架视图（frame view）上的属性类型与属性设置，如图 **??** 所示。

注意

库单元的 `design_type` 属性设置可以从该库单元的 LEF 文件导出。LEF 文件中宏类型为「PAD」时，生成的值为 `pad`。LEF 文件中宏类型为「COVER BUMP」时，生成的值为 `flip_chip_pad`。

可以用 IC Compiler II 工具中的 `set_attribute` 命令覆盖库参考单元的某些属性。在 IC Compiler II 工具中所做的属性设置不会写回参考库。

对于宏（macro），`design_type` 属性的默认设置是 `macro`，并且不会执行凸点分配。要对宏执行凸点分配，必须创建包含宏、宏的引脚或宏引脚端子的匹配类型。只要匹配宏的任一部分，就足以让 `place_io` 命令识别一个单元实例。要识别分配到凸点的引脚，执行下列操作之一：

- 在匹配类型中显式包含该引脚。这是推荐方法。
- 在库准备期间把任一端子引脚的 `class` 属性设为 `bump`，或在 IC Compiler II 工具中使用 `set_attribute` 命令。

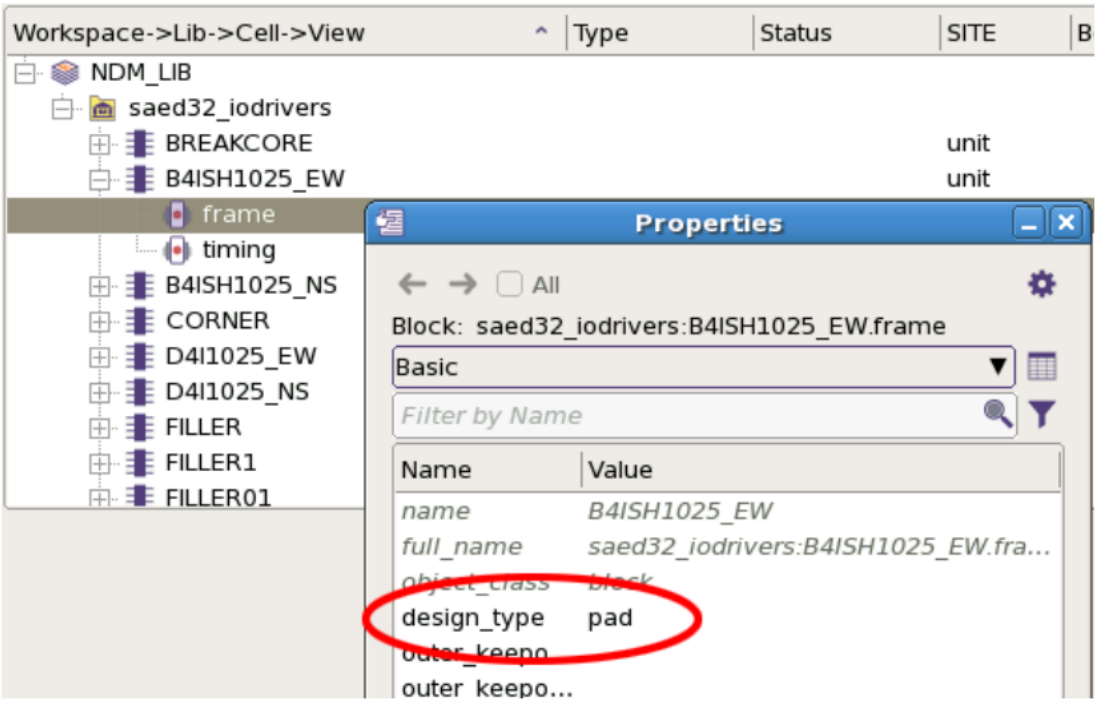


图 **6.2:** 驱动单元的 `design_type` 属性

6.2 创建 I/O 环或 I/O 引导

Creating I/O Rings or Guides

IC Compiler II 工具支持 I/O 引导（I/O guide）。I/O 引导构成 I/O 驱动器的布局区域。可以用四条 `create_io_guide` 命令在布图规划周边放置四条 I/O 引导来创建 I/O 环（I/O ring），也可以用一条 `create_io_ring` 命令创建。IC Compiler II 工具还支持多个 I/O 环，并支持在芯片同一条边上放置多条 I/O 引导。

只要遵守下列条件，就可以把 I/O 引导放在层次中的任意位置：

- 所有倒装芯片凸点单元必须放在顶层。
- 对于已经放置的块，I/O 引导不得跨越块边界。
- I/O 焊盘单元与其关联的 I/O 引导必须放在同一个物理块实例内。
- 只有与 I/O 引导处于同一逻辑层次的焊盘，才能用 `set_signal_io_constraints` 命令施加约束。
- I/O 引导不得重叠。

放置 I/O 单元之前，必须先创建 I/O 引导或 I/O 环。下列命令在布图规划周边创建包含四条 I/O 引导的环，并取出 `create_io_ring` 命令所创建的 I/O 引导名称。

```
icc2_shell> create_io_ring -name outer_ring -corner_height 300
{outer_ring}
icc2_shell> get_io_guides
{outer_ring.left outer_ring.bottom outer_ring.right outer_ring.top}
```

也可以用四条 `create_io_guide` 命令，在芯片的左、上、右、下四边创建 I/O 引导。

```
icc2_shell> create_io_guide -name "ring_left" -line {{0 300} 3000} \
   -side left
{ring_left}
icc2_shell> create_io_guide -name "ring_top" -line {{300 3640} 3000} \
   -side top
{ring_top}
icc2_shell> create_io_guide -name "ring_right" -line {{3680 3380} 3000} \
   -side right
{ring_right}
icc2_shell> create_io_guide -name "ring_bottom" -line {{3380 0} 3000} \
   -side bottom
{ring_bottom}
icc2_shell> get_io_guides
{ring_left ring_top ring_right ring_bottom}
```

也可以用 Floorplan Task Assistant 在 GUI 中创建 I/O 环与 I/O 引导。选择 **Task > Task Assistant**，再选择 **Floorplan Preparation > I/O Planning > I/O Ring and Guide**。输入 I/O 环或 I/O 引导的信息，然后单击 Apply，创建环或引导。

6.2.1 I/O 环与 I/O 引导任务

I/O Ring and I/O Guide Tasks

使用 `create_io_ring` 与 `create_io_guide` 命令的选项，控制环的创建方式：

- 用 `-name` 选项为环指定名称。该名称用作 I/O 引导名称的前缀。

```
icc2_shell> create_io_ring -name outer_ring
{outer_ring}
icc2_shell> get_io_rings
{outer_ring}
icc2_shell> get_io_guides
{outer_ring.left outer_ring.bottom outer_ring.right outer_ring.top}
```

- 用 `-inside` 选项在已有环的内部再创建一条环。已有环用作新环的外边界。

```
icc2_shell> create_io_ring -name inner -inside outer_ring
```

- 用 `-offset` 选项指定内环边界与外环边界之间的距离。

```
icc2_shell> create_io_ring -name inner -inside outer -offset 500
```

- 用 `-bbox` 选项在指定包围盒（bounding box）内创建环。

```
icc2_shell> create_io_ring -bbox {{1000 1000} {2000 2000}}
```

- 用 `-pad_cell_list` 选项指定要分配到该环的焊盘单元实例列表。

```
icc2_shell> create_io_ring -pad_cell_list {pad_iopad_0 pad_iopad_1}
```

- 用 `-guides` 选项把指定的 I/O 引导纳入环中。

```
icc2_shell> get_io_guides
{ring_left ring_top ring_right ring_bottom}
icc2_shell> create_io_ring -guides [get_io_guides]
{_default_io_ring1}
```

- 用 create_io_guide 命令的 -offset 选项，指定 I/O 引导起点与第一个 I/O 驱动器之间的最小距离，以及 I/O 引导终点与最后一个 I/O 驱动器之间的最小距离。

6.2.2 编辑、报告与删除 I/O 引导和 I/O 环

Editing, Reporting, and Removing I/O Guides and I/O Rings

- 用 add_to_io_guide 命令把 I/O 焊盘单元加入一条 I/O 引导。

```
icc2_shell> add_to_io_guide guide_left {pad_iopad_0 pad_iopad_1}
```

- 用 get_io_guides 命令创建 I/O 引导的集合，用 get_io_rings 命令创建 I/O 环的集合。

```
icc2_shell> get_io_guides
{guide_left guide_top guide_right guide_bottom}
icc2_shell> get_io_rings
{ring_outer}
```

- 用 remove_from_io_guide 命令从 I/O 引导中移除 I/O 焊盘单元。

```
icc2_shell> remove_from_io_guide guide_left {pad_iopad_0 pad_iopad_1}
total 2 cells removed from io guide
```

- 用 remove_from_io_ring 命令从 I/O 环中移除一条 I/O 引导。

```
icc2_shell> remove_from_io_ring ring_outer {guide_left guide_top}
```

- 用 report_io_rings 与 report_io_guides 命令报告块中的 I/O 环或 I/O 引导。
- 用 remove_io_rings 与 remove_io_guides 命令从块中删除 I/O 环或 I/O 引导。

6.3 创建凸点单元阵列

Creating Arrays of Bump Cells

IC Compiler II 工具支持倒装芯片设计中的凸点单元布局。可以根据 DEF 或 AIF 文件中的布局坐标放置凸点单元，也可以用 create_bump_array 命令创建凸点阵列（bump array）。应在放置 I/O 驱动单元之前放置凸点单元。在芯片驱动流程中，先为凸点单元布局创建一个区域，再由工具确定凸点单元的位置。下面的示例创建五个凸点阵列区域，分别对应芯片的左、上、右、下与中心区域，并用 BUMP 库单元填满每个区域。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {140 140} \
   -bbox {{410 450} {1175 3278}} -name left
icc2_shell> create_bump_array -lib_cell BUMP -delta {140 140} \
```

```
   -bbox {{2620 450} {3325 3278}} -name right
icc2_shell> create_bump_array -lib_cell BUMP -delta {140 140} \
   -bbox {{1100 450} {2650 1125}} -name bottom
icc2_shell> create_bump_array -lib_cell BUMP -delta {140 140} \
   -bbox {{1100 2555} {2650 3250}} -name top
icc2_shell> create_bump_array -bbox {{1100 1150} {2650 2550}} \
   -pattern staggered_1 -lib_cell {BUMP} -delta {140 140} -name pg_bump
```

执行上述命令之后的版图如图 **??** 所示。

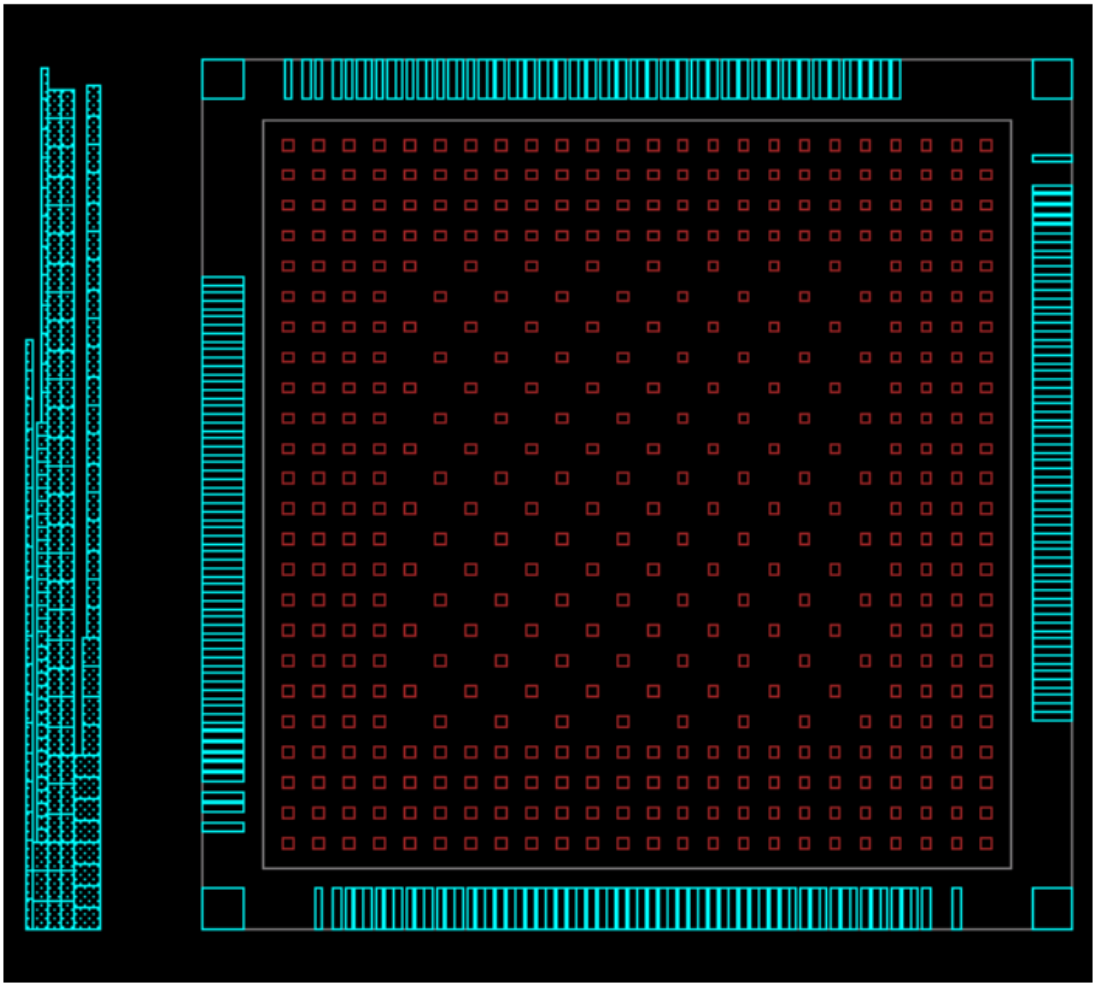


图 **6.3:** 执行 `create_bump_array` 之后的版图

6.3.1 凸点阵列选项

Bump Array Options

使用 `create_bump_array` 命令的选项，控制凸点阵列的创建方式：

- 用 `-delta` 选项指定相邻行之间的水平间距与垂直间距；`-delta` 与 `-lib_cell` 是必需选项。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150}
```

- 用 `-name` 选项为凸点阵列创建名称。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -name pg_bump
```

- 用 -bbox 选项指定容纳凸点单元的矩形包围盒。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -bbox {{1000 1000} {3000 3000}}
```

- 用 -boundary 选项在复杂的直线多边形内创建凸点阵列。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -boundary {{1000 1000} {3000 1000} {3000 3000} {1000 3000}}
```

- 用 -repeat 选项按列数或行数限制凸点阵列的尺寸。凸点在芯片或包围盒内居中。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -repeat {5 5}
```

- 用 -origin 选项设置凸点阵列包围盒左下角与第 0 列、第 0 行凸点单元实例左下角之间的间距。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -origin {1000 1000}
```

- 用 -orientation 选项指定凸点阵列中每个凸点单元的方向。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -orientation N
```

  合法的方向值为 N、W、S、E、FN、FS、FE 与 FW。

- 用 -pattern inline | staggered_1 | staggered_2 选项指定凸点单元占据每一个可能位置，还是占据交替位置。

```
icc2_shell> create_bump_array -lib_cell BUMP -delta {150 150} \
   -pattern staggered_1
```

  关键字 inline 把凸点单元放在 -delta 选项所指定阵列的每一个点上。关键字 staggered_1 把凸点单元放在列索引与行索引之和为偶数的点上。关键字 staggered_2 把凸点单元放在列索引与行索引之和为奇数的点上。图 **??** 是使用这三种布局图案的一个小型凸点布局示例。

6.4 创建凸点单元的行或列

Creating Rows or Columns of Bump Cells

在不使用 I/O 引导放置 I/O 焊盘单元的设计中，可以用 create_bump_pattern 命令例化凸点单元，并指定凸点间距、匹配类型以及其他信息。对于包含成簇 I/O 单元、并且要求把凸点单元排成与 I/O 焊盘单元垂直的行或列的设计，该命令很有用。

要为一组焊盘单元创建凸点图案，按本节所述创建凸点图案文件，再用 create_bump_pattern 命令加载该文件，如下所示：

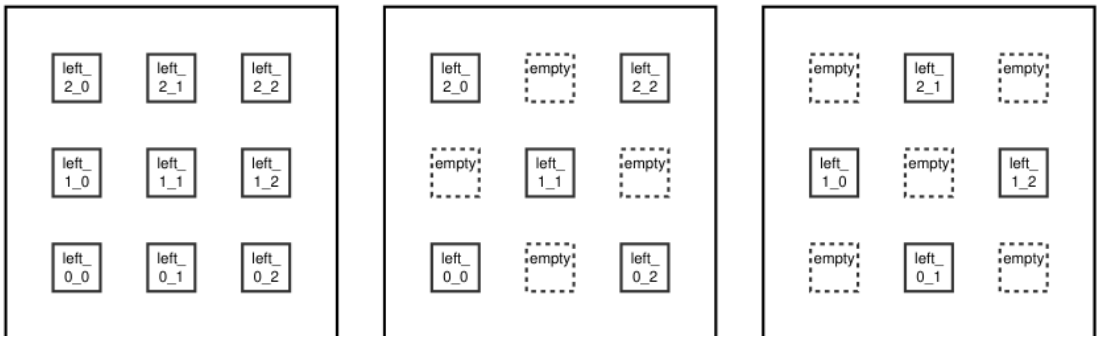


图 **6.4:** 凸点阵列布局图案选项

```
icc2_shell> create_bump_pattern -file_name bump_patterns.txt
12 bumps and 5 matching types are created successfully
```

本示例使用下面的凸点图案文件；版图结果如图 **??** 所示。

```
# Bump pattern file for area I/O pad cluster
bump_lib_cell BUMP_PAD   // Bump library cell
bump_pitch 150           // (A) Bump pitch

io_group {               // I/O group description
  {{bus_3}   {bus_2}       {bus_1}       {bus_0 s}
   {vss g0_} {vcc_main p_} {vss_io g1_}  {vcc_io p1_}
   {bus_7}   {bus_6}       {bus_5}       {bus_4}}
  {orientation row} // Valid orientations are row, column,
                    // or perpendicular
  {{-50}        // (B) Offset between center line of bumps
                //     and pad bbox center
   {100}        // (C) Spacing between the left or bottom
                //     of pad bbox and nearest bump cell center
   {s s p_ g0_} // Left or bottom bumps
   {75}         // (D) Spacing between the right/top
                //     of pad bbox and nearest bump cell center
   {s p1_ g1_   // Right or top bumps
    s s s s s}
   }
}
```

`-file_name` 选项所指定的图案文件定义凸点库单元、凸点布局节距（pitch）、间距、匹配类型，以及用于创建凸点图案的其他信息。以井号（`#`）开头的行为注释，以两条斜杠（`//`）开头的为行内注释。图案文件使用下列格式：

- `bump_lib_cell`:凸点库单元。把凸点库单元指定为`library_cell`,或`library_name/library_cell`。
- `bump_pitch`：凸点节距，单位为微米。
- `io_group`：I/O 组描述，用花括号括起，包含下列信息：
  - I/O 焊盘实例名：写在花括号内的实例名列表。每个实例名各自再用一对花括号括起，并可带一个可选的匹配类型前缀。若未指定匹配类型前缀，则使用前缀「s」。凸点实例名基于第

一个实例的 I/O 焊盘名。

在本示例中，`{vss g0_}` 这一项创建一个实例名为 `bus_3_n` 的凸点单元，以及一个名为 `g0_bus_3` 的匹配类型。「g0_」是匹配类型前缀。「bus_3」是所指定的第一个 I/O 焊盘，该图案文件创建的全部凸点单元实例都使用这一字符串。

- `orientation:`（可选）方向说明，取值为 `row`、`column` 或 `perpendicular`。

  若省略 `orientation` 关键字，命令会根据 I/O 焊盘包围盒的高度与宽度放置凸点单元。若宽度大于高度，工具把凸点单元排成列。若高度大于宽度，命令把凸点单元排成行。

- 焊盘顺序与间距：顺序与间距列表写在花括号内；一个图案文件可以包含多个顺序与间距段。说明如下：

  * 中心线偏移：新凸点单元中心线与由 I/O 焊盘定义的包围盒中心线之间的距离，单位为微米。
  * 左侧或底侧第一个凸点的间距：I/O 焊盘包围盒左边缘或底边缘到最近凸点单元中心的距离。
  * 左侧或底侧凸点单元：与 I/O 焊盘实例列表中所定义的匹配类型前缀相对应的匹配类型前缀列表。
  * 右侧或顶侧第一个凸点的间距：I/O 焊盘包围盒右边缘或顶边缘到最近凸点单元中心的距离。
  * 右侧或顶侧凸点单元：与 I/O 焊盘实例列表中所定义的匹配类型前缀相对应的匹配类型前缀列表。
  * `orientation:`（可选）该组的方向说明，取值为 `row`、`column` 或 `perpendicular`。

示例凸点图案文件所创建的版图结果如图 **??** 所示。

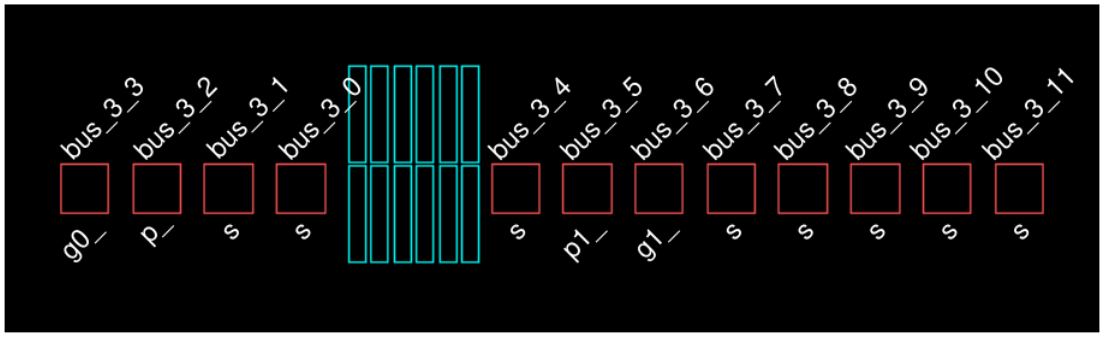


图 **6.5:** 凸点单元实例

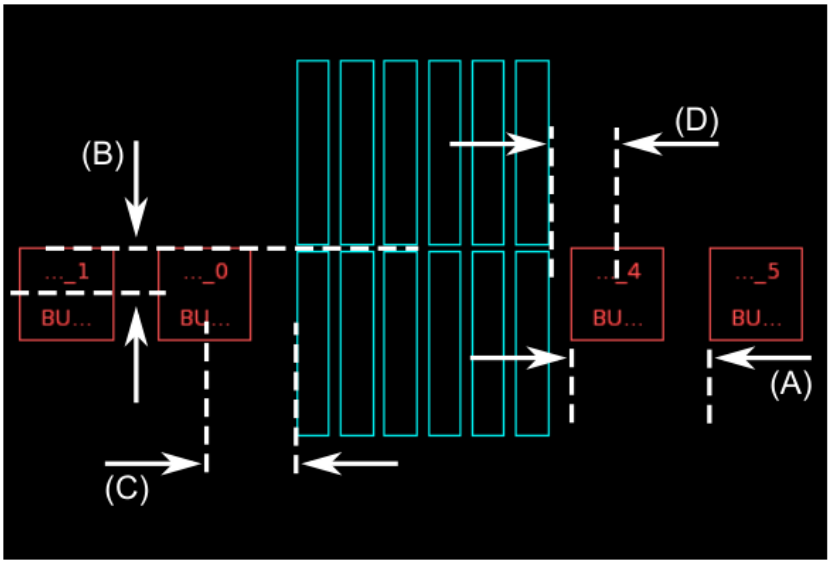
图案文件中的间距值如何对应版图中凸点单元与 I/O 焊盘单元的间距，如图 **??** 所示。`report_matching_types` 命令的输出给出前述图案文件所生成的匹配类型。

注意

本示例中已去掉 Uniquify 列与 Object Type 列。

包含多组行或列说明，就可以创建更复杂的凸点图案。下面的凸点图案文件通过三组偏移与匹配类型，创建三行凸点单元。每一行在 I/O 驱动器左侧有两个凸点，在右侧有两个凸点。

```
# Pattern file for three bump rows
```




```
bump_lib_cell BUMP_PAD
bump_pitch 150 // (A)
io_group {
 {{bus_3} {bus_2}
  ...}
 {orientation row}
 {{-50}   // (B)
 {100}   // (C)
 {s ...}  // Left bumps
 {75}    // (D)
 {s ...}  // Right bumps
 }
}
```

图 **6.6:** 凸点单元偏移与间距

表 **6.1:** 前述图案文件生成的匹配类型

匹配类型	对象名	匹配类型	对象名
sbus_3	bus_0	g0_bus_3	bus_3_3
	bus_1		vss
	bus_2	g1_bus_3	bus_3_6
	bus_3		vss_io
	bus_3_0	p1_bus_3	bus_3_5
	bus_3_1		vcc_io
	bus_3_10	p_bus_3	bus_3_2
	bus_3_11		vcc_main
	bus_3_4		
	bus_3_7		
	bus_3_8		
	bus_3_9		
	bus_4		
	bus_5		
	bus_6		
	bus_7		

```
bump_lib_cell BUMP_PAD   // bump lib cell
bump_pitch 200           // pitch

io_group {               // I/O group description
  {{bus_3}    {bus_2 }       {bus_1}       {bus_0 s}
   {vss g0_}  {vcc_main p_}  {vss_io g1_}  {vcc_io p1_}
   {bus_7}    {bus_6}        {bus_5}       {bus_4}}
  {orientation row}
  {{150}  {100} {s s}   {150} {p_ g0_}} // top row
  {{0}    {100} {s p1_} {150} {g1_ s}}  // middle row
  {{-150} {100} {s s}   {150} {s s}}    // bottom row
}
```

版图结果如图 **??** 所示，其中单元按匹配类型高亮。



图 **6.7:** 三行凸点图案

6.5 在预定位置放置凸点单元

Placing Bump Cells in Predetermined Locations

若凸点单元的位置由封装约束决定，使用 `read_aif` 命令读入凸点单元及其位置的列表。位置信息通常来自 DEF 或 AIF 文件，其中包含块中凸点单元的 x 坐标、y 坐标以及凸点单元名称。使用 `-hierarchy` 选项读入凸点单元与 I/O 焊盘的位置，并把 I/O 焊盘放到物理层次的任意一级。

```
icc2_shell> read_aif bumps.aif
```

AIF 文件的一个简短示例如下：

```
[DATABASE]
TYPE=AIF
VERSION=2.0
UNITS=UM
[DIE]
NAME=mydesign_die
WIDTH=3840.575700
HEIGHT=3690.006000
[PADS]
BUMP_N = POLY -17.000 46.500 -17.000 46.475 ...
[NETLIST]
;Netname Pad#    Type    Pad_X       Pad_Y       Ball#
-  left_0_0      BUMP_N  -1463.7878  -1348.5030
-  left_0_1      BUMP_N  -1323.7878  -1348.5030
...
-  right_0_0     BUMP_N  746.2122    -1348.5030
-  right_0_1     BUMP_N  886.2122    -1348.5030
```

```
...
-  bottom_0_0    BUMP_N  -773.7878   -1348.5030
-  bottom_0_1    BUMP_N  -633.7878   -1348.5030
...
-  top_0_0       BUMP_N  -773.7878   756.4970
-  top_0_1       BUMP_N  -633.7878   756.4970
...
-  pg_bump_0_0   BUMP_N  -773.7878   -648.5030
-  pg_bump_0_2   BUMP_N  -493.7878   -648.5030
...
```

6.6 创建电源 I/O 布局约束

Creating Power I/O Placement Constraints

电源 I/O 布局约束规定电源 I/O 驱动器的间距以及其他要求。使用 set_power_io_constraints 命令为 I/O 驱动焊盘的布局设置约束，并使用 place_io 命令把驱动器放进 I/O 引导。下面的示例为电源 I/O 驱动器设置布局约束。

```
icc2_shell> set_power_io_constraints \
   -io_guide_object [get_io_guides "*left *right"] \
    {{reference: VDD_EW}
     {ratio: 3}
     {prefix: VDD}
     {connect: {VDDIO VDDIO} {VDD VDD1} {VSS VSS1}}}
icc2_shell> set_power_io_constraints \
   -io_guide_object [get_io_guides "*left *right"] \
    {{reference: VSS_EW}
     {ratio: 7}
     {prefix: VSS}
     {connect: {VSSIO VSSIO} {VDD VDD1} {VSS VSS1}}}
Power IO constraints set successfully
1
```

在本示例中，第一条 set_power_io_constraints 命令规定下列约束：

- 该约束只作用于左侧与右侧 I/O 引导。
- 根据该约束插入电源焊盘单元 VDD_EW。
- 相邻 VDD_EW 焊盘之间的信号焊盘数量为三个。
- 把字符串「VDD」插入实例名。
- VDDIO 焊盘单元引脚连接到 VDDIO 网线，VDD 焊盘单元引脚连接到 VDD1 网线，VSS 焊盘单元引脚连接到 VSS1 网线。

第二条 set_power_io_constraints 命令为 VSS_EW 单元参考创建类似的约束，但规定相邻 VSS_EW 单元之间可以插入七个信号 I/O 焊盘。

执行 set_power_io_constraints 与 place_io 命令之后，版图左下角的一小段如图 **??** 所示。

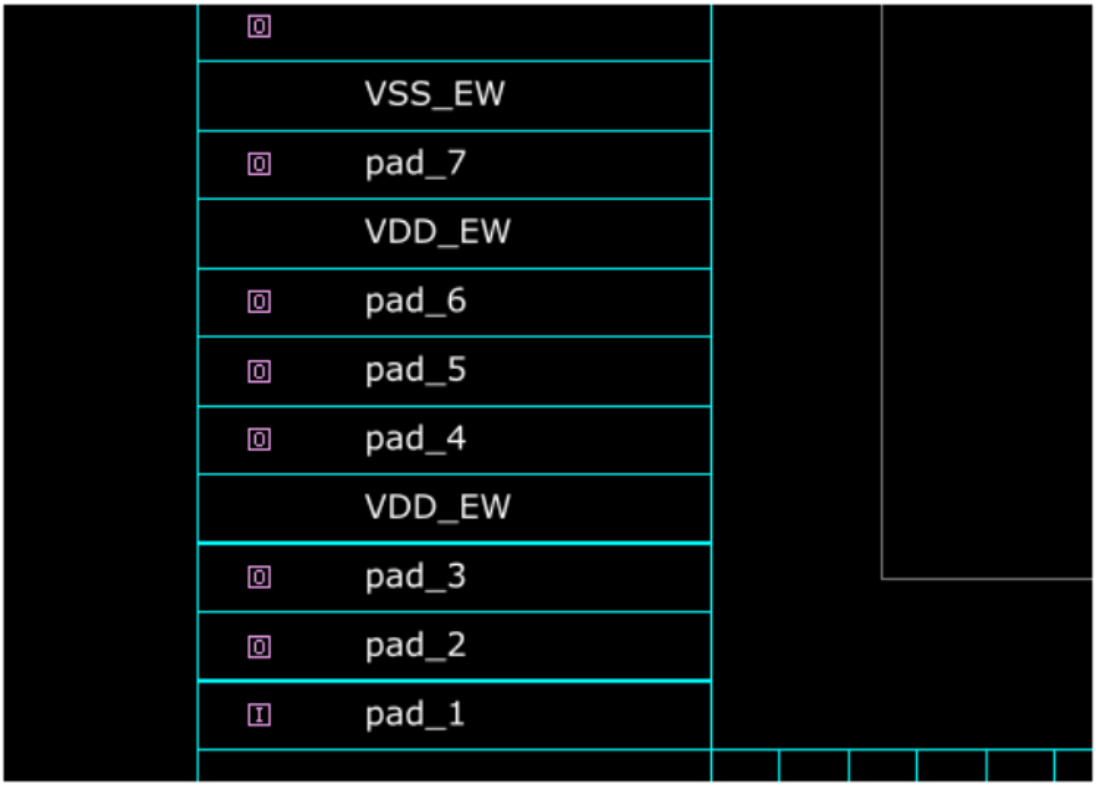


图 **6.8:** 带电源 I/O 约束的版图

若不用 `set_power_io_constraints` 命令指定约束，`place_io` 命令会把电源焊盘在 I/O 引导中均匀放置。

6.6.1 电源 I/O 布局约束选项

Power I/O Placement Constraint Options

使用 `set_power_io_constraints` 命令的选项，进一步控制电源信号 I/O 的布局。

- 用 `-share` 选项把多个电源焊盘分配给同一个凸点单元。

```
icc2_shell> set_power_io_constraints \
   -reference_cell {VDD_NS VSS_NS} -share {{2 VDD_NS} {3 VSS_NS}}
```

在本示例中，`-share {{2 VDD_NS} {3 VSS_NS}}` 允许最多两个 `VDD` 电源焊盘驱动同一个 `VDD` 凸点单元，并允许三个 `VSS` 电源焊盘驱动同一个 `VSS` 凸点单元。

- 用 `-ratio` 选项定义相邻电源焊盘之间可以放置的信号焊盘数量。

```
icc2_shell> set_power_io_constraints \
   -reference_cell {VDD_NS VSS_NS} -ratio {{7 VDD_NS} {6 VSS_NS}}
```

在本示例中，工具在 `VDD_NS` 焊盘单元之间放置的 I/O 焊盘不超过七个，在 `VSS_NS` 焊盘单元之间放置的 I/O 焊盘不超过六个。

- 用 `-spacing` 选项指定同一类型相邻电源焊盘之间的最大间距。

```
icc2_shell> set_power_io_constraints -reference_cell {VDD_NS VSS_NS} \
   -spacing {{100 VDD_NS} {100 VSS_NS}}
```

- 用 -offset 选项指定 I/O 引导起点与指定类型第一个电源焊盘最近边缘之间的最大距离。

```
icc2_shell> set_power_io_constraints -reference_cell {VDD_NS VSS_NS} \
   -offset {{100 VDD_NS} {100 VSS_NS}}
```

6.7 创建信号 I/O 布局约束

Creating Signal I/O Placement Constraints

要创建信号 I/O 布局约束，以规定信号 I/O 驱动器的顺序与间距，使用 set_signal_io_constraints 命令。该命令既可以为顶层的 I/O 引导指定约束，也可以为设计层次较低级别中的 I/O 引导指定约束。

在下面的示例中，set_signal_io_constraints 命令为 _default_io_ring1.left 这条 I/O 引导中的三个信号 I/O 驱动器指定仅顺序（order-only）约束：u_p/clk_pad_P1、u_p/sdclk_pad_P1 与 u_p/test_so_4_pad_P1。

```
icc2_shell> set_signal_io_constraints -constraint {{order_only} \
   u_p/clk_pad_P1 u_p/sdclk_pad_P1 u_p/test_so_4_pad_P1} \
   -io_guide_object {_default_io_ring1.left}
Signal IO constraints set successfully
1
```

-constraint {constraint_spec} 选项支持多种格式，用来指定信号 I/O 驱动器的布局。使用 -constraint {{order-only} pad_1 pad_2 ...} 选项，按指定顺序放置焊盘。

> **注意**
>
> 相邻焊盘之间可能会插入额外间距以及其他焊盘单元。

要在 I/O 驱动器之间创建固定间距，在信号名之间插入以微米为单位的间距值。例如，-constraint {pad_1 10 pad_2 20 pad_3} 选项在焊盘 pad_1 与 pad_2 之间插入 10 微米的间距，在焊盘 pad_2 与 pad_3 之间插入 20 微米的间距。要把每个信号 I/O 驱动器按指定节距放置，使用 -constraint {{pitch} pad_1 pad_2 ...} 这一选项格式。

也可以用 write_io_constraints -filename constraint_file_name 命令写出信号 I/O 布局约束，再用 set_signal_io_constraints -file constraint_file_name 命令加载这些约束。当信号约束数量很多时，约束文件很有用。下面的示例从约束文件 signal_io_constraints.txt 加载信号 I/O 布局约束。

```
icc2_shell> set_signal_io_constraints -file signal_io_constraints.txt
Signal IO constraints set successfully
1
icc2_shell> sh cat signal_io_constraints.txt
_default_io_ring1.left
{{order_only}
```

```
u_p/clk_pad_P1
u_p/sdclk_pad_P1
u_p/test_so_4_pad_P1};
```

信号排序的起点如图 **??** 所示：左边从左下焊盘开始，顶边从左上焊盘开始，右边从右上焊盘开始，底边从右下焊盘开始。

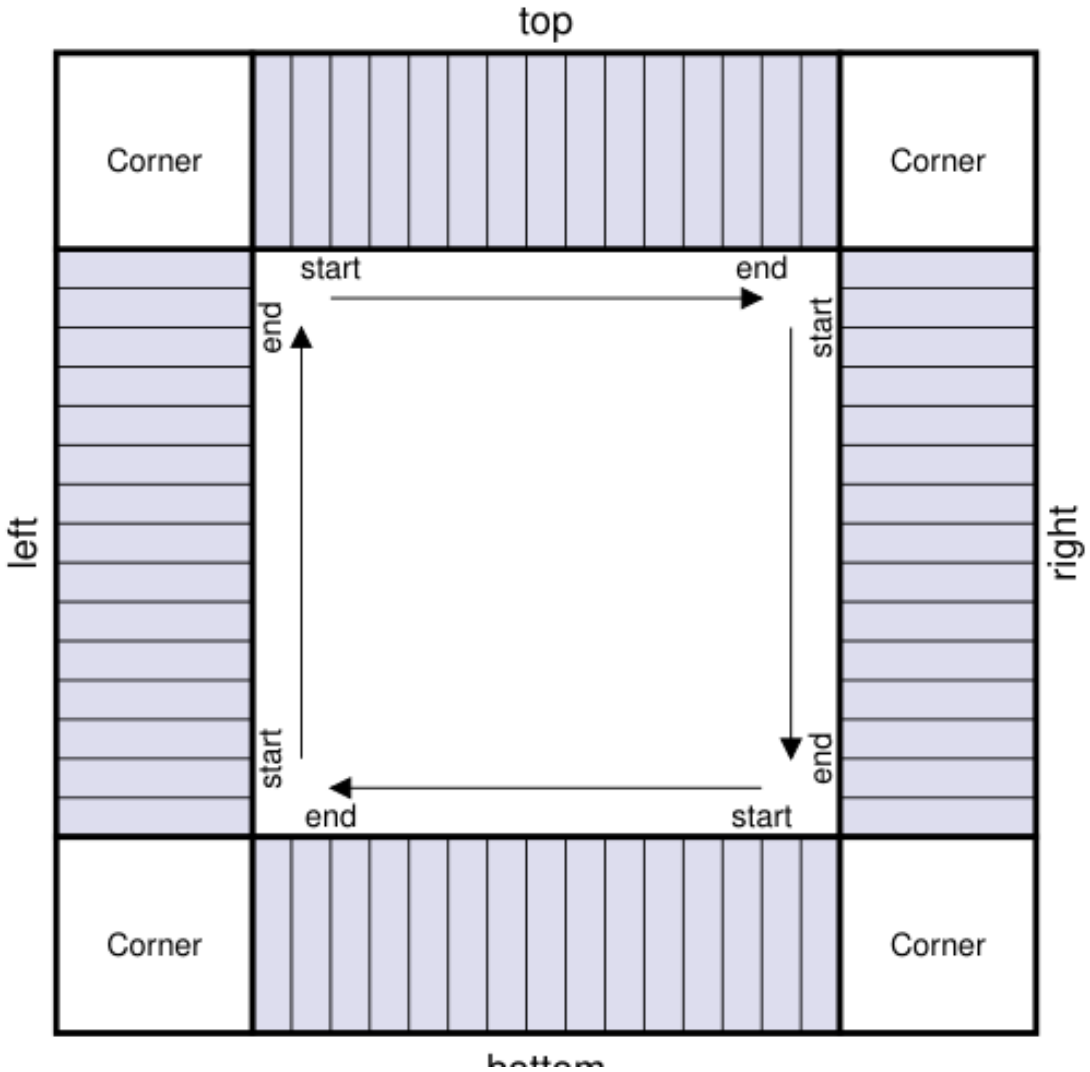


图 **6.9:** 信号 I/O 约束的各边排序

在下面的示例中，信号 I/O 约束文件为环中的顶部 I/O 引导创建顺序与间距约束。该约束在 I/O 引导起点与第一个信号 I/O 焊盘之间创建 80 微米的间隙。放置 `Pad_1`、`Pad_2` 与 `Pad_3` 这三个 I/O 驱动器时使用 55 微米的节距。该约束在 `Pad_3` 与 `Pad_4` 这两个 I/O 驱动器之间创建 10 微米的间距，在 `Pad_4` 与 `Pad_5` 之间创建 40 微米的间距，在 `Pad_5` 与 `Pad_6` 之间创建 60 微米的间距，并在 `Pad_6`、`Pad_7` 与 `Pad_8` 之间不留间距。示例 **??** 给出信号 I/O 约束文件，执行 `place_io` 命令之后的版图结果如图 **??** 所示。

```
ring.top
{80}
{{55} Pad_1 Pad_2 Pad_3}
{10}
```

```
{Pad_4 40 Pad_5 60 Pad_6}
{Pad_7 Pad_8}
;
```

代码 **6.1:** I/O 约束文件示例

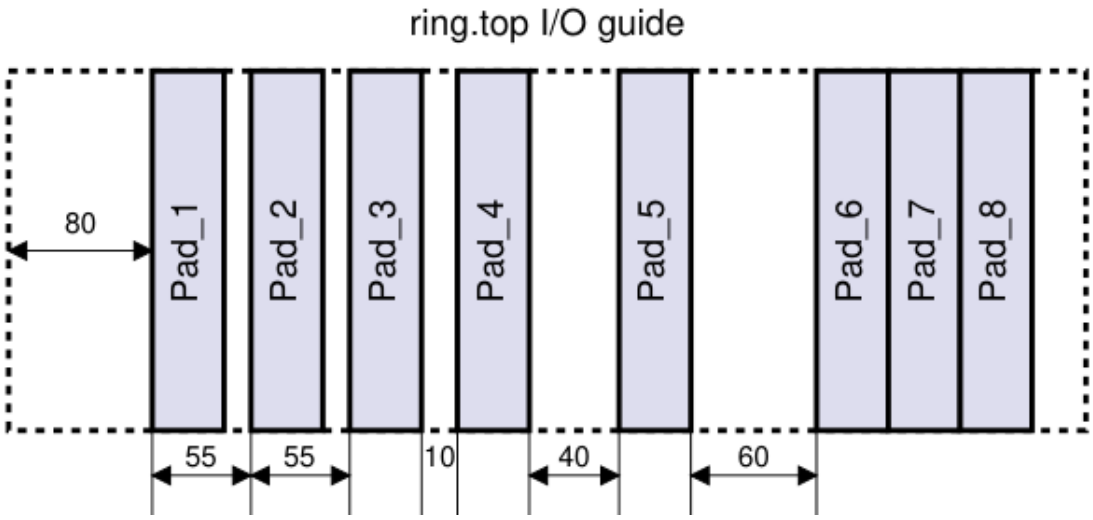


图 **6.10:** `ring.top` 的 I/O 布局

6.7.1 用 CSV 文件指定 I/O 布局约束

Specifying I/O Placement Constraints With a CSV File

除约束文件之外，也可以使用逗号分隔值（comma-separated values，CSV）文件，为某条特定的 I/O 引导定义 I/O 单元的顺序与位置。

注意

使用 CSV 文件放置 I/O 时请注意下列事项：

- CSV 文件必须包含一行表头，用来定义各字段的顺序
- CSV 文件中只有「Pad Name」字段是必需的，「Offset Value」字段是可选的
- CSV 文件中的其他字段会被读入并忽略
- 字段可以按任意顺序出现
- 字段不得重复
- 每一行的字段数量必须相同

要指定仅顺序约束，创建一个 CSV 文件，并从 I/O 引导的起点开始列出 I/O 焊盘单元名称。对 `set_signal_io_constraints` 命令使用 `-csv` 选项来指定 CSV 文件名。在下面的示例中，「ref_cell」字段仅供参考，`set_signal_io_constraints` 命令并不使用它。

```
icc2_shell> set_signal_io_constraints -csv iopads.csv \
   -io_guide_object io_guide_1
```

```
Signal IO constraints set successfully

icc2_shell> sh cat iopads.csv
ref_cell,Pad Name
B4ISH1025_EW,pad0
B4ISH1025_EW,pad1
B4ISH1025_EW,pad2
B4ISH1025_EW,pad3
```

要为每个 I/O 焊盘设置偏移值，如下例所示同时指定「Pad Name」与「Offset Value」字段。其他字段，例如本示例中的「Comment」字段，仅供参考，工具并不使用：

```
icc2_shell> sh cat iopads_offsets.csv
Pad Name,Offset Value,Comment
pad0,200,pad0 offset is 200
pad1,400,pad1 offset is 400
pad2,600,pad2 offset is 600
pad3,800,pad3 offset is 800
pad4,1000,pad4 offset is 1000
pad5,1200,pad5 offset is 1200
pad6,1400,pad6 offset is 1400
pad7,1600,pad7 offset is 1600
```

若 CSV 文件包含自定义列名，创建一个映射文件，其中每行是用逗号分隔的自定义列名与标准列名，并对 `set_signal_io_constraints` 命令使用 `-map_file` 选项指定该映射文件的名称。下面的示例在 CSV 文件中使用自定义列名 `REFERENCE_CELL`、`PAD_NAME` 与 `PAD_OFFSET`。映射文件列出自定义列名与标准列名，二者以逗号分隔。

注意

工具不使用 `REFERENCE_CELL` 列，映射文件中也不包含该列。

```
icc2_shell> sh head ioconstraints2.csv
REFERENCE_CELL,PAD_NAME,PAD_OFFSET
B4ISH1025_EW,pad_iopad_0,0
B4ISH1025_EW,pad_iopad_1,100
...

icc2_shell> sh cat map.txt
PAD_NAME,Pad Name
PAD_OFFSET,Offset Value

icc2_shell> set_signal_io_constraints -csv ioconstraints2.csv \
   -io_guide _default_io_ring1.left -map_file map.txt
Setting PAD_NAME as custom name for column Pad Name.
Setting PAD_OFFSET as custom name for column Offset Value.
Signal IO constraints set successfully
1
```

6.7.2 将 I/O 焊盘与固定对象一起放置

Placing I/Os Pads Together With Fixed Objects

默认情况下，place_io 命令把 I/O 焊盘放进 I/O 引导，并与任何已有的固定对象重叠。若要遵从放在 I/O 引导起点处的固定对象，并指定把 I/O 焊盘插在远离该对象的位置，创建一个同时包含绝对约束与基于顺序的约束的约束文件。绝对约束包含第一个要放置的焊盘以及间距值。基于顺序的约束同样包含第一个焊盘以及其他依赖顺序的焊盘。

下面的约束文件为名为 ring1.top 的顶部 I/O 环创建受约束的 I/O 布局。该约束把焊盘 pad_iopad_1 放在距 I/O 引导起点 200 微米处。

注意

pad_iopad_1 既出现在绝对约束中，也出现在基于顺序的约束中。

```
ring1.top
{{absolute} pad_iopad_1 200.0}
{
  pad_iopad_1
  pad_iopad_2
  pad_iopad_6
  pad_iopad_12
  pc_be_iopad_0
}
;
```

不使用上述约束与使用该约束时，执行 place_io 命令之后的设计如图 ?? 所示。

6.8 把 I/O 焊盘分配到凸点

Assigning I/O Pads to Bumps

IC Compiler II 工具同时支持自动的焊盘到凸点分配，以及用户指定的焊盘到凸点分配，说明见下列主题。

- 自动凸点分配
- 用匹配类型进行用户指定的凸点分配

6.8.1 自动凸点分配

Automatic Bump Assignment

在把倒装芯片驱动器布线到倒装芯片凸点单元之前，必须在 I/O 驱动器与凸点单元之间建立分配关系。工具既支持用 place_io 命令做自动分配，也支持用 create_matching_type 命令做用户生成的分配。要执行自动分配，对 place_io 命令指定 -bump_assignment_only 选项，如下所示：

```
icc2_shell> place_io -bump_assignment_only
```

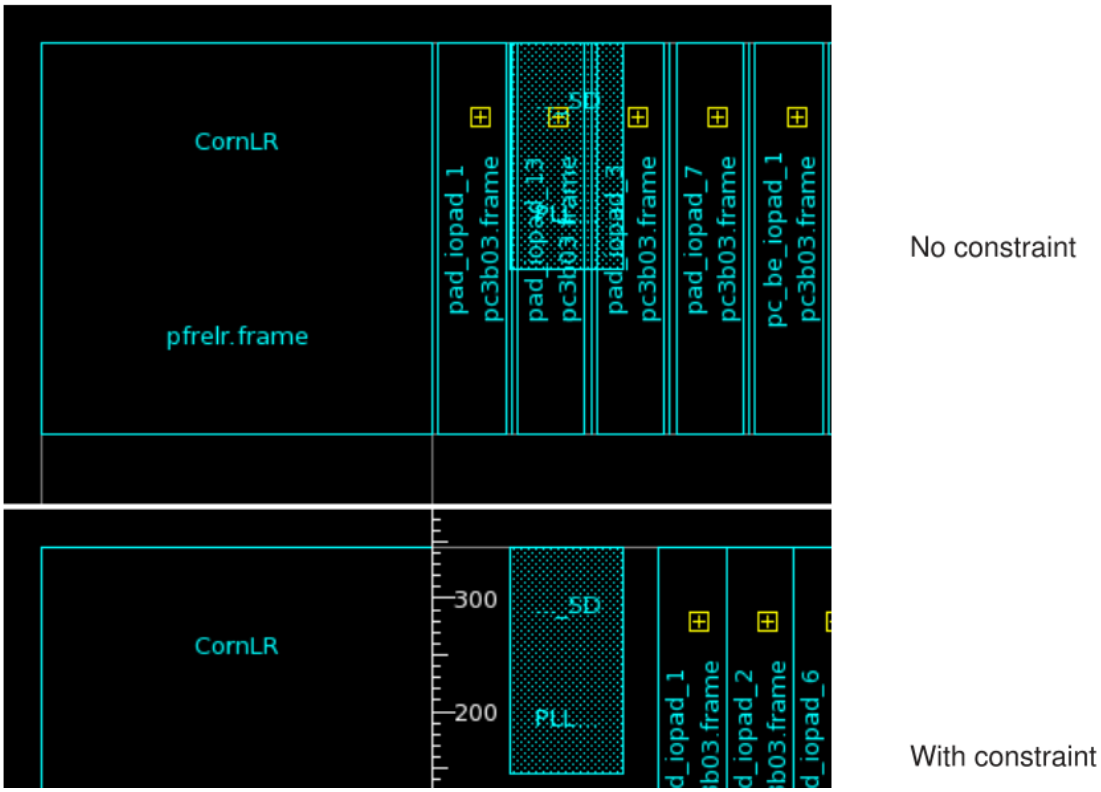


图 **6.11:** `place_io` 固定焊盘约束对比

要验证这些分配，使用 GUI 中的 RDL Flylines 面板查看分配结果。从菜单选择 **View > Flylines > RDL Flylines**，再选择 RDL 网线名称，以查看 RDL 飞线（flyline）。

也可以使用带 `-from_existing_assignment` 与 `-file_name` 选项的 `write_matching_types` 命令，写出由 `place_io` 命令创建的分配。查看该命令所写的文件，即可看到工具创建的分配。

```
icc2_shell> write_matching_types -from_existing_assignment \
   -file_name existing.tcl
1

icc2_shell> sh head existing.tcl
create_matching_type -name existing_assignment_0
   -uniquify 1 [list [get_cells { bottom_0_0 }] \
   [get_pins { __added_power_driver_1/VDDPAD }]]
create_matching_type -name existing_assignment_1
   -uniquify 1 [list [get_cells { bottom_0_1 }] \
   [get_pins { sdram_DQ_iopad_59/PADIO }]]
create_matching_type -name existing_assignment_2
   -uniquify 1 [list [get_cells { bottom_0_10 }] \
   [get_pins { sdram_DQ_iopad_38/PADIO }]]
...
```

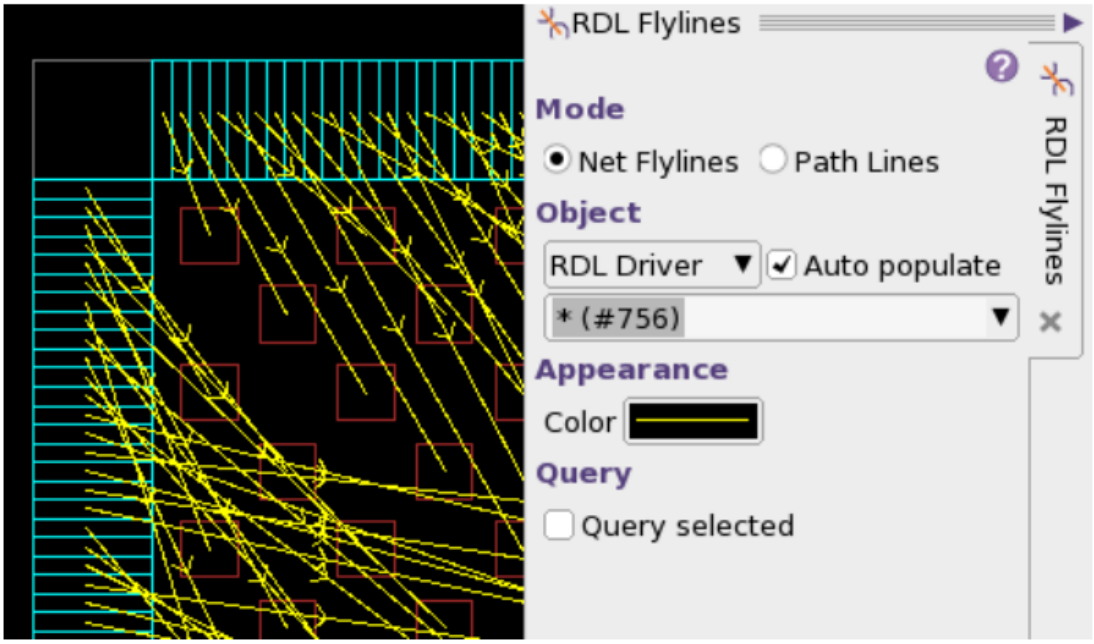


图 **6.12:** 默认 RDL 网线分配

6.8.2 用匹配类型进行用户指定的凸点分配

User-Specified Bump Assignment With Matching Types

除自动凸点分配之外，也可以通过创建一个或多个用户指定的匹配类型，把倒装芯片驱动器与凸点单元关联起来。单元的匹配类型有两项主要功能：支持驱动器到凸点的配对，以及支持驱动器布局。工具把倒装芯片驱动器分配给具有相同匹配类型的倒装芯片凸点单元。

下面的示例使用两条 `create_matching_type` 命令。第一条命令把焊盘单元 `pad_iopad_46` 至 `pad_iopad_49` 与第 25 行的凸点单元关联。第二条命令把焊盘单元 `pad_iopad_42` 至 `pad_iopad_45` 与第 24 行的凸点单元关联。`-uniquify` 选项指定可以分配给单个凸点单元的焊盘引脚数量；值为 0 表示每个凸点单元只能分配给一个焊盘单元。

```
icc2_shell> create_matching_type -name match1 \
   -uniquify 0 {left_25_0 left_25_1 left_25_2 left_25_3 \
   pad_iopad_46 pad_iopad_47 pad_iopad_48 pad_iopad_49}
icc2_shell> create_matching_type -name match2 \
   -uniquify 0 {left_24_0 left_24_1 left_24_2 left_24_3 \
   pad_iopad_42 pad_iopad_43 pad_iopad_44 pad_iopad_45}
```

创建匹配类型之后，使用带 `-matching_types` 选项的 `check_pre_place_io` 命令，检查下列各项：

- 顶层匹配类型的一致性
- 匹配类型在较低层次中不包含凸点单元、凸点引脚或凸点端子
- 每个对象只有一个匹配类型
- 每个匹配类型都有足够的凸点用于凸点分配
- 网表违例；同一匹配类型内的元件必须连接到同一条网线，或者不连接
- 所有多次例化块（multiply instantiated block）实例之间的匹配类型一致

```
icc2_shell> check_pre_place_io -matching_types
Warning: Matching type m1 and matching type m2 conflict
  over bump io_pg_0. (DPI-575)
Warning: bump io_pg_0 in matching type m1 cannot be skipped since
  it is connected to an IO pad pin. (DPI-547)
Warning: bump io_pg_0 in matching type m2 cannot be skipped since
  it is connected to an IO pad pin. (DPI-547)
0
```

`place_io` 命令把匹配类型分配应用到焊盘单元与凸点单元；要在 RDL Flylines 面板中查看更新后的飞线，必须执行该命令。`place_io` 命令所创建的 RDL 飞线连接如图 **??** 所示。在 GUI 中选择 **View > Flylines > RDL Flylines**，即可在版图中查看 RDL 飞线。

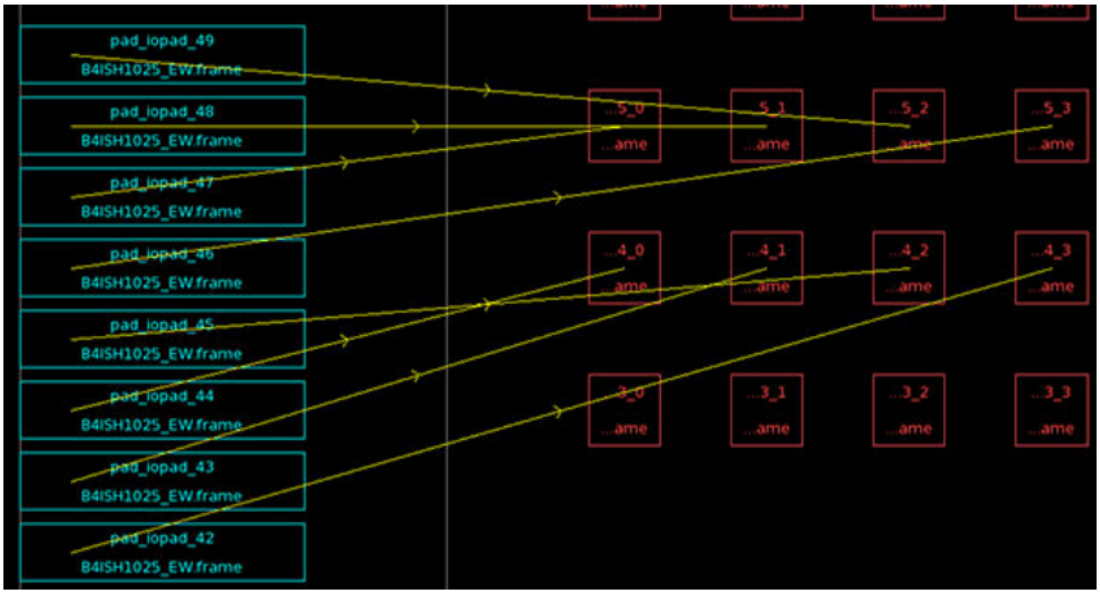


图 **6.13:** 匹配单元之间的 RDL 飞线

6.8.3 其他匹配类型命令示例

Other Matching Type Command Examples

使用下列命令修改、报告和删除匹配类型。

- 用 `get_matching_types` 命令返回匹配类型的集合。

```
icc2_shell> get_matching_types
{SIGNAL POWER}
```

- 用 `get_cells` 命令与 `matching_type.name` 属性，创建被指定匹配类型名称所分配的单元集合。

```
icc2_shell> get_cells -filter "matching_type.name == SIGNAL"
{pad_iopad_49 pad_iopad_48 ...
icc2_shell> get_cells -filter "matching_type.name =~ SIG*"
{pad_iopad_49 pad_iopad_48 ...
```

- 用 add_to_matching_type 命令把 I/O 焊盘或凸点单元加入匹配类型。

```
icc2_shell> add_to_matching_type match2 {pad_iopad_40 pad_iopad_41}
```

- 用 write_matching_types 命令写出用户定义的匹配类型分配。

```
icc2_shell> write_matching_types -file_name user_matching.tcl
```

若没有创建任何匹配类型分配，该文件为空。

- 用 write_matching_types -from_existing_assignment 命令写出工具所分配的默认匹配类型。

```
icc2_shell> write_matching_types -from_existing_assignment \
   -file_name existing.tcl
1

icc2_shell> sh head existing.tcl
create_matching_type -name existing_assignment_0
   -uniquify 1 [list [get_cells { bottom_0_0 }] \
   [get_pins { __added_power_driver_1/VDDPAD }]]
create_matching_type -name existing_assignment_1
   -uniquify 1 [list [get_cells { bottom_0_1 }] \
   [get_pins { sdram_DQ_iopad_59/PADIO }]]
create_matching_type -name existing_assignment_2
   -uniquify 1 [list [get_cells { bottom_0_10 }] \
   [get_pins { sdram_DQ_iopad_38/PADIO }]]
...
```

以后可以通过 source 由 write_matching_types 命令写出的文件，重新创建这些匹配类型。

```
icc2_shell> source existing.tcl
```

- 用 remove_matching_types 命令删除一个或多个匹配类型。

```
icc2_shell> remove_matching_types {match2}
1
icc2_shell> remove_matching_types -all
1
```

- 用 remove_from_matching_type 命令移除指定的 I/O 焊盘或凸点单元。

```
icc2_shell> remove_from_matching_type match2 {pad_iopad_42}
```

- 用 report_matching_types 命令创建匹配类型及其关联 I/O 焊盘或凸点单元的列表。

```
icc2_shell> report_matching_types match2
Matching Type  Uniquify  Object Type  Object Name
------------------------------------------------------
match2         0         cell         left_24_0
                         cell         left_24_1
                         cell         pad_iopad_42
                         cell         pad_iopad_43
...
```

6.9 放置 I/O 并写出约束

Placing I/Os and Writing Constraints

设置电源与信号 I/O 布局约束之后，可以放置 I/O 驱动器并写出布局约束。

要放置 I/O 并写出布局约束：

1. 用 `place_io` 命令放置 I/O。

```
icc2_shell> place_io
```

2. 用 `write_io_constraints` 命令，根据当前 I/O 布局写出 I/O 布局约束文件。

```
icc2_shell> write_io_constraints -filename io_constraints.txt
1
icc2_shell> sh cat io_constraints.txt
_default_io_ring1.left
{{order_only} u_p/test_si_13_pad_P1
u_p/data_pad_1_x0_3_P1
u_p/data_pad_2_x0_3_P1
u_p/data_pad_2_x0_2_P1
...
```

`place_io` 命令支持用于控制 I/O 放置方式的选项。使用 `-io_guide io_guide_list` 选项，只在指定的 I/O 引导中放置 I/O。若 I/O 引导不够长、容纳不下全部焊盘，焊盘会放在 I/O 引导终点之后。使用 `-rule rule_name` 选项，为再分布层布线指定单一的布线间距规则。使用 `-incremental` 选项，为非倒装芯片（nonflip-chip）设计放置额外的 I/O 驱动器。

注意

若 `physical_status` 属性未设为 `fixed`，`place_io` 命令会删除已有的填充单元。当倒装芯片 I/O 布局考虑全部焊盘时，就会出现这种情况。若只增量替换一部分焊盘，则不会删除填充单元。

若把某个匹配类型专门用于只包含凸点单元的集合，工具不会把这些凸点单元连接到 I/O 驱动器。使用这一特性，可以把某些凸点单元保留给电源或地，并避免把凸点单元连接到 I/O 驱动单元。若把匹配类型应用到 I/O 驱动器、I/O 驱动器引脚以及 I/O 驱动器端子，但没有为凸点指定匹配类型，`place_io` 命令会跳过凸点分配，并继续放置其他 I/O。若某个 I/O 驱动器引脚已经通过一条两引脚网线连接到凸点，该焊盘引脚会保持与凸点的连接。对于未分配到 I/O 引导的孤立焊盘单元，工具会找到最近的未分配凸点单元，并计算该焊盘单元与凸点单元之间的飞线路径。对于不在 I/O 引导内的成组焊盘单元，工具会创建一条虚拟 I/O 引导。

`write_io_constraints` 命令支持用于控制输出文件名称与内容的选项。使用 `-filename file_name` 选项指定输出约束文件名。使用 `-format order_only | spacing | pitch | fixed` 选项指定要写入输出文件的约束类型。参数 `order_only` 只把焊盘顺序信息写入文件。参数 `spacing` 把焊盘之间的间距值以及焊盘单元名称写入文件。参数 `pitch` 用单元节距而不是间距值写出焊盘单元位置，并要求全部焊盘单元在 I/O 引导中均匀放置。参数 `fixed` 指定全部焊盘单元固定在其当前位置。

6.10 添加并放置仅物理 I/O 单元

Adding and Placing Physical-Only I/O Cells

I/O 单元放置之后，会向 I/O 焊盘环中插入额外单元，填补 I/O 驱动单元之间的空隙，使芯片上的密度更加均匀。IC Compiler II 工具为此提供 `create_io_break_cells`、`create_io_corner_cell` 与 `create_io_filler_cells` 命令。带有焊盘、拐角、填充与间断单元的典型 I/O 环如图 **??** 所示。

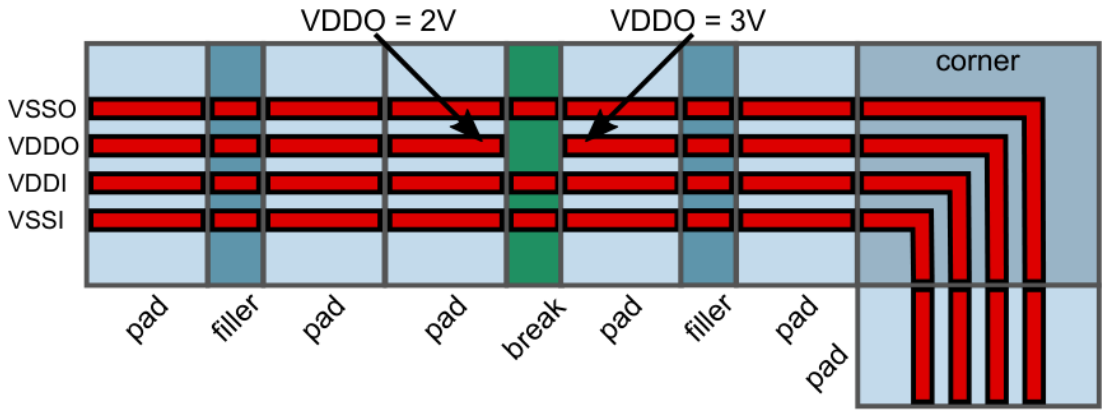


图 **6.14:** I/O 焊盘、拐角、填充与间断单元

参见

有关插入这些单元的信息，见下列主题：

- 插入拐角单元
- 插入间断单元
- 插入填充单元

6.10.1 插入拐角单元

Inserting Corner Cells

拐角单元放在两条互相垂直的 I/O 引导的交点或投影交点处。如下列示例所示，使用 `create_io_corner_cell` 命令的选项控制单元的插入方式。

- 在名为「left」与「bottom」的 I/O 引导交点处，插入名为「CORNER」的拐角单元参考。

```
icc2_shell> create_io_corner_cell \
   -reference_cell CORNER {left bottom}
```

- 在名为「bottom」与「right」的 I/O 引导交点处，放置已有的拐角单元实例。

```
icc2_shell> create_io_corner_cell -cell corner_cell_1 {bottom right}
```

- 在具有多层层次且块相互重叠的设计中，在名为「right」与「top」的 I/O 引导交点处插入名为「CORNER」的拐角单元参考，并把该拐角单元例化到 `I_ORCA_TOP` 块内。

```
icc2_shell> create_io_corner_cell \
   -reference_cell CORNER {right top} -force I_ORCA_TOP
```

6.10.2 插入间断单元

Inserting Break Cells

I/O 间断单元用来在 I/O 环中电源网线的紧贴连接之间留出空隙，以便使用不同电压。根据 I/O 驱动器的供电结构以及设计要求，一个库中可能包含多个 I/O 间断单元。

使用 `create_io_break_cells` 命令插入 I/O 间断单元。如下列示例所示，使用该命令的选项控制单元的插入方式。

- 在名为「left」的 I/O 引导起点之前，插入名为「BREAK」的间断单元参考。

```
icc2_shell> create_io_break_cells -reference_cells BREAK \
   -location start left
```

使用 `-location end` 选项，把间断单元放在 I/O 引导终点之后。使用 `-location both` 选项，把间断单元放在 I/O 引导的两端。

- 把已有的 `break_cell_1` 实例移到距名为「left」的 I/O 引导起点 600 微米处。

```
icc2_shell> create_io_break_cells -cells break_cell_1 \
   -location 600 left
```

- 在名为「top」与「right」的 I/O 引导交点处，插入名为「BREAK」的间断单元参考。

```
icc2_shell> create_io_break_cells -reference_cells BREAK {right top}
```

6.10.3 插入填充单元

Inserting Filler Cells

填充单元用来填满驱动单元之间的空白，并为 I/O 驱动器完成紧贴的电源连接。许多单元库包含多种宽度不同的填充单元参考；宽填充单元用来填补宽间隙，窄填充单元用来填补窄间隙。

如下列示例所示，使用 `create_io_filler_cells` 命令的选项控制单元的插入方式。

> **注意**
>
> 用 `-reference_cells` 选项指定多个填充单元参考时，工具会从列表中选择能够填满该空隙的最大单元。

- 把填充单元参考 `FILLER20A`、`FILLER10A`、`FILLER5A`、`FILLER1A`、`FILLER05A` 与 `FILLER0005A` 插入所有 I/O 引导中 I/O 驱动器之间的间隙，并在创建实例名时使用前缀「filler_」。最大的填充单元 `FILLER20A` 写在列表开头，其后按递减的填充单元尺寸依次指定。

```
icc2_shell> create_io_filler_cells \
   -prefix filler_ \
   -reference_cells {{ FILLER20A FILLER10A FILLER5A
                       FILLER1A FILLER05A FILLER0005A }}
```

- 在名为「top」与「right」的 I/O 引导上插入填充单元参考 FILLER10A。若插入填充单元之后仍有间隙，会发出一条错误消息。

```
icc2_shell> create_io_filler_cells -reference_cells FILLER10A \
   -io_guides {right top}
Error: In IO guide right, filler cells cannot cover the gap
at (6500,6090) with length 2. (DPI-089)
```

- 在名为「guide_1」的 I/O 引导上插入填充单元参考 FILLER10A，并强制把填充单元放在名为「top」的块内。在本示例中，guide_1 这条 I/O 引导与多个块重叠，且设计包含多层层次。

```
icc2_shell> create_io_filler_cells -reference_cells FILLER10A \
   -io_guides guide_1 -force top
```

6.11 检查 I/O 布局

Checking I/O Placement

用 place_io 命令放置 I/O 单元之后，使用 check_io_placement 命令验证 I/O 布局。该命令写出一份包含违例单元列表的报告，并返回这些单元的集合。check_io_placement 命令的选项控制执行哪些检查。

- 用 -cells 选项检查指定单元。
- 用 -unplaced 选项检查设计中是否存在未放置的 I/O 单元。

```
icc2_shell> check_io_placement -unplaced
---------- Start of Unplaced Pads Check ----------
Cell u13 is unplaced.
Total 1 cell is unplaced in current design.
---------- End of Unplaced Pads Check ----------
```

- 用 -overlap overlap_checks 选项检查重叠单元。其中 overlap_checks 是下列一项或多项：pad2pad、pad2filler、filler2filler、bump、corner2pad 或 all。

```
icc2_shell> check_io_placement -overlap {pad2pad bump}
---------- Start of Overlap Check ----------
Pad u488 and pad u547 overlap.
Pad u488 and pad u553 overlap.
Total 3 cells have overlap violation.
---------- End of Overlap Check ----------
```

- 用 -io_guides 选项检查指定的 I/O 引导。

```
icc2_shell> check_io_placement \
   -io_guides [get_io_guides {_default_io_ring1.bottom}]
---------- Start of Pad to Guide Assignment Check ----------
Pad u547 violates pad-to-guide constraints: assigned to
  IO guide _default_io_ring1.bottom, but placed outside
  of _default_io_ring1.bottom with 4089.49 gap.
```

```
Total 1 violation is found.
---------- End of Pad to Guide Assignment Check ----------
```

- 用 `-min_pitch` 选项，对照为 I/O 引导指定的最小节距检查 I/O 单元的布局。

```
icc2_shell> check_io_placement -min_pitch
---------- Start of Min Pitch Check ----------
bus_0 to bus_4 violates min_pitch 5. Current spacing is 28.
bus_2 to bus_7 violates min_pitch 5. Current spacing is 42.
Total 2 violations are found.
---------- End of Min Pitch Check ----------
```

- 用 `-flipping` 选项，对照焊盘的 `is_flipped` 属性检查 I/O 焊盘布局是否一致。

```
icc2_shell> check_io_placement -flipping
---------- Start of Pad Flip Check ----------
Pad bus_0 is not flipped, but has
  is_flipped attribute set to true.
Total 3 violations are found.
---------- End of Pad Flip Check ----------
```

- 用 `-gap` 选项检查 I/O 单元之间的空隙，这些单元包括焊盘、拐角、填充与间断单元。

```
icc2_shell> check_io_placement -gap
---------- Start of Gap Check ----------
Cell bus_0 and cell bus_4 have a gap between them.
Total 2 cells have gap violation.
---------- End of Gap Check ----------
```

- 用 `-signal_constraints` 选项，对照用 `set_signal_io_constraints` 命令设置的约束检查 I/O 焊盘布局。

```
icc2_shell> check_io_placement -signal_constraints
---------- Start of Signal I/O constraints Check ----------
The order of pad bus_1 and pad vss violates I/O guide
  default_guide_0 order constraint.
The order of pad bus_6 and pad bus_1 violates I/O guide
  default_guide_0 order constraint.
Total 2 violations are found.
---------- End of Signal I/O constraints Check ----------
```

信号 I/O 约束包括焊盘的顺序、节距、间距、偏移，以及边界间距。

- 用 `-pad_to_guide_assignment` 选项检查 I/O 焊盘是否放在正确的 I/O 引导上。

```
icc2_shell> sh cat padAssign
bus_5 default_guide_0 default_guide_2;
{bus_1 bus_6} default_guide_0;

icc2_shell> check_io_placement -pad_to_guide_assignment \
   -pad_assignment_file padAssign
```

```
---------- Start of Pad to Guide Assignment Check ----------
Pad bus_5 violates pad-to-guide constraints: assigned to
   IO guide default_guide_0, but placed outside of
   default_guide_0 with 500 gap.
Total 1 violation is found.
---------- End of Pad to Guide Assignment Check ----------
```

使用 -pad_assignment_file 选项指定一个文件，其中包含 I/O 焊盘及对应的 I/O 引导。该文件的格式为

```
pad1 guide1;
pad2 guide2 guide3;
{pad3 pad4} guide4;
```

其中，单个焊盘或一组焊盘可以指定一条或多条 I/O 引导，I/O 焊盘的集合用花括号括起。

- 用 -filename 选项把 I/O 布局报告写到指定文件。

```
icc2_shell> check_io_placement -filename io_placement_report.txt
```

- 用 -output_directory 选项把 I/O 布局违例报告写到指定目录。

```
icc2_shell> check_io_placement -output_directory io_checks
Writing IO placement checking reports to directory io_checks.

icc2_shell> ls io_checks/*
io_checks/bumpassignment_violations.rpt
io_checks/flip_violations.rpt
io_checks/gap_violations.rpt
io_checks/minPitch_violations.rpt
io_checks/overlap_violations.rpt
io_checks/pad2guide_violations.rpt
io_checks/powerIOConstraints_violations.rpt
io_checks/signalIOConstraints_violations.rpt
io_checks/unplacedpads_violations.rpt
```

该命令创建指定目录，并为每种违例类型分别写出文件。

6.12 布线 RDL 网线

Routing RDL Nets

放置凸点单元与 I/O 驱动单元之后，可以布线再分布层网线。
要布线 RDL 网线：

1. 为 RDL 网线布线定义布线规则。若计划为 RDL 布线添加网线屏蔽，按第 **??** 节「创建 RDL 网线屏蔽」所述指定 -shield_spacings 与 -shield_widths 选项。

```
icc2_shell> create_routing_rule rdlrule -widths {MRDL 2} \
   -spacings {MRDL 1}
```

2. 把该布线规则应用到倒装芯片凸点单元。

```
icc2_shell> set rdl_nets [get_nets -of_objects [get_pins \
   -of_objects [get_cells -filter "design_type==flip_chip_pad"]]]
{...}
icc2_shell> set_routing_rule [get_nets $rdl_nets] -rule rdlrule
```

3. 把应用选项 `flip_chip.route.layer_routing_angles` 设为 `90_degree` 或 `45_degree`，以指定 RDL 层的布线类型。

```
icc2_shell> set_app_options \
   -name flip_chip.route.layer_routing_angles \
   -value {{MRDL 90_degree}}
1
```

4. 布线 RDL 网线。

```
icc2_shell> route_rdl_flip_chip -layers {MRDL}
-- START GLOBAL ROUTING --
-- END GLOBAL ROUTING --

-- START DETAIL ROUTING --
-- END DETAIL ROUTING --
1
```

5. 检查 RDL 布线中的开路网线。

```
icc2_shell> report_rdl_routes -open_nets true
****************************************
Report : RDL routes
****************************************
Open Net:
DATA[7]
DATA[6]
...
```

6.（可选）为 GUI 中的 Error Browser 写出错误数据库。

```
icc2_shell> report_rdl_routes -create_error_data verification
```

对 `-create_error_data` 选项使用关键字 `verification`，写出开路与短路违例的错误数据。使用关键字 `flyline_only` 写出飞线数据。

RDL 布线之后的版图结果如图 **??** 所示。RDL 网线以 90 度角布线，并连接 I/O 驱动器与凸点单元。

6.12.1 其他 RDL 布线任务

Other RDL Routing Tasks

使用下列命令与应用选项执行不同的 RDL 布线任务。

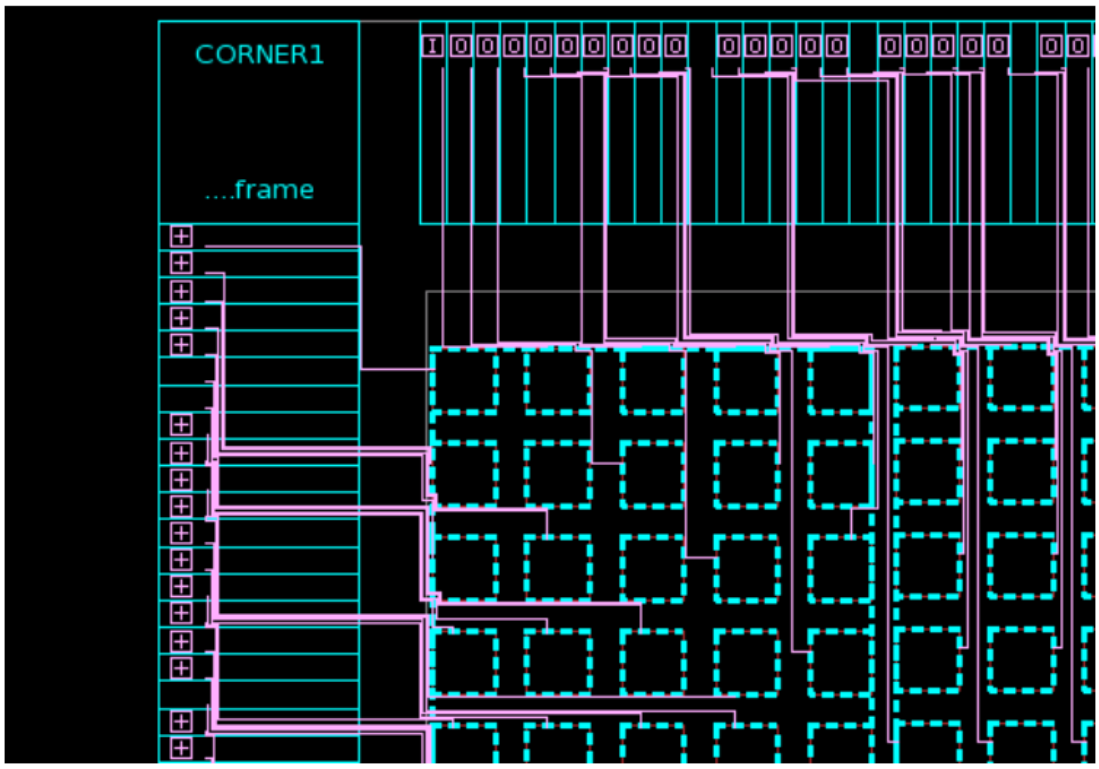


图 **6.15:** 倒装芯片布线之后的版图

- 在运行 `route_rdl_flip_chip` 命令之前，把应用选项 `flip_chip.route.layer_routing_angles` 设为 `45_degree`，以创建 45 度布线。

```
icc2_shell> set_app_options \
   -name flip_chip.route.layer_routing_angles \
   -value {{MRDL 45_degree} {M9 45_degree}}
icc2_shell> route_rdl_flip_chip -layers {MRDL M9} -nets $rdl_nets
```

工具创建的 RDL 布线如图 **??** 所示。

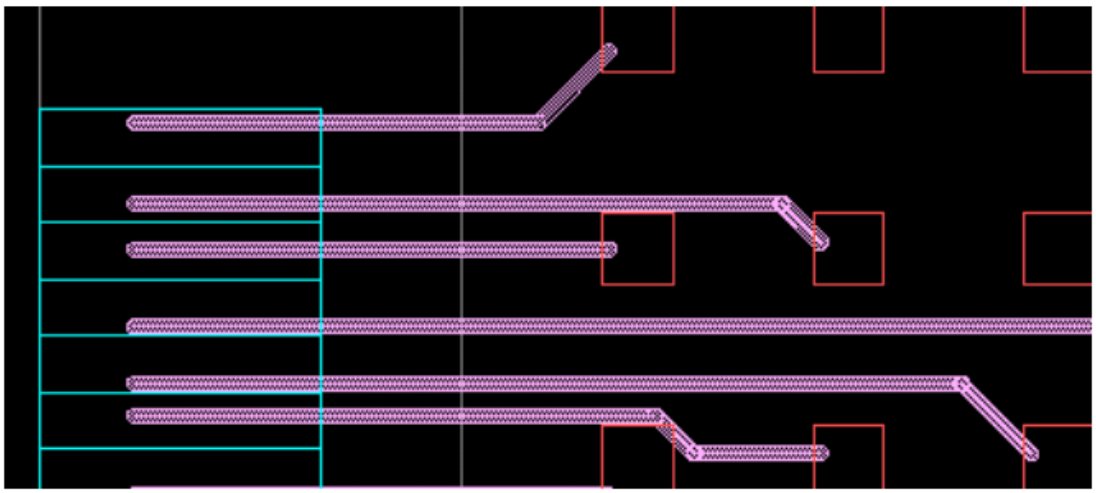


图 **6.16:** 45 度 RDL 布线

- 使用 `split_rdl_routes` 命令，在相邻层上创建一条与当前布线重叠的 RDL 布线。

```
icc2_shell> split_rdl_routes -nets {pad[48] pad[49]} \
   -mode adjacent_layer -via_interval 25 -from_layers MRDL \
   -to_layers M9
```

工具在 M9 层上添加一条平行布线并插入通孔，如图 **??** 中上面两条布线所示。

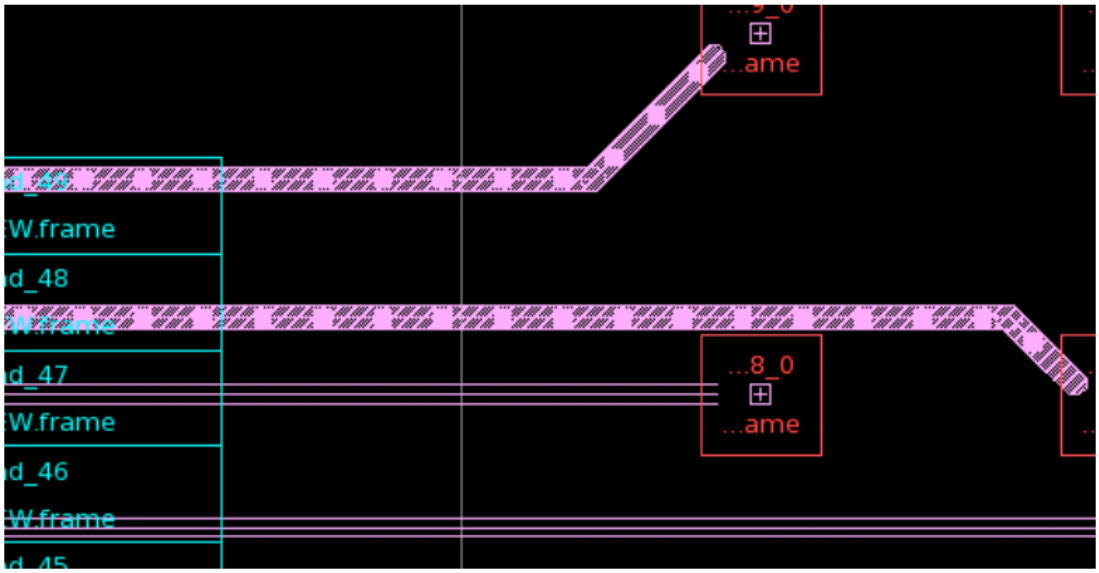


图 **6.17:** 分裂布线

- 使用带 -widths、-spacings 与 -number_of_routes 选项的 split_rdl_routes 命令，把已有布线在同一层上分裂为多条细布线。

```
icc2_shell> split_rdl_routes -nets {pad[46] pad[47]} \
   -widths {MRDL {2 2 2}} -spacings {MRDL {2 2}} \
   -number_of_routes {MRDL 3}
```

工具把已有布线分裂为多条细布线，如图 **??** 中下面两条布线所示。

- 使用 route_rdl_differential 命令布线 RDL 网线。

  下面的命令在指定层上为指定网线创建长度匹配的布线。

```
icc2_shell> route_rdl_differential -layers {MRDL} \
   -nets {pad[41] pad[43]}
```

工具为指定网线创建的布线如图 **??** 所示。

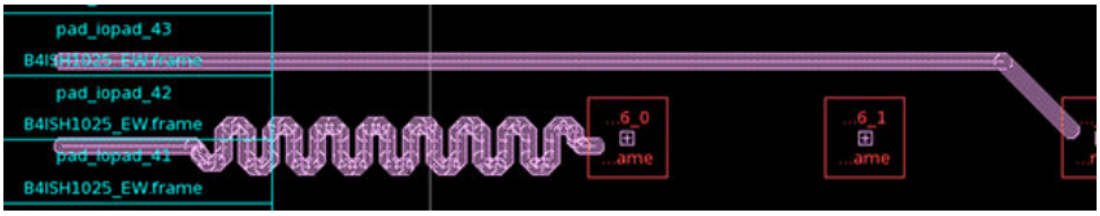


图 **6.18:** 差分布线

也可以使用 route_rdl_differential -mode parallel 命令，把 RDL 网线平行布线。

可以用 route_rdl_differential 命令的 -nets 选项,或应用选项 flip_chip.route.differential_groups，控制平行模式下的目标网线。

在下面的示例中，当工具以 -mode parallel 选项运行、并且目标网线由 -nets 选项指定时，有四条网线被布线，其中间部分有很长一段相互平行。在此模式下，工具只能布线一个组。

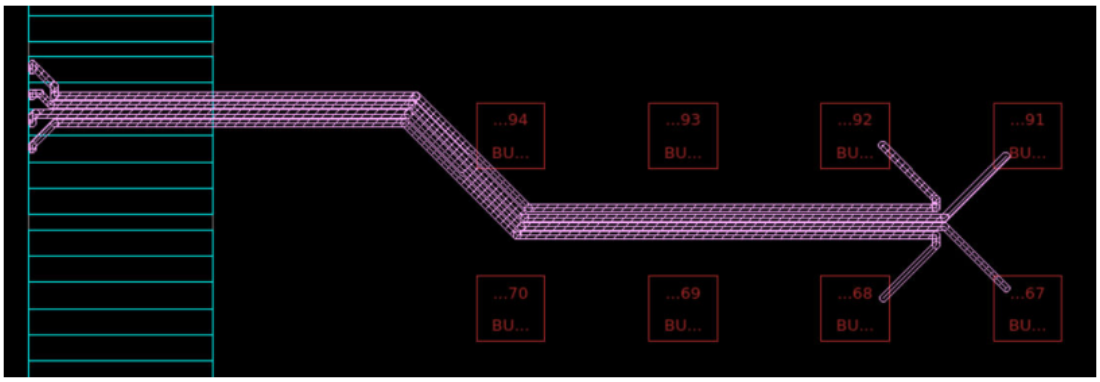


图 **6.19:** 差分布线：普通模式

然而,当使用 -mode parallel 选项、并且目标网线由应用选项 flip_chip.route.differential_groups 指定时，会一次布线多个网线组，每组有两条网线（如下图所示）。

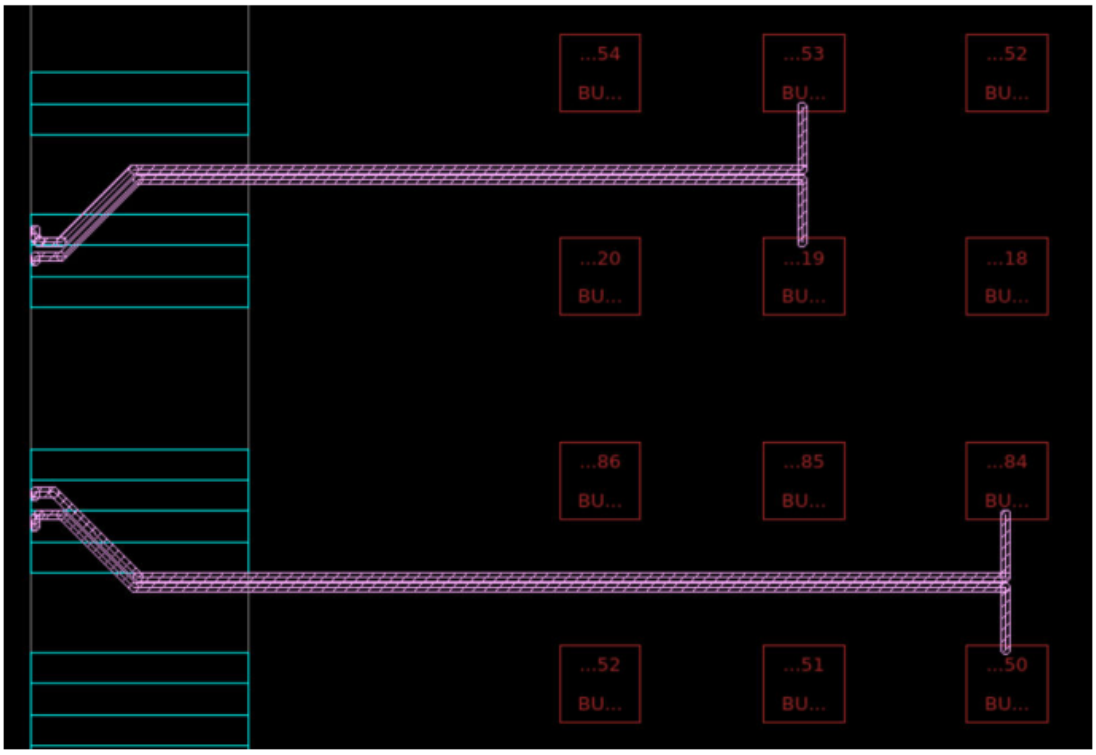


图 **6.20:** 差分布线：平行模式

- 用 create_routing_rule 命令设置渐变距离与宽度，以创建渐变布线（tapered route）。再用 route_rdl_flip_chip 命令创建该渐变布线。

```
icc2_shell> create_routing_rule taperrule \
   -widths {MRDL 10 M9 10} -spacings {MRDL 10 M9 10} \
   -rdl_taper_distances {MRDL 40 M9 40} \
```

```
   -rdl_taper_widths {MRDL 5 M9 5}
icc2_shell> set_routing_rule [get_nets $rdl_nets] -rule taperrule
icc2_shell> route_rdl_flip_chip -layers {MRDL M9} -nets $rdl_nets
```

工具创建的渐变布线如图 **??** 所示。

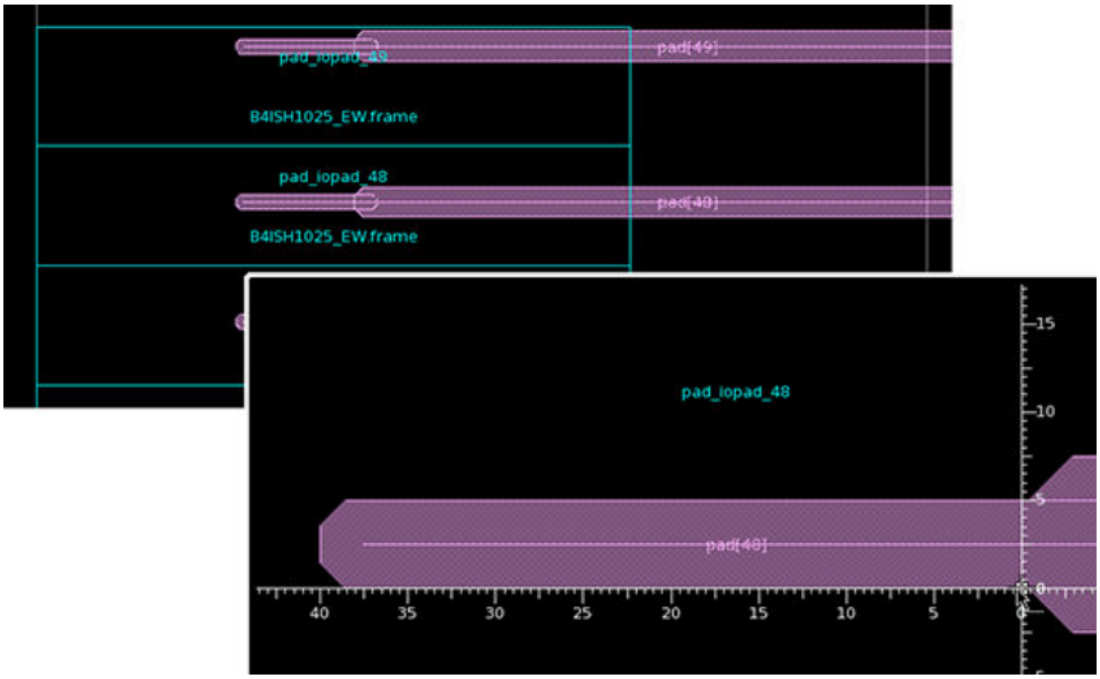


图 **6.21:** 渐变 RDL 布线

- 把应用选项 `flip_chip.route.route_to_ring_nets` 设为网线名，在凸点单元与电源环网线之间创建一条 RDL 布线。使用应用选项 `flip_chip.route.route_to_stripe_nets`，在凸点单元与电源地网格中的电源地网线形状之间创建一条 RDL 布线。下面的示例指定这些应用选项。

```
icc2_shell> set_app_options \
   -name flip_chip.route.route_to_ring_nets -value VDD1
icc2_shell> set_app_options \
   -name flip_chip.route.route_to_stripe_nets -value VDD1
icc2_shell> route_rdl_flip_chip -layers $rdl_layers -nets VDD1
```

典型倒装芯片布线的版图如图 **??** 左侧所示；由前述命令创建的倒装芯片布线版图如该图右侧所示。

- 如下图所示，设置应用选项 `flip_chip.route.extend_vias_on_pin`，在 RDL 布线与 I/O 驱动器引脚之间创建额外的叠孔（stacked via）。默认值为 `false`，工具只创建到引脚的简单连接。把该应用选项设为 `true`，以延伸导线并插入额外叠孔。导线延伸部分的宽度与布线规则所定义的导线宽度相同。把该应用选项设为 `full_pin`，以延伸并加宽导线，使其覆盖引脚，并插入额外叠孔。工具忽略布线规则所设置的布线宽度，并尝试覆盖整个 I/O 驱动器引脚。

```
icc2_shell> set_app_options \
   -name flip_chip.route.extend_vias_on_pin -value full_pin
```

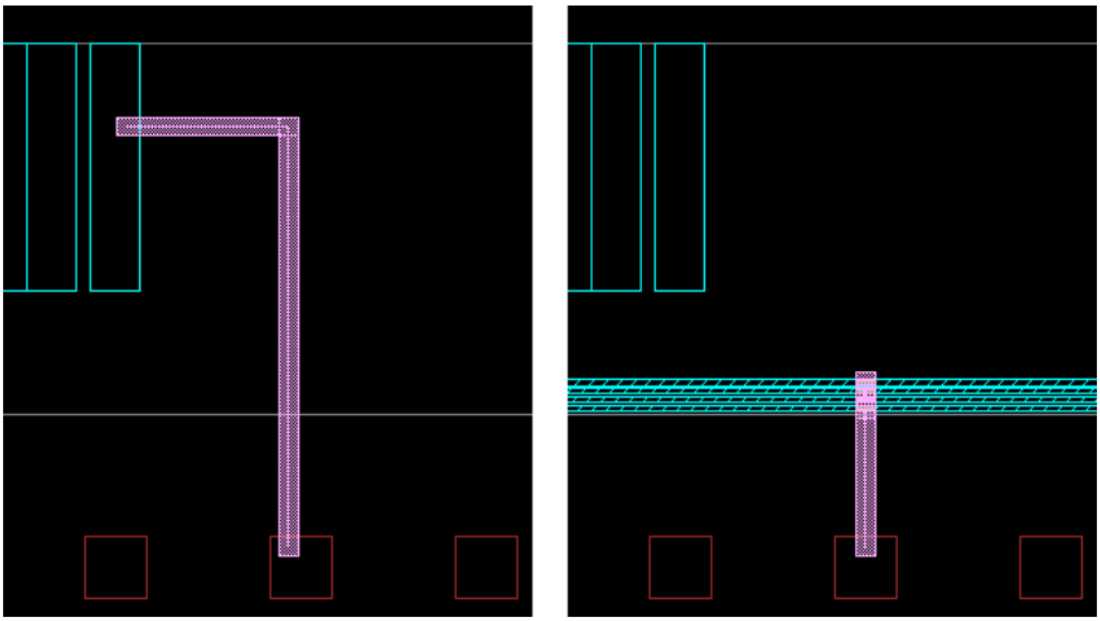


图 **6.22:** 到焊盘与电源地带的 RDL 布线

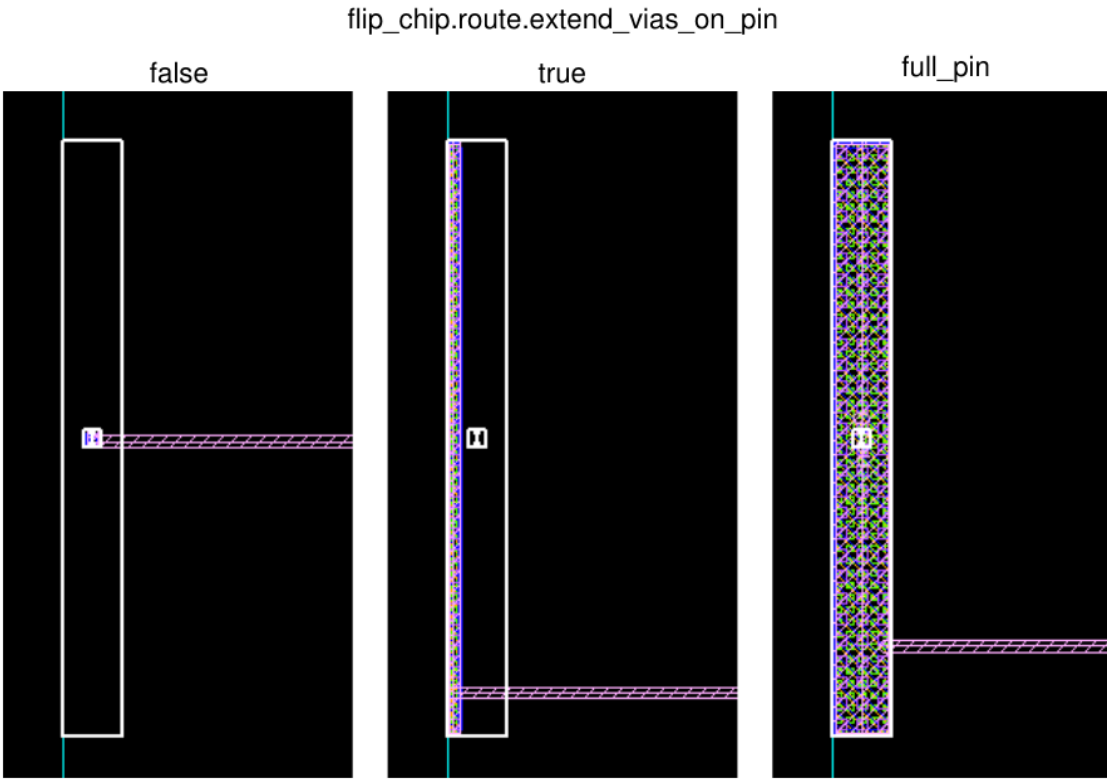


图 **6.23:** RDL 布线到 I/O 驱动器引脚的额外叠孔

- 使用应用选项 flip_chip.route.layer_bump_spacings，为每个倒装芯片布线层指定凸点到布线的间距值，单位为微米。也可以使用工艺文件规则 sameNetRDLMetalToPadMinSpacing 与 diffNetRDLMetalToPadMinSpacing 指定凸点到布线的间距。

```
icc2_shell> set_app_options \
   -name flip_chip.route.layer_bump_spacings \
   -value {{M9 10.0} {MRDL 10.0}}
```

设计规则 jogRDLKeepoutRange 指定凸点单元与布线之间的禁布范围（keepout range）。两种不同 jogRDLKeepoutRange 设置下的 RDL 布线结果如图 **??** 所示。

- 用 remove_routes -rdl 命令删除全部 RDL 布线。

```
icc2_shell> remove_routes -rdl
Successfully removed 321 route shapes.
```

6.13 优化 RDL 布线

Optimizing RDL Routes

用 route_rdl_flip_chip 命令或通过 GUI 创建 RDL 布线之后，使用 optimize_rdl_routes 命令改善 RDL 布线图案：减少块中 U 形与 Z 形布线的数量，并为 RDL 电源布线留出额外空间。执行下列任务以优化 RDL 布线：

- 改善 RDL 布线。

```
icc2_shell> optimize_rdl_routes -nets $rdl_nets
```

执行上述命令之前与之后的版图结果如图 **??** 所示。

- 用 -reserve_power_resources 选项移动信号布线，为电源与地布线留出额外的布线区域。

```
icc2_shell> optimize_rdl_routes -layer {MRDL} -nets $rdl_nets \
   -reserve_power_resources true
```

带此选项运行该命令之后，U 形与 Z 形布线的数量可能会增加。执行上述命令之前与之后的版图结果如图 **??** 所示。

6.14 创建 RDL 网线屏蔽

Creating RDL Net Shields

RDL 网线屏蔽（shield）为 RDL 网线提供隔离。可以在同一层上屏蔽网线（称为侧壁屏蔽，side-wall shielding），也可以在该层的上方与下方屏蔽网线（称为同轴屏蔽，coaxial shielding）。同轴屏蔽比侧壁屏蔽提供更好的信号隔离，但会占用更多布线资源。

下面的步骤创建 RDL 布线，并用侧壁屏蔽为这些布线加屏蔽：

1. 设置应用选项 flip_chip.route.shielding_net，定义屏蔽网线连接。

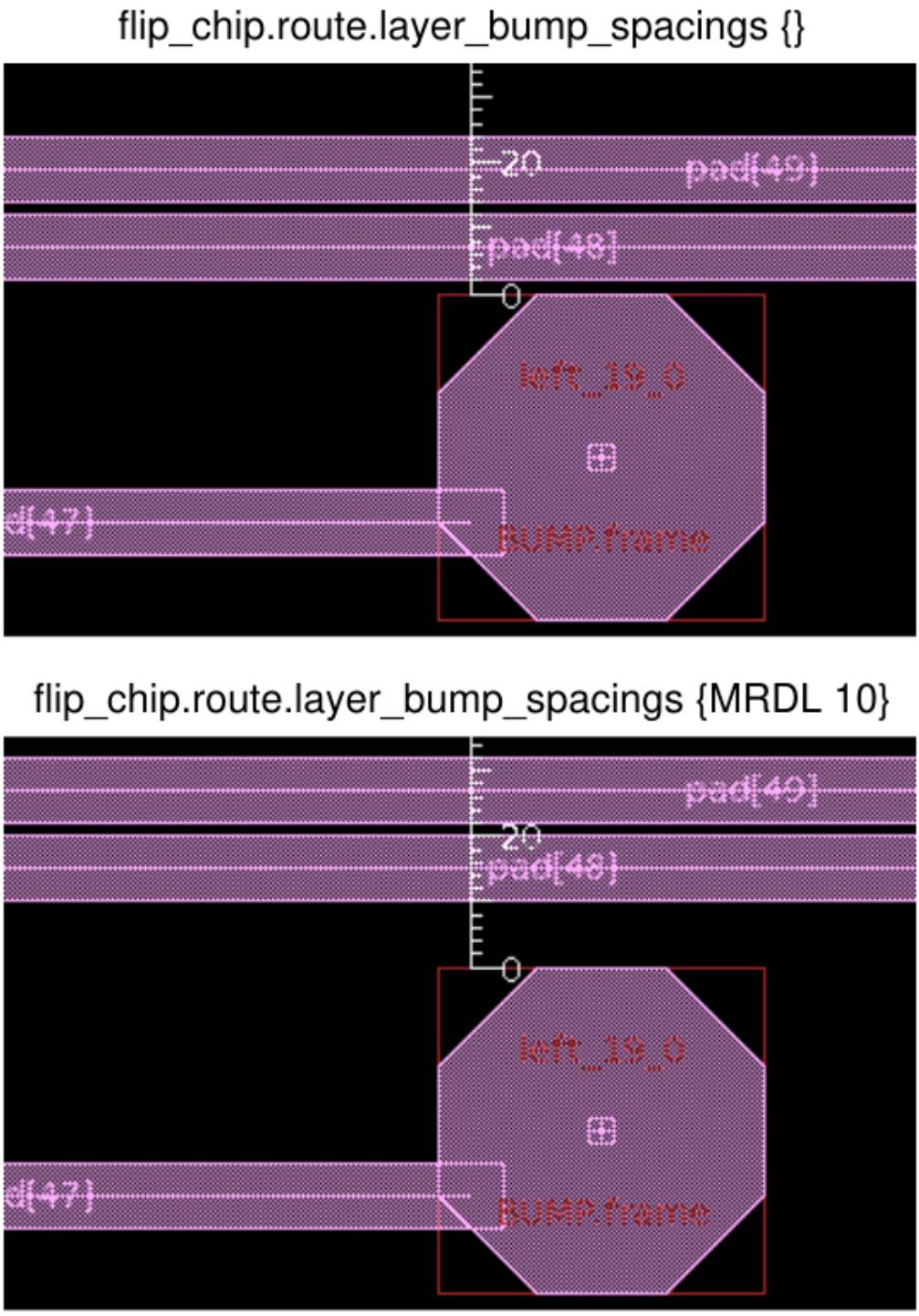
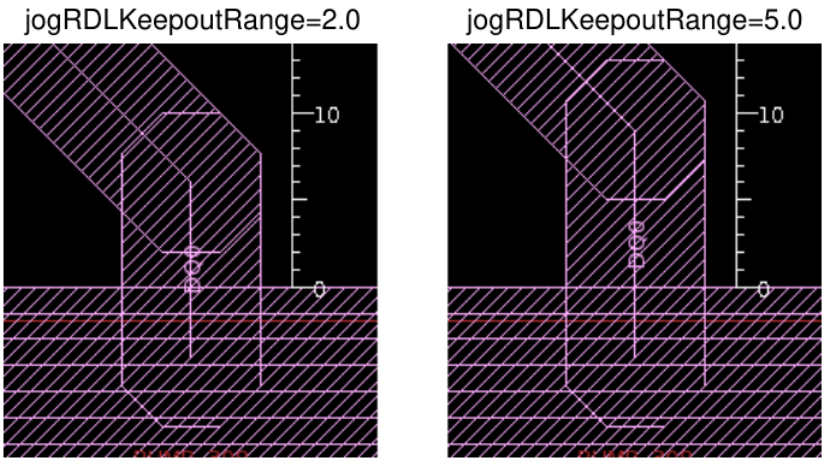


图 **6.24:** 凸点到布线的间距



- Remove all RDL routes with the `remove_routes -rdl` command.

```
icc2_shell> remove_routes -rdl
Successfully removed 321 route shapes.
```

Optimizing RDL Routes

After creating RDL routes with the `route_rdl_flip_chip` command or by using the GUI, use the `optimize_rdl_routes` command to improve the RDL routing patterns by

图 **6.25:** 折弯禁布范围

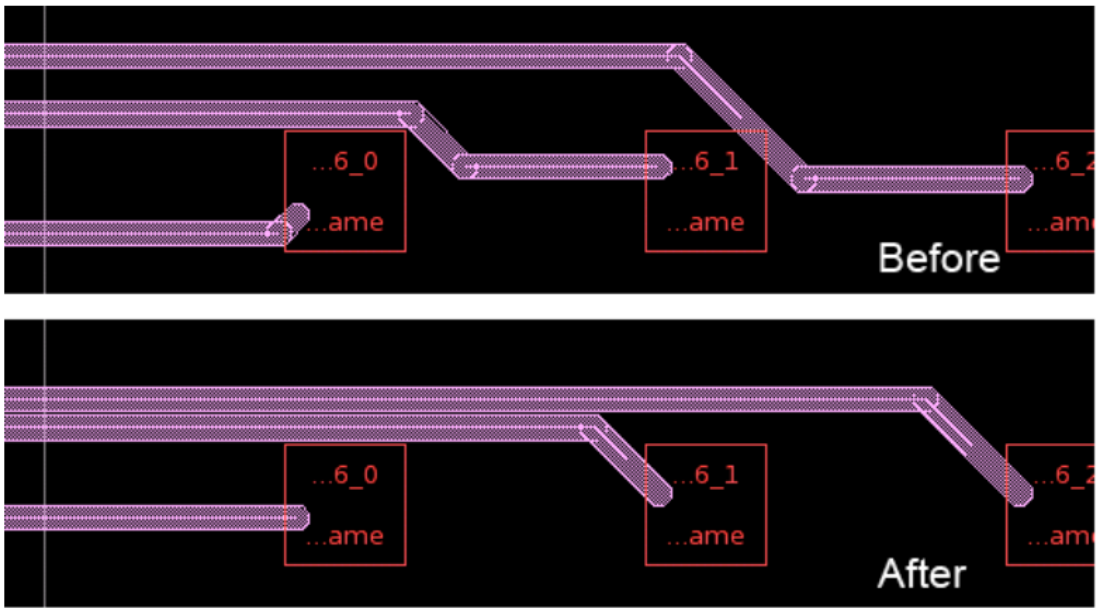


图 **6.26:** RDL 布线优化

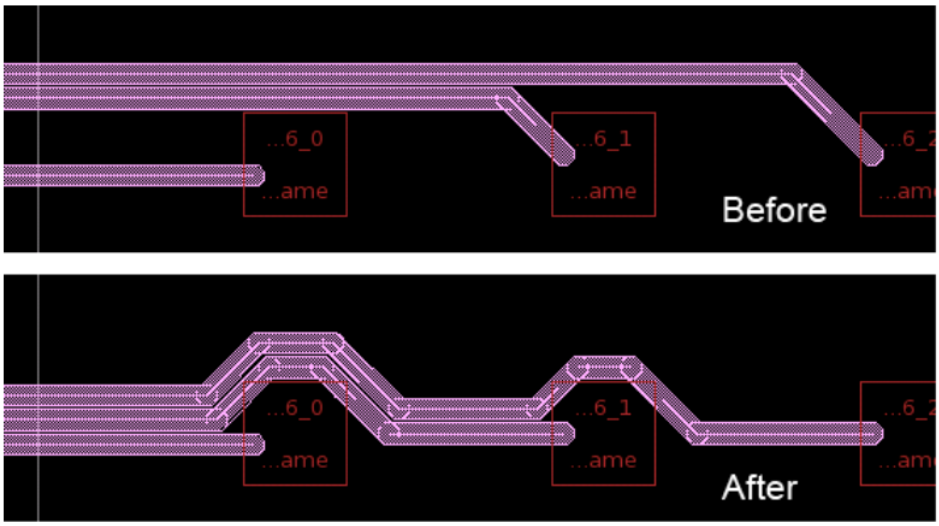


图 **6.27:** 为预留电源资源而做的 RDL 布线优化

```
icc2_shell> set_app_options -name flip_chip.route.shielding_net \
   -value VSS
```

2. 创建布线规则，并指定屏蔽间距与屏蔽宽度。

```
icc2_shell> create_routing_rule shieldrule \
   -shield_spacings {MRDL 2 M9 2} -shield_widths {MRDL 5 M9 5} \
   -widths {MRDL 6 M9 5}
```

3. 把该布线规则分配给 RDL 网线。

```
icc2_shell> set_routing_rule $rdl_nets -rule shieldrule
```

4. 布线 RDL 网线。

```
icc2_shell> route_rdl_flip_chip -layers {MRDL M9} -nets $rdl_nets
```

5. 创建 RDL 网线屏蔽。

```
icc2_shell> create_rdl_shields -layers {MRDL M9} -nets $rdl_nets
```

前述命令所产生的版图结果如图 **??** 所示。

6.14.1 RDL 网线屏蔽选项

RDL Net Shielding Options

使用 `create_rdl_shields` 命令的下列选项，控制屏蔽的创建方式。

- 把 RDL 网线列在一个文件中，并用 `-nets_in_file` 选项指定该文件名。

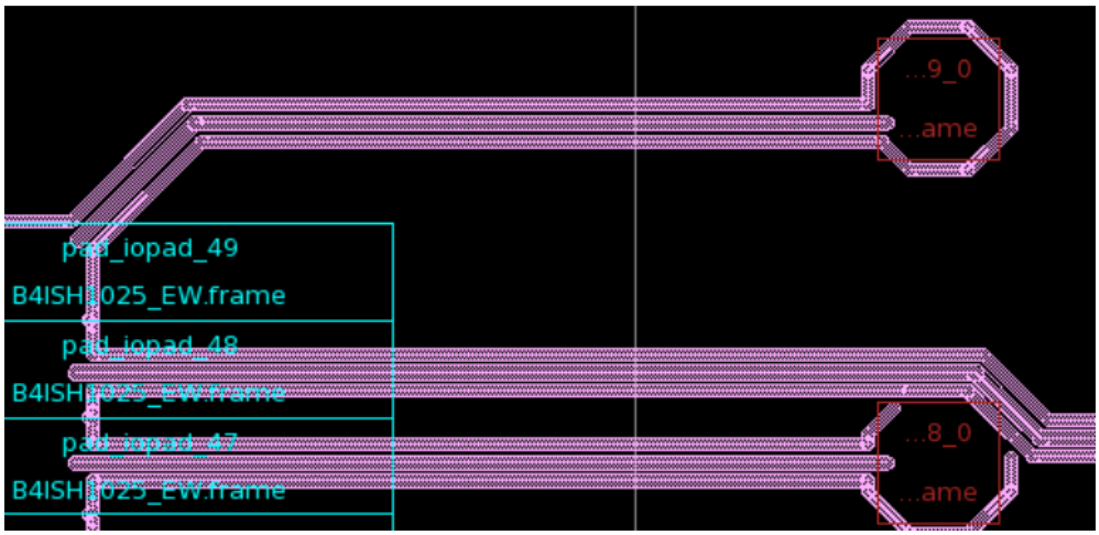


图 **6.28:** RDL 网线屏蔽

```
icc2_shell> create_rdl_shields -layers {MRDL M9} \
   -nets_in_file rdlnets.txt
icc2_shell> sh cat rdlnets.txt
pad[40]
pad[41]
...
```

- 用 `-shield_via_tie` 选项启用或禁用通孔连接上的屏蔽。
- 用 `-trim_floating true` 选项，在创建之后删除悬空屏蔽。

仅用于侧壁屏蔽时：

- 用 `-shield_on_bump` 选项启用或禁用凸点上的屏蔽。

```
icc2_shell> create_rdl_shields -layers {MRDL M9} -nets $rdl_nets \
   -shield_on_bump true
```

`-shield_on_bump` 选项设为 `true`（左）与 `false`（右）时的版图结果如图 **??** 所示。

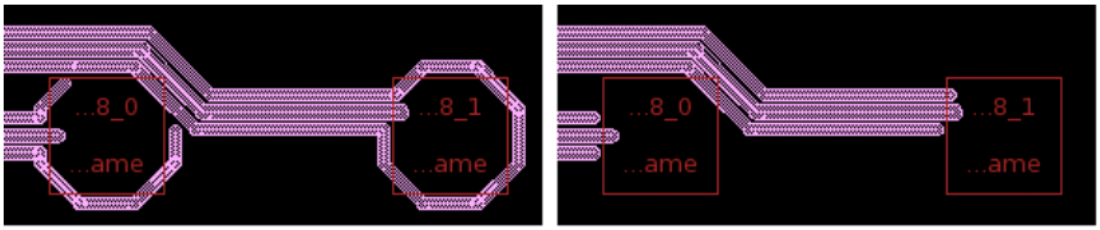


图 **6.29:** 凸点屏蔽：`-shield_on_bump` 选项

- 使用 `-half_shield` 选项并指定下列值之一，在网线的一侧创建侧壁屏蔽：`up`、`down`、`left` 或 `right`。要在水平飞线的上侧或下侧创建屏蔽，指定 `up` 或 `down`。要在垂直飞线的左侧或右侧创建屏蔽，指定 `left` 或 `right`。默认情况下，`create_rdl_shields` 命令在网线两侧都创建侧壁屏蔽。

仅用于同轴屏蔽时：

- 使用 `-reference_layer` 选项指定布线的参考层。目标层与参考层必须不同。要指定目标布线层，使用 `-layers` 选项。要指定网线屏蔽的中心线，使用 `-offset` 选项。
- 使用 `-report_coaxial_shield_ratio_offsets` 选项并指定下列层与偏移值，报告同轴屏蔽比例：`{shield_layer route_layer shield_route_offset}`。

下面的示例创建 `METAL7` 同轴屏蔽导线，以屏蔽 `METAL8` 布线。屏蔽到布线中心线的偏移为 5。

```
icc2_shell> create_rdl_shields -reference_layer METAL8 -layers METAL7 \
   -offset 5 -report_coaxial_shield_ratio_offsets {{METAL7 METAL8 5}}
```

同轴屏蔽之前与之后的网线如下图所示。

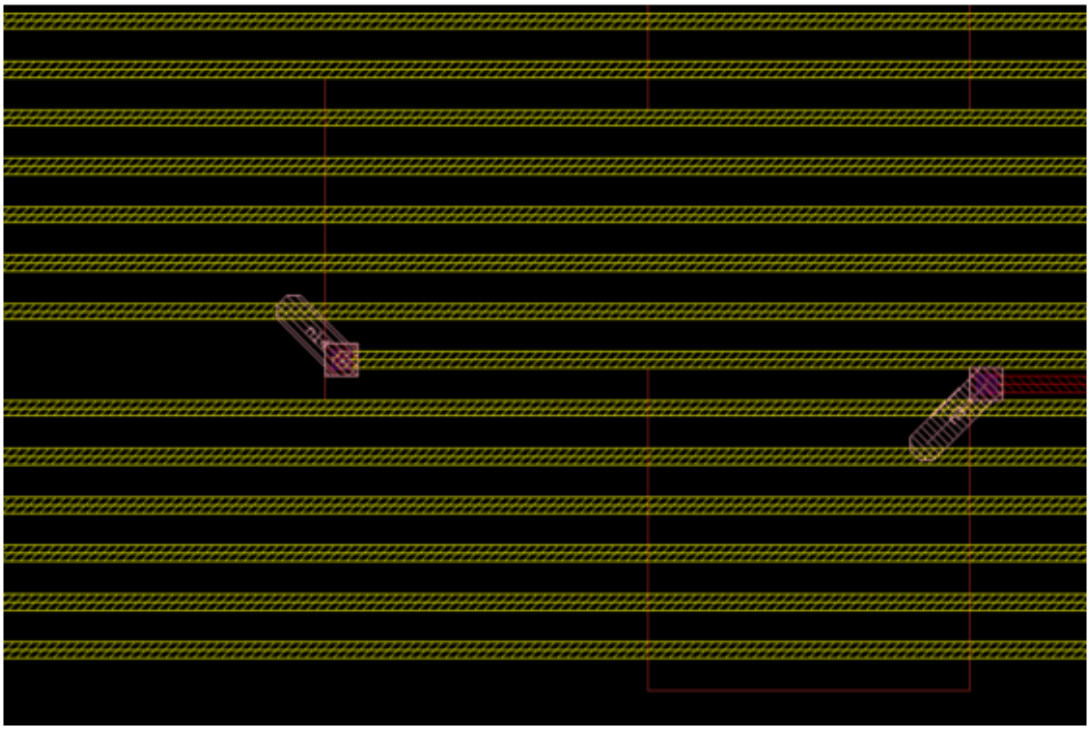


图 **6.30:** 同轴屏蔽之前

6.15 创建具有多层层次的倒装芯片设计

Creating a Flip-Chip Design With Multiple Levels of Hierarchy

对于包含多层物理层次的设计，工具支持顶层与块级的倒装芯片规划。对于这种设计风格，执行顶层倒装芯片布线时，会连接顶层的凸点单元，并忽略块内部的凸点单元。

注意

对于具有多层层次的设计，顶层倒装芯片布线还有下列额外考虑：

- 包含 I/O 凸点单元或焊盘单元的块必须预先放置。
- I/O 焊盘单元可以放在顶层，也可以放在块级。

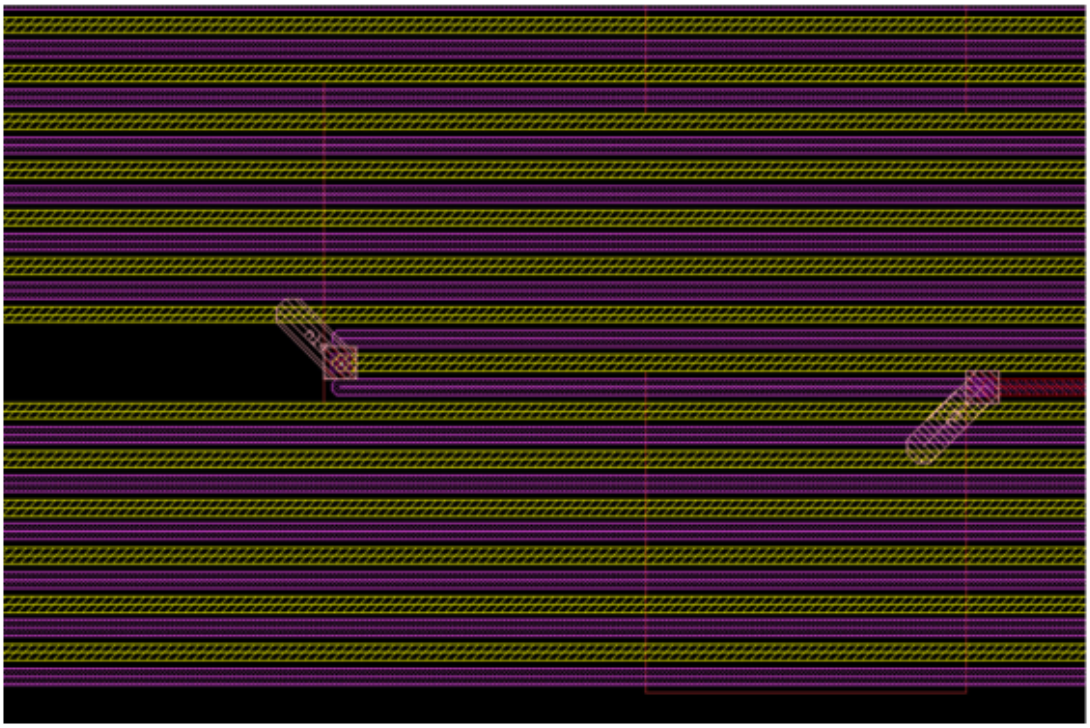


图 **6.31:** 同轴屏蔽之后

- 块级的 I/O 焊盘单元被视为固定。
- I/O 引导在顶层创建。
- 把顶层凸点分配给块级 I/O 单元的匹配类型必须在顶层创建。
- 对于包含 I/O 焊盘的块，I/O 焊盘放在块级。
- 块级 I/O 焊盘应当用 `create_matching_type` 命令指定。
- 对于未用 `create_matching_type` 命令指定的块级 I/O 焊盘，应在 IC Compiler II Library Manager 中把引脚形状的 `class` 属性设为 `bump`。

对于 N-2017.09 之前的 IC Compiler II 版本，可以使用基于框架的流程，把凸点单元分配给块中的 I/O 驱动器。要为 I/O 分配准备框架视图：

1. 进入包含 I/O 焊盘的块，或编辑该块。

```
icc2_shell> set_working_design -push [get_selection]
```

2. （可选）对于电源与地引脚，使用 `create_shape -port` 命令，根据焊盘单元的引脚形状创建端子形状。

   I/O 信号引脚的引脚形状会在下一步由 `create_frame` 命令自动创建。

3. 为这些块创建框架视图。

```
icc2_shell> create_frame
```

要缩短为多个块创建框架视图的运行时间，可以创建一个包含 create_frame 命令的脚本，并用 run_block_script 命令运行该脚本。

4. 返回顶层设计。

```
icc2_shell> set_working_design -pop
```

5. 选择该块，并把视图类型改为该块的框架视图。

```
icc2_shell> change_view -view frame [get_selection]
```

6. 把该块的 design_type 属性改为 flip_chip_driver。

```
icc2_shell> set_attribute [get_selection] design_type flip_chip_driver
```

7. 选择步骤 2 中在该块内创建的端子，并把端子的 class 属性改为 bump。

```
icc2_shell> set_attribute [get_selection] class bump
```

8. 继续执行正常的顶层倒装芯片流程。

Chapter 7

创建 3DIC 设计

Creating a 3DIC Design

在三维集成电路（3DIC）设计中，两颗或更多裸片（die）沿垂直方向直接堆叠，形成包含多颗裸片的完整设计或系统。在 2.5D IC 设计中，一颗或多颗裸片安装在中介层（interposer）上；中介层还把各裸片彼此连接，并连接到封装。中介层可由包括硅或玻璃在内的多种材料制造。

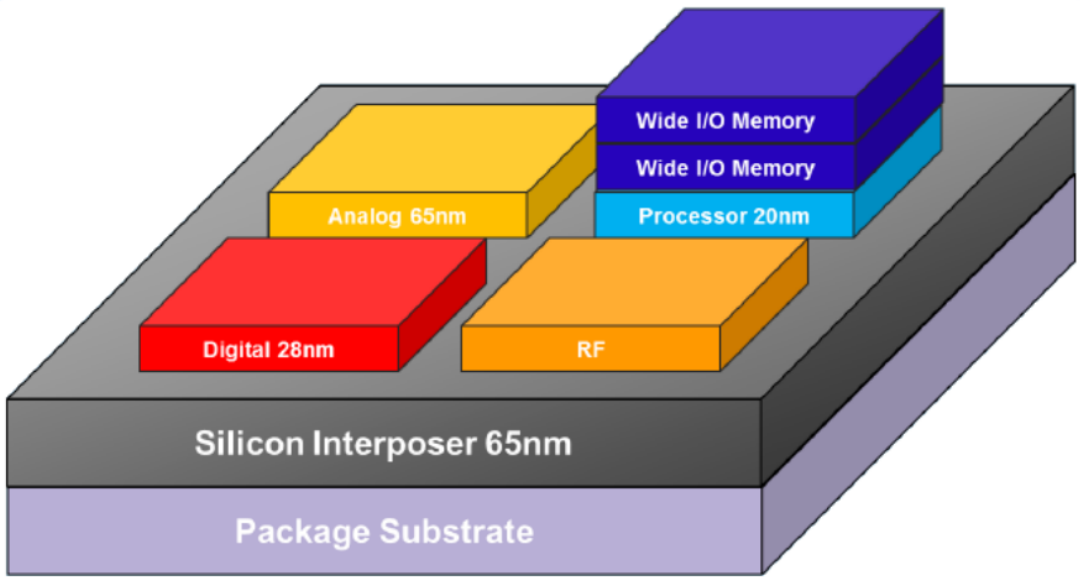


图 **7.1:** 复杂 3DIC 设计

IC Compiler II 工具支持 3DIC 流程以及用于创建包含多颗裸片的设计的命令。裸片可以分别放置在硅中介层上，也可以彼此堆叠并用硅通孔（through-silicon via，TSV）连接。该工具支持放置裸片、创建不同裸片之间的连接，以及验证 3DIC 设计的命令。

本节主题描述用于创建 2.5D 设计的流程。术语「有源裸片（active die）」用于描述包含有源器件的逻辑、存储器、射频或其他裸片。中介层是安装并连接有源裸片的硅裸片，不要求包含有源元件或逻辑。

有源裸片分别放置在无源硅中介层上、从而创建 2.5D 设计的流程如图 **??** 所示。

更多细节见下列主题：

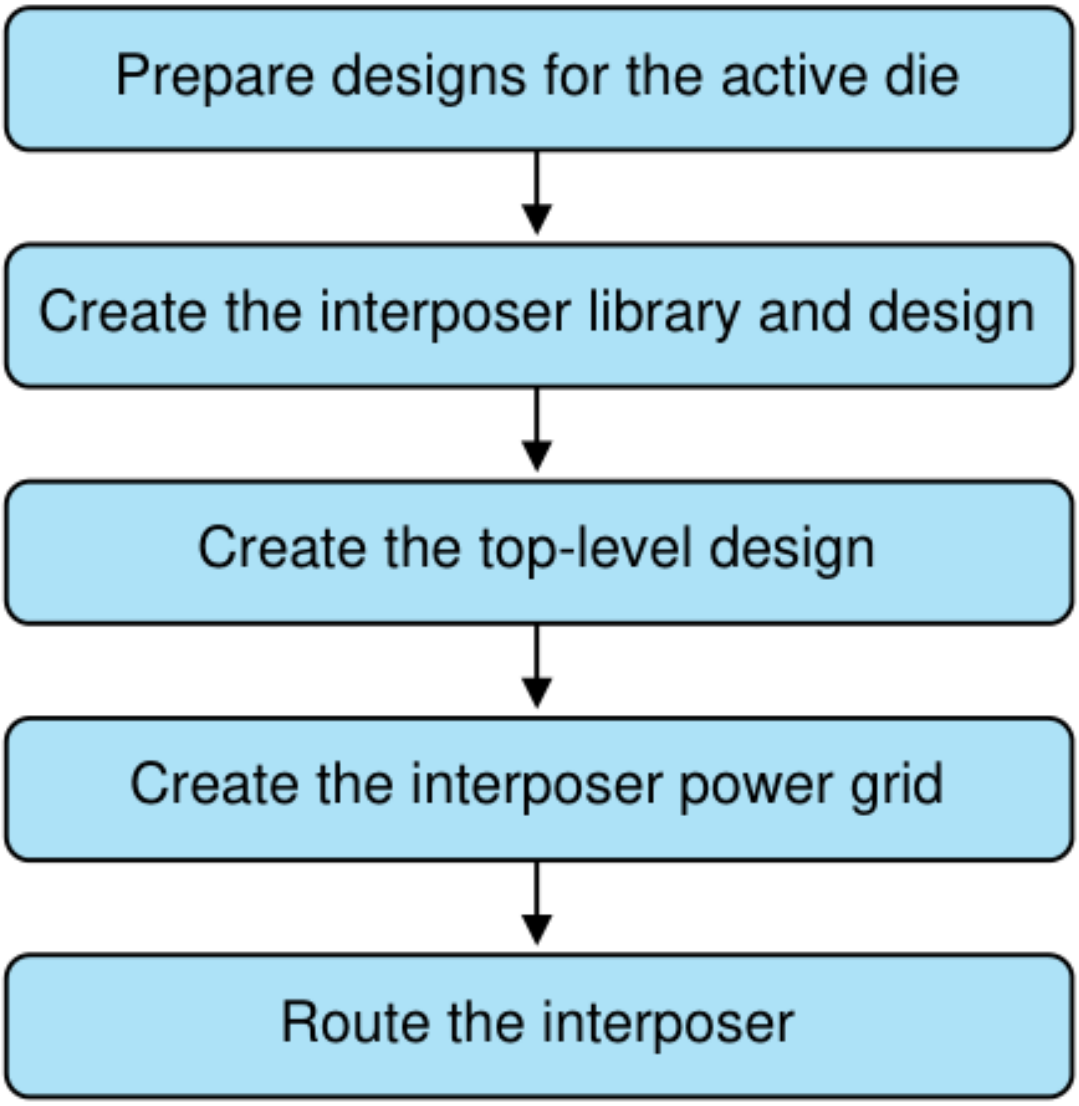


图 **7.2:** 2.5D 设计的 3DIC 流程

- 为 3D IC 设计准备库
- 为有源裸片准备设计
- 创建硅中介层设计
- 使用 CSV 文件读写凸点单元与 TSV 位置
- 使用 CSV 文件进行凸点规划
- 创建顶层设计
- 创建虚拟块
- 为中介层创建电源与地网格
- 布线中介层
- 3DIC 术语表

7.1 为 3D IC 设计准备库

Preparing Libraries for a 3D IC Design

3DIC 设计包含多个子设计，每个子设计可以引用各自的库并使用不同的工艺。这些库的缩放因子（scale factor）可能不同，但在完整的 3DIC 设计中需要一起使用。IC Compiler II 工具提供应用选项，以支持具有不同缩放因子的库。

要为 3DIC 设计中使用的库设置缩放因子，请在打开或创建第一个设计库之前指定 `design.session_scale_factor` 应用选项。3DIC 设计中各库的缩放因子必须是所有库缩放因子的公倍数。例如，若 `lib1` 库的缩放因子为 4000，`lib2` 库的缩放因子为 10000，则该应用选项必须设置为最小公倍数（20000），或 20000 的偶数倍。默认情况下不设置此选项。若打开缩放因子不同的库，命令会发出错误信息并退出。下列示例把缩放因子设置为 20000。

```
icc2_shell> set_app_options -name design.session_scale_factor \
   -value 20000
```

该工具还支持 3DIC 设计中库的半节点缩放（half-node scaling）。库的半节点缩放因子由库上的 `half_node_scale_factor` 属性指定。若在顶层设计上把 `design.is_3dic_mode` 应用选项设置为 `true`，则读入库时，库中的坐标会按 `half_node_scale_factor` 的设置进行缩放。

7.2 为有源裸片准备设计

Preparing the Designs for the Active Die

2.5D 流程从创建将放置在硅中介层上或彼此之上的各个有源裸片开始。每个有源裸片都有初始布图规划（floorplan），并包含用于把有源裸片连接到硅中介层或另一颗有源裸片的凸点单元（bump cell）。

下列简化流程可作为准备每个有源裸片的准则。这些步骤与创建倒装芯片（flip-chip）设计所用的步骤类似，更多信息见「创建凸点单元阵列」。由于把有源裸片放到中介层上时的关键信息是凸点单元位置与有源裸片尺寸，因此不必创建完全布局的设计。

1. 用 read_verilog 命令读入网表。

```
icc2_shell> read_verilog logic.v
```

2. 用 initialize_floorplan 命令初始化布图规划，或用 read_def 命令读入 DEF 文件。

```
icc2_shell> read_def logic.def
```

3. 用 create_bump_array 命令例化并放置凸点单元。

```
icc2_shell> create_bump_array -lib_cell BUMP -name BUMP_RING \
   -delta {225 225} -bbox {{100 100} {4000 4000}}
```

也可以用 read_aif 命令放置已有的凸点单元。

与倒装芯片设计类似，凸点单元位置来自下列来源之一：

- DEF 文件
- AIF 文件
- create_bump_array 命令

从焊盘单元到凸点单元的再分布层（redistribution layer，RDL）布线可以在后续设计阶段创建。

7.3 创建硅中介层设计

Creating the Silicon Interposer Design

要创建硅中介层设计，必须创建硅中介层的设计库、创建中介层网表、初始化布图规划，并创建电源与地连接。

创建中介层设计的步骤如下：

1. 用 create_lib 命令为中介层创建设计库，如下所示：

```
icc2_shell> create_lib interposer.ndm -technology interposer.tf \
   -ref_libs {c4_bump.nlib front_ubump.nlib physicalonly.nlib ...}
```

2. 为中介层创建空网表，并用 read_verilog 命令读入。

```
icc2_shell> sh cat interposer.v
module interposer (dummypin);
input dummypin;
endmodule
icc2_shell> read_verilog interposer.v
```

下列示例使用空的中介层网表。连接关系位于顶层网表中。IC Compiler II 工具会自动用端口和全部连接更新中介层网表，并按需要创建穿越。

3. 初始化中介层的布图规划。

```
icc2_shell> initialize_floorplan -control_type die \
   -boundary {{0 0} {15000 10000}}
```

4. 用 create_bump_array 命令创建凸点阵列，把封装的信号与电源焊盘连接到中介层。

```
icc2_shell> create_bump_array -lib_cell C4_BUMP \
   -name C4_logic_SIGNAL_B -origin {500 500} -delta {190 190} \
   -repeat {74 3}
...
```

5. 用 derive_3d_interface 命令，为上一步添加的每个凸点单元创建硅通孔（TSV）。

```
icc2_shell> derive_3d_interface -from [get_cells C4_logic_SIGNAL_B*] \
   -to_object_ref [get_via_defs -tech [get_techs *] VIAB1] \
   -name_prefix VIAB1_C4_logic_SIGNAL_B
...
```

derive_3d_interface 命令根据设计中已有结构的属性创建新的 3DIC 结构。图 **??** 显示上一条命令创建的 TSV 单元。

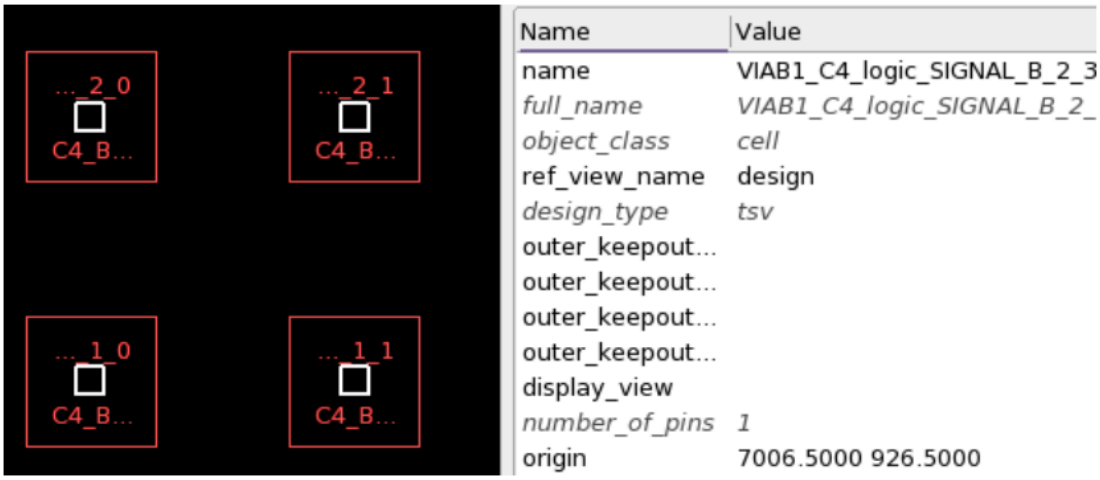


图 **7.3:** derive_3d_interface 创建的 TSV 单元

6. 为 RDL 布线创建信号、电源与地网络的匹配类型（matching type）。

```
icc2_shell> create_matching_type \
   -name Signal [all_connected [get_nets *SIGNAL*]]
```

完成这些步骤后，工具创建的中介层布图规划如图 **??** 所示。周边的凸点单元连接到有源裸片上的信号凸点。中央的两个凸点阵列连接到两颗有源裸片的电源与地。

7.4 使用 CSV 文件读写凸点单元与 TSV 位置

Reading and Writing Bump Cell and TSV Locations Using CSV Files

可以不设置布局约束与匹配类型来放置并连接凸点单元和硅通孔（TSV），而用一个或多个逗号分隔值（comma-separated values，CSV）文件指定位置与连接信息。使用 read_design_io 命令从 CSV 文件读入布局与连接信息，并把单元放到中介层上。使用 write_design_io 命令把位置信息写出到 CSV 文件。

下列示例写出当前设计中凸点单元与 TSV 的位置。

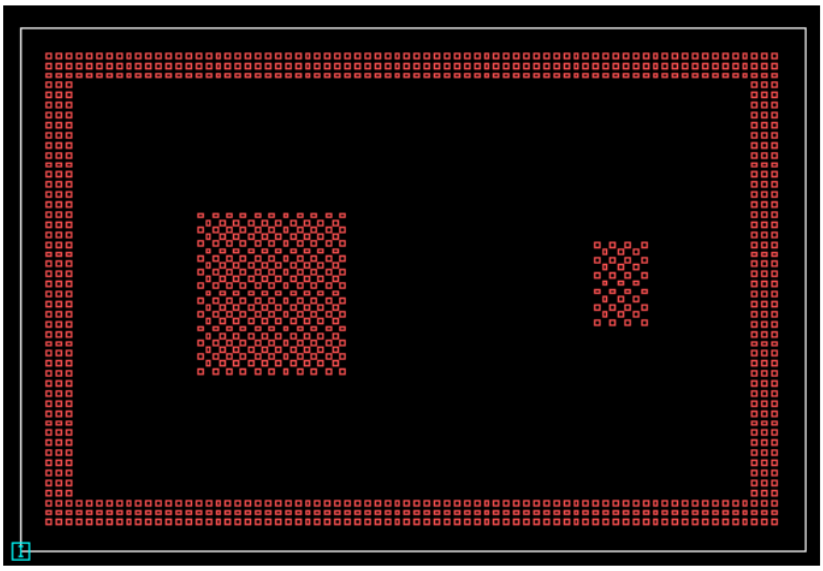


图 **7.4:** 中介层布图规划

```
icc2_shell> write_design_io -file_name bump_tsv_locations.csv
icc2_shell> sh cat bump_tsv_locations.csv
Reference_name,C4_inst,Ubump_inst,TSV_inst,X_origin,Y_origin,
  Orientation,Port_name,Net_name,Comments
C4_BUMP,C4_logic_SIGNAL_B_0_0,,,500,500,R0,,C4_logic_SIGNAL_B_0_0_net,
C4_BUMP,C4_logic_SIGNAL_B_0_1,,,690,500,R0,,C4_logic_SIGNAL_B_0_1_net,
C4_BUMP,C4_logic_SIGNAL_B_0_2,,,880,500,R0,,C4_logic_SIGNAL_B_0_2_net,
...
VIAB1,,,VIAB1_C4_mem_VSS_4_0,11165,5725,R0,,,
VIAB1,,,VIAB1_C4_mem_VSS_4_1,11465,5725,R0,,,
VIAB1,,,VIAB1_C4_mem_VSS_4_2,11765,5725,R0,,,
...
```

CSV 文件包含一行表头，以及每个单元实例一行数据。下列列表给出每个字段的说明，以及列表头中该字段的默认名称：

- 单元参考名（`Reference_name`）
- C4 凸点实例名（`C4_inst`）
- U 凸点实例名（`Ubump_inst`）
- TSV 实例名（`TSV_inst`）
- X 原点（`X_origin`）
- Y 原点（`Y_origin`）
- 单元方向（`Orientation`）
- 端口名（`Port_name`）
- 网络名（`Net_name`）

- 注释（Comments）

可以指定多个输入文件，并在每个文件中提供不同的字段。并非每个单元实例或连接都需要全部字段。表头行是必需的，它描述每一列的关键字。表头行之后，每一行描述一个单元实例或一条连接。单元行指定单元实例名、参考名、位置与方向。连接行指定端子之间的连接，端口行用于创建端口。

若单元实例不存在，工具会创建指定的单元实例。默认情况下，创建新单元实例时工具执行下列检查：

- 相同类型的单元不能重叠。
- C4 凸点单元可以与已有的 U 凸点单元、TSV 或标准单元重叠。
- U 凸点单元可以与已有的 C4 凸点单元、TSV 或标准单元重叠。
- TSV 可以与已有的 C4 凸点单元或 U 凸点单元重叠。

要禁用检查，使用 -overlap_check none 选项。要禁止任何重叠，使用 -overlap_check strict 选项。

若单元实例已经存在，则把它移动到指定位置。若单元实例与已有实例重叠，则跳过该实例。读到无效行时，工具发出警告信息并跳过该行。

默认情况下，若端口不存在，工具会忽略它。若要改为创建该端口，使用 -create_ports 选项。

默认情况下，若端口或网络已经存在，则保留其 port_type 或 net_type 属性的当前设置。要把特定端口或网络的该属性设置为电源，使用 -power 选项。要把该属性设置为地，使用 -ground 选项。若这些选项中指定的端口或网络不存在，工具会创建它们。

若 CSV 文件包含自定义列名，请创建一个映射文件，列出自定义列名与标准列名，然后用 -map_file 选项指定该映射文件。映射文件的每一行包含逗号分隔的自定义列名与对应的标准列名，如下例所示。

```
custom_reference_name,Reference_name
custom_C4_instance_name,C4_inst
...
```

在下列示例中，locations_mapped.csv 文件使用自定义列名，mapfile.txt 文件包含自定义列名与标准列名的列表。

```
icc2_shell> sh head mapfile.txt
re,Reference_name
c4,C4_inst
ub,Ubump_inst
ts,TSV_inst
...

icc2_shell> sh head locations_mapped.csv
re,c4,ub,ts,x_,y_,or,po,ne,co
C4_BUMP,C4_logic_SIGNAL_B_0_0,,,500,500,R0,,,
C4_BUMP,C4_logic_SIGNAL_B_0_1,,,690,500,R0,,,
...

icc2_shell> read_design_io -file_name locations_mapped.csv \
   -map_file mapfile.txt
```

```
...
1
```

配合 read_design_io 命令的其他选项，可以控制如何读取 CSV 文件。

- 用 -delimiter_char 选项指定分隔字符。

```
icc2_shell> sh head location_splat.csv
Reference_name!C4_inst!Ubump_inst!TSV_inst!X_origin!...
C4_BUMP!C4_logic_SIGNAL_B_0_0!!!500!500!R0!!!
C4_BUMP!C4_logic_SIGNAL_B_0_1!!!690!500!R0!!!
C4_BUMP!C4_logic_SIGNAL_B_0_2!!!880!500!R0!!!
...

icc2_shell> read_design_io -file_name location_splat.csv \
   -delimiter_char !
1
```

- 用 -c4_ref_name 选项指定 C4 凸点参考名。
- 用 -tsv_ref_name 选项指定 TSV 参考名。
- 用 -ubump_ref_name 选项指定 U 凸点参考名。
- 指定裸片原点与方向。
  默认情况下，工具把裸片的左下角用作原点，并假定没有旋转（R0 方向）。要用 -die_origin 选项指定原点坐标。要用 -die_orientation 选项指定裸片方向。合法值为 R0、R90、R180、R270、MX、MXR90、MY 与 MYR90。
- 指定单元原点位置。
  默认情况下，为单元原点指定的坐标被解释为单元的左下角。若要改为把坐标解释为单元中心，使用 -cell_origin_type center 选项。
- 指定 CSV 文件中使用的长度单位。
  默认情况下，命令假定 CSV 文件中的 x 坐标与 y 坐标使用设计库的长度单位。若 CSV 文件使用不同的长度单位，用 -units 选项指定一个乘数。命令把 CSV 文件中的值除以指定值，以得到以微米为单位的长度。乘数必须是大于或等于 1 的整数值。
- 用 -comment_char 选项指定注释字符。

用 write_design_io 命令的选项控制如何写出 CSV 文件：

- 指定要写入输出文件的对象。
  默认情况下，命令把所有 C4 凸点单元、U 凸点单元和 TSV 写入输出文件。要修改写入输出文件的对象，使用下列选项之一：
  - -contents：只输出单一对象类型。支持的类型为 c4、ubump 或 tsv。
  - -objects：只输出指定对象。指定对象必须是 C4 凸点单元、U 凸点单元或 TSV。
  - -within：只输出位于指定边界框内的 C4 凸点单元、U 凸点单元和 TSV。

- 指定单元原点位置。
  默认情况下，命令把单元的左下角用作原点。若要改为使用单元中心，使用 `-cell_origin_type center` 选项。使用此选项时，工具根据边界框计算单元中心。
- 指定 CSV 文件中使用的长度单位。
  默认情况下，命令使用设计库的长度单位把 x 坐标与 y 坐标写入 CSV 文件。若要使用不同的长度单位，用 `-units` 选项指定一个乘数。命令把以微米为单位的值乘以指定值，并把该值写入 CSV 文件。乘数必须是大于或等于 1 的整数值。
- 把字段名列表放入一个文件，并用 `-columns_file` 选项引用该文件，从而指定要写入 CSV 文件的数据字段。

```
icc2_shell> sh cat c4_columns.csv
Reference_name,C4_inst,X_origin,Y_origin

icc2_shell> write_design_io -file_name c4bumps.csv \
   -columns_file c4_columns.csv -contents c4
Parsing columns file c4_columns.csv

icc2_shell> sh head c4bumps.csv
Reference_name,C4_inst,X_origin,Y_origin
C4_BUMP,C4_logic_SIGNAL_B_0_0,500,500
...
```

7.5 使用 CSV 文件进行凸点规划

Bump Planning Using CSV Files

利用凸点图案、区域与伪凸点（pseudo bump）的概念进行凸点规划，提供了一种快速展开 3DIC 设计的方法。

凸点规划使用 `write_design_io -pseudo_bumps` 命令，以 CSV 文件形式的 ASCII 格式把凸点区域与伪凸点数据保存到磁盘。然后可以在新会话中用 `read_design_io -pseudo_bumps` 命令读入这些数据。例如，下列命令把凸点信息写出到 `ABC.csv` 文件。

```
icc2_shell> write_design_io -pseudo_bumps -file_name ABC.csv
```

在下列示例中，工具从 `ABC.csv` 文件读入凸点信息。

```
icc2_shell> read_design_io -pseudo_bumps -file_name ABC.csv
```

生成的 CSV 文件具有一组已定义的列，用于捕获凸点信息。

表 **7.1:** CSV 文件中的凸点区段列

列名	说明
`Name`	凸点区域的名称。此列为必填列，必须有值。
`Boundary`	X 与 Y 坐标对的列表。此列为必填列，必须有值。

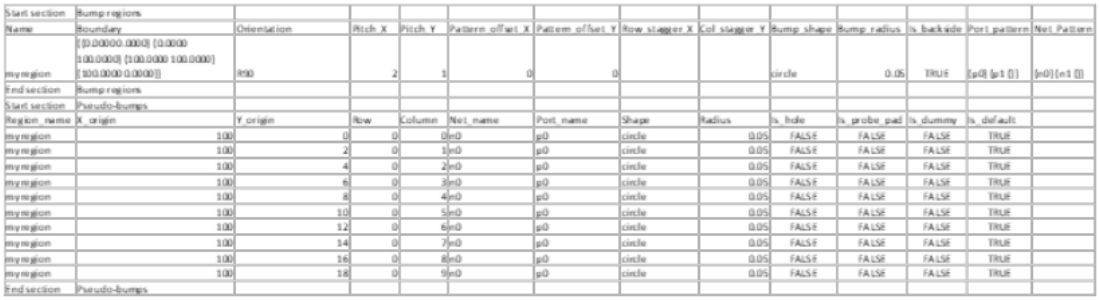
表 **7.1:** CSV 文件中的凸点区段列（续）

列名	说明
Pitch_X	第 i 列与第 $i+1$ 列之间循环距离的列表。此列为必填列，必须有值。
Pitch_Y	第 i 列与第 $i+1$ 列之间循环距离的列表。此列为必填列，必须有值。
Pattern_offset_X	从凸点区域左下角到图案起点的 X 方向增量。
Pattern_offset_Y	从凸点区域左下角到图案起点的 Y 方向增量。
Row_stagger_X	一行起点在 X 方向增量上的循环距离列表。
Column_stagger_Y	一列起点在 Y 方向增量上的循环距离列表。
Orientation	凸点的方向。合法值为 R0、R90、R180、R270、MX、MXR90、MY 与 MYR90。空白字段默认为 R0。
Bump_shape	区域内伪凸点的默认形状，除非被单个伪凸点覆盖。合法值为 triangle、square、pentagon、hexagon、septagon、octagon、circle。此列为可选列，默认值为 octagon。
Bump_radius	区域内伪凸点的默认半径，除非被单个伪凸点覆盖。此列为必填列，必须有值。
Is_backside	凸点区域是否位于背面。此列为可选列，默认值为 false。
Port_pattern	伪凸点所连接端口名的二维图案。
Net_pattern	伪凸点所连接网络名的二维图案。

表 **7.2:** CSV 文件中的伪凸点区段列

列名	说明
Region_name	此伪凸点所在区域的名称。
X_origin	凸点中心的位置（与 Y_origin 一起给出）。
Y_origin	凸点中心的位置（与 X_origin 一起给出）。
Row	行索引号，从底部的 0 开始，向上递增。
Column	列索引号，从底部的 0 开始，向上递增。
Net_name	连接到该凸点的网络名称。
Port_name	该伪凸点所连接的顶层逻辑端口名称。
Shape	合法值为 triangle、square、pentagon、hexagon、septagon、octagon、circle。若未指定，则默认为对应区域所指定的值。
Radius	以你所指定单位表示的半径值。若未指定，则默认为对应区域所指定的值。
Is_hole	合法值为 true 或 false。若未指定，则默认为 false。
Is_probe_pad	合法值为 true 或 false。若未指定，则默认为 false。
Is_dummy	合法值为 true 或 false。若未指定，则默认为 false。
Is_default	合法值为 true 或 false。若未指定，则默认为 false。

要在 CSV 文件中添加上表所述默认列以外的新列，步骤如下：



Start section	Bump regions												
Name	Boundary	Orientation	Pitch X	Pitch Y	Pattern offset X	Pattern offset Y	Row stagger X	Col stagger Y	Bump shape	Bump radius	Is backside	Port pattern	Net Pattern
myregion	{{0.0000 0.0000} {0.0000 100.0000} {100.0000 100.0000} {100.0000 0.0000}}	R90	2	1	0	0			circle	0.05	TRUE	{p0} {p1 {}}	{n0} {n1 {}}
End section	Bump regions												
Start section	Pseudo-bumps												
Region name	X origin	Y origin	Row	Column	Net name	Port name	Shape	Radius	Is hole	Is probe pad	Is dummy	Is default	
myregion	100	0	0	0	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	2	0	1	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	4	0	2	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	6	0	3	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	8	0	4	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	10	0	5	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	12	0	6	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	14	0	7	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	16	0	8	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
myregion	100	18	0	9	n0	p0	circle	0.05	FALSE	FALSE	FALSE	TRUE	
End section	Pseudo-bumps												

图 **7.5:** 凸点规划 CSV 文件示例

1. 定义该列（即用户属性）。
   用户属性可以是下列类型之一：Boolean、double、float、int 或 string。对于 string 类型，属性值不得包含任何逗号（,）。
   在下列示例中，向 CSV 文件添加名为 `my_bump_details` 的新列：

```
icc2_shell> define_user_attribute -type string -classes cell \
 -name my_bump_details
```

2. 把凸点细节保存到 CSV 文件。

```
icc2_shell> write_design_io -file_name ABC.csv -attribute \
 {my_bump_details}
```

要在新会话中恢复凸点规划数据，步骤如下：

1. 定义用户属性。
   用户属性可以是下列类型之一：Boolean、double、float、int 或 string。对于 string 类型，属性值不得包含任何逗号（,）。
   在下列示例中，定义名为 `my_bump_details` 的新属性：

```
icc2_shell> define_user_attribute -type string -classes cell \
 -name my_bump_details
```

2. 从 CSV 文件读入设计。

```
icc2_shell> read_design_io -file_name input.csv -attribute \
 {my_bump_details}
```

工具只读取已定义的用户属性。若未定义，则忽略这些属性。

7.5.1 使用 CSV 文件进行凸点与伪凸点 ECO

Bump and Pseudo Bump ECO Using CSV Files

可以用 `compare_design_io` 命令，把凸点布图规划的变更与设计进行比较。该命令以包含设计各版本 I/O 元素的两个 CSV 文件作为输入，对它们进行比较并生成差异。这些差异或者在图形界面中显示为用户表，或者写出到 CSV 文件。然后可以在新会话中用 `read_design_io -eco` 命令从该 ECO CSV 文件读入这些数据。例如，下列命令从 `DEF.csv` 文件读入 ECO 凸点信息。

```
icc2_shell> read_design_io -eco -file_name DEF.csv
```

`read_design_io -eco` 命令只读取真实凸点与伪凸点的 ECO CSV 文件。若对非 ECO 设计 CSV 文件指定 `-eco` 选项，或指定凸点或伪凸点 ECO CSV 文件时未指定 `-eco` 选项，命令会发出错误。当 `-pseudo_bumps` 选项与 `-eco` 选项一起使用时，读入命令把指定文件视为由 `compare_design_io` 命令生成的伪凸点 ECO 文件并加以解析。例如，要解析伪凸点 ECO CSV 文件：

```
icc2_shell> compare_design_io -pseudo_bumps -golden orig.csv -v2 new.csv \
 -output bump_planning_eco.csv
icc2_shell> read_design_io -pseudo_bumps -eco -file_name \
 bump_planning_eco.csv
```

若把 `-continue_on_error` 选项与 `read_design_io` 命令一起使用，它会忽略带有警告的行，并处理有效行。`-continue_on_error` 选项适用于凸点与伪凸点 CSV 文件。

读取凸点 ECO CSV 文件时适用下列准则：

- 只能通过 `-file_name_list` 选项指定一个 ECO 文件。
- 运行该命令时必须有当前设计。
- `read_design_io -eco` 支持撤销与重做操作。
- 概括而言，下列选项不能与 `-eco` 选项一起指定：
  - `-map_file`
  - `-comment_char`
  - `-units`

7.6 创建顶层设计

Creating the Top-Level Design

3DIC 设计包含两颗或更多有源裸片以及一个硅中介层；有源裸片与硅中介层被例化到顶层设计中。顶层设计还包含封装、硅中介层与有源裸片之间的连接，以及表示到封装连接的端口。

> **注意**
>
> 顶层设计只规定实际裸片与中介层之间的物理和逻辑连接；顶层设计本身从不在硅片上制造。

利用顶层设计，可以：

- 指定各裸片在硅中介层上的坐标与方向
- 在中介层与各裸片之间复制或镜像凸点
- 对整个系统执行三维物理与逻辑设计检查
- 根据最短路径把凸点分配给信号

下列顶层 Verilog 网表包含两个裸片实例和一个中介层实例，用于随后的示例。

```
module top (resetn , clk , pllref , ...);
input resetn ;
input clk ;
input pllref ;
...
leon3mp_chip logic_die_inst (.resetn(resetn), ...);
mem mem_die_inst (.sa(sa), .sd(sd), ...);
interposer interposer_inst ();
endmodule
```

要创建包含有源裸片与中介层的顶层设计：

1. 创建设计库以容纳顶层设计。

```
icc2_shell> create_lib top.ndm -technology top.tf \
   -ref_libs {mem.ndm leon3mp.ndm interposer.ndm}
```

因为顶层设计引用有源裸片与硅中介层，参考库必须包括硅中介层库以及每个有源裸片的库。

2. 读入顶层设计的 Verilog 网表。

```
icc2_shell> read_verilog top.v
```

3. 初始化布图规划。

```
icc2_shell> initialize_floorplan -boundary {{0 0} {15000 10000}}
```

4. 把当前块的 `design_type` 属性改为 `3dic`。

```
icc2_shell> set_attribute [current_block] design_type 3dic
```

5. 用 `set_cell_location` 命令放置中介层与有源裸片。请注意 `-z_offset` 选项，它指定中介层与有源裸片的相对 z 位置。

```
icc2_shell> set_cell_location interposer_inst \
   -coordinates {0 0} -z_offset 0 -orientation N
icc2_shell> set_cell_location logic_die_inst \
   -coordinates {2500 2600} -z_offset 1 -orientation FN
icc2_shell> set_cell_location mem_die_inst \
   -coordinates {10300 3000} -z_offset 1 -orientation FN
```

6. 用 `report_3d_chip_placement` 命令确认中介层与有源裸片的布局。

```
icc2_shell> report_3d_chip_placement -all
----------------------------------------------------------------------
chip_name        stack_z  location     orientation  scaling_factor
----------------------------------------------------------------------
logic_die_inst   1        (2500 2600)  FN           1
mem_die_inst     1        (10300 3000) FN           1
interposer_inst  0        (0 0)        N            1
----------------------------------------------------------------------
```

要查看有源裸片与中介层的 Z 序（z-order），也可以查询裸片实例上的 `z_order` 属性。或者，在 **View Settings** 面板中点击 **Layers** 选项卡，查看与每个裸片实例关联的 Z 序（Z level）设置，如图 **??** 所示。

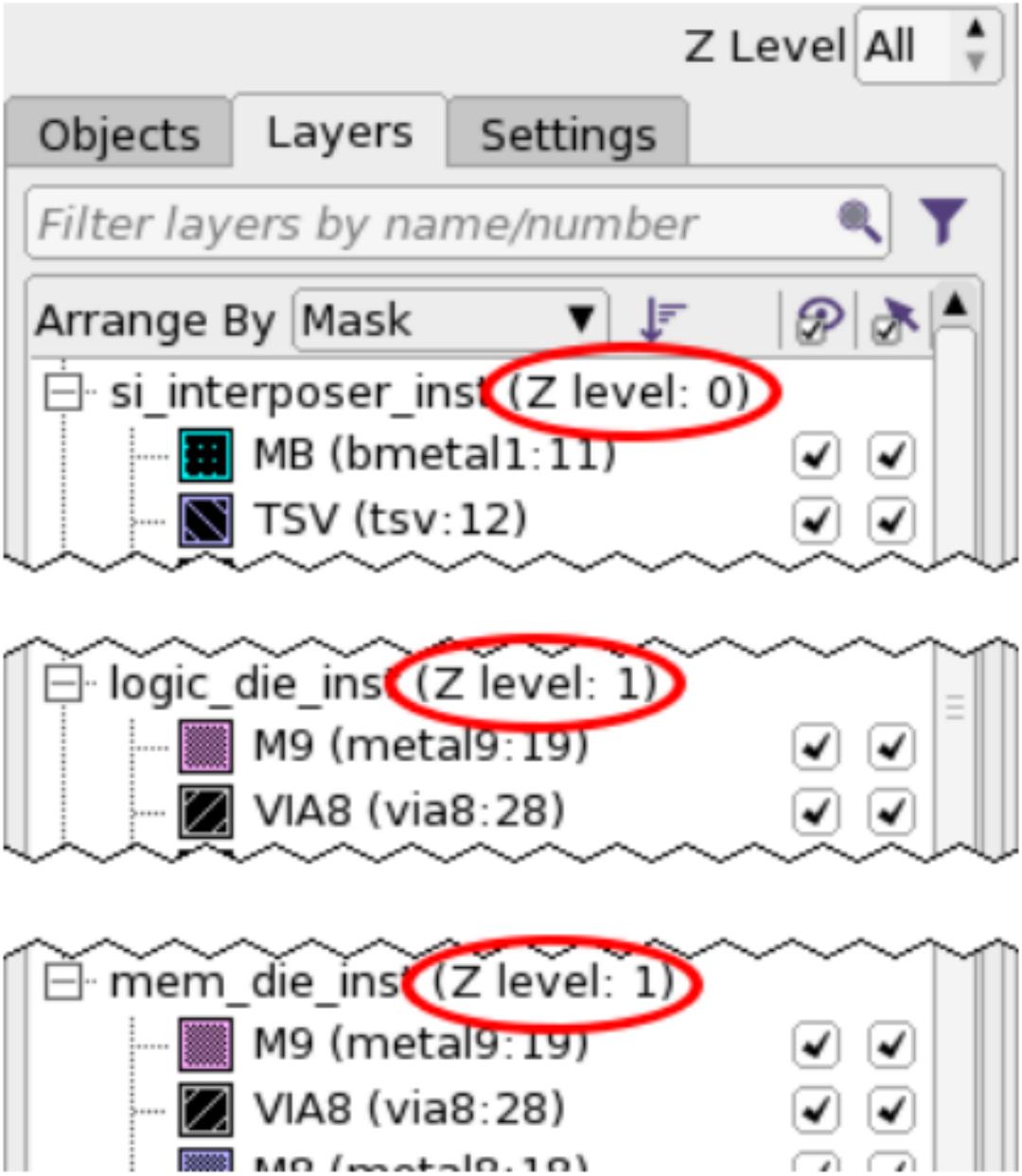


图 **7.6:** View Settings 面板中的 Z 序

7.（可选）用带 `-chip_placement` 选项的 `check_3d_design` 命令确定有源裸片之间是否存在重叠。

```
icc2_shell> check_3d_design -chip_placement
```

8. 保存设计、保存库并关闭库。

```
icc2_shell> save_block
icc2_shell> save_lib
icc2_shell> close_lib
```

9. 重新打开库。

```
icc2_shell> open_lib top.ndm -ref_libs_for_edit
```

10. 用 set_editability 命令确保各块可编辑。

```
icc2_shell> set_editability \
   -blocks leon3mp_chip.ndm:leon3mp_chip.design
icc2_shell> set_editability -blocks mem.ndm:mem.design
icc2_shell> set_editability -blocks siip.ndm:interposer.design
```

11. 通过从两颗有源裸片复制凸点单元位置，在中介层上创建凸点单元。

```
icc2_shell> create_3d_mirror_bumps -from mem_die_inst \
   -to si_interposer_inst -ref_cell BUMP -prefix mem
icc2_shell> create_3d_mirror_bumps -from logic_die_inst \
   -to si_interposer_inst -ref_cell UBUMP -prefix logic
```

当源裸片与目标裸片垂直堆叠时，此命令把凸点单元、凸点簇与键合焊盘从源裸片复制并放置到目标裸片。该工具还支持把凸点位置从中介层复制到各裸片的 3DIC 流程。
若要为硬宏单元的引脚创建镜像凸点单元，请对 create_3d_mirror_bumps 命令使用 -pins 选项来指定硬宏引脚。

12.（可选）用 propagate_3d_connections 命令，在上一步添加到中介层上的凸点单元或伪凸点与有源裸片上的凸点单元之间创建逻辑连接。

```
icc2_shell> propagate_3d_connections
Created 215 new nets after inter-chip nets splitting.
Propagate 67 nets.
```

若不希望允许端口打孔（port punching），使用 -no_port_punching 选项。要指定新创建端口的端口名，使用 -port_name_prefix 选项。使用 -set_terminals_for_updated_ports 为任何被更新的端口设置端子。关于全部可用命令选项，请参阅 propagate_3d_connections 命令的 man 页。

13. 在中介层上创建匹配类型。

```
icc2_shell> current_design si_interposer
{siip.ndm:si_interposer.design}

icc2_shell> add_to_matching_type Signal [get_cells *ubump*]
1
...

icc2_shell> create_matching_type -name Power [get_pins *VDD_CORE*/PAD]
{Power}
...
```

14. 在顶层创建匹配类型。

```
icc2_shell> current_design top
icc2_shell> create_matching_type \
   -name Signal { logic_die_inst/test_so ...}
```

```
icc2_shell> add_to_matching_type Signal { si_interposer_inst/a1 ...}
icc2_shell> add_to_matching_type Signal { mem_die_inst/sa[14] ...}
```

15. 用 propagate_3d_matching_types 命令，把单元、引脚与端子的匹配类型复制到其他引脚。

```
icc2_shell> propagate_3d_matching_types
============================================================
Matching types to be propagated from si_interposer_inst to
 mem_die_inst:
{ 'Signal' 'Power' 'Ground' }
============================================================
...
```

复制匹配类型之后，可以用 report_matching_types 命令显示由 propagate_3d_matching_types 命令分配的匹配类型。

16. 用 assign_3d_interchip_nets 命令，在凸点单元或伪凸点与最近的驱动单元之间分配连接。

```
icc2_shell> assign_3d_interchip_nets
...
Doing assignment between chip logic_die_inst and
  si_interposer_inst.
Successfully assigned 149 nets between chip logic_die_inst and
  si_interposer_inst.
...
Doing assignment between chip si_interposer_inst and Package.
Successfully assigned 149 nets between chip si_interposer_inst
  and Package.
```

assign_3d_interchip_nets 命令在逻辑上把 TSV、凸点单元和伪凸点连接到网络，根据顶层连接关系在中介层中创建端口与网络，为连接封装与裸片的信号创建穿越，并在中介层中创建裸片到裸片的网络连接。

若设计中同一芯片间网络有多个凸点单元，使用 -set_port_terminals incremental | all 选项来消除歧义，允许工具选择首选连接，并避免创建重叠端口。分配连接时，工具在端口上设置 preferred_pin 属性；使用 get_attribute 命令查看该分配。incremental 参数只在尚未具有该属性的端口上设置 preferred_pin 属性。all 参数重置并重新分配所有端口的首选引脚属性设置。类似地，该命令支持伪凸点与驱动之间的逻辑连接。使用 -bump_regions 与 -pseudo_bumps 选项指定要参与逻辑连接传播的伪凸点。使用 -no_port_punching 选项以不允许端口打孔。若要把电源地网络纳入分配，使用 -include_pg_nets 选项。要指定新创建端口的端口名，使用 -port_name_prefix 选项。要了解全部可用命令选项，请参阅 assign_3d_interchip_nets 命令的 man 页。

7.7 创建虚拟块

Creating Virtual Blocks

3DIC 设计包含由两颗裸片之间交叉耦合引起的电容；该电容影响顶层的网络。为捕获此电容，StarRC 等工具提取寄生电容，并把它存放在添加到设计数据库的虚拟接口块（virtual interface block）

中。在用 StarRC 提取电容之前，必须在 IC Compiler II 工具中用 create_3d_virtual_blocks 命令创建虚拟接口块。

虚拟接口块包含两颗堆叠裸片之间、以及裸片与中介层之间接口处的端口、网络、图形与通孔。create_3d_virtual_blocks 命令支持修改网表并连接接口信息的选项，或者只添加接口信息而不把它连接到已有块，如下列示例所示。

- 在 mem_inst 块与 interposer_inst 块之间创建虚拟接口块，并从 mem_inst 复制四层、从 interposer_inst 复制两层。

```
icc2_shell> create_3d_virtual_blocks \
   -blocks {{mem_inst:4} {interposer_inst:2}}
```

- 在 mem_inst 块与 logic_inst 块之间创建虚拟接口块，并从两个块各复制四层。

```
icc2_shell> create_3d_virtual_blocks \
   -blocks { logic_inst mem_inst} -layers 4
```

- 根据块的 z_offset 属性设置，在所有裸片与中介层之间创建虚拟接口块。这些虚拟接口块连接到顶层网表。

```
icc2_shell> create_3d_virtual_blocks -in_netlist
```

7.8 为中介层创建电源与地网格

Creating the Power and Ground Meshes for the Interposer

硅中介层安装有源裸片，并提供它们之间的连接。除信号之外，电源与地也经中介层布线，以便向有源裸片供电。为提供充足的电源并尽量减小 IR 压降（IR drop），应在中介层内部创建电源网格，以便把电源从封装高效分配到各裸片。

硅中介层通过背面的 C4 凸点连接到封装，然后把电源分配到正面的微凸点（microbump），这些微凸点再连接到裸片上的倒装芯片凸点。中介层中间布线层上的电源网格有助于尽量减小因中介层上额外布线而可能产生的 IR 压降。使用标准的 IC Compiler II 电源规划命令在中介层上创建电源网格。

为每个有源裸片在中介层上创建电源网格的步骤如下：

1. 用 create_pg_mesh_pattern 命令创建网格图案。

```
icc2_shell> create_pg_mesh_pattern mesh1 \
            -layers {{{vertical_layer: M4} {width: 12} \
             {spacing:88} {pitch: 200} {offset: 65}} \
             {{horizontal_layer: M3} {width: 12} \
             {spacing:88} {pitch: 200} {offset: 40}}}
```

2. 创建电源地策略，把该图案与有源裸片下方的中介层区域关联起来。

```
icc2_shell> set_pg_strategy strat1 \
   -pattern {{name: mesh1} {nets: VDD_mem VSS_mem}} \
   -polygon {{11060 3800} {12095 6755}}
```

3. 用 compile_pg 命令例化电源规划。

```
icc2_shell> compile_pg -strategies strat1
```

4. 对每个有源裸片重复前面的步骤。

7.9 布线中介层

Routing the Interposer

在硅中介层上放置凸点单元、创建硅通孔（TSV），并在有源裸片实例下方创建电源网格之后，可以对中介层进行布线。IC Compiler II 工具支持下列在中介层上创建布线的方法：用 Zroute 布线、用 RDL 布线器布线，以及组合这两种布线方法。

布线中介层之前，可以指定布线规划来放置通孔与网络。布线规划使用一个图案文件，其中包含用于创建并放置通孔与布线引导（route guide）的约束及其他数据。还可以修改图案文件并添加额外的布线规则，例如把凸点布线到预放置通孔，以及指定点到点布线连接，详见「创建布线规划」。

7.9.1 创建布线规划

Creating a Route Plan

为高带宽存储器设计布线中介层时，通常希望对通孔与网络的放置有更精细的控制。为此，使用 create_interposer_routeplan 命令创建布线规划。该命令分析一个中介层设计，其中包含预先放置并预先分配的正面凸点以及（可选的）背面凸点。根据凸点的配置，该命令创建并放置把凸点连接到第二层以供布线的通孔，并以线段桩预布线（布线引导）的形式创建布线指导。该命令还会对凸点阵列之间的中心区段进行布线。

布线引导由 RDL 布线器用来完成复杂的布线路径。已放置的凸点通孔与布线引导合称为布线规划。布线规划的目标是在布线步骤中优化布线并满足指定的布线要求。

create_interposer_routeplan 命令支持下列中介层风格模式：default、single_hbm（用于高带宽存储器设计）以及 fpga（用于 FPGA 设计）。默认情况下，该命令只插入通孔。要插入通孔和布线引导，用 -interposer_style 选项指定 single_hbm 或 fpga。

工具根据你选择的中介层风格处理不同类型的图案文件。图案文件包含用于创建并放置通孔与布线引导的约束及其他数据。

可以通过修改图案文件来指定额外的布线指导。例如，可以：

- 在凸点单元与中介层上的金属层之间创建通孔
- 指定通孔定义、通孔位置、单个通孔或堆叠通孔，以及通孔间距规则
- 把线段桩布线到其他线段桩和通孔（点到点布线）
- 把预放置的着陆通孔布线到凸点

图案文件是包含约束与指示的 ASCII 文件，用于驱动 create_interposer_routeplan 命令。它包含一个或多个区段，每个区段以一个关键字开始，后跟一对花括号。约束写在花括号内部。除非你用新约束修改文件并用 -pattern_file 选项指定更新后的文件，否则工具使用默认图案文件。

要查看 single_hbm 中介层风格的图案文件，见「图案文件示例」。

下列示例为所有凸点单元在 RDL 布线层与顶层金属之间创建单个通孔。

```
icc2_shell> create_interposer_routeplan -pattern_file interposer.pat
```

7.9.2 布线高带宽存储器设计

Routing High-Bandwidth Memory Designs

3DIC 中介层设计支持高带宽存储器，这是一种使用堆叠 RAM 的高性能 RAM 接口。高带宽存储器设计常常与 CPU 或 GPU 一起放在硅中介层裸片上。这些设计用更宽的数据通道把存储器移近处理器，从而加快数据吞吐、减少驱动信号所需的功耗，并减小 RC 延时。

包含高带宽存储器的中介层设计对布线有具体而详细的物理设计要求，比其他中介层配置更为严格。这些要求包括：

- 高密度布线，布线轨道利用率常常接近 100%
- 布线层数量有限（常常为两层）
- 高度并行的总线布线
- 关键数据信号的长度匹配
- 同轴屏蔽（coaxial shielding）

  下列布线策略有利于高带宽存储器设计的布线。请只对高密度、高带宽存储器设计使用此策略。

- 把凸点布线到着陆通孔。
  可以指示布线器在与凸点同一网络上搜索最近的通孔，并把凸点连接到该通孔。接触凸点的通孔被视为已连接，除非凸点与通孔之间存在设计规则检查（DRC）违例，否则不需要布线。

- 点到点布线。
  把凸点布线到着陆通孔之后，可以在布线之前于布线规划文件中提供下列信息，以指定点到点连接，从而在凸点、着陆通孔与线段桩之间布线：

  ```
  #netName #layer #point1_x #point1_y #point2_x #point2_y
  ```

  布线规划文件指定要布线的网络、连接点，以及点到点连接的目标布线层。
  连接点必须满足下列要求：

  – 这些点必须落在同一网络的图形之内，例如线段、通孔或引脚图形。
  – 对于线段图形，该点必须恰好位于线段的端点。不过，布线器并不一定恰好在文件中指定的点上连接到引脚和通孔。
  – 这些点必须位于最小网格上。
  – 这些点必须位于没有 DRC 违例的坐标上，也就是说，把线段布线到指定点时，与相邻图形之间没有 DRC 违例。存在 DRC 违例的布线对保持未布线。

  在下列示例中，RDL 布线器在 AP 层上把 `Net1` 网络从点 (10.5, 15.0) 布线到点 (20.5, 20.0)：

  ```
  Net1 AP 10.5 15.0 20.5 20.0
  ```

  下列示例为高带宽存储器设计布线中介层：

```
### Open the floorplanned interposer block

open_lib interposer_floorplan
open_block interposer

### Set the design style to a 3DIC interposer

set_app_options -name flip_chip.route.design_style \
   -value 3dic_interposer

### Create the interposer RDL vias and route guides (routeplanning)
### Set the style to HBM and specify the pattern constraint file

create_interposer_routeplan -interposer_style single_hbm \
   -pattern_file HBM_pattern.txt

### Define the HBM channel signals to be routed

set net_all "DQa24 DMa2 DMa3 DQa16 DQa27 DQa26 DQa25 DQa20 \
   DQa21 RDQSa0_t DQa28 DQa29 DERRa0 RDQSa0_c DQa18 DQa19 DQa17 \
   RDa1 DQa22 DQa23 DBIa3 DQa31 DQa30 DBIa2 DMe2 DQe25 DMe3 \
   RDQSe0_t DQe18 DQe19 DQe16 DERRe0 DQe26 DQe27 DQe24 DQe17 \
   DQe30 DQe23 DQe21 DQe20 RDQSe0_c DQe31 DQe29 DQe28 RDe1 DBIe2 \
   DQe22 DBIe3"

### Define the routing rules

set fc_route_layer_m4 {M4 AP}
set fc_route_space_m4 {M4 2.96 AP 3}
set fc_route_width_m4 {M4 1.6 AP 5}

set fc_route_layer_m2 {M2 AP}
set fc_route_space_m2 {M2 2.96 AP 3}
set fc_route_width_m2 {M2 1.6 AP 5}

create_routing_rule rdlrule_m4 -widths $fc_route_width_m4 \
   -spacings $fc_route_space_m4
create_routing_rule rdlrule_m2 -widths $fc_route_width_m2 \
   -spacings $fc_route_space_m2

set_routing_rule [get_nets $net_m2] -rule rdlrule_m2
set_routing_rule [get_nets $net_m4] -rule rdlrule_m4

### Set the shape type for pre-placed vias to user_route
### to preserve vias until the routing stage

set_attribute [get_vias *] shape_use user_route

set_app_options -name flip_chip.route.connect_via_center \
```

```
   -value true

### Specify bump-to-via routing and route the
### RDL layer

set_app_options -name flip_chip.route.bump_via_landing_mode \
   -value true

### Route the nets
### Use the -coordinates option to speed up routing by
### routing inside the specified region only

route_3d_rdl -nets [get_nets $net_all]  -layer AP \
  -coordinates {{12300 11980} {12515 18814}}

### Set the options for via-to-stub routing on the lower layers

set_app_options -name flip_chip.route.bump_via_landing_mode \
  -value false

### Specify the point-to-point connection file created by the
### route plan command for the first layer (M2)

set_app_options \
   -name flip_chip.route.point_to_point_connection_file_name \
   -value hbm_p2p.txtM2

### Route the via-to-stub nets on the specified layer (M2)

route_3d_rdl -nets [get_nets $net_m2] -layer M2 \
  -coordinates {{12300 11980} {12515 18814}}

### Set the point-to-point connection file created by the route
### plan command for the second layer (M4)

set_app_options \
   -name flip_chip.route.point_to_point_connection_file_name \
   -value hbm_p2p.txtM4

### Route the via-to-stub nets on the specified layer (M4)

route_3d_rdl -nets [get_nets $net_m4] -layer M4 \
   -coordinates {{12300 11980} {12515 18814}}
```

7.9.3 组合 Zroute 与 RDL 布线器的布线流程

Routing With a Combined Zroute and RDL Router Flow

若 M1 到 Mn 层上的布线不需要高带宽存储器设计所需的特殊配置，并且需要连接到硅通孔（TSV）引脚，则用组合的 Zroute 与 RDL 布线器流程来布线中介层。

用 Zroute 与 RDL 布线器布线中介层的步骤如下：

1. 创建中介层 RDL 通孔（默认模式下的布线规划）。

```
icc2_shell> create_interposer_routeplan -pattern_file via_pattern.txt
icc2_shell> set_attribute [get_vias *] shape_use user_route
```

2. 创建布线规则。在本例中，AP 是 RDL 金属层，MB 是背面金属层。

```
icc2_shell> create_routing_rule Wide \
   -widths {AP 5 MB 5 M1 2 M2 2 M3 2 M4 2}
{Wide}
icc2_shell> set_routing_rule $nets -rule Wide
```

3. 在 RDL 层上把凸点布线到着陆通孔。

```
icc2_shell> set_app_options -name \
 flip_chip.route.bump_via_landing_mode \
   -value true
icc2_shell> route_3d_rdl -nets $nets -layer AP
icc2_shell> set_app_options -name \
 flip_chip.route.bump_via_landing_mode \
   -value false
```

4. 用 route_group 命令布线中介层上 M1 到 Mn 层的网络。

```
icc2_shell> set_ignored_layers -min_routing_layer M1 \
 -max_routing_layer M4
icc2_shell> route_group -nets $nets
```

5.（如需要）布线背面层上的网络。

```
icc2_shell> route_3d_rdl -nets $nets -layer MB
```

7.9.4 图案文件示例

Pattern File Example

下列示例给出一个 single_hbm 图案文件。图案文件包含一个或多个区段，每个区段以一个关键字开始，后跟一对花括号内指定的一条或多条约束。可以通过添加约束（关键字与值）来修改图案文件，以指定额外的布线规则。

```
config {
{via_def 0 VIA3AP_1cut}
}
```

```
exception {
{hbm_bump_array_box {5194.670 4225.350} {6337.725 4362.85}
{9119.675 4387.700} {10357.175 4525.2}}
{hbm_right_array_routing_layer M3}
(hbm_left_array_routing_layer M2}
{hbm_wire_width 1.8}
{hbm_wire_spacing 3.5}
{hbm_channel_direction horizontal}
{hbm_mid_shielding HBM_VSS}
{hbm_middle_turn_offset 72.0}
{hbm_boundary_stub_corners auto_align}
{hbm_via_bump_abutment false}
{hbm_rdl_via_offset 0}
{hbm_stub_alignment_type staggered_before}
}
```

7.9.5 在高带宽存储器设计中创建 RDL 网络屏蔽

Creating RDL Net Shields in High-Bandwidth Memory Designs

可以在同一层上屏蔽网络（称为侧壁屏蔽，side-wall shielding），也可以在该层的上方与下方屏蔽网络（称为同轴屏蔽，coaxial shielding）。同轴屏蔽比侧壁屏蔽提供更好的信号隔离，但占用更多布线资源。在高带宽存储器设计中，最好组合使用侧壁屏蔽与同轴屏蔽。在此流程中，`create_interposer_routeplan` 命令创建线段桩（布线引导），并尽可能为屏蔽线留出空间。

为高带宽存储器设计创建 RDL 布线并屏蔽这些布线的步骤如下：

1. 设置 `flip_chip.route.shielding_net` 应用选项，以定义屏蔽网络连接。

```
icc2_shell> set_app_options -name flip_chip.route.shielding_net \
   -value VSS
```

2. 创建布线规则。

```
icc2_shell> create_routing_rule rdlrule -widths {MRDL 6 M9 5}
```

3. 把布线规则分配给 RDL 网络。

```
icc2_shell> set_routing_rule $rdl_nets -rule rdlrule
```

4. 布线 RDL 网络。

```
icc2_shell> route_3d_rdl -layer {MRDL} -nets $rdl_nets
```

5. 创建布线规则，并指定屏蔽间距与屏蔽宽度。

```
icc2_shell> create_routing_rule shieldrule -shield_spacings {M9 2} \
   -shield_widths {M9 5} -widths {MRDL 6 M9 5}
```

6. 把布线规则分配给 RDL 网络。

```
icc2_shell> set_routing_rule $rdl_nets -rule shieldrule
```

7. 创建 RDL 网络屏蔽。

```
icc2_shell> create_rdl_shields -layers {M9} -reference_layer {MRDL} \
   -offset {8} -nets $rdlnets
```

要只在特定区域内布线以加快布线速度，使用 `-coordinates` 选项并指定矩形列表。

7.10 3DIC 术语表

3DIC Glossary of Terms

3DIC 设计中使用下列术语与约定：

- 中介层（Interposer）：提供到封装的连接并安装各裸片的专用硅裸片。无源中介层通常不含逻辑，但包含 TSV、C4 凸点、微凸点与布线。
- 有源裸片（Active dies）：用倒装芯片技术正面朝下接触中介层的裸片。
- 正面（Front side）：硅裸片的顶面。正面与衬底和器件层相对，大部分标准单元金属布线在正面完成。
- 背面（Back side）：硅裸片的底面。底面包含衬底和器件层，背面金属层在此处创建。
- 硅通孔（Through-silicon via）：穿过衬底把背面金属与器件层连接到正面金属的通孔。硅中介层上需要 TSV，以便把信号从封装衬底经中介层传播到各裸片。
- 微凸点（Microbump）：正面或背面、带有单个引脚的单元。该单元包含用于外部接触的 RDL 金属与钝化开口。微凸点可用于到另一颗裸片的信号以及电源和地连接。
- C4 凸点（C4 bump）：正面或背面、带有单个引脚、用于连接到封装的单元。该单元包含用于外部接触的背面金属与钝化开口。C4 凸点用于到封装的信号以及电源和地连接。
  C4 表示可控塌陷芯片连接（Controlled Collapse Chip Connect）。

Chapter 8

管理设计块

Managing Design Blocks

为管理设计块，IC Compiler II 工具支持若干操作，以便在设计流程早期方便地划分设计，并把逻辑层次单元提交（commit）为物理层次块。提交为块之后，可以为设计块创建多个经过优化的抽象视图（abstract view），其中只包含执行布局、时序及其他任务所需的信息。这种方法使你能够尽量降低高效分布并处理超大规模设计所需的系统需求。

提交块并创建抽象的流程如图 **??** 所示。

更多细节见下列主题：

- 探索设计层次
- 创建模块边界
- 提交设计块
- 创建块布局抽象
- 更改块边界与块原点
- 在顶层与块层之间移动对象
- 为网表编辑生成 ECO 脚本

8.1 探索设计层次

Exploring the Design Hierarchy

IC Compiler II 工具提供用于探索设计层次、并确定要把哪些逻辑层次单元提交为物理层次块的工具。图形界面中的层次浏览器（Hierarchy Browser）显示设计层次的视图，并提供每个层次单元的其他信息。要用层次浏览器探索设计层次：

1. 在图形界面中选择 **View > Hierarchy Browser**。
2. 点击层次单元旁边的 [+] 符号以展开或折叠层次。
3.（可选）点击并拖动列标题以重新排列表格。

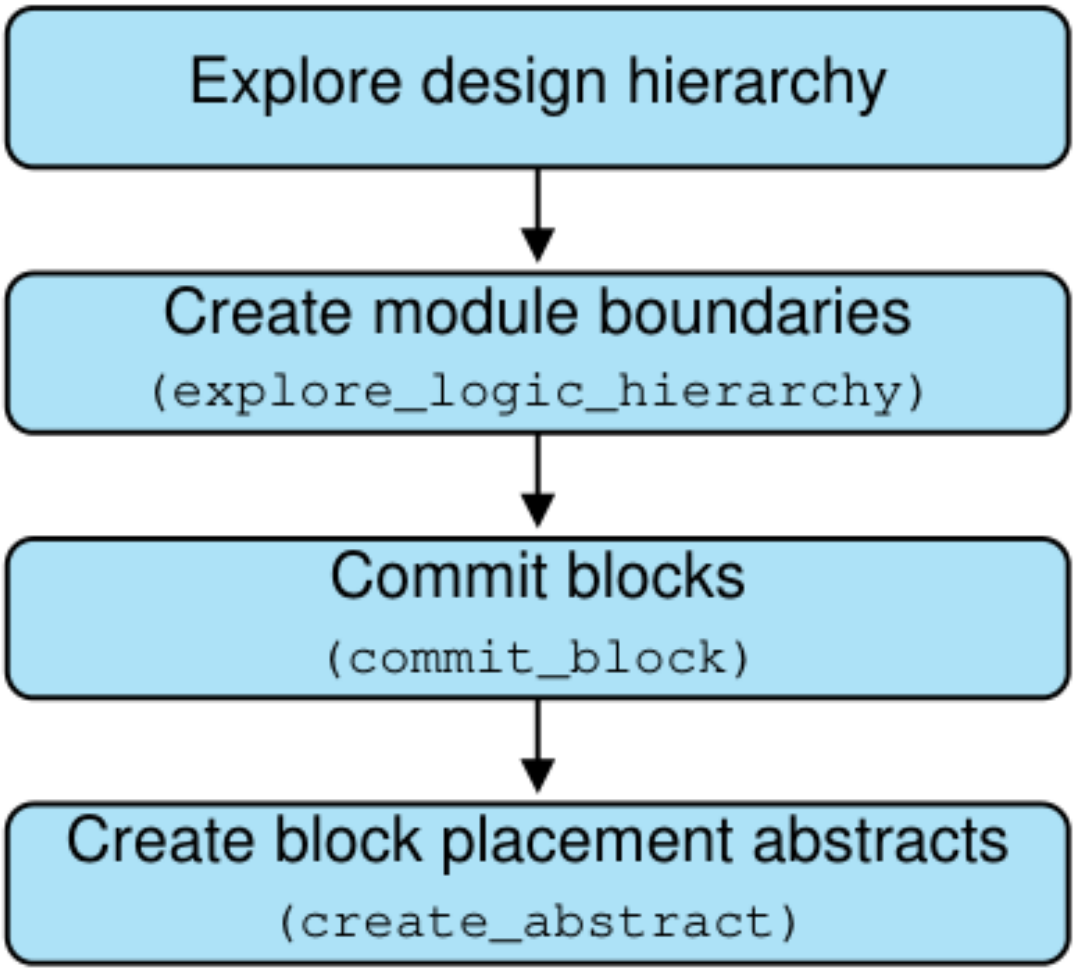

Explore design hierarchy
Create module boundaries
(explore_logic_hierarchy)
Commit blocks
(commit_block)
Create block placement abstracts
(create_abstract)

图 **8.1:** 管理块的流程

4. 点击块名以对该块执行其他操作。

层次浏览器中显示的层次示例如图 **??** 所示。

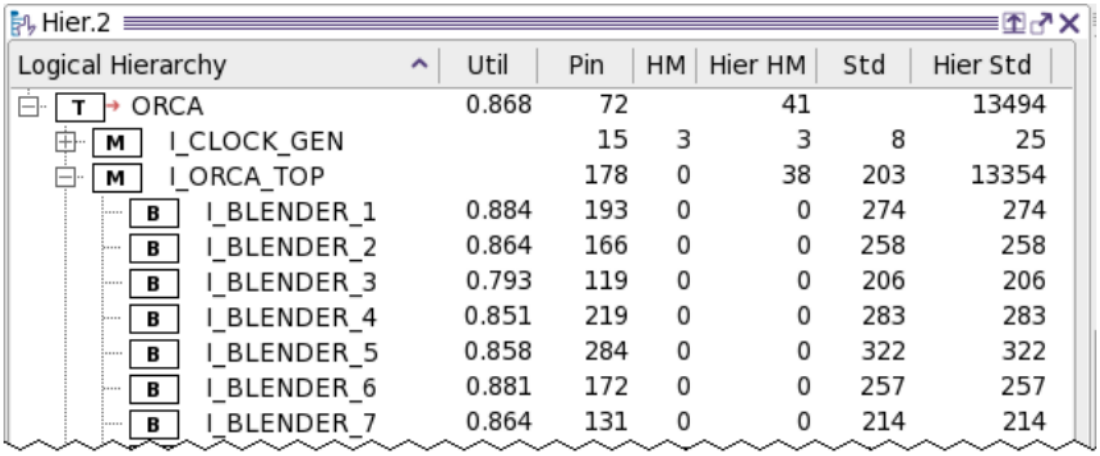


图 **8.2:** 层次浏览器

层次浏览器显示单元名、利用率、引脚数、硬宏数、整个层次中的标准单元数以及其他信息。

8.2 创建模块边界

Creating Module Boundaries

在把逻辑层次单元提交为物理层次块之前，可以在图形界面中创建模块边界（module boundary）。模块边界表示逻辑层次中尚未提交的单元（模块）。利用模块边界，可以快速评估顶层块在给定利用率下所需的近似块尺寸，并在布图规划内手工排列模块边界，以便粗略估计块的布局。还可以设置不同参数，以控制在此视图中哪些顶层块被当作模块边界。

要创建模块边界：

1. 使用 `explore_logic_hierarchy` 命令，并指定要为其创建模块边界的单元名。

```
icc2_shell> explore_logic_hierarchy -create_module_boundary \
   -cell { I_ORCA_TOP/I_BLENDER_1 I_ORCA_TOP/I_BLENDER_2 \
   I_ORCA_TOP/I_BLENDER_3 I_ORCA_TOP/I_BLENDER_4 \
   I_ORCA_TOP/I_BLENDER_5 I_ORCA_TOP/I_BLENDER_6 \
   I_ORCA_TOP/I_BLENDER_7 }
```

工具根据你指定的选项创建模块边界。默认情况下，工具根据顶层布图规划的利用率确定边界尺寸。创建模块边界之后，工具更新图形界面中的显示，如图 **??** 所示。原始布图规划边界显示在左侧，I/O 位于布图规划边界上方，模块边界放置在右下方，任何未布局的宏单元放置在模块边界上方。可以用 `-nested` 选项保留父模块边界，并从父层次内部创建额外的模块边界。指定此选项时，工具保留父级模块边界，并为更低一层的层次创建新的模块边界。下列示例创建两个模块边界：一个用于 `I_ORCA_TOP`，一个用于 `I_ORCA_TOP/I_BLENDER_0`。

```
icc2_shell> explore_logic_hierarchy -create_module_boundary \
   -cell {I_ORCA_TOP}
icc2_shell> explore_logic_hierarchy -create_module_boundary \
   -cell {I_ORCA_TOP/I_BLENDER_0} -nested
```

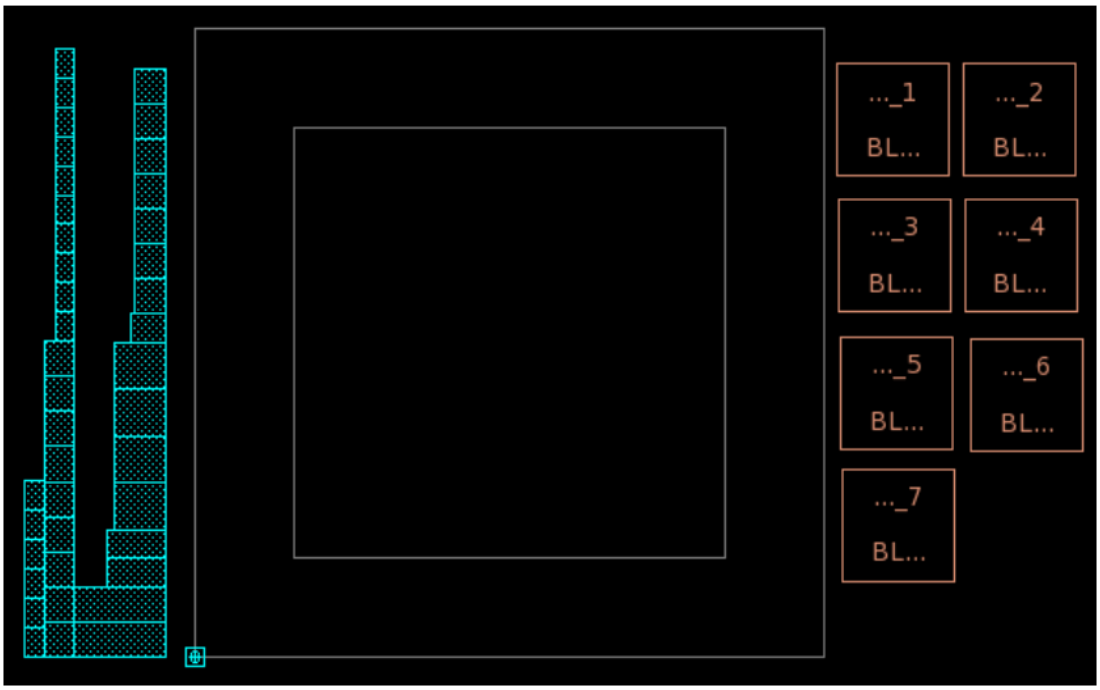


图 **8.3:** 模块边界

2.（可选）选择一个模块边界并把它移入布图规划边界，以试验不同的块布局，如图 **??** 所示。

3. 重置模块边界位置，或移除模块边界，以探索其他层次。

```
icc2_shell> explore_logic_hierarchy -organize
icc2_shell> explore_logic_hierarchy -remove
```

`-organize` 选项把模块边界移到布图规划之外、由 `explore_logic_hierarchy -create_module_boundary` 命令创建的位置，如图 **??** 所示。`-remove` 选项移除所有模块边界。也可以把 `-remove` 与 `-cell` 选项组合使用，以移除特定的模块边界。

4. 为其他单元创建模块边界，重复该流程，以探索不同的层次。

要用图形界面创建初始模块边界：

1. 选择 **View > Assistants > Hierarchy Exploration** 打开图形界面中的 Hierarchy Exploration 面板，或在 **Task Assistant** 中选择 **Floorplan Preparation > Hierarchy Exploration**。
2. 选择要为其创建模块边界的块。
3. 从上下文菜单中点击 **Create Module Boundary**，如图 **??** 所示。

工具为所选逻辑层次创建模块边界。

8.2.1 为扁平设计创建模块边界

Creating Module Boundaries for Flat Designs

`explore_logic_hierarchy` 命令也可以为扁平设计创建模块边界。把 `plan.place.hierarchy_by_name` 应用选项设置为 `true`；这使 `explore_logic_hierarchy` 命令能够按名称提取虚拟层次，并为这些虚

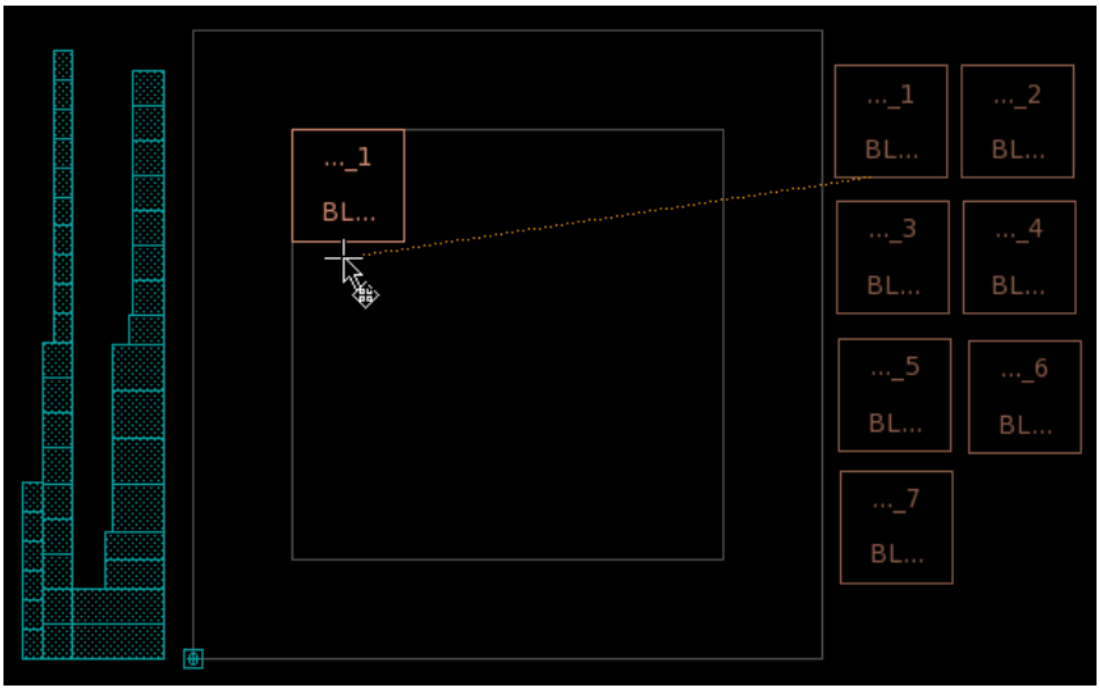


图 **8.4:** 移动模块边界

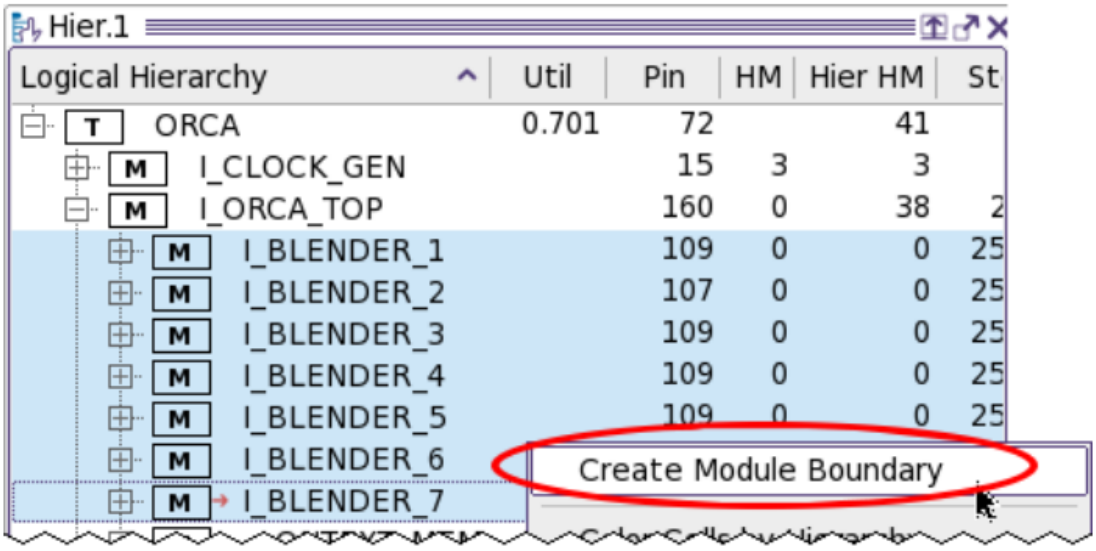


图 **8.5:** 在层次探索器中创建模块边界

拟层次创建移动边界（move bound）。在创建新的移动边界之前，会移除任何已有的模块边界。要为每个模块创建单独的移动边界，用 -cell 选项显式指定这些模块，如下所示：

```
icc2_shell> explore_logic_hierarchy -create_module_boundary \
   -cell {I_ORCA_TOP/I_PCI_TOP I_ORCA_TOP/I_SDRAM_TOP}
```

要创建单个移动边界以把两个或更多模块编为一组，按如下方式指定 -virtual_group 选项：

```
icc2_shell> explore_logic_hierarchy -name group1 \
   -virtual_group {I_ORCA_TOP/I_PCI_TOP I_ORCA_TOP/I_SDRAM_TOP}
```

按如下方式使用 get_attribute 命令，报告与分组移动边界关联的各个模块：

```
icc2_shell> get_attribute [get_bounds group1] -name flat_hier_names
```

8.3 提交设计块

Committing Design Blocks

在 IC Compiler II 设计规划流程中，在设计过程早期把逻辑层次单元提交为物理层次块。可以利用单元的连接关系、单元尺寸以及其他因素，确定要提交哪些逻辑层次单元。决定要把哪些逻辑层次单元提交为物理层次块之后，可以把它们转换为抽象表示，用于布局、电源规划、引脚放置与时序预算。交互工具通过显示不同模块之间的网络连接来支持你的分析。

要提交设计中的一个块：

1. （可选）创建一个新的设计库来容纳该块，并保存该库。这样每个块都可以利用分布式处理并行处理。

   ```
   icc2_shell> create_lib ../lib/orca -technology orca.tf \
      -ref_libs $libs
   icc2_shell> save_lib
   icc2_shell> set_ref_libs -add orca
   ```

2. 提交该块。块被保存到指定的设计库。

   ```
   icc2_shell> commit_block -library orca ORCA
   ```

在本例中，该命令为名为 orca 的块创建新的物理层次，并把该块写入 orca 设计库。图形界面更新为显示已提交块的轮廓视图。

8.4 创建块布局抽象

Creating Block Placement Abstracts

把逻辑层次单元提交为物理层次块之后，可以创建块布局抽象，为块塑形与初始宏单元布局做准备。块布局抽象是物理层次块的轻量表示，包含硬宏、原始标准单元数量与面积信息，以及足以表示块 I/O 接口的标准单元与寄存器。要创建块布局抽象，必须加载顶层设计的完整网表表示，而不包含已提交块的细节。

要创建块布局抽象：

1. 设置约束映射文件。

```
icc2_shell> sh cat split/mapfile
BLENDER_0  CLKNET  BLENDER_0/clocknets.tcl
BLENDER_0  SDC     BLENDER_0/top.tcl
BLENDER_0  UPF     BLENDER_0/top.upf
...
BLENDER_6  CLKNET  BLENDER_6/clocknets.tcl
BLENDER_6  SDC     BLENDER_6/top.tcl
BLENDER_6  UPF     BLENDER_0/top.upf
ORCA       CLKNET  ORCA/clocknets.tcl
ORCA       SDC     ORCA/top.tcl
ORCA       UPF     TOP/top.upf
```

约束映射文件由 split_constraints 命令自动生成，指定块的列表及其关联的 SDC 与 UPF 约束文件。更多信息见「划分约束的输出文件」。

2. 确认当前加载的顶层块表示是 outline。若当前表示已经是 design，则进入第 3 步。

```
icc2_shell> current_design
{orca:ORCA.outline}
```

current_design 命令报告当前块名（ORCA）、表示（outline）以及包含该设计的设计库（orca）。

3. 用 expand_outline 命令展开顶层块的轮廓表示，并保存设计库。

```
icc2_shell> expand_outline
...
icc2_shell> current_design
{orca:ORCA.design}
icc2_shell> save_lib -all
```

在本例中，expand_outline 命令读入网表并展开顶层。请注意，current_design 命令报告的表示从 outline 变为 design。

注意

要使该命令正确工作，必须把原始 Verilog 文件保持在 read_verilog_outline 命令所引用的同一位置，或者使该文件可在 search_path 中找到。

4. 为分布式计算设置主机选项。

```
icc2_shell> set_host_options -name block_script host_settings
```

注意

可以用 check_host_options 命令确认当前的主机选项设置。

5. 利用分布式处理，用下列命令为所有块创建块布局抽象。

```
icc2_shell> create_abstract -placement \
   -host_options block_script -all_blocks
```

```
Submitting job for block BLENDER_0 ...
Submitting job for block BLENDER_1 ...
...
Running distributed create_abstract ...
...
1
```

使用 abstract.allow_all_level_abstract 与 plan.flow.design_view_only 应用选项，控制层次块的抽象创建。当 abstract.allow_all_level_abstract 应用选项为 true 时，只要该子块的 plan.flow.design_view_only 应用选项为 false，就可以为层次中任意一级的任何子块创建抽象视图。当 abstract.allow_all_level_abstract 应用选项为 false 时，工具只允许为 plan.flow.design_view_only 应用选项设置为 false 的最低层块创建抽象视图。

省略 -host_options 选项，即可不使用分布式处理来创建块布局抽象。若只为一部分块创建块布局抽象，使用 -blocks {blocks} 选项代替 -all_blocks 选项。

8.5 更改块边界与块原点

Changing the Block Boundary and Block Origin

通过设置 boundary 属性，或用图形界面更改块的形状或位置来重塑块时，origin 属性不会改变。origin 属性指定块原点，若重塑或重新放置该块，可能需要修改它。

要连同块边界一起更改块原点，使用 set_block_boundary 命令并指定 -origin 选项。若省略 -origin 选项，该命令把原点放在该块边界框的左下角。

```
icc2_shell> set_block_boundary -cell U0 \
   -boundary {{2500 100} {3000 100} {3000 1000} {2500 1000}} \
   -origin {2500 100} -orientation R0
Information: Block instance U0 boundary is set. (DPP-057)
1
```

若只更改当前顶层块的原点，使用 move_block_origin 命令。你指定的位置相对于当前原点。例如，要把当前顶层块的原点移到块的中心，而当前原点位于左下角、块的边界框为 {0 0} {1000 1000}，使用下列命令：

```
icc2_shell> move_block_origin -to {500 500}
```

8.6 在顶层与块层之间移动对象

Moving Objects Between the Top and Block Levels

可以在顶层与块层之间移动单元、凸点区域、布线、布线引导、布线走廊（routing corridor）、阻挡（blockage）和引脚引导等对象，如下列主题所述：

- 将对象从顶层下推到块层
- 将对象从块层上弹到顶层

移动对象是为了利用完全布线的信号或线段桩作为引脚标记来规划层次接口，并移动没有关联网络的布线对象。

8.6.1 将对象从顶层下推到块层

Pushing Objects Down From the Top Level to the Block Level

要把对象从顶层移到块层，使用 set_push_down_object_options 与 push_down_objects 命令。默认情况下，由 push_down_objects 命令创建的对象带有后缀 _PUSHDOWN_n。要为端口名与网络名指定不同的后缀，使用 plan.pins.new_port_name_tag 应用选项。要为单元名指定不同的后缀，使用 plan.pins.new_cell_name_tag 应用选项。

要把对象从顶层下推到块层：

1. 用 set_push_down_object_options 命令指定要移动的对象类型以及如何复制它们。

```
icc2_shell> set_push_down_object_options -object_type signal_routing \
   -block_action {copy create_pin_shape}
```

关于下推对象选项的信息，见「下推对象选项」。

2.（可选）用 report_push_down_object_options 命令确认选项设置。

```
icc2_shell> report_push_down_object_options \
   -object_type signal_routing
****************************************
Report : report_push_down_object_options
****************************************
--- push_down_objects options for signal_routing ---
Top action: remove
Block action(s): { copy create_pin_shape }
Routing overlap check: false
Ignore misalignment: false
```

3.（可选）在下推到块层之前，用 check_objects_for_push_down 命令检查网络是否存在潜在布线问题。

```
icc2_shell> check_objects_for_push_down \
   [get_nets -of_objects [get_cells CLK_BUFFER*]]
0 nets determined to have problems
Overall runtime: 0.008u 0.000s 0:00.01e 100.9%
```

check_objects_for_push_down 命令检查将要下推的布线或充电站（charging station）是否存在潜在问题。该命令检查共线布线与通孔、与网络逻辑定义不兼容的物理布线问题，以及顶层不完整的布线。充电站会针对潜在的 UPF 问题进行检查。check_objects_for_push_down 命令在检查设计时使用由 set_push_down_object_options 命令指定的选项设置。

4. 用 push_down_objects 命令把指定对象从顶层下推到块层。

```
icc2_shell> push_down_objects \
   [get_nets -of_objects [get_cells CLK_BUFFER*]]
Examined 8 nets for push-down
Pushed down routing for 8 nets
Overall runtime: 0.008u 0.000s 0:00.01e 106.6%
```

当下推来自顶层的布线和其他对象，或在块上创建穿越时，会创建一个影子网表（shadow netlist），其中包含被下推或被创建的对象。在接收新对象的块内部，工具插入这些对象，并且：

- 把这些对象上的 `is_shadow` 属性设置为 `true`。从顶层下推的穿越，其穿越网络的引脚、网络与缓冲器上的 `is_shadow` 属性被设置为 `true`。
- 把 `shadow_status` 属性设置为 `pushed_down`。其他可能的影子状态属性值为 `copied_down`、`copied_up`、`normal`、`pulled_up`、`pushed_down` 与 `virtual_flat`。

参见

- 下推对象选项
- 下推穿越网络
- 下推期间用通孔位置作为新引脚位置
- 将下推对象写入网表

下推凸点区域

Pushing Down Bump Regions
只能下推方向为 R0、MX、MY 或 R180 的凸点区域。
当下推凸点区域时，工具会：

- 重新计算新区域的边界、图案偏移、方向、节距与交错，使它们从顶层看起来与之前相同
- 切割任何重叠的块边界
- 检查多次例化块的对齐。若多个区域重叠且属性不同，则不下推这些凸点区域，工具显示一条警告信息
- 把与 TSV 相关的信息添加到凸点区域的伪凸点。工具还会把此信息映射到下推块内新创建的凸点区域。该映射确保下推之后 TSV 与伪凸点的对齐保持不变

下推之后：

- 对新创建的顶层区域，关联的网络矩阵保持原样；对子区域，则映射到所连接的子网络与端口
- 顶层区域锚定到相同的对象与位置

下推之前与下推之后的对照如下：下推之后，工具把凸点区域切成三部分，并把每一部分下推到一个重叠的块中。本例为两层多次例化物理层次（MPH）块，块的方向为 R180。凸点区域的方向为 R180，并具有双值边界边距、图案偏移、变化的节距、交错、网络矩阵以及一个锚点。

下推对象选项

Push Down Object Options
用下列选项配合 `set_push_down_object_options` 命令，控制 `push_down_objects` 命令的行为。

- 用 `-object_type` 选项指定对象类型。`set_push_down_object_options` 命令支持下列对象类型：`pg_routing`、`signal_routing`、`routing_guide`、`routing_corridor`、`blockage`、`pin_guide`、`pin_blockage`、`cells`、`charging_station`、`bump_region` 与 `marker_layer`。

```
icc2_shell> set_push_down_object_options -object_type {cells blockage}
```

- 用 -top_action 选项指定对象在顶层如何处理。合法的顶层动作为 keep 或 remove。

```
icc2_shell> set_push_down_object_options -object_type blockage \
   -top_action keep
```

- 用 -block_action 选项指定对象在块层如何处理。合法的块层动作视 -object_type 选项指定的对象类型而定，为 center_pin_from_wire、copy、create_blockage、create_pin_shape、create_route_blockage_from_cell 与 retain_obsolete_ports。

```
icc2_shell> set_push_down_object_options -object_type cells \
   -top_action remove -block_action {copy retain_obsolete_ports}
```

- 用 -routing_overlap_check 选项检查与已有布线对象的重叠。当用 -object_type 选项指定 pg_routing 与 signal_routing 对象类型时，可以使用此选项。

```
icc2_shell> set_push_down_object_options \
   -object_type pg_routing -block_action {create_pin_shape copy} \
   -routing_overlap_check true
```

- 用 -ignore_misalignment 选项，在把未对齐的电源地布线对象下推到多次例化块时发出警告信息。

```
icc2_shell> set_push_down_object_options \
   -object_type pg_routing -block_action {copy create_pin_shape} \
   -ignore_misalignment true
```

- 通过指定 -allow_multi_rail_cells 选项，下推具有多条电源轨的多电压单元。

```
icc2_shell> set_push_down_object_options -object_type cells \
   -allow_multi_rail_cells true
```

- 用 -collinear_margin 选项，把位于块边界之外、但在指定距离设置之内的平行电源地带状线视为共线布线。请注意，这些电源地带状线在子块中作为原始顶层带状线的副本被创建，原始带状线在顶层保持不动。

```
icc2_shell> set_push_down_object_options \
   -object_type pg_routing -block_action {copy create_pin_shape} \
   -ignore_misalignment true -collinear_margin 2.1 \
   -pin_meet_fatwire_rule true
```

- 用 -pin_meet_fatwire_rule 选项调用粗线间距规则，如上例所示。
- 用 -location_based_terminal_naming 选项指定该命令是否根据引脚的层与位置创建唯一的引脚名。

下推穿越网络

Pushing Down Feedthrough Nets

若块包含穿越网络，push_down_objects 命令会在穿越网络所跨越的较低层块上创建一个电气等效端口。在顶层，该命令把顶层网络连接到块上这个新的电气等效引脚。若要在较低层块中创建穿越端口而不是电气等效端口，对 create_pin_constraint 命令指定 -allow_feedthroughs true 选项。若在下推期间只允许某些类型的网络上存在穿越，按如下示例应用过滤器：

```
icc2_shell> create_pin_constraint -type individual \
   -nets [get_nets * -filter "net_type == signal"] \
   -allow_feedthroughs true
```

在下列示例中，push_down_objects 命令把与布线走廊 corridor_a 关联的所有网络下推到该网络所跨越的块中。请注意，必须启用穿越创建。图 ?? 显示下推该布线之前与之后的设计块示意图。工具在 Block2 中创建纯穿越，并在网络与块边界的交点处为详细布线的网络创建物理引脚。

```
icc2_shell> push_down_objects [get_nets \
   -of_objects [get_routing_corridors corridor_a]]
```

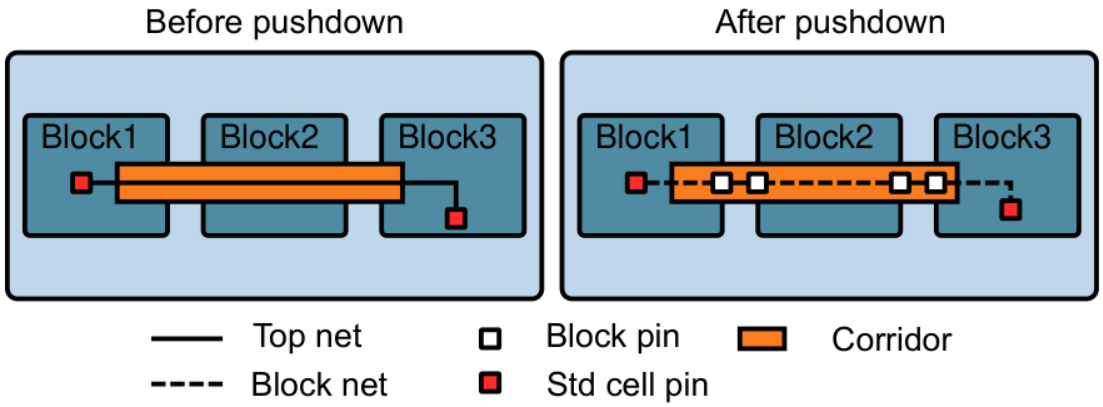


图 **8.6:** 下推全局布线

参见

创建单独的引脚约束

下推期间用通孔位置作为新引脚位置

Using Via Locations as New Pin Locations During Push Down

默认情况下，下推与网络相关的对象时，push_down_objects 命令按需要在块边上创建引脚。为避免在块边上创建引脚，而把通孔当作正在下推的网络上的新引脚位置，请执行下列步骤。请注意，把通孔位置用作引脚位置，有助于限制为包含多次例化块的设计所创建的新引脚位置数量。

1. 对 set_push_down_object_options 命令指定 -block_action {copy create_pin_shape} 选项。

```
icc2_shell> set_push_down_object_options \
   -object_type signal_routing -top_action remove \
   -block_action {copy create_pin_shape}
```

2. 把通孔下推到 -cells 选项指定的块中。

```
icc2_shell> push_down_objects [get_vias VIA_S_5] -cells u1/u2
...
Examined 1 nets for push-down
Pushed down routing for 1 nets
...
```

将下推对象写入网表

Writing Pushed Down Objects to the Netlist

push_down_objects 命令设置 is_shadow 与 shadow_status 属性，以标识被下推的对象。

- is_shadow 属性被设置为 true。
- shadow_status 属性被设置为 pushed_down 或 copied_down。

默认情况下，write_verilog 命令把影子网表对象写出到较低层块的网表。可以用 write_verilog 命令的 -include shadow_netlist 或 -exclude shadow_netlist 选项明确包含或排除这些对象。

若要以 ECO 脚本的形式而不是影子网表的形式写出影子信息，使用 write_shadow_eco 命令。该 ECO 脚本可供 IC Compiler II 工具这类物理设计工具使用；该脚本也可以为 Design Compiler 工具这类前端工具提供有用信息。请注意，Design Compiler 工具消费 ECO 脚本的方式与 IC Compiler II 工具不同。面向 IC Compiler II 工具时使用 -command_style icc2 选项，面向 Design Compiler 工具时使用 -command_style dc 选项。

用下列步骤查询较低层块中被下推的对象：

1. 打开包含被下推对象的块。

```
icc2_shell> open_block lib/myblock:myblock.design
```

2. 返回被下推单元与网络的列表。

```
icc2_shell> get_cells -filter is_shadow
{x0_mem0_ram0_0_x0__PUSHDOWN x0_mem0_ram0_0_x1__PUSHDOWN ...

icc2_shell> get_nets -filter is_shadow
{from_top_1____PUSHDOWN_0 ...
```

3. 按需要检查上一条命令返回的对象上的 shadow_status 属性。

```
icc2_shell> get_attribute \
   [get_cells x0_mem0_ram0_0_x0__PUSHDOWN] shadow_status
pushed_down

icc2_shell> get_attribute [get_nets from_top_1____PUSHDOWN_0] \
   shadow_status
pushed_down
```

参见

将对象从顶层下推到块层

8.6.2 将对象从块层上弹到顶层

Popping Objects Up From the Block Level to the Top Level

要把对象从块层移到顶层，使用 set_pop_up_object_options 与 pop_up_objects 命令。
要把对象从块层上弹到顶层：

1. 用 set_pop_up_object_options 命令指定要移动的对象类型以及如何复制它们。

```
icc2_shell> set_pop_up_object_options -object_type blockage \
   -block_action keep
```

关于上弹对象选项的信息，见「上弹对象选项」。

2.（可选）用 report_pop_up_object_options 命令确认选项设置。

```
icc2_shell> report_pop_up_object_options -object_type blockage
****************************************
Report : report_pop_up_object_options
****************************************
--- pop_up_objects options for blockage ---
Block action: keep
```

3. 用 pop_up_objects 命令把特定单元中的布局阻挡上弹到顶层。
在下列示例中，来自 cell34 的布局阻挡被上弹到顶层。

```
icc2_shell> pop_up_objects [get_placement_blockages cell34/*]

--- Process cell34: Popping up blockages from block block1 ---
Block action: keep
Number of Placement blockages processed: 6
Overall runtime: 0.017u 0.000s 0:00.02e 103.7%
```

对象被上拉之后，工具把 shadow_status 属性设置为 pulled_up。其他可能的影子状态属性值为 copied_down、copied_up、normal、pulled_up、pushed_down 与 virtual_flat。

上弹对象选项

Pop Up Object Options
用下列选项配合 set_pop_up_object_options 命令，控制 pop_up_objects 命令的行为。

- 用 -object_type 选项指定对象类型。合法的对象类型为 pg_routing、signal_routing、routing_guide、routing_corridor、blockage、pin_guide、pin_blockage、cells、charging_station、bump_region 与 marker_layer。

```
icc2_shell> set_pop_up_object_options -object_type {cells blockage}
```

- 用 -block_action 选项指定对象在块层如何处理。合法的块层动作为 keep 或 remove。

```
icc2_shell> set_pop_up_object_options -object_type cells \
   -top_action remove -block_action {keep}
```

- 用 -routing_overlap_check 选项检查与已有布线对象的重叠。当用 -object_type 选项指定 pg_routing 与 signal_routing 对象类型时，可以使用此选项。

```
icc2_shell> set_pop_up_object_options -object_type pg_routing \
   -block_action {keep} \
   -routing_overlap_check true
```

- 用 -pins_as_terminals 选项指定 pop_up_objects 命令是否在父级创建顶层端子。

上弹凸点区域

Popping Bump Regions
上弹凸点区域时，工具会：

- 把子凸点区域作为顶层上各自独立的区域上弹，并且不合并它们
- 重新计算新区域的边界、图案偏移、方向、节距与交错，使它们从顶层看起来与之前相同
- 把任何与 TSV 相关的映射信息复制到顶层块，并把它与每个被上弹的凸点区域关联

若块层动作设置为：

- keep：上弹之后，工具不移除原始的块层凸点区域
- remove：上弹之后自动移除子凸点区域
- keep：新区域的影子状态属性根据块层动作设置为 copied_up 或 pulled_up

对于顶层动作，允许的值只有 copy，它在上弹时创建一个新的顶层凸点区域。

8.7 为网表编辑生成 ECO 脚本

Generating ECO Scripts for Netlist Editing

可以创建工程变更指令（ECO）脚本来对网表做下列更改。使用 ECO 脚本提供了便利并增强了能力。

- 将物理多扇出网络的分支上推到顶层
- 将标准单元树下推一层
- 在子块上创建备用端口与网络

8.7.1 将物理多扇出网络的分支上推到顶层

Push Up the Branching of a Physical Multiple-Fanout Net to the Top Level

物理多扇出网络是连接到多于一个物理块的网络。要生成 ECO 脚本，把物理多扇出网络的分支移到顶层，使用 write_split_net_eco 命令。默认情况下，该命令拆分块中的所有物理多扇出网络。

要拆分特定网络，用 -nets 选项指定这些网络。网络必须连接到多个物理块引脚才能被拆分。若某条网络符合拆分条件，分支网络会被移到尽可能高的层次。此命令既移动逻辑封装的分支，也移动物理网络的分支。

默认情况下，该命令把 ECO 脚本写到控制台。要把脚本写入 Tcl 文件，用 -filename 选项指定文件名。用 source 命令载入该脚本。

下列各图显示物理多扇出网络的分支在被上推到顶层之前与之后的情况。

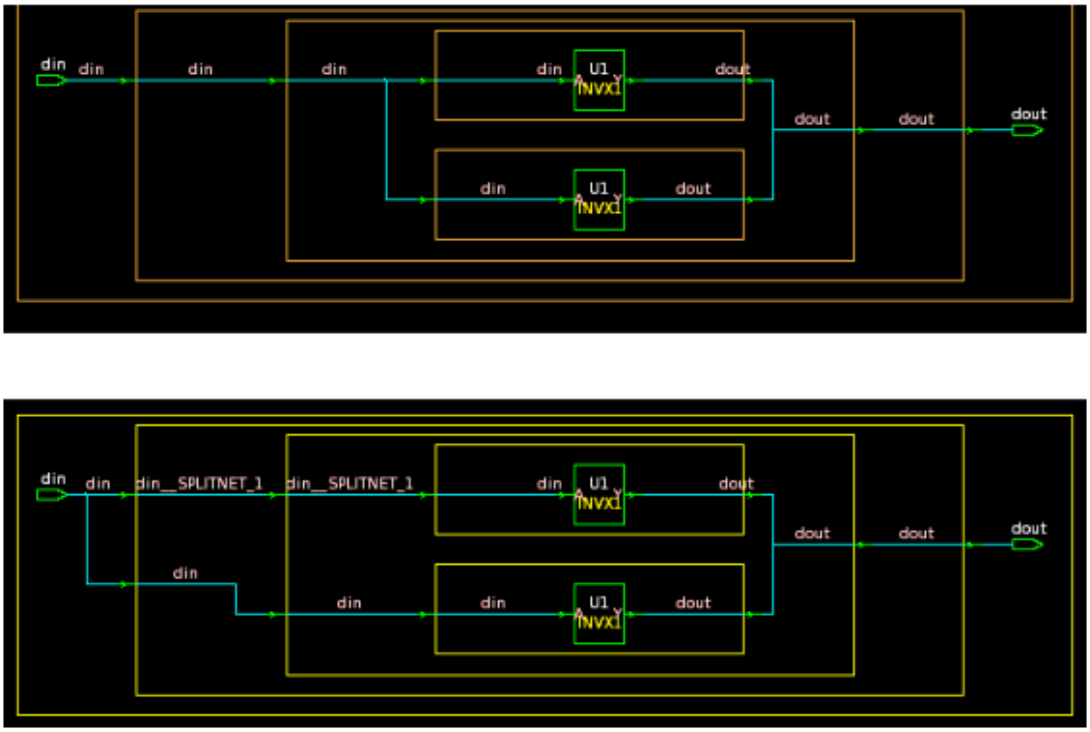


图 **8.7:** 上推到顶层之前与之后

8.7.2 将标准单元树下推一层

Push Down a Tree of Standard Cells One Level

要生成 ECO 脚本，把一棵标准单元树从顶层物理层向下移动一层、进入下一物理层上与之重叠的块，使用 write_push_down_eco 命令。该脚本提供下推或门树的方法；push_down_objects 命令不具备此能力。

使用 write_push_down_eco 命令时，指定要下推的单元。只有当一个单元完全落在某个块的边界之内时，才能把它下推到该块中。

工具把标准单元下推到块中时，会在该块上创建新端口。原始端口不会被复用。任何未使用的输入端口都被绑到地（VSS）。

在物理布图规划中，三个 or2 门全部与 block_0 重叠。载入 ECO 脚本之后，这些单元被下推到 block_0 块中。

write_push_down_eco 命令为每个块生成一个 ECO 文件。默认情况下，ECO 文件被写到名为 push_down_eco_scripts 的目录。可以用 -dir 选项指定目录名。要为所有块运行 ECO 脚本，用 source 命令载入顶层块的 Tcl 脚本。

8.7.3 在子块上创建备用端口与网络

Create Spare Ports and Nets on a Child Block

要生成 ECO 脚本，在子块上创建备用（哑）端口与网络，使用 write_spare_ports_eco 命令并指定：

- 用 cell_list 参数给出的单元列表。
- 用 -name 选项给出的端口名。默认情况下，该命令使用 dummyFT。
- 用 -direction 选项给出的端口方向，指定 in、out 或 inout。默认值为 in。
- 用 -bits 选项给出的总线宽度。

创建端口与网络之后，该命令创建顶层网络，并把它们连接到子块的引脚。

write_spare_ports_eco 命令为每个块生成一个 ECO 文件。默认情况下，ECO 文件被写到名为 create_spare_ports 的目录。可以用 -dir 选项指定目录名。要为所有块运行 ECO 脚本，用 source 命令载入顶层块的 Tcl 脚本。

下列示例为 U* 单元实例写出脚本，并指定名为 spareFT 的新端口：

```
icc2_shell> write_spare_ports_eco [get_cells U*] -direction in \
          -dir test_path -bits {0 4} -name spareFT
```

Chapter 9

执行块整形与宏布局

Performing Block Shaping and Macro Placement

块整形（block shaping）流程根据层次探索期间定义的粗略矩形或直线多边形（rectilinear）外形，细化块的边界（boundary）。创建块外形时，工具考虑目标利用率（utilization）、通道宽度（channel width）与禁布（keepout）等设计约束，同时尽量减少馈通（feedthrough）并缩短接口线长（interface wire length）。可以为设计创建可选的块网格（block grid）；工具会把块外形对齐到该网格。

块整形流程如图 **??** 所示。

更详细的说明见下列主题：

- 设置宏约束
- 为宏与宏阵列创建相对布局约束
- 设置宏禁布区
- 在相连块之间为宏预留空间
- 创建宏阵列
- 整形块
- 用 Tcl 约束执行块整形
- 为块整形创建通道约束
- 为多次实例化块创建块网格
- 创建初始宏布局
- 以岛式放置宏
- 使宏远离块边界
- 执行基于机器学习的宏单元布局
- 使用宏布局助手
- 执行时序驱动与拥塞驱动布局

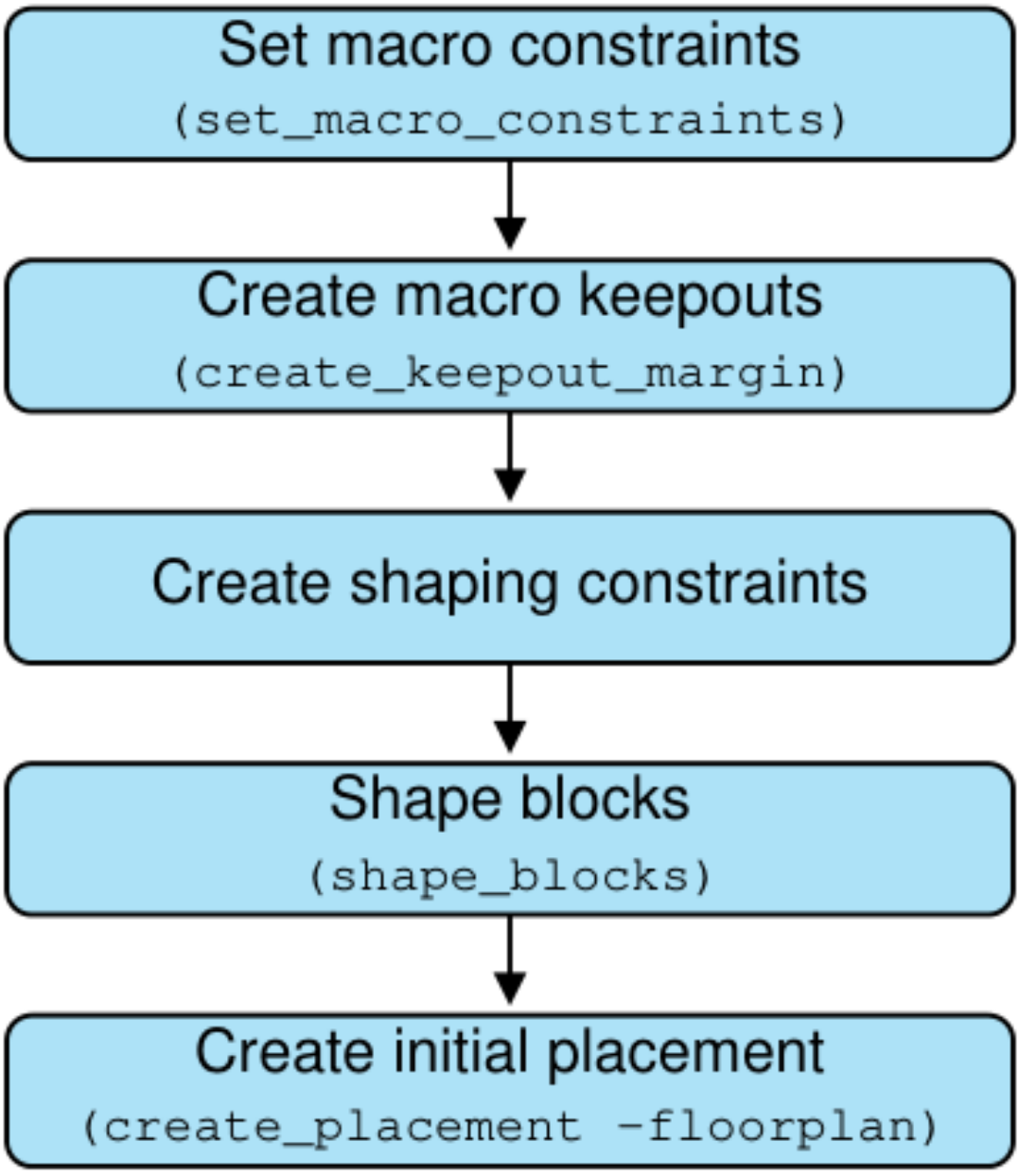

Set macro constraints
(set_macro_constraints)
Create macro keepouts
(create_keepout_margin)
Create shaping constraints
Shape blocks
(shape_blocks)
Create initial placement
(create_placement -floorplan)

图 **9.1:** 块整形流程

9.1 设置宏约束

Setting Macro Constraints

块整形之前，可以为部分或全部块与宏（macro）指定宏布局约束与禁布边距。布局约束限制宏可以被放置的方向（orientation）。设置宏布局约束的步骤如下：

1. 创建宏单元集合。下例把一个变量赋给包含设计中全部宏单元的集合。

```
icc2_shell> set macro_cells [get_cells -physical_context \
   -filter "is_hard_macro && !is_physical_only" -quiet]
```

2. 用适当参数指定 set_macro_constraints 命令。下例把全部硬宏（hard macro）的布局方向限制为 R0 或 R180。

```
icc2_shell> set_macro_constraints \
   -allowed_orientations {R0 R180} $macro_cells
```

3.（可选）用 report_macro_constraints 命令核对当前约束设置。对 report_macro_constraints 使用 -allowed_orientations、-preferred_location、-alignment_grid、-align_pins_to_tracks 中的一个或多个选项，可以把报告限制为只含这些约束。

```
icc2_shell> report_macro_constraints
****************************************
Report : report_macro_constraints
Design : ORCA
****************************************
  macro     allowed           legal
  name      orientations      orientations
  ----------------------------------------
  I_ORCA_TOP/I_RISC_CORE/I_REG_FILE/REG_FILE_D_RAM
            R0,R180           all
  I_ORCA_TOP/I_RISC_CORE/I_REG_FILE/REG_FILE_C_RAM
            R0,R180           all
...
```

用 set_macro_constraints 命令的下列选项约束设计中硬宏与 I/O 单元的布局。表 **??** 汇总这些选项。

表 **9.1:** 指定宏约束选项

要完成的操作	使用此选项
指定一组允许的方向值，以进一步限制参考库中规定的合法布局方向	-allowed_orientations {list}
把宏的信号管脚对齐到管脚图形所属层的布线轨道上来放置宏。按需求使用 -align_pins_to_tracks 或 -alignment_grid grid_name 之一，不要同时使用	-align_pins_to_tracks

表 **9.1:** 指定宏约束选项（续）

要完成的操作	使用此选项
把宏对齐到指定的布局网格。按需求使用 `-alignment_grid grid_name` 或 `-align_pins_to_tracks` 之一，不要同时使用	`-alignment_grid grid_name`
只把同一条命令中指定的 `-alignment_grid` 或 `-align_pins_to_tracks` 应用于以指定方向集合放置的宏。方向集合为 R0：{R0, R180, MX, MY}，或 R90：{R90, R270, MXR90, MYR90}	`-alignment_orientation_set R0 | R90 | all`
指定宏上必须落在网格上的一个位置。若不指定坐标，默认值为 {0 0}。本选项须与 `-alignment_grid grid_name` 一起使用，否则工具忽略 `-alignment_point {x y}` 设置	`-alignment_point {x y}`
为指定宏指定首选布局位置，其中 `x_range` 与 `y_range` 的取值从 0 到 1	`-preferred_location {x_range y_range}`
当应用选项（application option）`plan.macro.style` 设为 `hybrid` 时，指定放置每个宏所用的风格。合法值为 `auto`、`on_edge` 与 `freeform`。默认值为 `auto`，表示由工具决定沿边界放置还是以自由形态放置。自动选择依据宏的尺寸以及被阻挡的布线层数。工具把沿边选择限制为那些不应位于标准单元区域内的宏。关于 `plan.macro.style` 应用选项的更多信息见 man 页	`-style on_edge | freeform | auto`

9.2 为宏与宏阵列创建相对布局约束

Creating Relative Placement Constraints for Macros and Macro Arrays

设计规划期间，宏与标准单元（standard cell）由 `create_placement -floorplan` 命令放置。要把某个宏或宏阵列（macro array）约束为相对于设计中另一个对象放置，使用 `set_macro_relative_location` 命令。`create_placement -floorplan` 命令遵守 `set_macro_relative_location` 所设约束，并按距锚点对象（anchor object）的指定偏移距离放置目标宏或宏阵列。

创建宏布局约束的步骤如下：

1. 用 `set_macro_relative_location` 命令设置约束。

```
icc2_shell> set target .../I_RISC_CORE/I_REG_FILE/REG_FILE_D_RAM
icc2_shell> set anchor .../I_RISC_CORE/I_REG_FILE/REG_FILE_B_RAM
icc2_shell> set_macro_relative_location \
   -target_object [get_cells $target] \
   -target_orientation R0 -target_corner bl \
   -anchor_object [get_cells $anchor] -anchor_corner tl \
   -offset {0.0000 35.2550} -offset_type fixed
```

2.（可选）用 report_macro_relative_location 命令确认约束设置正确。

```
icc2_shell> report_macro_relative_location
****************************************
Report : report_macro_relative_location
Design : ORCA
****************************************
Macro relative location constraints in block ORCA/dp:
------------------------------------------------------------
target  target       target  anchor  anchor  anchor  offset...
name    orientation  corner  name    type    corner  X     ...
------------------------------------------------------------
.../I_RISC_CORE/I_REG_FILE/REG_FILE_D_RAM
        R0           bl      .../I_RISC_CORE/I_REG_FILE/REG_FILE_B_RAM
                                      macro   tl      0.0000 ...
```

图 **??** 是运行 set_macro_relative_location 与 create_placement -floorplan 命令之后的版图。

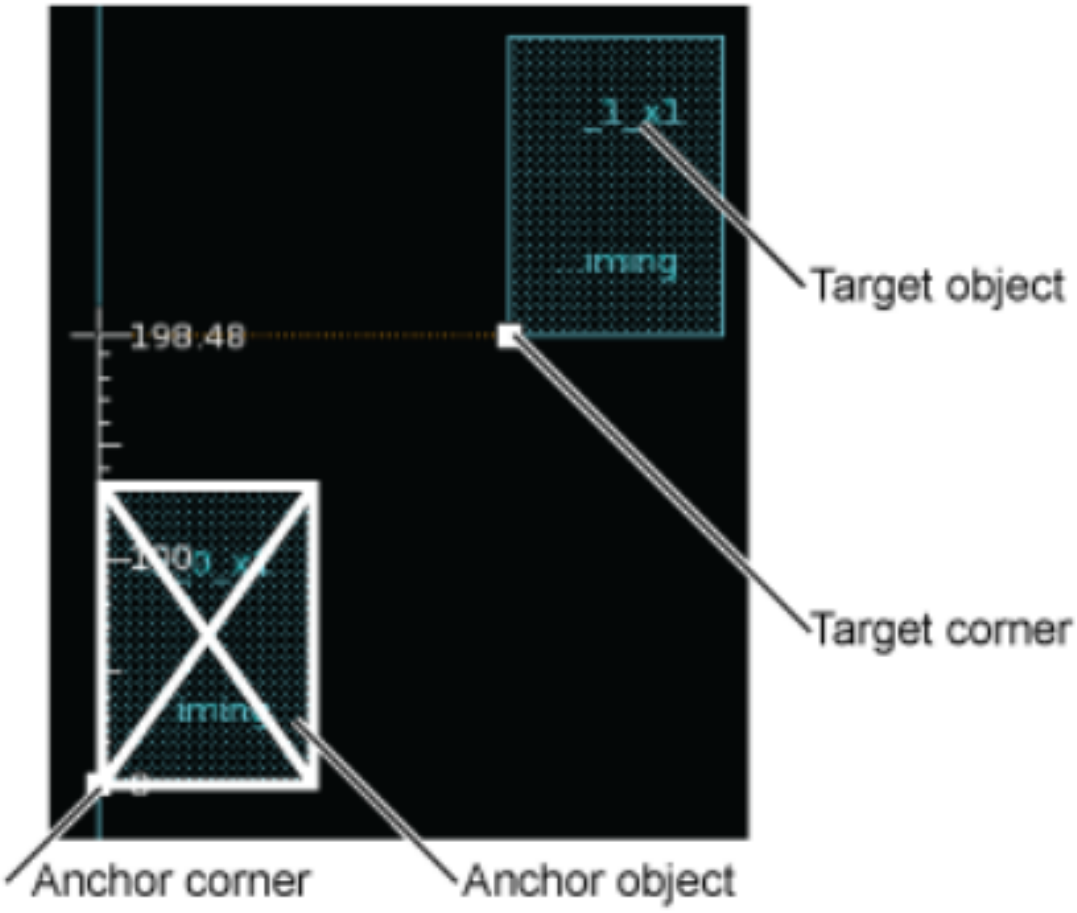


图 **9.2:** 宏相对位置

set_macro_relative_location 命令把对象的布局约束为相对于锚点对象。若未指定锚点对象，则约束相对于包含目标对象的父级设计。支持下列锚点对象：

- 硬宏（Hard Macro）

- 宏阵列（Macro Array）
- 块（Block）
- I/O 焊盘（I/O Pad）
- 块管脚（Block Pin）
- 核心区域（Core Area）
- 移动约束（Move Bound）
- 电压区域（Voltage Area）

可用下列 `set_macro_relative_location` 命令选项更改相对位置约束：

- 指定创建锚点对象与目标对象之间偏移时使用锚点对象的哪一个角。合法角值为 `bl`（左下）、`br`（右下）、`tl`（左上）与 `tr`（右上）。

```
icc2_shell> set_macro_relative_location -anchor_corner bl ...
```

- 指定用作目标对象参考位置的对象实例名。

```
icc2_shell> set_macro_relative_location -anchor_object {u1/u2} ...
```

- 指定目标对象相对于锚点对象的 x 与 y 偏移。

```
icc2_shell> set_macro_relative_location -offset {200 200} ...
```

- 指定施加约束的目标宏的角。合法角值与 `-anchor_corner` 选项相同。

```
icc2_shell> set_macro_relative_location -target_corner tr ...
```

- 指定要约束的目标宏实例名。

```
icc2_shell> set_macro_relative_location -target_object {u3/u4} ...
```

- 指定目标宏的方向。合法方向为 R0、R90、R180、R270、MX、MXR90、MY 与 MYR90。所指定的方向必须是该宏的允许方向。

```
icc2_shell> set_macro_relative_location -target_orientation R0 ...
```

- 指定相对锚点对象的可缩放偏移比，可为正或负。

```
icc2_shell> set_macro_relative_location -offset_type scalable \
   -offset {0.5 0.5} ...
```

可缩放偏移按下列公式计算：

$$target\ position = anchor\ position + offset \times (scale_edge_length - used_length)$$

- 为 x 方向与 y 方向指定不同的偏移类型。

```
icc2_shell> set_macro_relative_location \
   -offset_type {fixed scalable} ...
```

9.3 设置宏禁布区

Setting Macro Keepouts

禁布边距在块与宏周围保持间距，有助于避免拥塞（congestion）与设计规则检查（design rule check，DRC）错误。下例先创建全部硬宏的集合，再用 create_keepout_margin 命令创建四种禁布边距：

```
icc2_shell> set all_hm [get_cells -hierarchical \
   -filter "is_hard_macro==true"]
icc2_shell> create_keepout_margin -type hard \
   -tracks_per_macro_pin 0.05 $all_hm
icc2_shell> create_keepout_margin -type hard_macro \
   -tracks_per_macro_pin 0.5 $all_hm
icc2_shell> create_keepout_margin -type routing_blockage \
   -layers "M2 M3 M4" -tracks_per_macro_pin 0.05 $all_hm
icc2_shell> create_keepout_margin -type soft \
   -tracks_per_macro_pin 1.0 $all_hm
```

create_keepout_margin 命令提供若干选项来控制禁布边距。要在块或宏外侧创建禁布边距，使用 -outer {left bottom right top} 选项。要在块或宏内侧创建禁布边距，使用 -inner {left bottom right top} 选项。要按指定的布线轨道数创建禁布边距，使用 -tracks_per_macro_pin tracks 选项。使用该选项时，命令把所指定的轨道数乘以宏该侧的管脚数，以计算禁布边距尺寸。可以同时指定 -min_padding_per_macro size 与 -max_padding_per_macro size 选项和 -tracks_per_macro_pin 选项，进一步限制禁布尺寸。要指定施加 route_blockage 禁布的层，使用 -layers {layer_list} 选项。

禁布本身不是布局阻挡（placement blockage），但行为与布局阻挡类似，并随硬宏一起移动。

工具支持四种禁布边距：hard、soft、hard_macro 与 routing_blockage。表 ?? 汇总这四种类型。

表 9.2: 禁布类型汇总

禁布类型	说明
hard（默认）	阻止标准单元被放置到禁布边距之内。
soft	阻止标准单元被放置到禁布边距之内。不过，布局器可以在优化期间把单元移入禁布区。
hard_macro	阻止在禁布边距内放置硬宏或其他硬宏禁布边距。该类型只影响硬宏布局，并保证硬宏之间有最小通道。整个宏与禁布都保持在站点阵列之内。可以用这种禁布类型为通向宏管脚的布线分配空间。
routing_blockage	阻止电源与地（power and ground，PG）布线器插入会挡住宏管脚出线的通孔阵列。该类型需要一份要纳入阻挡的层列表。

9.4 在相连块之间为宏预留空间

Reserving Space for Macros Between Connected Blocks

若宏被放在靠近块边界处，块之间可用的布线资源会减少。要用 create_placement -floorplan 命令控制宏在块边界附近的放置方式，把块上的应用选项 plan.macro.cross_block_connectivity_planning 设为下列值之一：

- hard：工具分析子块之间的连接关系，并避免把宏放在块与块之间边界的某一段上。空出的边可在流程后期用于管脚布局。
- soft：该设置与 hard 约束类似，但创建的是可在需要时被违反的软约束。
- unset：应用选项设置从父块继承。
- false：该约束设置关闭跨块连接规划（cross-block connectivity planning）。

下例在块 BLK1 上创建硬约束，以启用跨块连接规划。

```
icc2_shell> set_app_options -block BLK1 \
   -name plan.macro.cross_block_connectivity_planning -value hard
```

对移动约束与电压区域也可以做类似的连接规划，方法是在对象上设置 interface_connectivity_planning 属性。该属性接受的值与应用选项 plan.macro.cross_block_connectivity_planning 相同。下例在电压区域 VA1 上设置硬接口连接规划约束。

```
icc2_shell> set_attribute -name interface_connectivity_planning \
   -value hard [get_voltage_areas VA1]
```

9.5 创建宏阵列

Creating Macro Arrays

宏阵列可以把两个或更多宏作为单个单元来管理。用宏阵列为组内全部宏强制统一的方向、一致的通道宽度、宏翻转与填充图案。要创建宏阵列，指定 create_macro_array 命令以及阵列的行数与列数。

```
icc2_shell> create_macro_array -num_rows 2 -num_cols 6 [get_selection]
{MACRO_ARRAY_0}
```

创建宏阵列之后，可以用 GUI 中的宏阵列工具旋转并交换宏，也可以用该工具旋转整个宏阵列。要启动宏阵列工具，在宏阵列中选择两个或更多宏，再从菜单选择 **Edit > Macro Array**。宏阵列编辑器如图 **??** 所示。

要移除宏阵列，用 remove_edit_groups 命令移除与该宏阵列关联的编辑组。

```
icc2_shell> remove_edit_groups {MACRO_ARRAY_0}
```

参见

创建初始宏布局，见第 **??** 节。

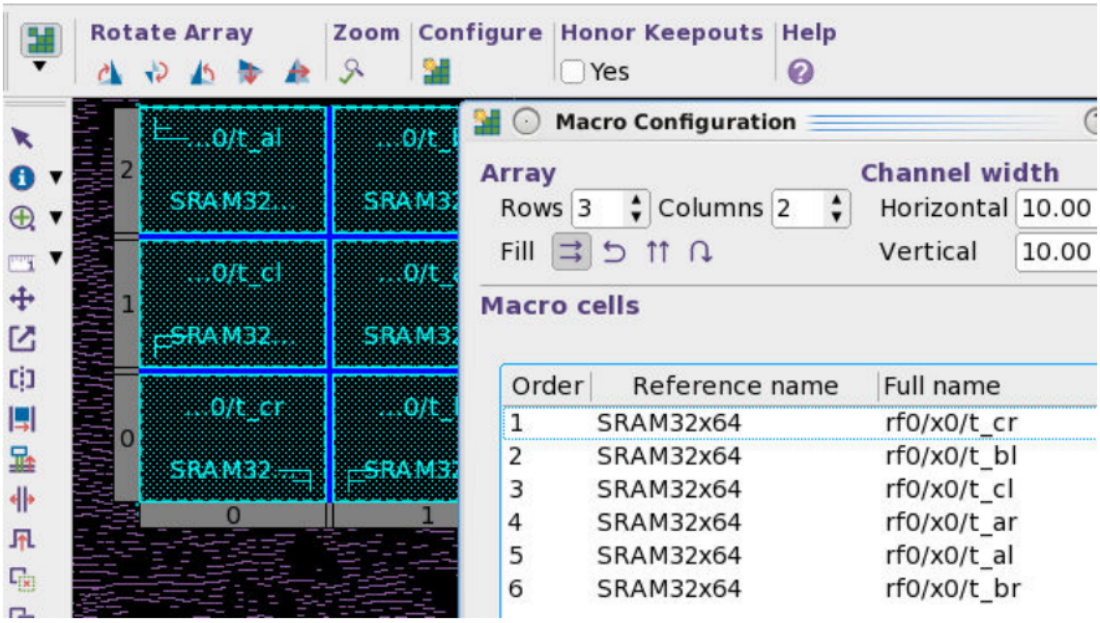


图 **9.3:** 宏阵列编辑器

9.6 整形块

Shaping Blocks

把逻辑层次单元提交为物理层次块之后，可以创建物理层次块的粗略布局。工具根据指定的利用率、通道尺寸与约束，对物理块（包括电源域与电压区域）进行整形并放置。

放置并整形已提交的物理层次块的步骤如下：

1. 用 `set_shaping_options` 命令设置块布局选项。

```
icc2_shell> set_shaping_options -keep_top_level_together true
```

用 `set_shaping_options` 命令设置的任何选项在本次会话中保持有效。可以用 `report_shaping_options` 命令核对当前整形设置。

2. 用 `shape_blocks` 命令整形并放置块。

```
icc2_shell> shape_blocks -channels true
```

本例中，`-channels true` 选项指示命令在块之间插入通道。请注意，`shape_blocks` 命令的选项不会持久保存。图 **??** 给出块整形前后的 floorplan。

3. 用 `report_block_shaping` 命令报告块布局期间发生的违例。

块整形期间，工具尽量减少馈通数量并缩短接口线长。该命令用宏打包来放置顶层宏。若已指定块网格，工具还会把块外形对齐到块网格。若已按「创建块布局抽象」一节所述创建了块布局抽象，可以用这些抽象缩短布局所需的运行时间。

`report_block_shaping -flyline_crossing` 命令评估并比较不同 floorplan 之间的整形结果质量（quality of results，QoR）。使用 `-flyline_crossing` 选项时，工具计算并报告全部块之间的飞线（flyline）交叉。报告中的数值越小，表示网线交叉越少、拥塞越轻。例如，在下图情形中，若从 A

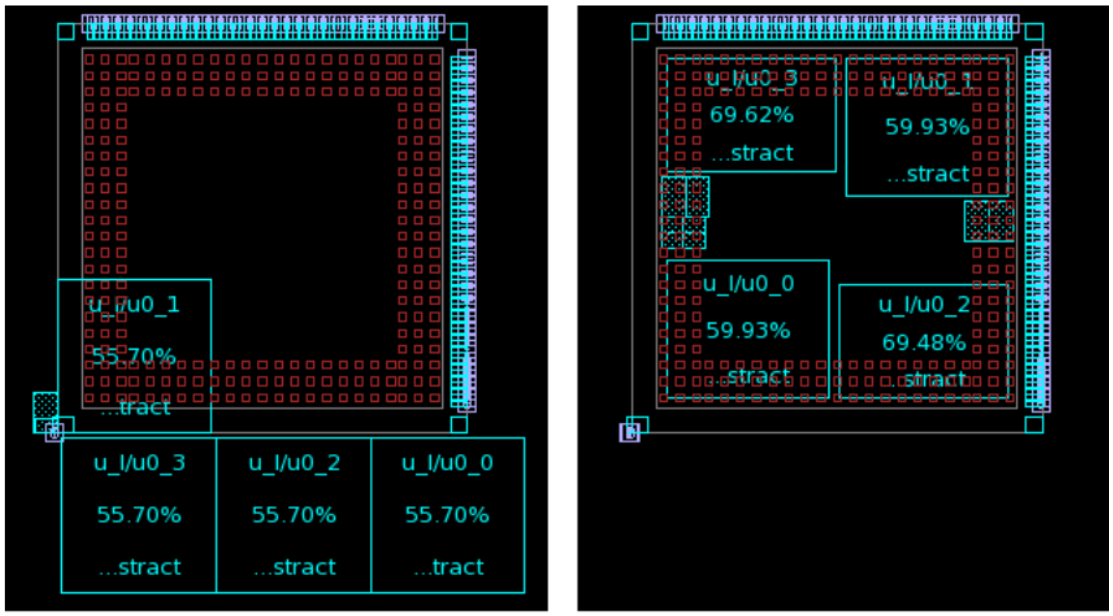


图 **9.4:** 块整形之前（左）与之后（右）

到 D 有 100 条飞线，从 C 到 B 有两条飞线，则该命令报告 2×100 次飞线交叉。若没有从 A 到 D 的飞线，只有从 C 到 B 的飞线，则命令不报告交叉飞线。另外，若相邻块的飞线不与任何其他飞线交叉，则没有可供工具报告的交叉飞线。

```
icc2_shell> report_block_shaping -core_area_violations -overlaps \
   -flyline_crossing
****************************************
Report : report_block_shaping
Design : ORCA
****************************************
  Block core area violation report
  ================================
  Core area violations in design ORCA: 0

  Block overlap report
  ====================
  Block overlaps in design ORCA: 0

  Block crossing flyline report
  =============================
  Maximum flyline crossings is between block pairs
    (I_ORCA_TOP/I_BLENDER_2 I_ORCA_TOP/I_BLENDER_1) and
    (I_ORCA_TOP/I_BLENDER_7 I_ORCA_TOP/I_BLENDER_4): 4x19 = 76
  Flyline crossings in design ORCA: 400
1
```

`report_block_shaping` 命令为当前块整形结果生成 QoR 报告。用该命令检查块重叠、超出核心边界的块，以及过多的飞线交叉。该命令还会给出未对齐管脚数量与网线绕行的估计。

块整形之后可以使用 `report_block_shaping` 命令的大多数选项。不过，该命令的 `-detour_estimate` 选项只应在顶层单元已经放置之后使用，以便得到更准确的 QoR 报告。

-detour_estimate 选项报告由块引起的估计网线绕行数。

4.（可选）从步骤 1 起重复该流程，调整整形选项以得到更高的 QoR。

shape_blocks 命令支持不同选项，用来修改并细化整形结果。用 -channels true | false 选项指定命令是否在不同块之间，或在块与核心边界之间插入通道。用 -constraint_file file_name 选项为每个块施加利用率、首选位置、分组与固定外形约束。用 -incremental congestion_driven | target_utilization_driven 选项按拥塞优先或目标利用率优先做增量整形。

9.6.1 创建块整形约束

Creating Block Shaping Constraints

要为 shape_blocks 命令指定首选的块与电压区域布局位置，创建块整形约束文件，并用 -constraint_file 选项指定该文件。工具根据目标利用率、通道尺寸以及文件中指定的其他约束，确定指定区域的尺寸与位置。

整形约束文件支持下列语法：

```
# Global shaping constraints
utilization { target: 0.2; }
guard_band { width: 0; height: 0; }
channel_size { size=5; }

# TOP constraints
# block constraints cannot be nested
block TOP {
    # Define channel constraints for TOP
    # and blocks within TOP
    # Channel constraints can be defined for
    # block_inst, voltage_area, group, or current_block
    channel_size {
        # Define constraint for TOP/U1
        block_inst U1;
        # Default channel is 20 micron
        # You can also specify "min = 20, max = 20"
        size = 20;
        # Left and right channels are between 30 and 50 microns
        left, right: min = 30, max = 50;
    }
    # Create a guard band constraint for voltage area VA1
    # of 50 microns, and set the voltage area boundary
    # to {{0 0} {300 600}}
    voltage_area VA1 {
        contents: voltage_area VA2, block_inst U3;
        # Define spacing around voltage area VA2
        guard_band {
            width: 50; height: 50;
        }
        boundary {
            type: rigid;
```

```
            shape: {{0 0} {300 600}};
        }
    }
    # Define spacing around any other voltage area
    # beneath TOP
    guard_band {
        width: 40;
        height: 40;
    }
    # Define a group that contains a set of blocks or
    # voltage areas within TOP
    define_group GROUP1 {
        # Organize blocks and voltage areas in a row or column
        # Legal directions are:
        # east (from left to right)
        # west (from right to left)
        # north (from bottom to top)
        # south (from top to bottom)
        arrange_in_array {
            contents: block_inst U1, block_inst U2, voltage_area VA3;
            direction: east;
        }
        # Specify a relative placement location
        # for instance U4 in the middle of TOP block,
        # alignment_point is optional
        arrange_in_box {
            block_inst U4;
            location: x = 0.5, y = 0.5;
            alignment_point: x = 1, y = 0;
        }

        # aspect_ratio specifies the relative width
        # and height. Use aspect_ratio together with
        # arrange_in_box
        aspect_ratio {
            block_inst U1;
            target: width = 1, height = 2;
        }
    }
}

# BLOCK2 constraints
block BLOCK2 {
    # Reserve a space between the current block
    # boundary and any regions shaped inside the block
    boundary_channel_size {
        current_block;
        left: min = 3, max = 5;
        right, bottom: min = 5, max = 10;
    }
```

```
    # Specify a 50% target utilization
    # and a maximum of 70%
    utilization {
        target: 0.5;
        max: 0.7;
    }
}
```

下例用整形约束文件组织设计中的块。整形约束文件指定两个组，每组包含五个块。五个块水平堆叠，两个组垂直堆叠。块之间施加宽度为 100 的保护带（guard band）。

```
icc2_shell> sh cat shaping.con
# Shaping constraint file
channel_size { size=100; }
block top {
    define_group UPPER {
        arrange_in_array {
            contents: block_inst u0, block_inst u1, block_inst u2,
               block_inst u3, block_inst u4;
            direction: east;
        }
    }
    define_group LOWER {
        arrange_in_array {
            contents: block_inst u5, block_inst u6, block_inst u7,
               block_inst u8, block_inst u9;
            direction: east;
        }
    }
    define_group TOP {
        arrange_in_array {
            contents: group UPPER, group LOWER;
            direction: south;
        }
    }
}

icc2_shell> shape_blocks -constraint_file shaping.con
```

图 **??** 是运行 shape_blocks 命令之后的版图。

下例为设计中的一个块指定固定边界。边界用 shape: 与 type: rigid 约束指定。坐标是相对的，工具可以把该外形移动到父块中的新位置。块在设计中的位置由 arrange_in_array 或 arrange_in_box 约束指定。

```
icc2_shell> sh cat fixedblock.con
# Shaping constraints with fixed block
channel_size { size=50; }
block block100a {
    boundary {
        type: rigid;
        shape: {{350 500} {350 1360} {550 1360}
```

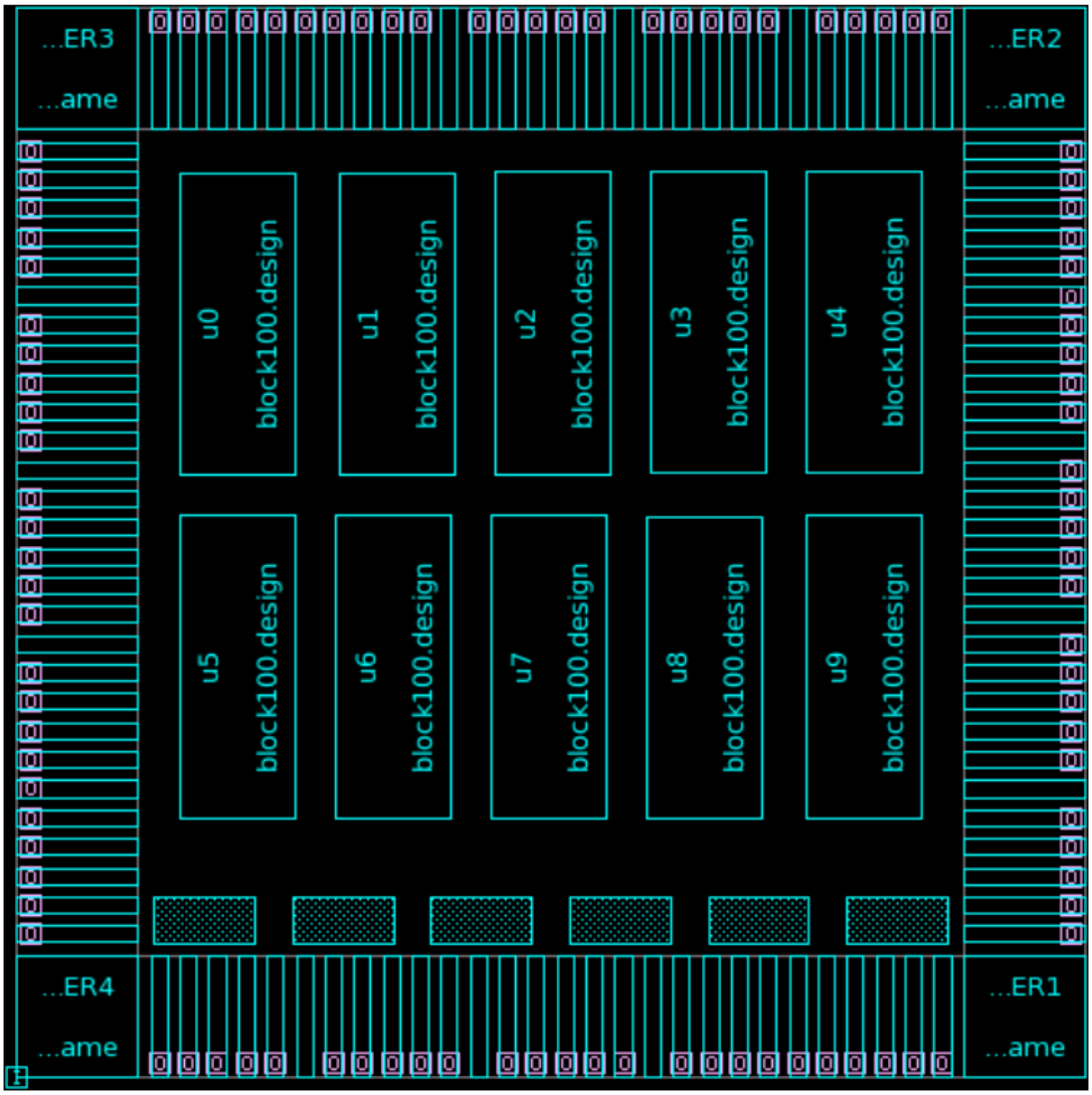


图 **9.5:** 带整形约束的版图

```
                {550 900} {850 900} {850 500}};
    }
}
block top {
    define_group UPPER {
        arrange_in_array {
            contents: block_inst u0, block_inst u1, block_inst u2,
                      block_inst u3, block_inst u4;
            direction: east;
        }
    }
    define_group LOWER {
        arrange_in_array {
            contents: block_inst u5, block_inst u6, block_inst u7,
                      block_inst u8, block_inst u9;
            direction: east;
        }
    }
    define_group TOP {
        arrange_in_array {
            contents: group UPPER, group LOWER;
            direction: south;
        }
    }
}
```

图 **??** 是运行 shape_blocks 命令之后的版图。请注意 block100a 块的外形，它在设计中实例化为实例 u0。

9.7 用 Tcl 约束执行块整形

Performing Block Shaping With Tcl Constraints

除块整形约束文件之外，工具还支持基于 Tcl 的约束。这些约束与 shape_blocks 命令配合，用于指定基于约束的块、阻挡、通道、移动约束与电压区域整形。可以通过指定 Tcl 命令来创建这些约束，也可以用 GUI 在版图中创建约束。

整形约束可以施加于单个对象，或异类对象组成的组。组还可以包含其他组，从而形成层次。用 create_group 命令创建包含组内对象的组，并用 get_groups 与 report_groups 命令查询这些组。

下例创建如图 **??** 所示的十个块的特定排列。块 u1、u2、u3、u4 与 u5 被分组，并在版图上半部分从左到右排列。块 u6、u7、u8、u9 与 u10 从左到右排列，并放在版图下半部分。

1. 把应用选项 plan.shaping.import_tcl_shaping_constraints 设为 true，使 shape_blocks 命令遵守基于 Tcl 的约束。

```
icc2_shell> set_app_options \
   -name plan.shaping.import_tcl_shaping_constraints -value true
```

2.（可选）把应用选项 plan.shaping.report_import_constraints 设为 file、log 或 both，把块整形约束的详细摘要写到控制台、日志文件或二者。该应用选项默认关闭。

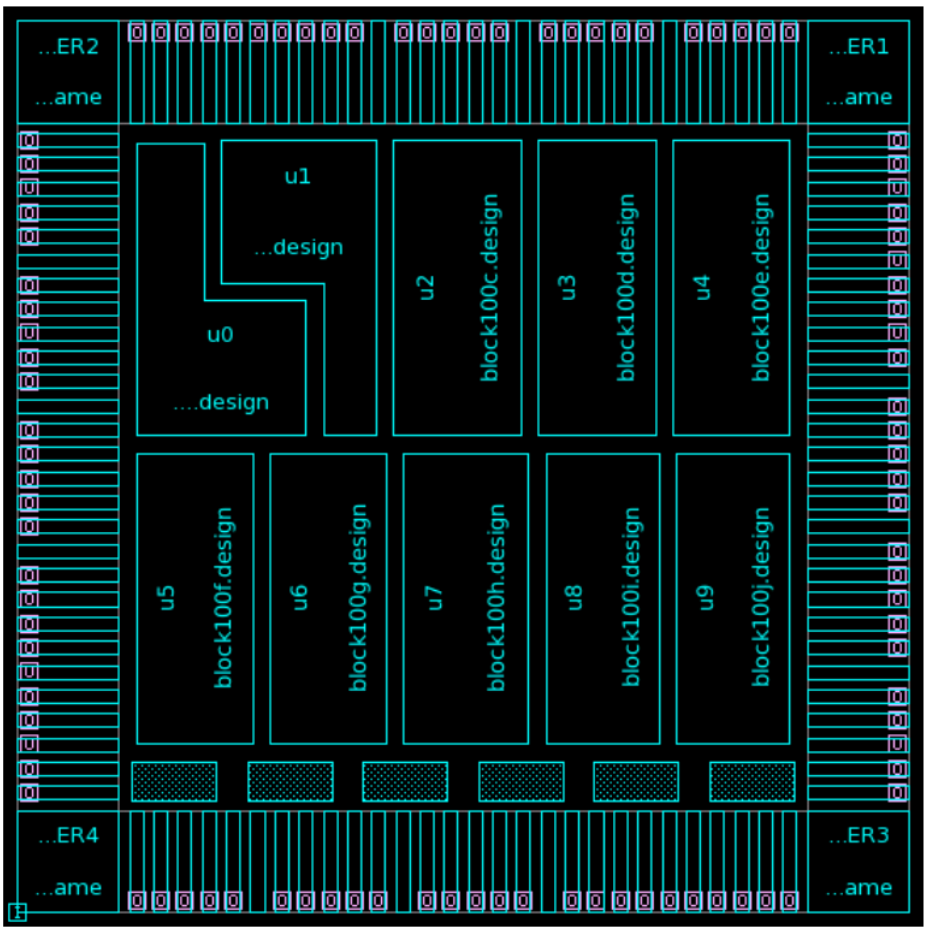


图 **9.6:** 带固定边界的版图

3. 创建一个名为 TOP_ROW 的组以包含上方一组块，再创建一个名为 BOTTOM_ROW 的组以包含下方一组块。

```
icc2_shell> create_group -name TOP_ROW \
   -shaping [get_cells {u1 u2 u3 u4 u5}]
{TOP_ROW}
icc2_shell> create_group -name BOTTOM_ROW \
   -shaping [get_cells {u6 u7 u8 u9 u10}]
{BOTTOM_ROW}
```

4. 创建一个名为 TOP_AND_BOTTOM 的父组，以包含 TOP_ROW 与 BOTTOM_ROW 组。

```
icc2_shell> create_group -name TOP_AND_BOTTOM \
   -shaping [get_groups -shaping {TOP_ROW BOTTOM_ROW}]
{TOP_AND_BOTTOM}
```

注意

可以用 `set_shaping_group_order` 命令修改整形组内对象的顺序。该命令可以移动指定对象，或为整个组指定新顺序。

5. 为 TOP_ROW 组创建一个整形约束，再为 BOTTOM_ROW 组创建另一个整形约束，以指定每组的版图类型。`east` 阵列版图按 `create_group` 命令的 `-shaping` 选项所指定的顺序，把对象从左到右排列。

```
icc2_shell> create_shaping_constraint [get_groups \
   -shaping TOP_ROW] -type array_layout -array_layout east
{TOP_ROW/SHAPING_CONSTRAINT_0_array_layout}
icc2_shell> create_shaping_constraint [get_groups \
   -shaping BOTTOM_ROW] -type array_layout -array_layout east
{BOTTOM_ROW/SHAPING_CONSTRAINT_1_array_layout}
```

6. 创建一个整形约束，指定把 BOTTOM_ROW 组放在 TOP_ROW 组之下。

```
icc2_shell> create_shaping_constraint [get_groups \
   -shaping TOP_AND_BOTTOM] -type array_layout -array_layout south
{TOP_AND_BOTTOM/SHAPING_CONSTRAINT_2_array_layout}
```

7. 用 `shape_blocks` 命令执行块整形。

```
icc2_shell> shape_blocks
```

要移除整形约束，使用 `remove_shaping_constraints` 命令。下列命令移除 u1/u2 块上的全部整形约束：

```
icc2_shell> remove_shaping_constraints u1/u2/*
```

要报告当前块或指定块的整形约束，使用 `report_shaping_constraints` 命令。下例报告当前块及其全部层次嵌套块上、`shape_blocks` 命令可见的全部整形约束：

```
icc2_shell> report_shaping_constraints
```

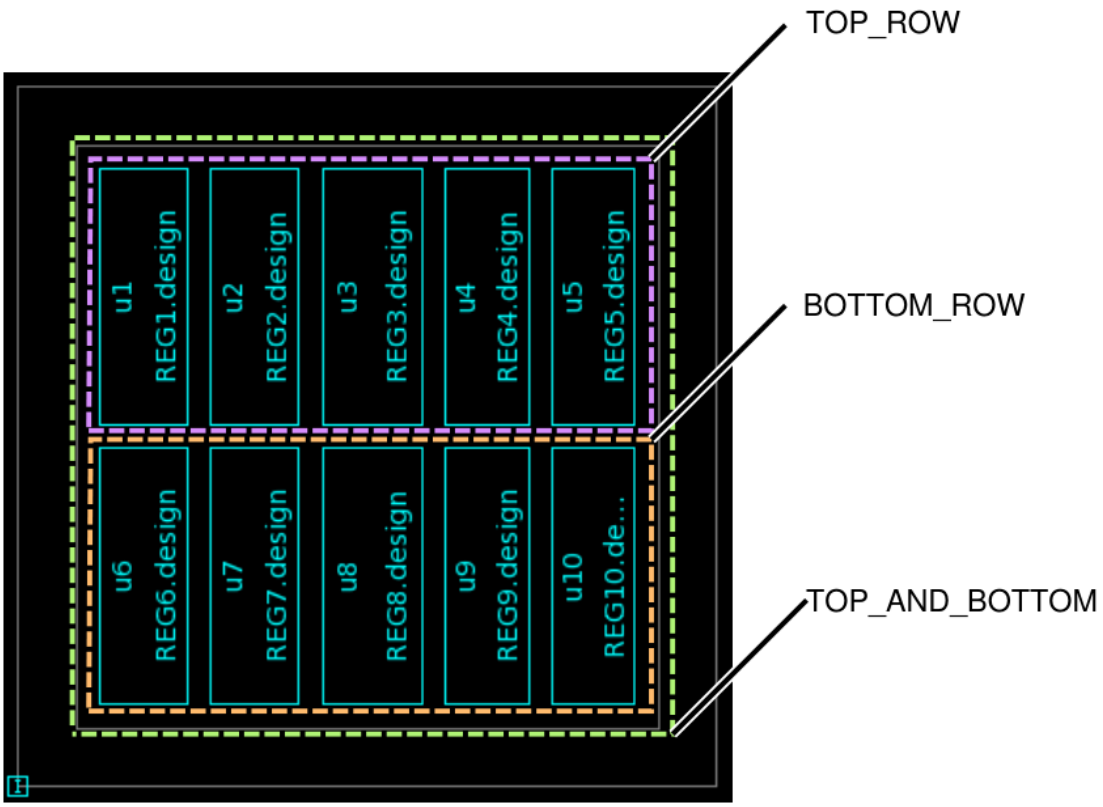


图 **9.7:** 块整形示例

GUI 中的 Design Assistant 显示施加于设计的基于 Tcl 的整形约束。从菜单选择 **View > Assistants > Design Assistant** 打开 Design Assistant。在 Design Assistant 中选择 **View > Shapeable Objects**，查看对象与约束。用该工具创建整形约束、重排整形组中的对象、从整形组中移除对象，以及删除整形组。图 **??** 是上一示例在 Design Assistant 中的可整形对象页。

下例把基于 Tcl 的整形约束施加于两个块和一个电压区域。`create_shaping_constraint` 命令用于在每个被整形对象周围创建 50 微米通道。请注意，这里用 Tcl 的 `list` 命令为 `create_group` 命令的 `-shaping` 选项收集异类对象。

```
set_app_options \
   -name plan.shaping.import_tcl_shaping_constraints -value true

create_group -name G1 \
   -shaping [list [get_cells u0_1] \
                  [get_voltage_areas u_m/PD_LEON3_misc] \
                  [get_cells U2]]

create_group -name USG_1 \
   -shaping [list [get_groups -shaping G1] \
                  [get_cells U1]]

create_shaping_constraint \
   [get_blocks leon3mp.nlib:leon3mp/shaping.design] \
   -type boundary_channels \
   -object_channel [create_shaping_channel -left_min 50 \
   -left_max 50 -bottom_min 50 -bottom_max 50 -right_min 50 \
```

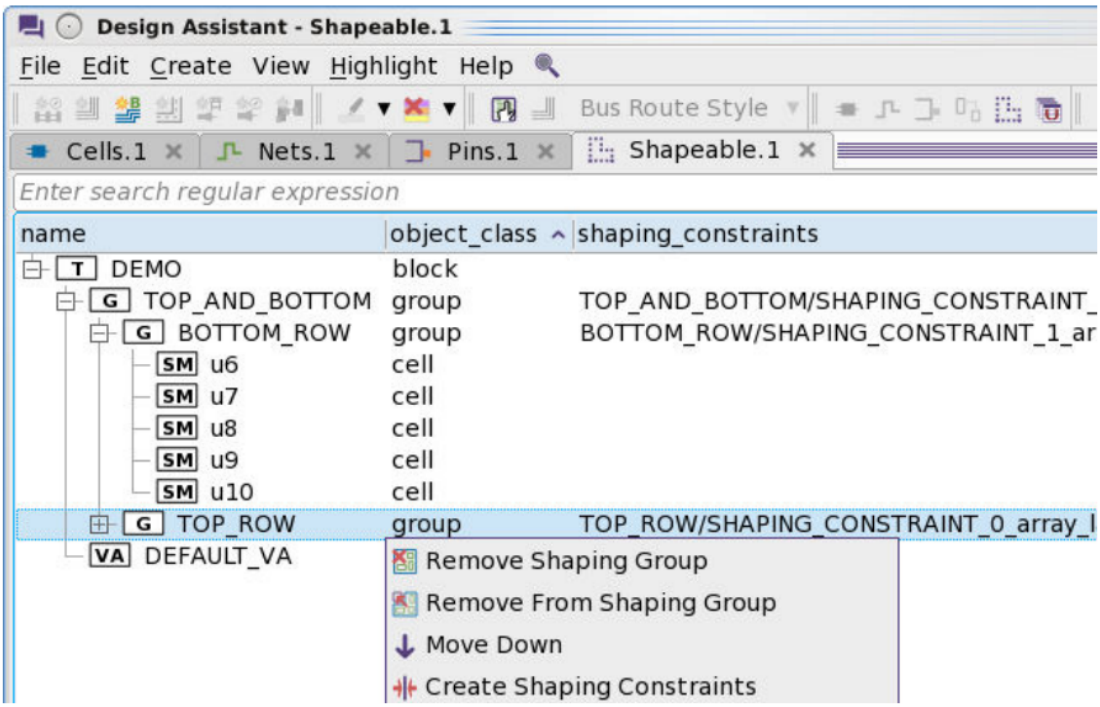


图 **9.8:** Design Assistant 中的可整形对象

```
   -right_max 50 -top_min 50 -top_max 50]

create_shaping_constraint [get_groups -shaping G1] \
   -type array_layout -array_layout east

create_shaping_constraint [get_groups -shaping USG_1] \
   -type array_layout -array_layout south

shape_blocks
```

该示例创建图 **??** 所示的版图。

9.8 为块整形创建通道约束

Creating Channel Constraints for Block Shaping

块整形通道用于在版图中的不同对象之间创造间距，这些对象包括块、电压区域、移动约束与整形组。要创建整形通道，使用 `create_shaping_channel` 命令，并用选项定义对象顶边、底边、左边与右边的最小与最大通道宽度。用 `create_shaping_channel` 命令的 `-neighbor` 选项，指定相对于版图中某个相邻对象的间距约束。创建整形通道之后，用 `create_shaping_constraint` 命令把该通道赋给某个对象。

可以用 `create_shaping_constraint` 命令的 `-type` 选项指定三种通道：

- `boundary_channel`：为顶层设计创建边界通道约束。
- `external_channels`：沿对象各边创建通道约束。请注意，不能把外部通道约束赋给当前顶层块（由 `current_block` 命令返回）。

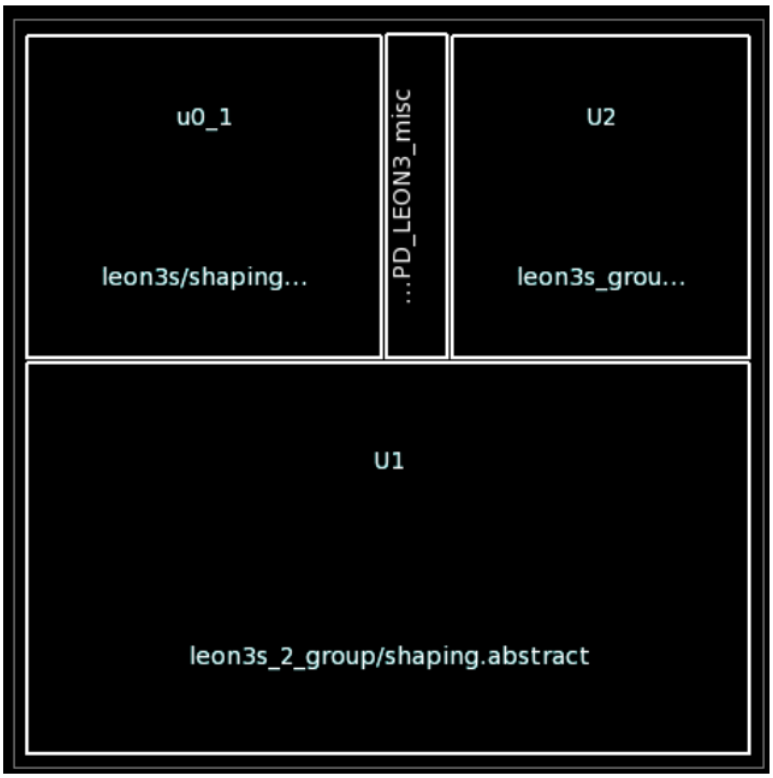


图 **9.9:** 块与电压区域整形

- `child_default_channels`：为指定块、电压区域、移动约束或整形组的嵌套子区域创建默认外部通道。

下例在核心边界与顶层对象之间创建 100 微米的边界通道。为块 u0_1 创建相邻通道约束，并把它赋给块 U1。`shape_blocks` 命令根据这些约束整形块与通道。

```
# Create the TOP_ROW shaping group to contain blocks
# u0_1 and U2, and voltage area u_m/PD_LEON3_misc.
# Create the TOP_AND_BOTTOM group to contain TOP_ROW
# group and block U1
set_app_options \
   -name plan.shaping.import_tcl_shaping_constraints -value true
create_group -name TOP_ROW \
   -shaping [list [get_cells u0_1] \
                  [get_voltage_areas u_m/PD_LEON3_misc] \
                  [get_cells U2]]
create_group -name TOP_AND_BOTTOM -shaping \
   [list [get_groups -shaping TOP_ROW] [get_cells U1]]

# Create the boundary shaping channel and
# neighbor shaping channel
set boundary_channel_100 \
   [create_shaping_channel -left_min 100 \
       -right_min 100 -top_min 100 -bottom_min 100]
set neighbor_channel \
   [create_shaping_channel -left_min 200 -left_max 250 \
```

```
       -right_min 200 -right_max 250 \
       -top_min 200 -top_max 250 \
       -bottom_min 200 -bottom_max 250 \
       -neighbor [get_cells u0_1]]

# Assign the boundary channel to the current block and
# the neighbor channel to block U1
create_shaping_constraint [current_block] -type boundary_channel \
   -object_channel $boundary_channel_100
create_shaping_constraint [get_cells U1] -type external_channels \
   -neighbor_channels $neighbor_channel

# Arrange the objects in the TOP_ROW group in
# left-to-right order
create_shaping_constraint [get_groups -shaping TOP_ROW] \
   -type array_layout -array_layout east

# Arrange the objects in the TOP_AND_BOTTOM group in
# top-to-bottom order
create_shaping_constraint [get_groups -shaping TOP_AND_BOTTOM ] \
   -type array_layout -array_layout south
shape_blocks
```

这些约束产生图 **??** 中的版图结果。

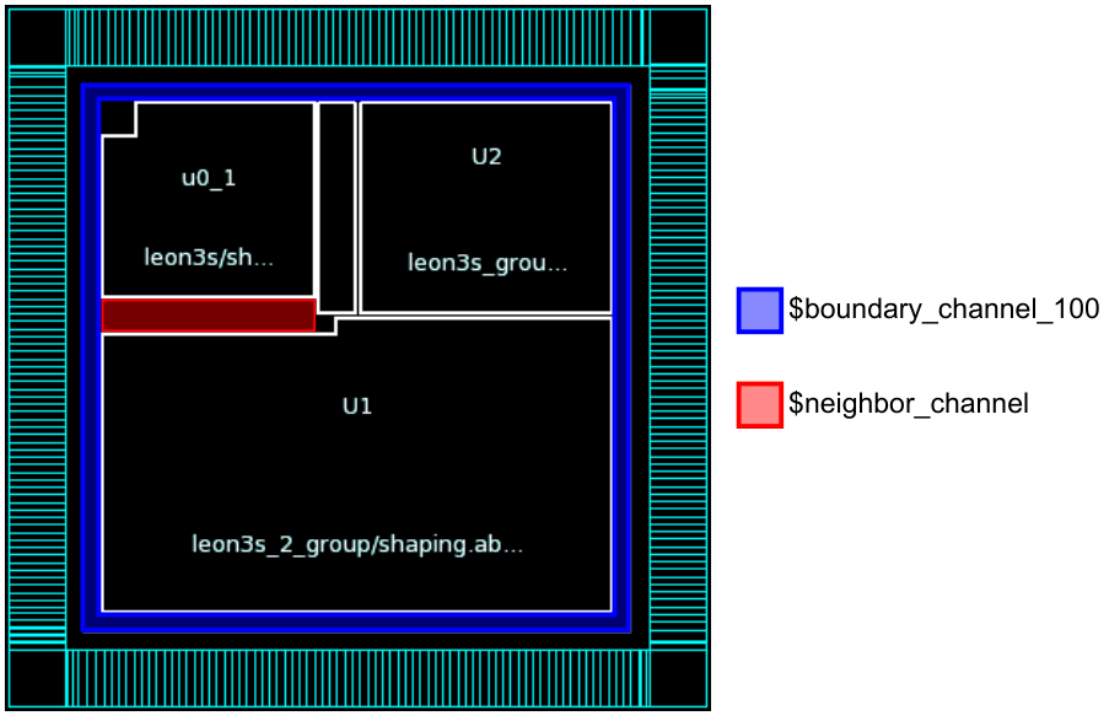


图 **9.10:** 块整形与通道约束

要移除整形通道，使用 `remove_shaping_channels` 命令。下列命令移除与某个特定整形约束对象关联的全部通道：

```
icc2_shell> remove_shaping_channels \
```

```
    DEFAULT_VA/SHAPING_CONSTRAINT_0_external_channels/*
```

要报告由 create_shaping_channel 与 create_shaping_constraint 命令创建的整形通道，使用 report_shaping_channels 命令。下例报告与移动约束 MB2 相关的整形通道：

```
icc2_shell> report_shaping_channels -of_objects [get_bounds MB2]
```

9.9 为多次实例化块创建块网格

Creating the Block Grid for Multiply Instantiated Blocks

对于包含多次实例化块（multiply instantiated block，MIB）的设计，必须创建块网格，以便工具把 MIB 实例与其他设计对象正确对齐。正确的 MIB 对齐保证对象在被下推到块中时，仍能与原始块保持对齐。用 shape_blocks 命令做块整形时，工具把 MIB 实例对齐到块网格。

块网格可以从电源规划策略导出，也可以从 floorplan 中已有的站点行（site row）导出。也可以通过指定原点与网格间距来手工定义网格。创建网格之后，必须把块参考与某个块网格关联，以保证工具以相同方式把全部实例对齐到该网格。

从已有站点行与电源地策略设置创建块网格的步骤如下：

1. 用 create_pg_mesh_pattern 与 set_pg_strategy 命令定义电源网格（power grid）。不必用 compile_pg 命令实例化该网格。

```
icc2_shell> create_pg_mesh_pattern pat_mesh \
   -layers { \
      {{vertical_layer: ME8} {width: 3} \
       {spacing: 3} {pitch: 20} {offset: 2}} \
      {{horizontal_layer: ME7} {width: 3} \
       {spacing: 3} {pitch: 20} {offset: 2}}}
icc2_shell> set_pg_strategy s_mesh -core \
   -pattern {{name: pat_mesh} {nets: VDD90 VSS}}
```

2. 用带 -site_rows 与 -pg_strategy 选项的 create_grid 命令创建网格。

```
icc2_shell> create_grid grid1 -type block \
   -site_rows [get_site_rows] -pg_strategy s_mesh1
Calculating block grid:
...
{grid1}
```

若设计包含的是站点阵列而不是站点行，用下列命令创建网格：

```
icc2_shell> create_grid grid1 -type block \
   -site_arrays [get_site_array] -pg_strategy s_mesh1
Calculating block grid:
...
{grid1}
```

3.（可选）用 report_grids 命令确认网格创建正确。

```
icc2_shell> report_grids
****************************************
Report : report block grid
Design : ORCA
****************************************
Auto Block grid grid1:
   Step in X direction: 20.0000 micron
   Step in Y direction: 20.0000 micron
   X offset: 0.0000 micron
   Y offset: 0.0000 micron
   Allowed orientations: R0
   Derived from PG strategies.
        Strategies Used  Layers:
         s_mesh1         \
```

4. 用 set_block_grid_references 命令把块网格与 MIB 参考关联。

```
icc2_shell> set_block_grid_references -designs {ORCA} \
   -grid grid1
```

也可以通过指定网格的 x、y 偏移与步长来手工创建块网格。用 report_grids 与 set_block_grid_references 命令核对网格，并把网格与 MIB 参考关联。手工创建网格如下：

```
icc2_shell> create_grid -type block grid2 -x_step 20.0 \
   -y_step 20.0 -x_offset 19.0 -y_offset 13.0
{grid2}
```

请注意，若手工创建网格，则不必指定电源规划图案或策略。

9.9.1 设置多次实例化块的吸附点

Setting the Snap Point for a Multiply Instantiated Block

为了保证多次实例化块可以正确翻转与旋转，同时仍落在网格上，可以更改块的吸附点（snap point）。工具把块的吸附点对齐到最近的网格点。默认情况下，设计的吸附点为 (0,0)。要设置吸附点并核对设置，按下列步骤使用 set_block_grid_references 命令，再用 get_attribute 命令：

1. 为多次实例化设计设置吸附点。

```
icc2_shell> set_block_grid_references -grid grid1 -designs BLENDER_2 \
   -snap_point {100.0 100.0}
```

2.（可选）检查多次实例化块实例上的属性，以核对设置。

```
icc2_shell> get_attribute [get_cells I_BLENDER_1] snap_point
100.0000 100.0000
icc2_shell> get_attribute [get_cells I_BLENDER_1] block_grid
grid1
```

3.（可选）用下列命令之一把多次实例化块吸附到网格。

```
icc2_shell> snap_cells_to_block_grid -grid grid1
icc2_shell> snap_cells_to_block_grid -designs BLENDER_2
icc2_shell> snap_cells_to_block_grid \
   -cells [get_cells I_BLENDER_1]
```

吸附点可以设在块上的任意位置。在图 **??** 中，小加号表示网格上的点，大加号（红圈标出）表示为该设计定义的吸附点。

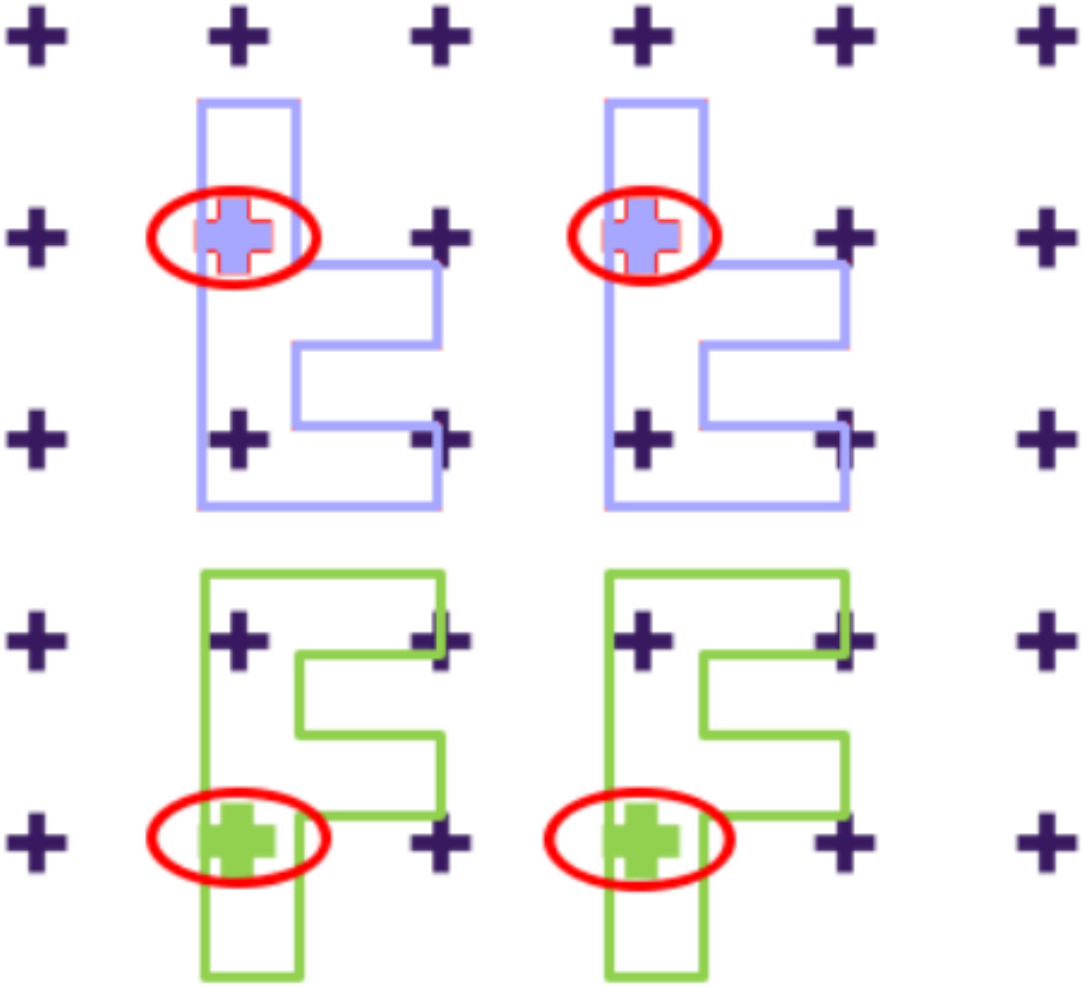


图 **9.11:** 带吸附点的多次实例化块实例

9.10 创建初始宏布局

Creating the Initial Macro Placement

块整形之后，可以用 `create_placement` 命令创建初始的全局宏布局。宏布局的目标是决定把哪个宏放在何处。宏布局在设计规划流程的早期完成。

创建初始宏布局的步骤如下：

1.（可选）用 `set_host_options` 命令为创建 floorplan 级布局设置主机选项。

```
icc2_shell> set_host_options -name distributed ...
```

用 set_host_options 命令创建设置，以进行分布式处理。

2.（可选）把应用选项 plan.macro.macro_place_only 设为 true，以只做宏布局。

```
icc2_shell> set_app_options -name plan.macro.macro_place_only \
   -value true
```

3.（可选）查看布局器相关应用选项的设置，并做必要修改。

可以用 report_app_options plan.place* 命令报告当前与布局器相关的设置。用 set_app_options 命令更改某项设置。

4.（可选）用 set_editability 命令把布局限制在设计层次的特定级别。下例禁止在设计层次的较低级别布局，并指定只在顶层布局。

```
icc2_shell> set_editability -from_level 1 -value false
```

5. 用 create_placement -floorplan 命令执行全局宏布局。

```
icc2_shell> create_placement -floorplan -host_options distributed
```

可选地：

- 若在步骤 1 中设置了主机选项，用 -host_options 选项启用分布式处理。
- 用 -effort low | medium | high 选项指定施加于布局的努力级别。
  提高努力级别时，工具增加内部循环次数，以创建结果质量尽可能好的布局。为了更好收敛，工具在后续布局遍中减小允许单元移动的平均距离。
- 在更改宏块边界之后，用 -incremental 选项更新布局。
  使用该选项时，命令尽量在当前位置附近创建合法的宏布局。若当前宏布局已经合法，则予以保留；否则把宏移到其他合法位置。非法位置的例子包括宏重叠、伸出核心，或不在网格上。
- 用 -congestion 选项降低拥塞。
  使用该选项时，工具通过移动宏并加宽通道来尽量降低拥塞。

6. 用 report_placement 命令验证布局。

```
icc2_shell> report_placement -physical_hierarchy_violations all \
   -wirelength all -hard_macro_overlap
****************************************
Report : report_placement
Design : ORCA
****************************************
Wire length report (all)
==================
wire len in design ORCA/dp: 828662.662 microns.
wire len in design ORCA/dp (see through blk pins): 950442.075 microns.
```

```
Physical hierarchy violations report
====================================
Violations in design ORCA/dp:
Information: cell .../I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_5
  overlaps block .../I_BLENDER_1 (DPP-403)
Information: cell .../I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_5
  overlaps block .../I_BLENDER_5 (DPP-403)
2 cells have placement violation.

Voltage area violations report
==================================
Voltage area placement violations in design ORCA/dp:
   0 cells placed outside the voltage area which they belong to.

Hard macro to hard macro overlap report
=======================================
Information: .../I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_4 and
  .../I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_7 overlap (DPP-405)
Information: .../I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_3 and
  .../I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_6 overlap (DPP-405)
HM to HM overlaps in design ORCA/dp: 2
---------------------------------------
Total hard macro to hard macro overlaps: 2
```

7.（可选）若设计中存在重叠，用 `report_placement` 命令生成重叠实例的完整列表。

```
icc2_shell> report_placement -verbose high > report.txt
```

可以查看 `report_placement` 命令生成的报告，并调查这些重叠。

8.（可选）要移除由 `create_placement` 命令创建的布局，使用 `reset_placement` 命令。

```
icc2_shell> reset_placement
```

关于拥塞驱动与时序驱动布局的更多信息，见第 **??** 节。

9.10.1 识别对象之间的通道

Identifying Channels Between Objects

要识别并报告指定对象之间存在的通道：

1. 指定要在其间识别通道的对象。

```
icc2_shell> set macros {I_CONTEXT_RAM_3_1 I_CONTEXT_RAM_3_2 \
   I_CONTEXT_RAM_3_3 I_CONTEXT_RAM_3_4}
```

2. 运行 `identify_channels` 命令，写出包含这些对象之间通道坐标的文件。

```
icc2_shell> identify_channels -cross_area $macros
Creating channel file channels.txt
```

工具把通道写入 channels.txt 文件。该文件列出通道类型（horizontal、vertical 或 crossed_area）以及通道坐标，如下所示：

```
icc2_shell> sh cat channels.txt
horizontal    (848.17   1632.765) (970.3    1669.995)
horizontal    (1009.175 1632.765) (1131.305 1669.995)
vertical      (970.3    1669.995) (1009.175 1774.075)
vertical      (970.3    1528.685) (1009.175 1632.765)
crossed_area  (970.3    1632.765) (1009.175 1669.995)
```

在图 **??** 的版图中，通道被勾出轮廓，编号与 channels.txt 文件中该通道所在行号对应。

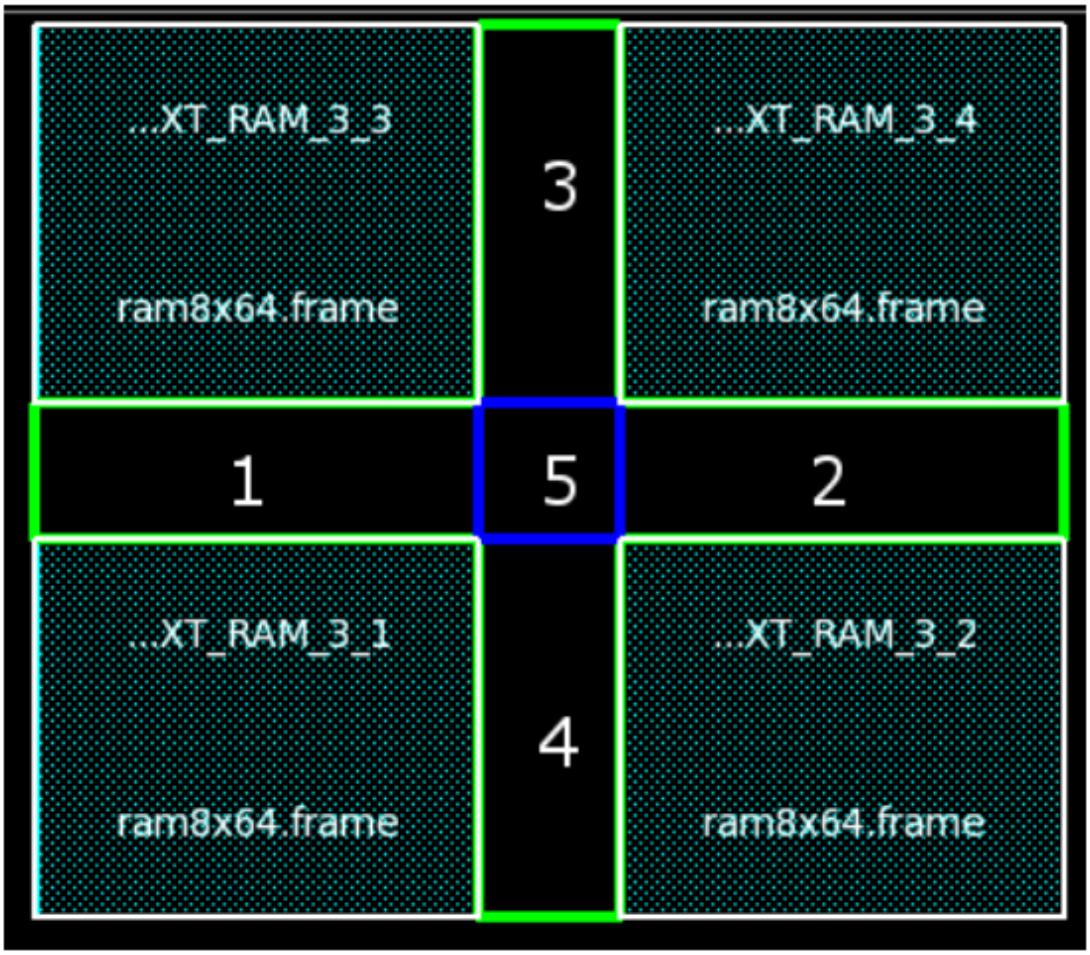


图 **9.12:** 已识别的通道

用 identify_channels 命令的选项控制通道的报告方式：

- 用 -horizontal_threshold 选项报告高度小于指定值的通道。
- 用 -vertical_threshold 选项报告宽度小于指定值的通道。
- 用 -output_filename 选项指定输出文件名。
- 用 -cross_area 选项报告交叉区域。交叉区域出现在一个对象的左下边与另一个对象的右上边之间，反之亦然。
- 用 -cross_area_height 选项报告高度小于指定值的交叉通道。

- 用 -cross_area_width 选项报告宽度小于指定值的交叉通道。

9.10.2 宏布局应用选项

Application Options for Macro Placement

表 **??** 列出宏布局应用选项。

表 **9.3:** 宏布局应用选项

要完成的操作	使用此应用选项
指定管脚是在 x 与 y 两个方向上都对齐，还是只与其所在层的方向对齐	plan.macro.align_pins_on_parallel_edges
指定自动生成的宏阵列的最大高度	plan.macro.auto_macro_array_max_height
指定自动生成的宏阵列的最大列数	plan.macro.auto_macro_array_max_num_cols
指定自动生成的宏阵列的最大行数	plan.macro.auto_macro_array_max_num_rows
指定自动生成的宏阵列的最大宽度（微米）	plan.macro.auto_macro_array_max_width
指定只有一侧有管脚的宏是翻转到与相邻宏的管脚相接（pins_in），还是翻转到与无管脚的一侧相接（pins_out），或不作改变（none）	plan.macro.auto_macro_array_minimize_channels
指定阵列打包程度：none、low、medium 或 high	plan.macro.auto_macro_array_size
指定当宏堆叠高度达到或超过应用选项 plan.macro.max_buffer_stack_height 所指定的值时，所创建通道的高度	plan.macro.buffer_channel_height
指定当宏堆叠宽度达到或超过应用选项 plan.macro.max_buffer_stack_width 所指定的值时，所创建通道的宽度	plan.macro.buffer_channel_width
指定在 create_placement 命令使用 -congestion 选项估计拥塞时所用的迭代次数	plan.macro.congestion_iters
指定用于创建临时 PG 网格的命令或脚本，以在宏布局期间帮助降低拥塞。可选地，可以在脚本中加入代码，在窄通道中与 PG 网格一起插入临时电源开关单元	plan.macro.create_temporary_pg_grid
指定要运行的用户脚本，用以移除为改善拥塞估计而插入的临时 PG 网格与临时电源开关单元	plan.macro.remove_temporary_pg_grid
指定在块之间的通道中避免沿块边界放置宏时所用的约束类型（soft 或 hard）或行为（false、true 或 unset）	plan.macro.cross_block_connectivity_planning
指定 create_placement -floorplan 命令创建临时 PG 网格，以提高拥塞估计的精度	plan.macro.estimate_pg

表 **9.3:** 宏布局应用选项（续）

要完成的操作	使用此应用选项
指定把宏对齐到网格时若发生错误会怎样。指定 continue 则打印警告并继续 floorplan 布局。指定 macro_place_only 则打印警告，并在完成宏布局后停止。指定 quit 则在发生对齐错误时立即打印错误信息并停止	plan.macro.grid_error_behavior
指定 create_placement -floorplan 命令按逻辑层次对宏分组。为 true 时，若层次尺寸与块尺寸之比超过应用选项 plan.macro.hierarchy_area_threshold 所指定的阈值，宏布局器把同一层次内的全部宏放入同一组	plan.macro.grouping_by_hierarchy
指定 create_placement -floorplan 命令在放置宏之后是停止还是继续	plan.macro.macro_place_only
指定宏的首选放置方式。该应用选项支持 on_edge、islands、freeform 与 hybrid 风格。默认值为 on_edge，即把宏沿块、电压区域与移动约束的边界放置。若设为 islands，部分宏会被放在远离划分边界的岛上，也就是浮动的紧密打包宏组。若为 freeform，宏被单独地、较松散地放在标准单元之间。若为 hybrid，工具把大宏沿划分边界放置，较小的宏则用自由形态放置。不过，工具也可能把全部宏都沿边放置，或全部以自由形态放置（例如，若全部宏都很大、阻挡很多布线层，或二者兼有，工具可能把它们沿边界放置；相反的原因则可能使工具把全部宏以自由形态放置）。hybrid 风格与 islands 风格的差别在于：hybrid 风格中浮动的宏是单独以自由形态放置的，而不是被分组并打包成岛；另外，hybrid 风格中可以有任意数量的宏浮动，而 islands 风格中只有设计面积的一小部分可以成为岛	plan.macro.style
指定宏堆叠的高度，超过该高度后工具为缓冲器布局创建通道	plan.macro.max_buffer_stack_height
指定宏堆叠的宽度，超过该宽度后工具为缓冲器布局创建通道	plan.macro.max_buffer_stack_width
指定为保证宏管脚可布线性而创建的宏禁布的最小尺寸	plan.macro.min_macro_keepout
指定一个宽度（垂直层）或高度（水平层），工具在该范围内尝试使管脚图形与轨道的着色匹配	plan.macro.pin_shape_track_match_threshold
为相邻宏指定一对距离：精确距离与最小距离。相邻宏必须按该精确垂直距离放置，或者垂直间距大于该最小距离	plan.macro.spacing_rule_heights

表 **9.3:** 宏布局应用选项（续）

要完成的操作	使用此应用选项
为相邻宏指定一对距离：精确距离与最小距离。相邻宏必须按该精确水平距离放置，或者水平间距大于该最小距离	`plan.macro.spacing_rule_widths`
控制是否在硬对象与软对象之间的通道中自动创建软布局阻挡。硬对象包括硬宏、硬禁布边距、块边界、电压区域边界与硬布局阻挡。软对象包括软布局阻挡与软禁布边距	`plan.place.auto_create_blockages`
控制是否自动创建垂直通道阻挡	`plan.place.auto_create_blockage_channel_widths`
控制是否自动创建水平通道阻挡	`plan.place.auto_create_blockage_channel_heights`
控制在确定阻挡时，是否把略呈直线多边形的宏视为其矩形包围盒的形状	`plan.place.auto_generate_blockages_smooth_recti`
控制在包含块的设计做顶层布局时，是否允许标准单元聚成一团	`plan.place.auto_max_density`
指定为降低拥塞而考虑的单元类型：`macro`、`std_cell` 或二者	`plan.place.congestion_driven_mode`
控制工具是否为全部宏创建默认硬禁布，以强制宏之间留出间距	`plan.place.default_keepout`
控制工具是否为全部宏创建默认硬禁布，以强制宏与标准单元之间留出间距	`plan.place.auto_create_hard_keepout`
指定宏布局追踪信号时所用的追踪方法。指定 `normal` 则只把缓冲器与反相器追踪到寄存器；指定 `dfa` 则把缓冲器、反相器与组合逻辑追踪到寄存器	`plan.place.trace_mode`

9.10.3 执行自由形态宏布局

Performing Freeform Macro Placement

随着宏数量增加，把宏放在边界上的传统做法扩展性不好。为解决该问题并改善 QoR 与易用性（ease of use，EoU），工具允许把宏放在标准单元之间，这称为自由形态（freeform）宏布局风格。与沿边宏布局相比，自由形态宏布局有下列好处：

- 考虑拥塞驱动的通道尺寸
- 向宏通道添加部分阻挡
- 计算通道拥塞时考虑 PG 资源
- 提供自动管脚保护阻挡
- 支持带标准单元的硬包含约束（hard inclusive bound）

- 遵守任何与功能安全相关的双核锁步（Dual Core Lock Step，DCLS）或 n 核锁步（nCLS）约束

要使用自由形态宏布局，设置下列应用选项：

```
icc2_shell> set_app_options -name plan.macro.style -value freeform
```

用下列应用选项对自由形态宏布局做更细的控制：

- plan.macro.create_auto_pin_blockages：指定运行自由形态宏布局时是否在管脚周围创建阻挡。这些阻挡类型为 hard_macro，用于保护设计管脚，避免宏过于靠近。
- plan.macro.remove_auto_pin_blockages：指定是否移除自由形态宏布局期间放在管脚周围的临时阻挡，以便于接近管脚。
- plan.macro.auto_pin_blockages_width：指定自由形态宏布局期间所创建管脚阻挡的宽度（管脚在垂直块边上时）或高度（管脚在水平块边上时）。

要用感知缓冲的时序模型启用时序驱动布局，使用 create_placement -buffering_aware_timing_driven 命令。该方法估计设计规划流程后期对长网线与高扇出网线做缓冲的影响，为后续时序优化提供更好的起点。

```
icc2_shell> create_placement -floorplan -congestion \
   -buffering_aware_timing_driven
```

若由于下列一种或多种原因发生了变化，之后要做增量自由形态宏布局，使用 create_placement -from 命令：

- 网表变化（优化期间因时序或面积变化）
- 约束变化（例如硬宏禁布）
- 拥塞降低

-from 选项让你选择宏移动的程度，从最激进、最耗时的 combined_placement 到最不激进的 channel_sizing。值为 combined_placement 时，工具运行完整的自由形态流程。默认值为 macro_legalization。请确保 -from 选项与 -floorplan、-incremental 选项一起使用。

对于增量的拥塞驱动自由形态宏布局，在 create_placement 命令中使用下列选项：

- -floorplan
- -incremental
- -congestion
- （可选）下表中的 -from 选项

表 **9.4:** 拥塞驱动增量自由形态宏布局的起点

增量自由形态宏布局的起点	使用此选项
增量组合布局	-from combined_placement
合法化宏（增量自由形态宏布局的默认起点）	-from macro_legalization
宏通道尺寸调整，以及增量标准单元布局	-from channel_sizing

例如：

```
icc2_shell> create_placement -floorplan -incremental -congestion \
   -from combined_placement
```

对于非拥塞驱动的自由形态宏布局：

表 **9.5:** 非拥塞驱动增量自由形态宏布局的起点

增量自由形态宏布局的起点	使用此选项
增量组合布局	`-from combined_placement`
合法化宏，以及增量标准单元布局	`-from macro_legalization`

9.10.4 集成自由形态宏布局

Integrated Freeform Macro Placement

为了在宏布局期间获得更好的运行时间效率，可以使用集成自由形态宏单元布局（integrated freeform macro cells placement，iFFMP）。在该方法中，宏单元被放在标准单元之间，并且宏的布局会随时间优化步骤中出现的拥塞与时序变化而调整。

通常，自由形态宏布局（freeform macro placement，FFMP）在运行 `place_opt` 流程之前执行。这种独立 FFMP 的局限是运行时间效率不高、结果不能自适应，以及在运行 `place_opt` 流程时可能重复工作。

为克服这些局限，工具支持集成自由形态宏布局（iFFMP），即把自由形态宏单元布局与 `place_opt` 流程集成。这种集成流程的优点是运行时间效率更好，并且能在 `place_opt` 流程期间调整宏单元布局以改善 QoR。

可以对单遍、两遍以及手工两遍 `place_opt` 流程运行 iFFMP。除此之外，iFFMP 还支持 PG 网格插入与原生芯片收尾，而不需要改动 `place_opt` 流程。

要在不支持 PG 网格、开关或抽头插入的情况下运行 iFFMP，执行下列步骤：

1. 打开一个设计。
2. 确保全部待放置的宏单元都未固定。例如：

```
icc2_shell> set_attribute [get_flat_cells -filter is_hard_macro] \
   unplaced
```

3. 移除任何边界单元、开关单元与抽头单元。

```
icc2_shell> remove_cells -force \
   [get_flat_cells -quiet -all -filter \
   "(is_power_switch|| is_physical_only) && !is_io"]
```

4. 设置下列应用选项以启用 iFFMP 流程：

```
icc2_shell> set_app_options -name plan.macro.style -value freeform
icc2_shell> set_app_options -name plan.macro.integrated -value true
```

注意

把 plan.macro.integrated 设为 true 时：

- 若 place_opt.initial_place.two_pass 为 false，工具启用单遍流程。
- 若 place_opt.initial_place.two_pass 为 true，工具启用两遍流程。

运行手工两遍流程时，把应用选项 plan.macro.integrated 设为 two_pass。

5. 运行 place_opt 命令，以 iFFMP 启动 place_opt 流程。
6. 用 set_fixed_objects 命令确保 place_opt 流程之后全部宏都已固定。例如：

```
icc2_shell> set_fixed_objects [get_flat_cells -filter is_hard_macro]
```

9.10.5 带临时 PG 网格插入的 iFFMP

Performing iFFMP With Temporary PG Grid Insertion

在宏单元布局之前插入 PG 网格会增加运行时间，因为宏单元对 PG 网格有显著影响。为改善性能，工具支持带临时 PG 网格创建的 iFFMP，以便在宏与标准单元布局期间为电源网格的拥塞效应建模。

临时 PG 网格流程更快，因为它插入时不做 PG 通孔 DRC 检查。place_opt 流程的 final_place 步骤完成后，PG 网格被移除并重新插入，以分析 PG 网格对可用于布线的轨道容量的影响。

要执行带临时 PG 网格插入的 iFFMP：

1. 打开一个设计。
2. 确保全部待放置的宏单元都未固定。例如：

```
icc2_shell> set_attribute [get_flat_cells -filter is_hard_macro] \
   unplaced
```

3. 移除任何边界单元、开关单元与抽头单元。

```
icc2_shell> remove_cells -force \
   [get_flat_cells -quiet -all -filter \
   "(is_power_switch|| is_physical_only) && !is_io"]
```

4. 设置下列应用选项以启用 iFFMP 流程与 PG 网格插入：

```
icc2_shell> set_app_options -name plan.macro.style -value freeform
icc2_shell> set_app_options -name plan.macro.integrated -value true
set_app_options -name plan.macro.create_temporary_pg_grid \
   -value <pg_script.tcl|Tcl code block>
set_app_options -name plan.macro.remove_temporary_pg_grid \
   -value <pg_script.tcl|Tcl code block>
```

5. 运行 place_opt 命令，以 iFFMP 启动 place_opt 流程。
6. 用 set_fixed_objects 命令确保 place_opt 流程之后全部宏单元都已固定。例如：

```
icc2_shell> set_fixed_objects [get_flat_cells -filter is_hard_macro]
```

7. 移除并重新插入带通孔 DRC 检查的 PG 网格。

9.10.6 带 floorplan 收尾的 iFFMP

Performing iFFMP With Floorplan Finishing

运行自由形态或混合宏布局时，可以用应用选项 `plan.macro.floorplan_finishing` 做 floorplan 收尾，以插入边界单元、抽头单元、开关单元等。

在 iFFMP 流程中为这些单元建模很重要，因为这些单元的布局对设计的可布局面积有显著影响。为提高 iFFMP `place_opt` 流程的精度，工具在启用 iFFMP 流程时使用应用选项 `plan.macro.floorplan_finishing`。可以用该应用选项指定插入边界单元、抽头单元与开关单元的脚本文件或 Tcl 代码。

要执行带 floorplan 收尾的 iFFMP：

1. 打开一个设计。
2. 确保全部待放置的宏单元都未固定。例如：

```
icc2_shell> set_attribute [get_flat_cells -filter is_hard_macro] \
   unplaced
```

3. 移除任何边界单元、开关单元与抽头单元。

```
icc2_shell> remove_cells -force \
   [get_flat_cells -quiet -all -filter \
   "(is_power_switch|| is_physical_only) && !is_io"]
```

4. 设置下列应用选项以启用 iFFMP 流程、PG 网格插入与 floorplan 收尾：

```
icc2_shell> set_app_options -name plan.macro.style -value freeform
icc2_shell> set_app_options -name plan.macro.integrated -value true
set_app_options -name plan.macro.create_temporary_pg_grid -value \
   <pg_script.tcl|Tcl code block>
set_app_options -name plan.macro.remove_temporary_pg_grid -value \
   <pg_script.tcl|Tcl code block>
set_app_options -name plan.macro.floorplan_finishing \
   -value <floorplan_finishing.tcl|Tcl code block>
```

5. 运行 `place_opt` 命令，以 iFFMP 启动 `place_opt` 流程。
6. 用 `set_fixed_objects` 命令确保 `place_opt` 流程之后全部宏单元都已固定。例如：

```
icc2_shell> set_fixed_objects [get_flat_cells -filter is_hard_macro]
```

7. 移除并重新插入带通孔 DRC 检查的 PG 网格。

9.11 以岛式放置宏

Placing Macros Island Style

默认的宏布局是沿块、电压区域与移动约束的边界。对某些设计，把一组宏放在远离这些对象边界的位置，或把宏放在标准单元之间，可能改善宏布局。岛式布局可以改善那些较小、又需要相对靠近设计不同边上对象的宏的布局质量。

默认情况下，工具用时序与连接信息决定把哪些宏分到一组。可以按下列示例的步骤 2 与步骤 3，创建一个约束来包含这组宏，从而指定宏分组。

要在宏布局期间把宏放在远离块、电压区域与移动约束边界的位置：

1. 把应用选项 `plan.macro.style` 设为 `islands`。

```
icc2_shell> set_app_options -list {plan.macro.style islands}
```

2.（可选）用 `create_bound` 命令为岛式宏集合创建组约束。

```
icc2_shell> create_bound -effort high \
   -name bound1 { I_ORCA_TOP/I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_7
   I_ORCA_TOP/I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_6
   I_ORCA_TOP/I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_5 }
```

3.（可选）若创建了约束来指定要分在一起的宏，把该组约束上的 `is_off_edge` 属性设为 `true`。

```
icc2_shell> set_attribute -objects [get_bounds bound1] \
   -name is_off_edge -value true
```

4. 用 `create_placement -floorplan` 命令执行全局宏布局。

```
icc2_shell> create_placement -floorplan
```

图 **??** 左侧是默认的沿边宏布局。右侧创建了三个组约束，并用上述流程为设计创建岛式宏布局。

9.12 使宏远离块边界

Placing Macros Away From Block Edges

要避免把宏放在某个块之上，用 `create_placement_blockage -type hard_macro` 命令在该块上创建硬宏布局阻挡。工具避免把宏放在阻挡之内，但可能把宏紧贴阻挡放置。要在阻挡周围再预留空间，把硬宏布局阻挡上的 `is_pin_protect` 属性设为 `true`。该设置使工具在硬宏布局阻挡区域周围为管脚布局预留空间。默认情况下 `is_pin_protect` 属性为 `false`，宏布局器只避免把宏单元放在硬宏布局阻挡区域之上；允许在布局阻挡边缘放置宏单元。例如，下列命令创建布局阻挡，并把该阻挡上的 `is_pin_protect` 属性设为 `true`：

```
icc2_shell> set pb [create_placement_blockage -type hard_macro \
   -boundary {{350 350} {500 500}}]
icc2_shell> set_attribute -objects $pb -name is_pin_protect \
   -value true
```

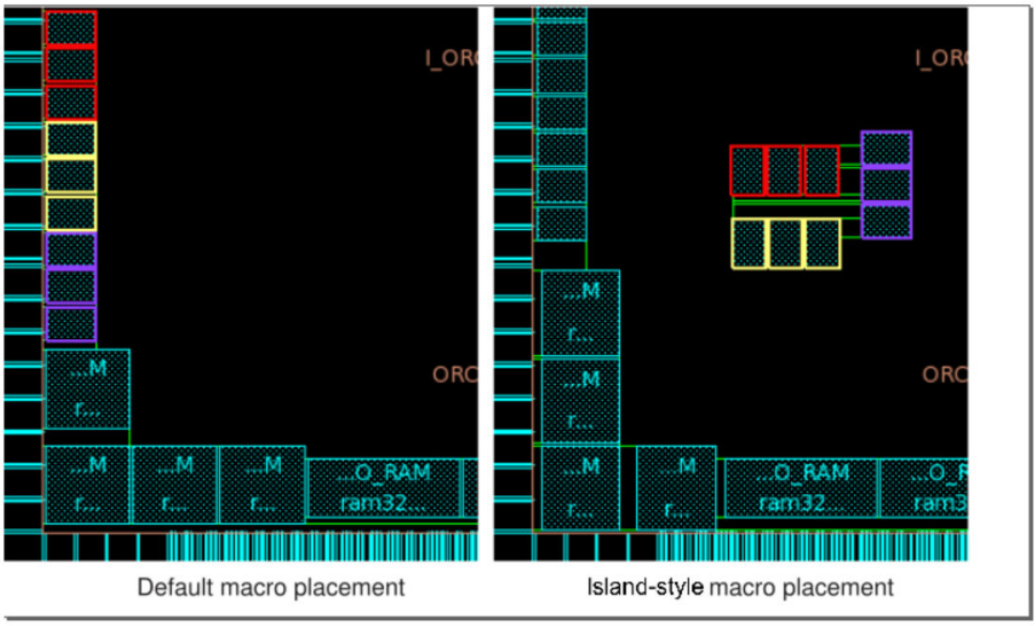


图 **9.13:** 默认与岛式宏布局

图 **??** 给出三种版图结果：无布局阻挡、`hard_macro` 类型布局阻挡，以及 `is_pin_protect` 属性设为 `true` 的 `hard_macro` 类型布局阻挡。

9.13 执行基于机器学习的宏单元布局

Performing Machine Learning-Based Macro Cells Placement

在典型的片上系统（system on chip，SoC）中，宏单元主导许多块，这些单元的位置对 QoR 影响很大。因此，这些宏单元的布局在物理设计中起着重要作用。把宏单元放到尽可能好的位置，既费时间也费精力。

不同的宏布局会导致截然不同的拥塞、线长以及随之而来的动态功耗，并可能对时序产生非常大的影响。例如见图 **??**。

本节包括以下主题：

- 基于机器学习的宏单元布局的一般术语
- 基于机器学习的宏单元布局流程
- 调用基于机器学习的宏布局流程
- 创建机器学习数据
- 基于机器学习的宏布局的已知限制
- 执行基于机器学习的宏布局时要考虑的要点
- 用 DSO.ai 获得更好的功耗、性能与面积

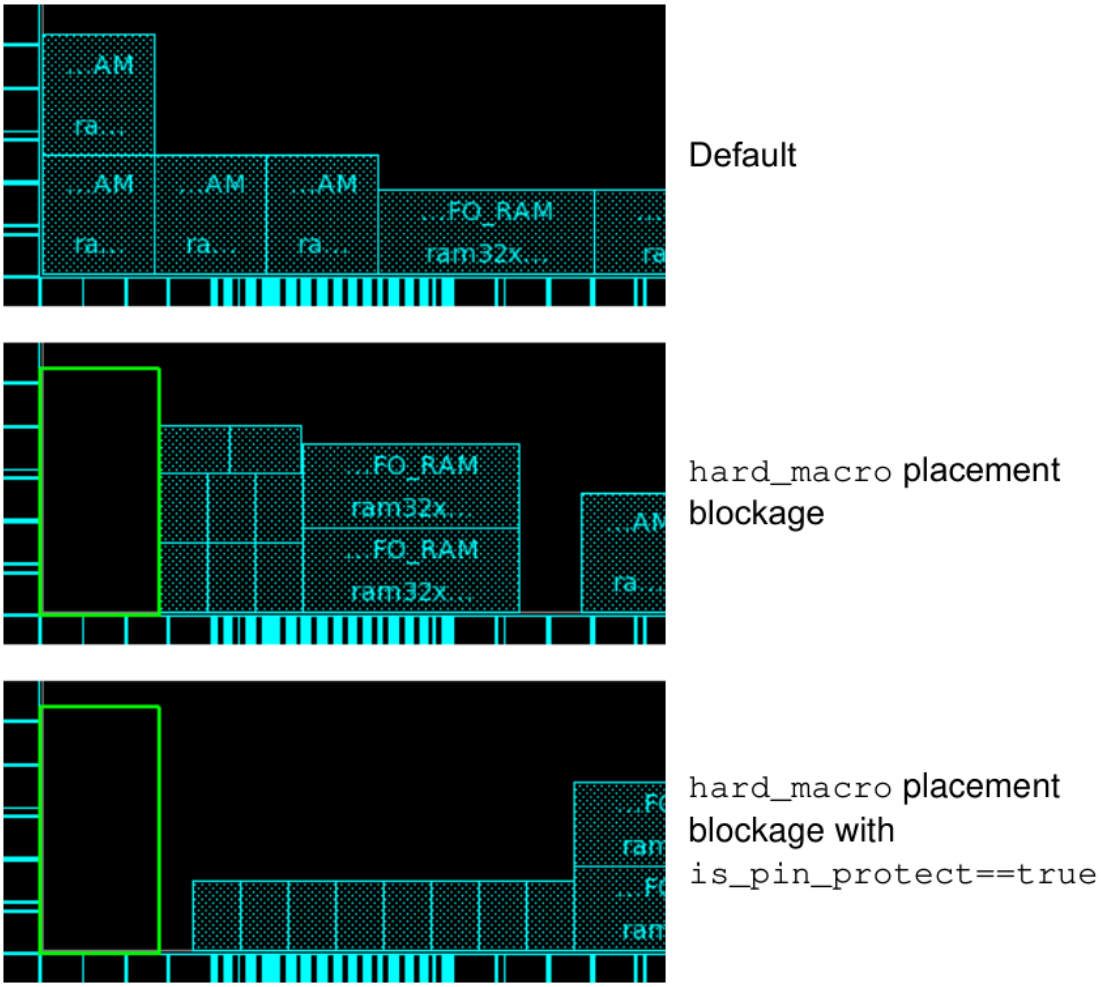


图 **9.14:** 远离块边界放置的宏

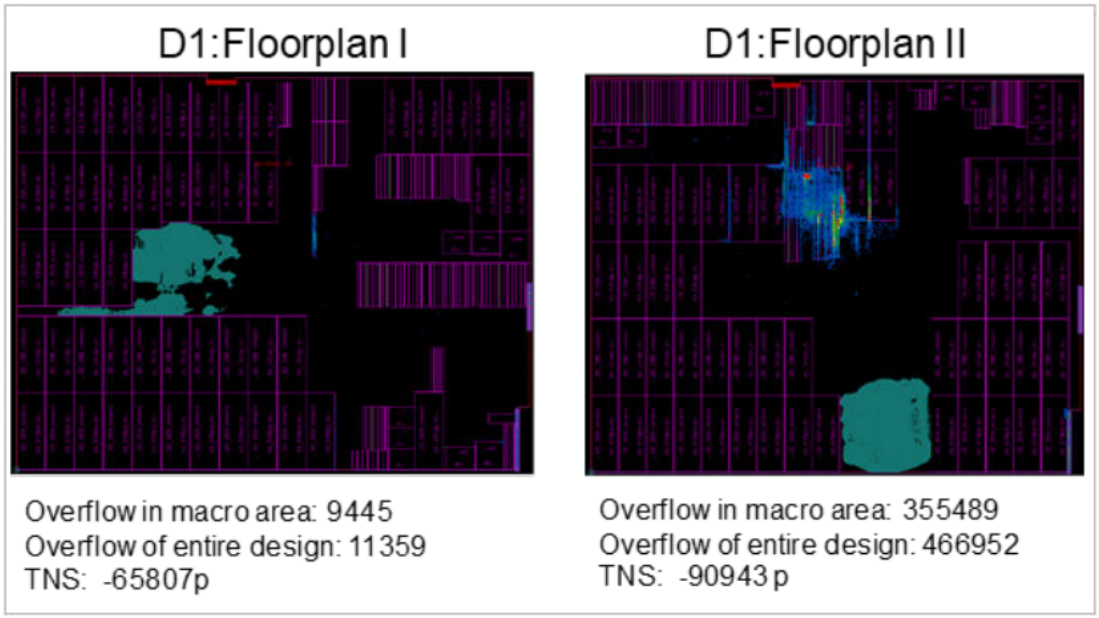


图 **9.15:** 不同宏布局产生的不同 floorplan

9.13.1 基于机器学习的宏单元布局的一般术语

General Terminology for ML-Based Macro Cells Placement

表 **9.6:** 基于机器学习的宏单元布局术语

术语	说明
基于机器学习的宏布局（ML-based macro placement）	该方法使用先前宏布局的数据来预测 QoR，并应用机器学习创建具有最佳宏布局的 floorplan。
机器学习数据（ML data）	它是某个 floorplan 的标签。使用基于机器学习的宏布局时，工具会创建若干 floorplan。每个 floorplan 称为一份机器学习数据。
训练模型（trained model）	该模型预测优化后的拥塞与总负松弛（total negative slack，TNS）。工具根据机器学习数据创建该模型。可用数据越多，模型越准确。

9.13.2 基于机器学习的宏单元布局流程

ML-Based Macro Cells Placement Flow

基于机器学习的宏布局期间，工具用分布式处理探索可能的宏布局结果，并创建大量 floorplan。然后工具创建训练模型，在所创建的 floorplan 中预测 QoR，并得出最佳 floorplan。

9.13.3 在 IC Compiler II 中执行基于机器学习的宏布局

Performing ML-Based Macro Placement in IC Compiler II

IC Compiler II 中基于机器学习的宏布局流程如下：

1. 工具生成大量 floorplan，并从每个 floorplan 提取机器学习数据。
   (a) 提取宏连接关系、宏分组、宏与标准单元区域之间的关系等信息。
   (b) 选择某些 floorplan，并执行标准单元布局、合法化、全局布线与优化。
   (c) 分别从全局布线与优化的结果中提取总体拥塞与 TNS 数据。
   (d) 把提取的拥塞与 TNS 数据保存为机器学习数据，随后可用于不同设计上的最优基于机器学习的宏布局。
2. 用上一步以及任何其他机器学习数据（例如来自工具库，或你从先前设计提供的数据）创建机器学习训练模型。

   工具用该模型预测拥塞与 TNS，并得出宏布局最佳的 floorplan。

   基于机器学习的宏布局期间，可以通过应用选项提供 PG 脚本，以便更好地预测拥塞。

   基于机器学习的宏布局完成之后，可以继续插入纯物理单元并创建 PG 网格。

   若使用了自由形态或混合宏布局，则继续用 `place_opt -from initial_drc`。否则使用 `place_opt -from initial_place`。

   继续流程，做时钟树综合与布线。

注意

即使在基于机器学习的宏布局期间提供了 PG 脚本，也要重新创建 PG 网格。这是因为基于机器学习的宏布局完成后，工具会移除任何 PG 网格。

9.13.4 调用基于机器学习的宏布局流程

Invoking ML-Based Macro Placement Flow

要执行基于机器学习的宏布局，使用 `place_macro_ml` 命令。按需要使用下列选项。

表 **9.7:** `place_macro_ml` 命令选项

命令选项 说明
`-host_options`（必需选项） 指定以启用分布式处理，从而改善运行时间。
`-mode`

表 **9.7:** `place_macro_ml` 命令选项（续）

命令选项 说明
指定优化目标。 • `congestion`：工具针对总体拥塞优化 floorplan。 • `tns`：工具针对总负松弛（TNS）优化 floorplan。 • `power`：工具针对功耗优化 floorplan。 • `both`（默认）：工具同时针对总体拥塞与 TNS 优化 floorplan。 • `all`：工具针对拥塞、TNS 与功耗优化 floorplan。 • `wirelength`：工具针对总线长优化 floorplan。 • `thermal`：工具针对温度优化 floorplan。工具执行宏布局以降低最高温度。在为温度优化而运行宏布局之前，须用应用选项 `thermal.hier_block_power_profile` 为硬宏设置功耗值。示例见本表之后。
`-work_directory` 指定机器学习内部数据（临时文件与其他数据）的存放位置。若不提供值，默认工作目录为 `mlmp_dir`。生成的机器学习数据存放在 `mlmp_dir/design_local_data/hm_local.csv` 文件中。
`-effort`

表 **9.7:** `place_macro_ml` 命令选项（续）

命令选项 说明
指定优化努力级别。 • `low`：运行时间最短，精度较低。此模式下，工具不为当前设计生成任何机器学习数据，而是使用任何已有数据（工具库中的机器学习数据，以及可选的用户提供数据）来得出最佳 floorplan。 • `medium`（默认）：在运行时间与 QoR 之间取得最佳平衡。 • `high`：以最高运行时间为代价提供最佳精度。以高努力级别运行机器学习宏布局时，工具在 `place_opt` 期间使用默认设置。若需要在 `place_opt` 期间使用定制设置，可以用应用选项 `plan.macro.ml_place_opt_setting` 指向包含该设置的 Tcl 文件。 在 `medium` 与 `high` 努力级别下，工具除任何已有机器学习数据之外，还使用从当前设计生成的机器学习数据来得出最佳 floorplan。这通常会得到更好的 floorplan。
`-style`

表 **9.7:** `place_macro_ml` 命令选项（续）

命令选项 说明
把宏单元放在边上（`on_edge`，默认）、以混合布局、以自由形态放置，或用 `auto` 让工具寻找最佳宏布局风格。混合风格中，部分宏单元沿边放置，其余以自由形态放置。`auto` 风格中，工具同时探索混合与自由形态宏布局风格以优化设计。
`-on_edge_ratio` 指定应被识别为沿边宏的宏，占全部可用宏的比例。该选项只用于混合宏布局。若未设置该选项，工具自动确定该值。要知道工具所用的值，在工具日志文件中搜索文本 `On_edge ratio:`。要查看根据 `-on_edge_ratio` 设置哪些宏会被识别为沿边宏，打开 GUI，再使用命令 `identify_on_edge_macros -ratio <ratio>`。
`-incremental` 仅当你需要相对现有宏布局做小改动时指定该选项。
`-exploration_file` 指定定制的、与宏布局相关的应用选项文件。使用该选项时，工具探索文件中设置的全部组合。示例见本表之后。

`-mode thermal` 的功耗设置示例：

```
icc2_shell> set_app_options \
   -name thermal.hier_block_power_profile \
   -value { SRAM1RW256x32 0.1 SRAM2RW32x39 0.2 \
 SRAM2RW64x32 0.3 }
```

下例中，工具探索 ml_app_options.tcl 文件中的全部设置：

```
icc2_shell> set_host_options -name mlmp \
   -submit_command "qsub -P bnormal -l mem_free=30G" \
   -max_cores 8
icc2_shell> place_macro_ml -host_options mlmp \
   -exploration_file ml_app_options.tcl
```

下例是带应用选项设置的 Tcl 文件：

```
plan.macro.max_group_area_percentage {0.3 0.2 0.1}
plan.macro.grouping_by_hierarchy {true false}
plan.place.auto_create_blockages {soft partial}
plan.macro.auto_buffer_channels {true false}
plan.place.create_auto_pin_blockages {true false}
```

9.13.5 IC Compiler II 示例脚本

Example Script for IC Compiler II

下面是 IC Compiler II 的示例脚本：打开一个已经做完 floorplan 的设计，运行基于机器学习的宏布局，然后继续 place_opt。

```
# Load the floorplanned design
open_lib design.nlib
open_block floor.design
# Set up distributed processing
set_host_options -name mlmp -submit_command [list qsub -P ae -pe mt 4 -l \
 mem_free=100G]
# Set up macro auto-blockages, spacing, keepouts, etc.
set_app_options -list {plan.place.auto_create_blockages  partial}
set_app_options -list {plan.place.auto_create_blockage_channel_widths \
 {2um 25um}}
set_app_options -list {plan.place.auto_create_blockage_channel_heights \
 {1um 25um}}

# Run ML macro placement in hybrid mode, with 50% macros on edge,
# optimized for both congestion and timing
place_macro_ml -host_options mlmp -mode both -style hybrid -on_edge_ratio \
 0.5

# fix macro cells, create the PG mesh, continue with the flow
set_fixed_objects [get_cells -hier -filter is_hard_macro]
source boundary_tap.tcl
source pg.tcl
# If you used freeform or hybrid, skip the initial_place stage
place_opt -from initial_place  or  place_opt -from initial_drc
```

9.13.6 创建机器学习数据

Creating ML Data

机器学习数据是构建训练模型的基础：可用的机器学习数据越多，训练模型越准确。要创建机器学习数据，可以按方法 1（从已优化设计创建机器学习数据）或方法 2（在运行 place_macro_ml 命令时从机器学习宏布局创建机器学习数据）。

本节还包括：

- 如何提供用户机器学习数据
- 从先前的机器学习宏布局运行中选择最佳设计

9.13.7 方法 1：从已优化设计创建机器学习数据

Method 1: Creating ML Data From Optimized Designs

为通过训练机器学习模型增强可预测性，可以从已优化设计提供额外的机器学习数据。已优化设计可以是跑过 place_opt 的设计，也可以是已完成时钟树综合、甚至更好是已完成 route_opt 的设计。

要捕获机器学习数据，使用 create_mlmp_data -output 命令。例如：

```
icc2_shell> create_mlmp_data \
   -output /remote/central_location/MLMP/design1/hm.csv
```

本例在 /remote/central_location/MLMP/design1 目录中创建下列文件：

- hm.csv：包含当前 floorplan 的机器学习数据
- .hm.csv.signature：包含工具用来解码机器学习数据的信息

工具训练机器学习模型时需要这两个文件。

9.13.8 方法 2：运行 place_macro_ml 时创建机器学习数据

Method 2: Creating ML Data From ML Macro Placement When Running the place_macro_ml Command

要创建机器学习数据，可以运行 place_macro_ml -work_directory 命令。例如：

```
icc2_shell> place_macro_ml -host_options mlmp \
   -style hybrid -work_directory mlmp_dir_medium
```

本例在 mlmp_dir_medium/design_local_data 目录中创建下列文件：

- hm_local.csv 文件：包含多份机器学习数据。每份机器学习数据对应一个被探索的 floorplan
- .hm_local.signature 文件：包含工具用来解码机器学习数据的信息

9.13.9 如何提供用户机器学习数据

How to Provide User ML Data

要把从先前流片设计创建的机器学习数据用于当前机器学习模型训练：

1. 确保把全部必要文件放在一个文件夹下。该文件夹应包含 *.csv 与 .*.csv.signature 文件。
2. 调用 place_macro_ml -work_directory 命令时指定该目录。

   例如，若已把全部必要的机器学习数据文件放在 /remote/central_location/ML_data，则使用命令：

```
icc2_shell> place_macro_ml -host_options mlmp -style hybrid \
   -work_directory /remote/central_location/ML_DATA
```

   机器学习数据目录的内容示例：

```
/remote/central_location/ML_data> ls -al
drwxr-xr-x  4 snps synopsys  4096 Aug 24 08:59 ./
drwxr-xr-x 60 snps synopsys 36864 Aug 26 07:55 ../
-rw-r--r--  1 snps synopsys  7744 Aug 24 08:59 hm_local.csv
-rw-r--r--  1 snps synopsys    48 Aug 24 08:59 .hm_local.csv.signature
drwxr-xr-x  2 snps synopsys  4096 Aug 24 09:07 leon3mp_design_local_data/
drwxr-xr-x  2 snps synopsys  4096 Aug 23 16:22 orca_design_local_data/
```

9.13.10 从先前的机器学习宏布局运行中选择最佳设计

Selecting the Best Design From Previous ML Macro Placement Runs

用 select_mlmp_result 命令从先前基于机器学习的宏布局运行中选择最佳设计。工具根据相应宏布局的拥塞与时序值选择最佳设计。用 -mode 选项指定优化目标。例如，用该选项可以指示工具选择针对下列一项或多项优化的最佳设计：拥塞、TNS、功耗或线长。

用该选项指定一个文件，其中包含先前基于机器学习的宏布局运行的全部相应 mlmp_dir 目录。下例中，工具从 mlmp_result_file 所指定的目录中选择最佳结果。

```
icc2_shell> select_mlmp_result -result_file_name mlmp_result_file
```

示例文件如下：

```
/u/hardmacro/designs/orca/mlmp_dir_medium_hybrid
/u/hardmacro/designs/orca/mlmp_dir
/u/hardmacro/designs/orca/mlmp_dir_auto
/u/hardmacro/designs/orca/mlmp_dir_medium_encoded
```

9.13.11 基于机器学习的宏布局的已知限制

Known Limitations of ML-Based Macro Placement

基于机器学习的宏单元布局不支持：

- 层次化设计
- 带抽象视图或设计视图的软块

9.13.12 执行基于机器学习的宏布局时要考虑的要点

Points to Consider While Performing ML-Based Macro Placement

执行基于机器学习的宏布局时，可以考虑下列要点：

- 宏单元周围的禁布边距
- 为窄通道创建布局阻挡
- 在顶层管脚周围创建布局阻挡
- 控制宏单元之间的间距
- 保护直线多边形 floorplan 中的凹角
- 沿边界分布宏单元
- 禁布边距与布局阻挡的比较
- 在基于机器学习的宏单元布局期间插入并处理临时 PG 网格

9.13.13 宏单元周围的禁布边距

Keepout Margins Around Macro Cells

虽然基于机器学习的宏单元布局流程遵守全部宏布局约束与选项，仍可能在宏周围看到 DRC 违例。该问题可能是由于通道尺寸或禁布边距不足，如下图所示。

- 当 DRC 错误因空间不足而发生时，可以通过在宏之间预留更多空间来解决。例如可以：
  - 设置 `hard_macro` 类型禁布边距，或
  - 用硬布局阻挡覆盖窄通道。

  这些操作可以在运行边界单元插入之前完成。
- 当 DRC 错误发生在宏阻挡与标准单元之间时，可以通过设置 `hard_macro` 类型禁布边距来解决。该操作在宏与标准单元之间预留更多空间。

一般而言，用宏禁布边距改善布局与布线 QoR。许多情况下，可能需要为指定的物理设计与单元创建禁布边距。

表 **9.8:** 禁布类型、受影响对象与用途

禁布类型	受影响对象
用途	
hard	影响标准单元布局，包括优化期间新加入的任何单元
阻止标准单元靠近宏单元边界	

表 **9.8:** 禁布类型、受影响对象与用途（续）

禁布类型 用途	受影响对象
soft 把时序优化产生的任何新单元放在软禁布边距或软布局阻挡之上	影响布局期间宏单元周围的标准单元布局
hard_macro 保证宏之间有最小通道；阻止宏重叠，包括 `hard_macro` 禁布；为通向宏管脚的布线分配空间；确保没有硬宏或其他硬宏禁布边距被放在禁布边距之内	影响硬宏之间的布局
route_blockage 阻止预布线阻挡	影响预布线网线

9.13.14 为窄通道创建布局阻挡

Creating Placement Blockages for Narrow Channels

宏布局之后，很可能会形成许多窄通道，其中并非全部都能当作正常的标准单元区域使用。这是因为：

- 许多通道可能正对宏管脚
- 有些区域必须预留给布线等用途

这种情况下，用软布局阻挡或部分布局阻挡覆盖这些窄通道。不过，若通道非常窄，并且在该通道上放置标准单元会引起 DRC 错误，则用硬布局阻挡覆盖该通道。

注意

一般建议是用硬阻挡覆盖窄通道，用部分阻挡或软阻挡覆盖宽通道。另外，自由形态宏布局使用部分阻挡。

应用选项 `plan.place.auto_create_blockages` 控制布局阻挡的类型。

若要更精细的控制，可以使用应用选项 `plan.place.auto_create_blockage_channel_widths {min max}` 与 `plan.place.auto_create_blockage_channel_heights {min max}`。它们分别让你控制垂直方向与水平方向上的部分阻挡尺寸。每个应用选项接受两个值：

- 当通道尺寸小于第一个数时，工具用硬布局阻挡覆盖该通道。
- 当通道宽度介于第一个数与第二个数之间时，工具用软阻挡或部分阻挡覆盖该通道。

  近似的部分阻挡通道百分比 $= (1 - channel_width/(max - min)) \times 100\%$。

  例如，若通道宽度为 5 μm，最小值与最大值分别为 2 μm 与 30 μm，则该 5 μm 通道的阻挡百分比为 $(1 - 5/28) \times 100 = 82\%$。
- 当通道宽度大于第二个数时不创建阻挡，因为这通常意味着通道已经足够大，可以用来放置标准单元。

9.13.15 在顶层管脚周围创建布局阻挡

Creating Placement Blockages Around Top Pins

当设计在顶层有管脚时，在这些管脚周围创建布局阻挡很重要。自由形态宏布局期间，管脚布局阻挡默认被创建。不过，对于沿边宏布局，必须把应用选项 `plan.macro.ml_create_auto_pin_blockages` 设为 `true`。这样可以保证稍后不会出现布线问题。

在管脚周围创建布局阻挡的流程如下：

1. 打开设计。
2. 选择需要布局阻挡的管脚。
3. 把应用选项 `plan.macro.ml_create_auto_pin_blockages` 设为 `true`。

9.13.16 控制宏单元之间的间距

Controlling Spacing Between Macro Cells

有些设计可能要求满足间距规则，以便通道可以用来放置标准单元或插入电源开关单元。

要控制宏单元之间的间距，可以使用下列应用选项：一个控制垂直间距，另一个控制水平间距。

- `plan.macro.spacing_rule_heights {exact_distance at_least_distance}`
- `plan.macro.spacing_rule_widths {exact_distance at_least_distance}`

每个应用选项接受两个值。在下列示例中，工具把水平通道间距设为 2 μm 或至少 8 μm，把垂直通道间距设为 2 μm 或至少 9 μm。

```
icc2_shell> set_app_options -name plan.macro.spacing_rule_heights -value \
 {2 8}
icc2_shell> set_app_options -name plan.macro.spacing_rule_widths -value \
```

```
{2 9}
```

9.13.17 保护直线多边形 floorplan 中的凹角

Protecting Concave Corners in Rectilinear Floorplans

当堆叠在凹角上的宏伸进核心区域并引起拥塞时，可能需要凹角保护。

要避免在凹角上打包宏,在执行基于机器学习的宏布局之前把应用选项 `plan.macro.ml_use_concav_blockage` 设为 `true`。该应用选项设为 `true` 时，阻止把宏放在角上。默认值为 `false`。

9.13.18 沿边界分布宏单元

Distributing Macro Cells Around Boundaries

边界周围的宏太多可能引起拥塞或时序问题，在直线多边形设计中尤其如此。

应用选项 `plan.macro.ml_distribute_evenly` 有助于克服该问题。设为 `true`（这是默认值）时，工具把宏均匀分布在边界周围，并在管脚周围创建阻挡。

9.13.19 禁布边距与布局阻挡的比较

Comparison Between Keepout Margin and Placement Blockage

表 **??** 比较禁布边距与布局阻挡。

表 **9.9:** 禁布边距与布局阻挡的比较

禁布边距	布局阻挡
附着在宏单元上。因此，宏单元从一处移到另一处时，禁布边距的位置随之改变	与 floorplan 中的一块固定区域关联
保护宏以及宏周围的区域	确保没有单元被放在靠近宏的地方
它确保没有单元被放在靠近宏的地方	它阻止在布局阻挡所覆盖的区域中放置单元
对宏单元使用禁布边距	对固定区域使用布局阻挡

9.13.20 在基于机器学习的宏单元布局期间插入并处理临时 PG 网格

Inserting and Handling Temporary PG Grid During ML-Based Macro Cells Placement

在基于机器学习的宏布局之前插入临时 PG 网格，以便得到更准确的拥塞驱动通道尺寸。宏布局完成之后，必须移除所插入的 PG 网格。

可以使用下列应用选项：

- `plan.macro.create_temporary_pg_grid`：指定用于插入临时 PG 网格的 PG 脚本
- `plan.macro.remove_temporary_pg_grid`：指定用于移除临时 PG 网格的 PG 脚本

9.13.21 用 DSO.ai 获得更好的功耗、性能与面积

Achieving Better PPA Using DSO.ai

可以用设计空间优化（Design Space Optimization，DSO.ai™）工具得到功耗、性能与面积（power, performance, and area，PPA）更好的 floorplan。

`explore_macro_placement` 命令提供基于机器学习的宏布局（MLMP）与 DSO.ai 之间的接口。运行该命令时，工具根据机器学习排序输出五个 floorplan，而不是一个。该命令还创建一个名为 `mlmp_compile_permutons.tcl` 的设置文件。DSO.ai 用该设置文件加载所生成的 floorplan，并优化它们，以在拥塞、时序与功耗方面达到最佳。关于运行 DSO.ai 的更多信息，见 *DSO.ai User Guide*。

下例中，工具用 `on_edge` 风格探索宏布局。

```
icc2_shell> set_host_options -name mlmp -submit_command \
   "qsub -P bnormal -l mem_free=30G" -max_cores 8
icc2_shell> explore_macro_placement -host_options mlmp -style on_edge
```

9.14 使用宏布局助手

Using the Macro Placement Assistant

手工放置宏时，可以用宏布局助手（Macro Placement Assistant）改善布局结果质量：对宏分组，并试验不同的宏布局。用该工具减少版图中的窄通道数量，并改善布线拥塞。要启动宏布局助手，从菜单选择 **View > Assistants > Macro Placement Assistant**，或单击工具栏上的 Macro Placement Assistant 按钮。启动宏布局助手之后，先单击 Initialize 按钮再继续。

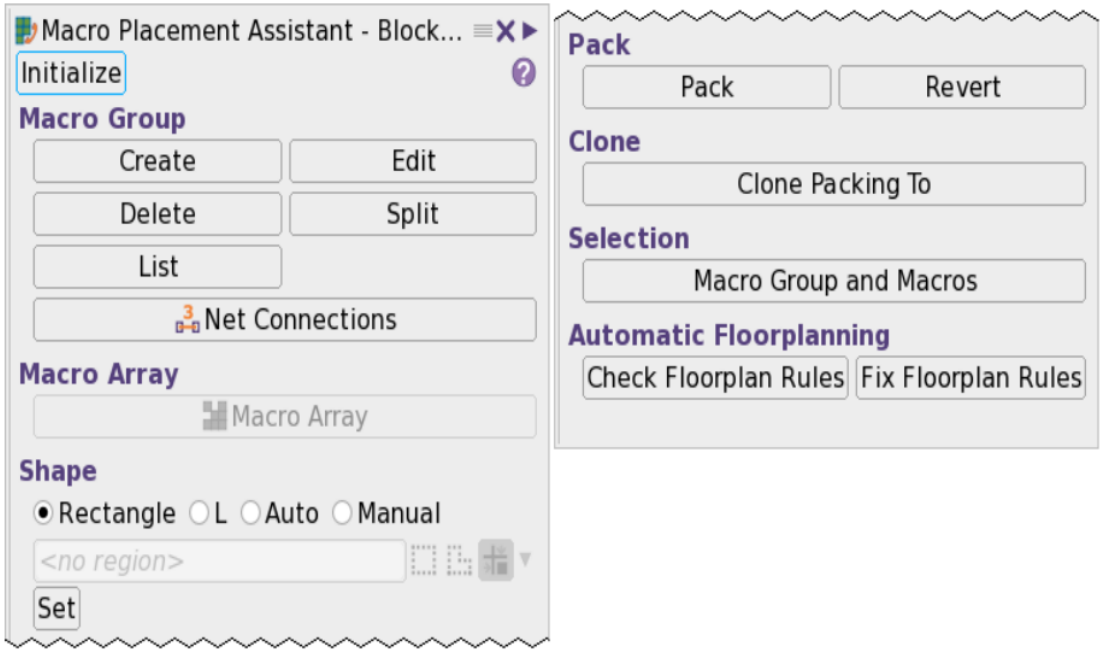


图 **9.16:** 宏布局助手面板

可以用图 **??** 所示流程创建初始宏布局，并用宏布局助手细化宏布局。

该流程为：执行自由形态宏布局；创建或编辑宏组；分析宏组；整形宏组；打包宏组。

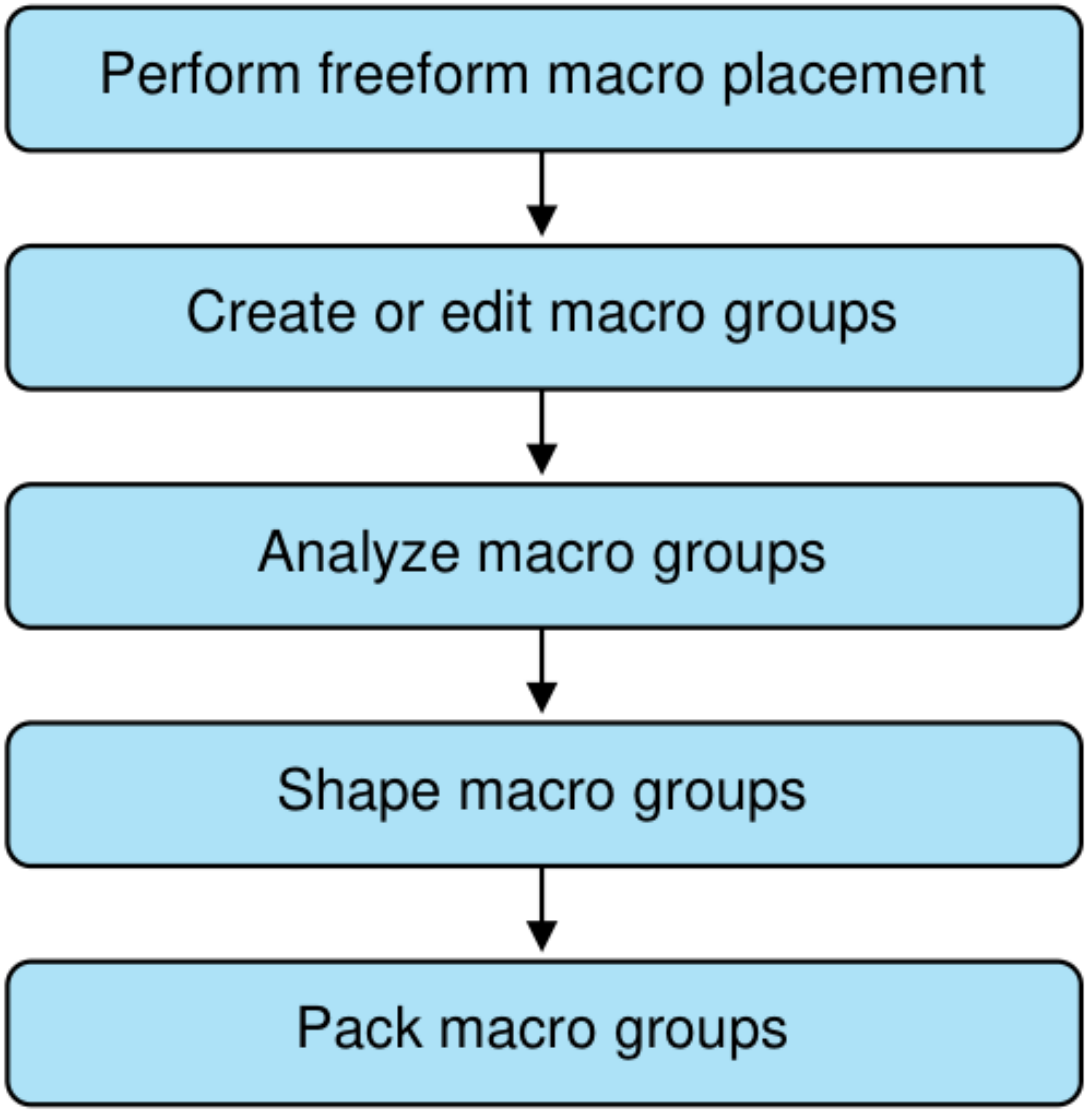

Perform freeform macro placement
Create or edit macro groups
Analyze macro groups
Shape macro groups
Pack macro groups

图 **9.17:** 宏布局流程

9.14.1 创建宏组

Creating Macro Groups

宏组是应当在版图中放在一起的宏单元与宏阵列的集合。用宏布局助手创建宏组时，工具应用 `create_macro_groups` 命令来创建约束。宏布局助手支持下列创建宏组的方法；要使用这些方法，单击 Create 按钮打开 Create Macro Groups 对话框。

自动（**Auto**）

根据层次、连接关系与粗布局，创建一个或多个组以产生最佳分组。在 GUI 中单击 Auto，再单击 OK。也可以用下列 Tcl 命令创建宏组：

```
create_macro_groups -method auto -macros
```

按连接关系（**By connectivity**）

根据版图中宏与触发器之间的连接创建一个或多个组。在 GUI 中单击 By connectivity，再单击 OK。要为新宏组指定名称，在 Group name 框中输入名称，以空格分隔。也可以用下列 Tcl 命令创建宏组：

```
create_macro_groups -method by_connectivity -names {M1 M2 M3}
```

按层次（**By hierarchy**）

为包含宏的每一级层次创建一个组。在 GUI 中单击 By hierarchy，再单击 OK。要为新宏组指定名称，在 Group name 框中输入名称，以空格分隔。也可以用下列 Tcl 命令创建宏组：

```
create_macro_groups -method by_hierarchy -names {M1 M2 M3}
```

按名称模式（**By name pattern**）

根据正则表达式创建一个宏组。在 GUI 中，在版图里选择一个宏，单击 By name pattern，并按需要调整正则表达式模式。工具根据所选宏的实例名导出一个正则表达式。要为新宏组指定名称，在 Group name 框中输入名称，以空格分隔。也可以在正则表达式框中输入模式字符串；模式应包含完整路径名，并用 `?` 或 `.` 字符以匹配多个宏。单击 Show Macros，按该模式填充宏列表。单击 OK 创建宏组。也可以用下列 Tcl 命令：

```
create_macro_groups -method by_pattern \
   -pattern {I_ORCA_TOP/I_RISC_CORE/I_REG_FILE/REG_FILE_?_RAM}
```

下例中，模式字符串 `I_ORCA_TOP/I_RISC_CORE/I_REG_FILE/REG_FILE_?_RAM` 匹配 `I_ORCA_TOP/I_RISC_CORE/I_REG_FILE` 下的宏 REG_FILE_A_RAM、REG_FILE_B_RAM、REG_FILE_C_RAM 与 REG_FILE_D_RAM。

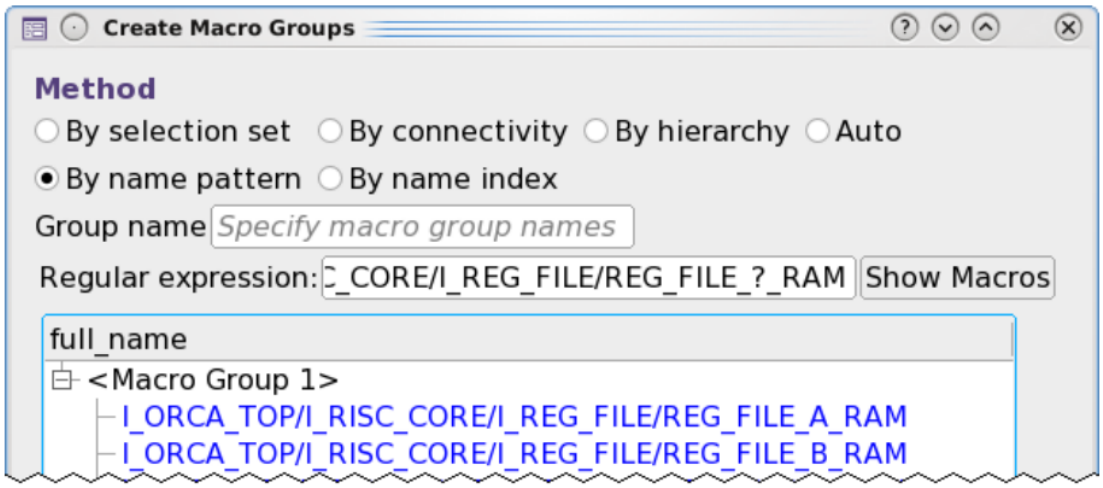


图 **9.18:** 按名称模式分组

按名称索引（**By name index**）

根据宏索引的正则表达式创建一个或多个宏组。在 GUI 中，在版图里选择一个宏并单击 By name index。工具尝试根据所选宏实例名末尾的索引导出索引模式。也可以在索引模式框中输入模式字符串；模式应包含完整路径名，并用 ? 或 . 字符以匹配多个宏。单击 Show Macros，按该模式填充宏列表。也可以用下列 Tcl 命令创建宏组：

```
create_macro_groups -method by_index_pattern \
   -index_pattern {I_ORCA_TOP/I_CONTEXT_MEM/I_CONTEXT_RAM_?_*}
```

下例中，模式字符串 `I_ORCA_TOP/I_CONTEXT_MEM/I_CONTEXT_RAM_?_*` 匹配版图中的 16 个宏。工具创建四个组，每组四个宏，索引划分如下：{0_1 0_2 0_3 0_4}、{1_1 1_2 1_3 1_4}、{2_1 2_2 2_3 2_4} 与 {3_1 3_2 3_3 3_4}。

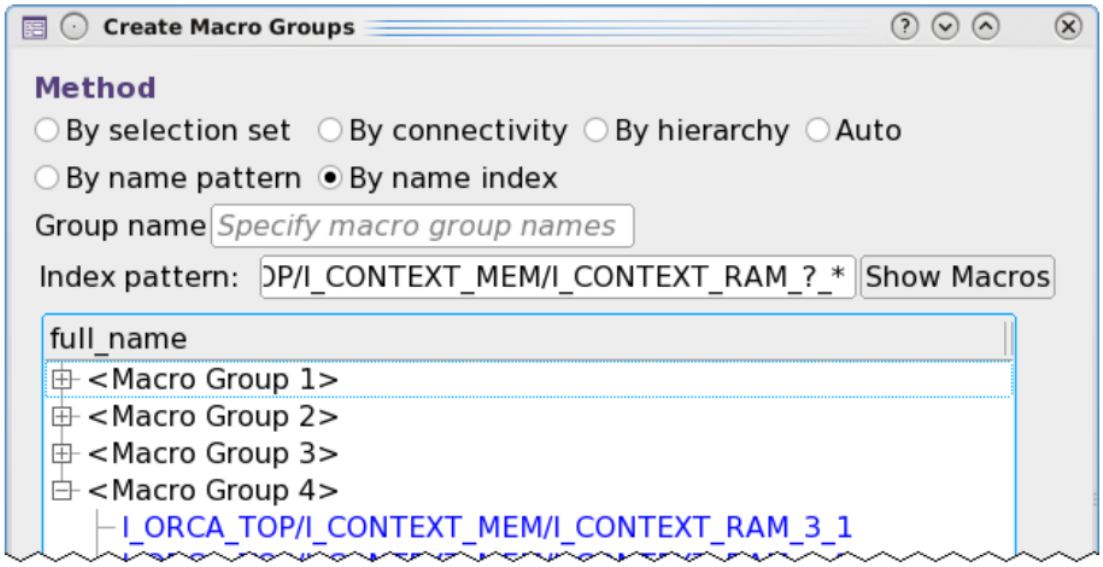


图 **9.19:** 按名称索引分组

按选择集（**By selection set**）

创建一个包含当前所选宏的单个组。在 GUI 中选择这些宏，单击 By selection set，再单击 OK。在版图中选择一个宏并单击 [+] 按钮，可以把新宏加入列表。若该宏已经属于某个组，它会被移到新组。在列表中选择一个或多个宏并单击 [-] 按钮，可以从列表中移除它们。也可以用下列 Tcl 命令创建宏组：

```
create_macro_groups -method by_macros -macros [get_selection]
```

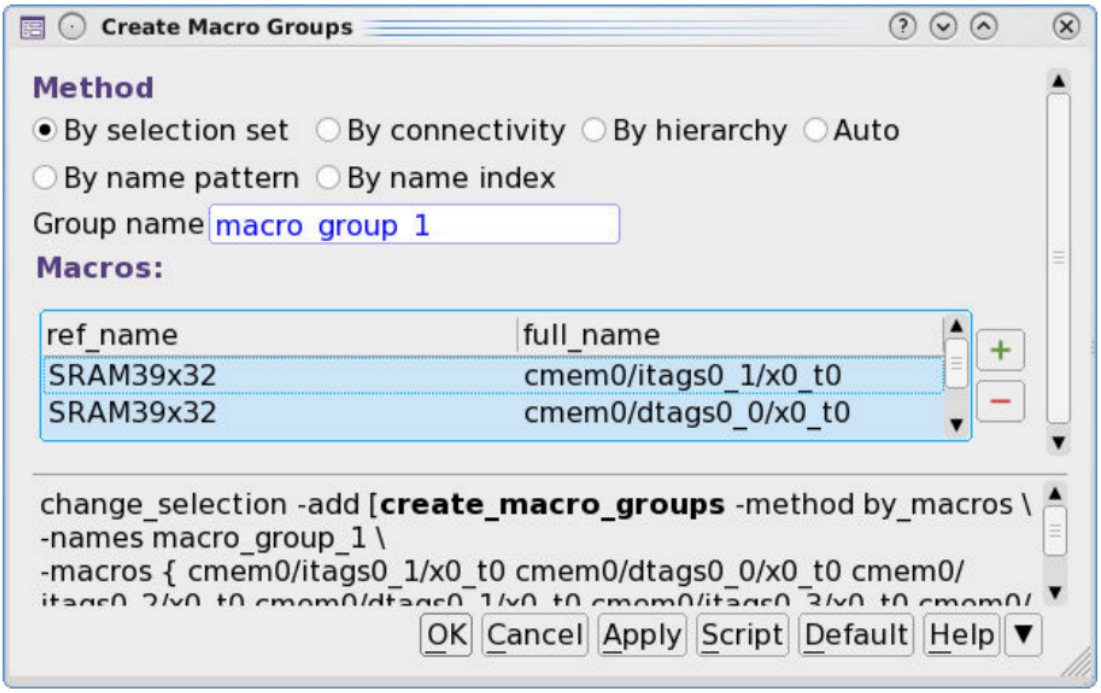


图 **9.20:** 创建宏组

9.14.2 对宏执行其他任务

Performing Other Tasks on Macros

宏布局助手还支持可以对宏与宏组执行的其他任务。

删除（**Delete**）

选择一个或多个宏组，单击 Delete 按钮，移除当前所选的宏组。

编辑（**Edit**）

在版图中选择一个宏组并单击 Edit，向该组添加或从该组删除宏。在该工具中可以执行下列任务：

- 在左侧或右侧框中选择一个或多个宏，再用箭头把所选宏从一侧移到另一侧。
- 在左侧或右侧框中选择一个或多个宏，并单击 Flip 部分中的方向按钮，把所选宏的方向改为 MX 或 MY。

完成后单击 OK。

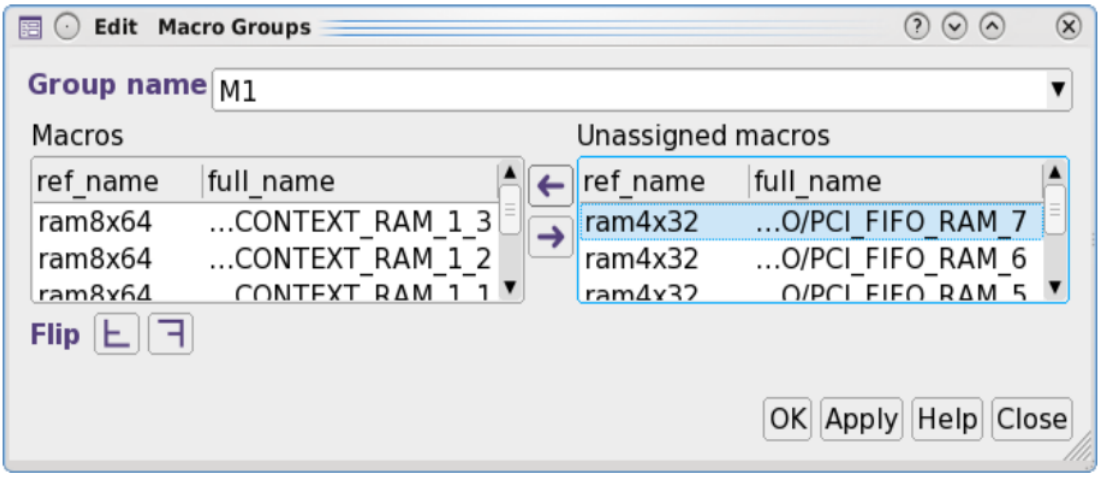


图 **9.21:** 编辑宏组

列表（**List**）

单击 List 按钮，显示当前宏组的列表。单击列表中的实例名，可以在版图中选择宏组、宏集合或单个宏。

网线连接（**Net Connections**）

单击 Net Connections 按钮，打开 Net Connections 面板。

拆分（**Split**）

创建一个新组，并根据连接关系把当前所选组中的部分宏移入新组。

宏阵列（**Macro Array**）

在版图中选择两个或更多宏，单击 Macro Array 按钮以启动宏阵列工具。也可以从菜单选择 **Edit > Macro Array**，打开 Macro Configuration 对话框。

宏组外形（**Macro Group Shape**）

在版图中选择一个或多个宏组外形，单击 Rectangle、L、Auto 或 Manual，再单击 Set 以设置外形。也可以用 `set_macro_group_shape` 命令设置外形。

打包（**Pack**）

在版图中选择一个或多个宏组外形，单击 Pack 按钮，把宏放入宏组外形。也可以用 `pack_macro_group` 命令。

还原（**Revert**）

单击 Revert 按钮，撤销最近一次打包操作。可以多次单击 Revert 按钮以执行多次撤销。

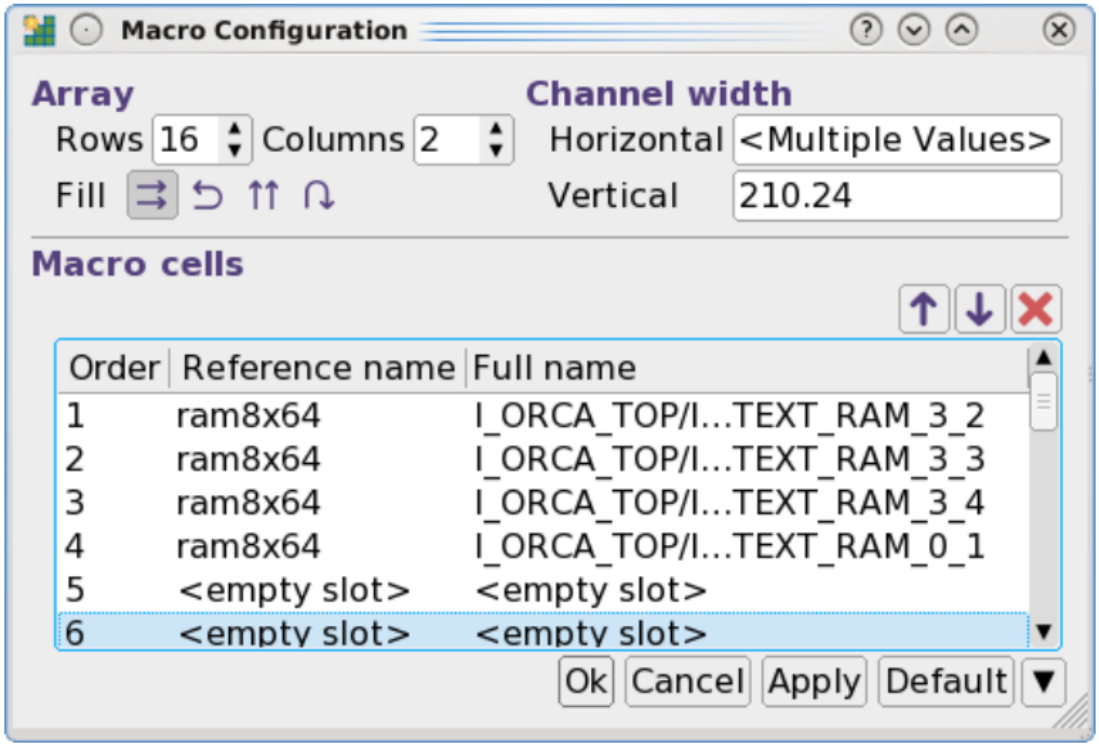


图 **9.22:** 宏配置

克隆（**Clone**）

单击 Clone 按钮，把宏布局从一个宏组复制到另一个宏组。首先选择源宏布局组。然后单击 Clone Packing to 按钮，从列表中选择目标组，再单击 OK。

宏组与宏（**Macro Group and Macros**）

在版图中选择一个宏，单击 Macro Group and Macros 按钮以选择相应的宏组。也可以选择一个宏组，再单击 Macro Group and Macros 按钮以选择该组中的宏。

9.15 执行时序驱动与拥塞驱动布局

Performing Timing-Driven and Congestion-Driven Placement

`create_placement` 命令同时支持宏与标准单元的时序驱动布局与拥塞驱动布局。用 `create_placement` 命令的选项，以及应用选项 `plan.place.timing_driven_mode` 与 `plan.place.congestion_driven_mode`，控制如何执行时序驱动与拥塞驱动布局。

9.15.1 拥塞驱动布局

Congestion-Driven Placement

要创建拥塞驱动布局，在 `create_placement` 命令中同时使用 `-congestion` 与 `-floorplan` 选项。使用 `-congestion` 选项时，还可以用 `-congestion_effort low | medium | high` 选项控制施加于

拥塞驱动布局的努力级别。可以在同一条 create_placement 命令中同时指定时序驱动布局与拥塞驱动布局。

根据应用选项 plan.place.congestion_driven_mode 的设置，拥塞驱动布局可以执行三种拥塞降低：

- macro：只降低宏之上以及宏之间的拥塞
- std_cell：降低标准单元之间的拥塞
- both：同时降低宏之间与标准单元之间的拥塞

下例只在硬宏之间施加拥塞降低，并以高努力级别执行粗布局：

```
icc2_shell> set_app_options -name plan.place.congestion_driven_mode \
   -value macro
icc2_shell> create_placement -floorplan -congestion \
   -congestion_effort high
```

9.15.2 时序驱动布局

Timing-Driven Placement

要创建时序驱动布局，在 create_placement 命令中使用 -timing_driven 与 -floorplan 选项。可以在同一条 create_placement 命令中同时指定时序驱动布局与拥塞驱动布局。

根据应用选项 plan.place.timing_driven_mode 的设置，时序驱动布局可以执行三种布局：

- macro：只对宏执行时序驱动布局
- std_cell：只对标准单元执行时序驱动布局
- both：对宏与标准单元都执行时序驱动布局

下例只对标准单元施加时序驱动布局；宏按常规方式放置。

```
icc2_shell> set_app_options -name plan.place.timing_driven_mode \
   -value std_cell
icc2_shell> create_placement -floorplan -timing_driven
```

9.15.3 同时执行拥塞驱动与时序驱动布局

Combined Congestion-Driven and Timing-Driven Placement

要用同一条 create_placement 命令同时执行拥塞驱动布局与时序驱动布局，同时使用 -congestion 与 -timing_driven 选项，并指定应用选项 plan.place.congestion_driven_mode 与 plan.place.timing_driven_mode。下例对宏与标准单元都执行拥塞驱动与时序驱动布局。

```
icc2_shell> set_app_options -name plan.place.congestion_driven_mode \
   -value both
icc2_shell> set_app_options -name plan.place.timing_driven_mode \
   -value both
icc2_shell> create_placement -floorplan -congestion -timing_driven
```

Chapter 10

执行电源规划

Performing Power Planning

电源规划（power planning）包括电源网络布线（power network routing）与电源网络分析（power network analysis），是做出电源完整性（power integrity）良好的设计所必需的。稳健的电源与地（power and ground，PG）网格可以提供足够数量的电源地焊盘（pad）与电源轨（rail），从而降低 IR 压降（IR drop）与电迁移（electromigration）。电源规划可用于评估电源网所消耗的布线资源（routing resource），并判断电源规划对可布线性（routability）的影响。可以试验不同的电源规划，或通过修改命令选项设置并重新生成电源规划来微调已有规划。

创建设计电源规划的流程如图 **??** 所示。

基于图案的电源规划（pattern-based power planning）把电源环（power ring）与电源网格（power mesh）的物理实现细节（层、宽度、间距）与这些结构所插入的设计区域分开。在典型设计中，该流程会运行多遍。第一遍定义并创建电源环，第二遍定义并创建电源网格，依此类推。

`create_pg_ring_pattern`、`create_pg_mesh_pattern`、`create_pg_macro_conn_pattern` 与 `create_pg_std_cell_conn_pattern` 命令创建图案规格（pattern specification），用以定义电源规划的物理实现细节，并把宽度、间距、偏移以及过孔规则规格与设计中的金属层关联起来。图案定义之后，可以在设计的不同电源区域之间方便地复用。

`set_pg_strategy` 命令把图案规格赋给设计中特定的电源网与区域。也可以用该命令定义图案起点的偏移、电源带（power strap）延伸规格以及阻挡（blockage）规格。

基于图案的电源规划过程中可以创建并实现复杂的过孔规则。`set_pg_via_master_rule` 命令定义接触孔代码（contact code）、接触孔代码在过孔结构内的偏移、切割间距，以及定制过孔结构的其他参数。电源过孔主单元（via master）通过在相应的 `create_pg_*_pattern` 命令上指定 `-via_rule` 选项，与特定的电源规划策略（strategy）关联。

`create_pg_*_pattern` 与 `set_pg_strategy` 命令规定电源规划，但并不修改设计。创建图案与策略定义之后，使用 `compile_pg` 命令把电源结构实例化到设计中。在创建电源环与电源网格之后若要修改电源规划，用 `compile_pg -undo` 删除环与网格，再修改过孔规则、图案规格或电源规划策略。

实例化电源规划之后，可以检查设计规则检查（design rule check，DRC）与连通性（connectivity）违例，以验证电源规划的完整性。使用 `check_pg_drc` 命令报告违反布线设计规则的电源结构。使用 `check_pg_connectivity` 命令检查电源网络的物理连通性。

下列各节说明这些主题：

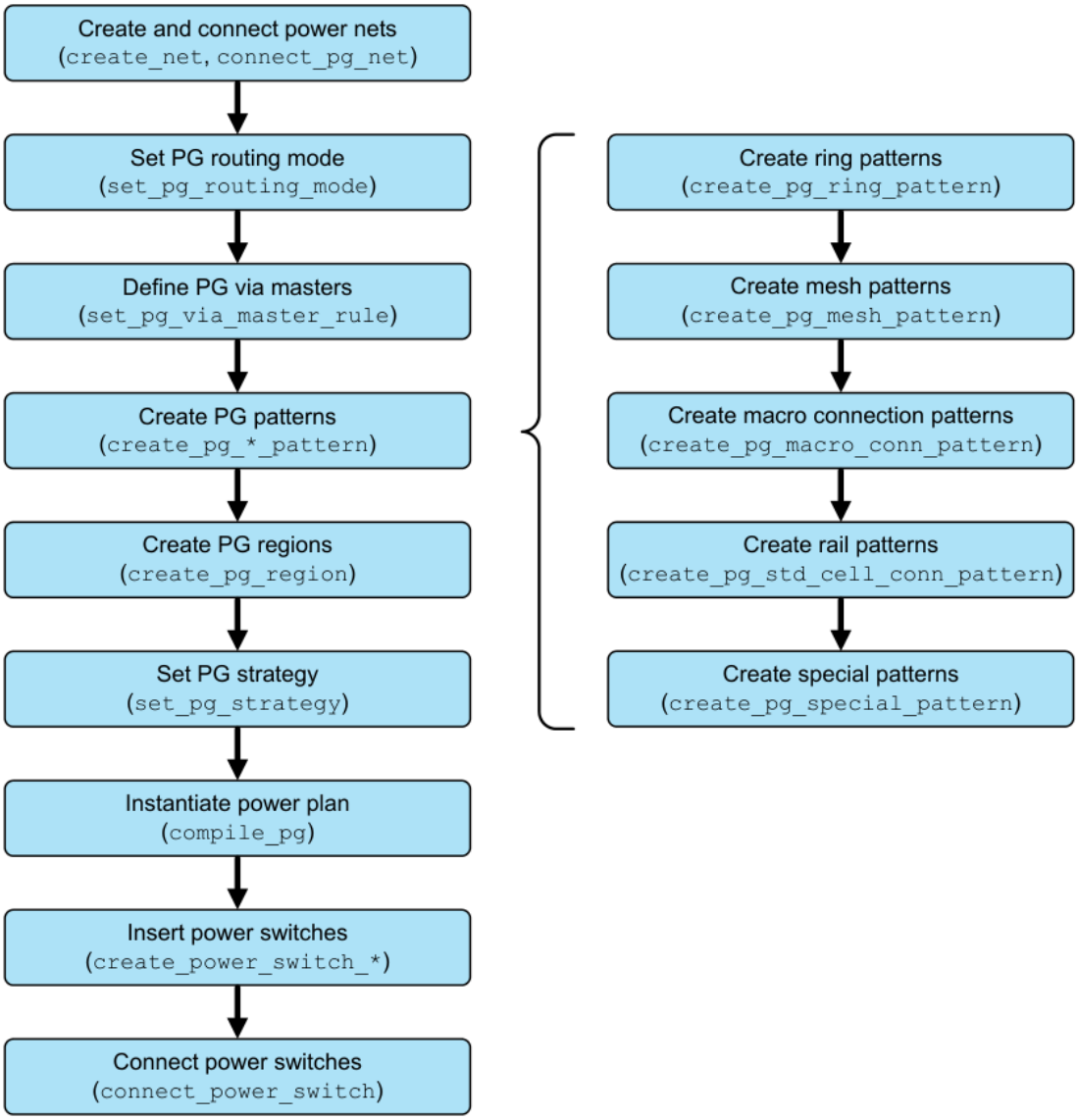


图 **10.1:** 基于图案的电源规划流程

- 创建并连接电源网
- 设置电源地布线模式
- 定义电源地过孔主单元
- 创建电源与地环图案
- 创建电源与地网格图案
- 创建电源与地宏单元连接
- 创建电源与地标准单元电源轨
- 创建通道与对齐电源带
- 创建复杂复合图案
- 定义电源规划区域
- 设置电源规划策略
- 在不同策略之间创建过孔规则
- 实例化电源规划
- 基于图案的电源网络布线示例
- 电源地脚本生成器
- 手工创建电源带与过孔
- 使用辅助电源地约束
- 遵守相对阻挡的金属与过孔间距规则
- 使用电源地图案形状
- 使用过孔矩阵
- 调试因 DRC 违例而被删除的过孔与电源地带
- 合并电源地网格中的形状
- 向已有电源网格插入钉扎过孔
- 把过孔阶梯连接到电源与地
- 插入并连接电源开关
- 修剪电源与地网格
- 检查电源网络完整性
- 分析电源规划
- 执行分布式电源网络布线

10.1 创建并连接电源网

Creating and Connecting Power Nets

开始基于图案的电源规划时，先在设计中定义电源网与地网，并把它们连接到电源引脚与地引脚。若使用统一功耗格式（Unified Power Format，UPF）描述电源网络，可以跳过这些步骤。

1. 用 `create_net` 命令定义电源网与地网。

```
icc2_shell> create_net -power VDD
{VDD}
icc2_shell> create_net -ground VSS
{VSS}
```

2. 用 `connect_pg_net` 命令把电源网与地网连接到电源引脚与地引脚。

```
icc2_shell> connect_pg_net -net VDD [get_pins -physical_context *VDD]
icc2_shell> connect_pg_net -net VSS [get_pins -physical_context *VSS]
```

若设计带有电源网络的 UPF 描述，可以指定 `connect_pg_net -automatic`，从电源域（power domain）规格直接推导电源网与地网并完成连接。

10.2 设置电源地布线模式

Setting the PG Routing Mode

根据当前所处的设计规划阶段，可以设置电源地布线模式，使工具按指定模式优化电源地布线。

使用 `set_pg_routing_mode` 命令，把电源地布线模式设为表 **??** 中的某一取值。设置为这些取值之一时，工具会自动更改部分与电源地布线相关的应用选项（application option）的默认值（见表 **??**），以达到设计中预期的电源地布线效果。

表 **10.1:** 电源地布线模式

要完成的工作	使用
保持与电源地布线相关的应用选项的默认值不变	`default`
确保设计 DRC 干净并可以流片（tapeout）	`tapeout`
在电源地规划开始时快速检查电源地结构。此模式下，工具可能跳过过孔创建的 DRC 修复，甚至禁用过孔创建。	`fast_exploration`
通过过孔定义选择或交点选择创建尽可能多的电源地过孔。此模式下工具创建更多过孔，电阻更低，因而电流更大。	`maximize_via`

表 **10.2:** 已设置电源地布线模式时，与电源地布线相关的应用选项取值

应用选项	默认值	特定模式下的默认值
plan.pgroute.discard_stackvia_with_drc	false	fast_exploration 模式为 true
plan.pgroute.disable_via_creation	false	fast_exploration 模式为 true
plan.pgroute.disable_stapling_via_fixing	false	fast_exploration 模式为 true
plan.pgroute.fix_via_drc_multiple_viadef	false	maximize_via 模式为 true
plan.pgroute.honor_ndr_same_net_spacing	false	tapeout 模式为 true
plan.pgroute.honor_ndr_spacing_drc	false	tapeout 模式为 true
plan.pgroute.honor_signal_route_drc	false	tapeout 模式为 true
plan.pgroute.honor_std_cell_drc	false	tapeout 模式为 true
plan.pgroute.load_frame_view	false	fast_exploration 模式为 true
plan.pgroute.create_via_cellalign_maxpin_only	true	maximize_via 模式为 false

要查看当前块中与电源地布线相关的应用选项取值，运行 report_pg_routing_mode。例如，下面的报告给出电源地布线模式为 default 时的应用选项默认值。

```
icc2_shell> report_pg_routing_mode -nosplit
****************************************
Report : report_pg_routing_mode
Design : ORCA_TOP
Version: T-2022.03-SP3-BETA
Date   : Wed Jun 22 19:25:20 2022
****************************************
--------------------------------------------------------------------------
Mode                          default
--------------------------------------------------------------------------
App option                                    Current  Mode     Default
                                              value    default  value
--------------------------------------------------------------------------
plan.pgroute.create_via_cellalign_maxpin_only true     true     true
plan.pgroute.load_frame_view                  false    false    false
plan.pgroute.honor_ndr_same_net_spacing       false    false    false
plan.pgroute.honor_std_cell_drc               false    false    false
plan.pgroute.honor_signal_route_drc           false    false    false
plan.pgroute.honor_ndr_spacing_drc            false    false    false
plan.pgroute.disable_stapling_via_fixing      false    false    false
plan.pgroute.disable_via_creation             false    false    false
plan.pgroute.fix_via_drc_multiple_viadef      false    false    false
plan.pgroute.discard_stackvia_with_drc        false    false    false
```

```
1
```

下面的报告给出电源地布线模式为 tapeout 时的应用选项默认值。

```
icc2_shell> report_pg_routing_mode -nosplit
****************************************
Report : report_pg_routing_mode
Design : ORCA_TOP
Version: T-2022.03-SP3-BETA
Date   : Sun Jun 26 21:07:58 2022
****************************************
Mode                          tapeout
App option                                        Current  Mode     Default
                                                  value    default  value
plan.pgroute.create_via_cellalign_maxpin_only     true     true     true
plan.pgroute.load_frame_view                      false    false    false
plan.pgroute.honor_ndr_same_net_spacing           true     true     false
plan.pgroute.honor_std_cell_drc                   true     true     false
plan.pgroute.honor_signal_route_drc               true     true     false
plan.pgroute.honor_ndr_spacing_drc                true     true     false
plan.pgroute.disable_stapling_via_fixing          false    false    false
plan.pgroute.disable_via_creation                 false    false    false
plan.pgroute.fix_via_drc_multiple_viadef          false    false    false
plan.pgroute.discard_stackvia_with_drc            false    false    false
```

10.3 定义电源地过孔主单元

Defining PG Via Masters

若设计需要工艺文件中未定义的过孔主单元，可以用 set_pg_via_master_rule 命令为电源网络创建过孔主单元。在 create_pg_composite_pattern、create_pg_macro_conn_pattern、create_pg_mesh_pattern 或 create_pg_ring_pattern 命令的 -via_rule 选项中引用该过孔主单元，即可在电源网络中使用它。

要为电源网络创建新的过孔主单元，按下面的示例使用 set_pg_via_master_rule 命令。

```
icc2_shell> set_pg_via_master_rule VIA78_2x2 -contact_code VIA78 \
   -via_array_dimension {2 2} -offset {3 1}
```

要用相对过孔包围盒原点的偏移，使用 -offset {x y} 选项。要指定本过孔所使用的过孔主单元或接触孔代码，使用 -contact_code code_name 选项。要指定本规则的过孔阵列尺寸，使用 -via_array_dimension {column row} 选项。要用接触孔代码或过孔阵列的多次重复来填充过孔规则时，使用 -allow_multiple {x_pitch y_pitch} 选项指定所用的水平节距与垂直节距。要指定简单阵列过孔中切割之间的水平间距与垂直间距，使用 -cut_spacing {horizontal_spacing vertical_spacing} 选项。

要在设计中使用新规则，在 create_pg_composite_pattern、create_pg_macro_conn_pattern、create_pg_mesh_pattern 与 create_pg_ring_pattern 命令的 via_master 关键字之后指定过孔规则名。例如，下面的命令使用前一条命令所定义的 VIA78_2x2 过孔规则。

```
icc2_shell> create_pg_mesh_pattern mesh_pat \
   -layers {layer_spec} \
   -via_rule { \
      {{layers: M6} {layers: M7} {via_master: default}} \
      {{layers: M7} {layers: M8} {via_master: VIA78_2x2}}}
```

10.3.1 使用总切割面积最大的过孔主单元

Using the Via Master With the Maximum Total Cut Area

要根据走线交点的包围盒，使用总切割面积最大的过孔主单元或 ContactCode，把应用选项 `plan.pgroute.maximize_total_cut_area` 设为下列取值之一：

- `all`：考虑全部可能的 ContactCode。
- `default`：使用默认 ContactCode。
- `Vs`：使用方形切割的 ContactCode。
- `Vh`：使用水平条形的 ContactCode。
- `Vv`：使用垂直条形的 ContactCode。

10.4 创建电源与地环图案

Creating Power and Ground Ring Patterns

环图案规定用于创建电源环与地环的水平层名与垂直层名、环宽度、间距、过孔以及转角桥接。可以在核（core）、设计块、宏单元（macro）、电源地区域，或所指定的直线多边形周围创建环图案。按如下方式使用 `create_pg_ring_pattern` 命令定义环图案：

```
icc2_shell> create_pg_ring_pattern ring_pat -horizontal_layer M7 \
   -horizontal_width {10} -horizontal_spacing {2} \
   -vertical_layer M8 -vertical_width {10} \
   -vertical_spacing {2} -corner_bridge false
```

也可以用任务助手（Task Assistant）定义环图案。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 在 Task Assistant 中选择 **PG Planning > Ring/Hard Macro/Std Cell Pattern**。
3. 单击 Ring 选项卡。
4. 在 Task Assistant 中输入图案名、层名、宽度以及其他环参数。
5. 单击 Apply 创建图案。

也可以用参数定义 `create_pg_ring_pattern` 命令的宽度、间距及其他取值。要创建参数，加上 `-parameters` 选项并后跟参数名，再用参数名替换命令选项的取值。参数由参数名前面加上 `(@)` 字符来标识。

在用 set_pg_strategy 命令设置之前，参数的取值保持未定义。参数的设置顺序与 -parameters 选项所指定的顺序相同。采用这种方式，可以复用该命令所指定的图案，并用 set_pg_strategy 命令施加不同的参数值。

下面的示例用参数定义环图案的金属层、宽度、间距与转角桥接设置。

```
icc2_shell> create_pg_ring_pattern ring_pat -horizontal_layer @hlayer \
   -horizontal_width {@hwidth} -horizontal_spacing {@hspace} \
   -vertical_layer @vlayer -vertical_width {@vwidth} \
   -vertical_spacing {@vspace} -corner_bridge @cbridge \
   -parameters {hlayer hwidth hspace
                vlayer vwidth vspace cbridge}

icc2_shell> set_pg_strategy ring_strat -core \
   -pattern {{name: ring_pat} {nets: {VDD VSS}}
             {offset: {3 3}} {parameters: {M7 10 2 M8 10 2 true}}} \
   -extension {{stop: design_boundary}}
```

10.5 创建电源与地网格图案

Creating Power and Ground Mesh Patterns

网格图案规定用于创建电源网格与地网格的水平层名与垂直层名、金属宽度、金属间距、金属节距、过孔以及走线修剪。按如下方式使用 create_pg_mesh_pattern 命令定义网格图案：

```
icc2_shell> create_pg_mesh_pattern mesh_pat -layers {
   {{vertical_layer: M8} {width: 5}
       {spacing: interleaving} {pitch: 32}}
   {{vertical_layer: M6} {width: 2}
       {spacing: interleaving} {pitch: 32}}
   {{horizontal_layer: M7} {width: 5}
       {spacing: interleaving} {pitch: 28.8}}} \
   -via_rule {
       {{layers: M6} {layers: M7} {via_master: default}}
       {{layers: M8} {layers: M7} {via_master: VIA78_3x3}}}
```

也可以用 Task Assistant 定义网格图案。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 在 Task Assistant 中选择 **PG Planning > Mesh Pattern**。
3. 输入图案名并定义过孔规则。
4. 选择层、方向、宽度、间距、节距与偏移取值。
5. 单击表行中的条目以编辑该设置。
6. 单击 Preview，在 Tcl Command 框中查看 Tcl 命令。
7. 单击 Run Tcl Command 创建网格图案。

也可以按「创建电源与地环图案」一节所述，用参数定义电源网格的设置。下面的示例用参数定义 create_pg_mesh_pattern 命令的金属宽度。

```
icc2_shell> create_pg_mesh_pattern mesh_pat -layers {
   {{vertical_layer: M8} {width: @width8}
       {spacing: interleaving} {pitch: 32}}
   {{vertical_layer: M6} {width: @width6}
       {spacing: interleaving} {pitch: 32}}
   {{horizontal_layer: M7} {width: @width7}
       {spacing: interleaving} {pitch: 28.8}}} \
   -via_rule {
       {{layers: M6} {layers: M7} {via_master: default}}
       {{layers: M8} {layers: M7} {via_master: VIA78_3x3}}} \
   -parameters {width6 width7 width8}

icc2_shell> set_pg_strategy mesh_strat -core -pattern {
   {name: mesh_pat} {nets: {VDD VSS}}
      {parameters: {32 28.8 32}}}
```

10.6 创建电源与地宏单元连接

Creating Power and Ground Macro Connections

宏单元连接图案规定电源网与地网、布线方向、金属层、层宽、层间距、金属节距以及其他信息，用以创建到宏单元的连接。按下面的示例使用 `create_pg_macro_conn_pattern` 定义宏单元连接图案。

```
icc2_shell> create_pg_macro_conn_pattern macro_pat -nets {VDD VSS} \
   -direction horizontal -width 1 -layers M5 -spacing minimum \
   -pitch 5 -pin_conn_type long_pin
```

也可以用 Task Assistant 定义标准单元电源轨图案。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 选择 **PG Planning > Ring/Hard Macro/Std Cell Pattern**。
3. 按硬宏引脚连接类型，单击 HM scattered pin、HM long pin 或 HM ring pin 选项卡。
4. 在表单中输入图案名、层、宽度以及其他信息。
5. 单击 Apply 创建图案。

也可以按「创建电源与地环图案」一节所述，用参数定义标准单元电源轨的设置。下面的示例用参数定义标准单元电源轨图案的金属宽度，并用 `set_pg_strategy` 命令为参数赋值。

```
icc2_shell> create_pg_macro_conn_pattern macro_pat -nets {VDD VSS} \
   -direction horizontal -width @width -layers M5 -spacing minimum \
   -pitch @pitch -pin_conn_type long_pin -parameters {width pitch}

icc2_shell> set_pg_strategy macro_strat -core \
   -pattern {{pattern: macro_pat} {nets: {VDD VSS}}
             {parameters: {1 5}}}
```

10.7 创建电源与地标准单元电源轨

Creating Power and Ground Standard Cell Rails

标准单元电源轨图案规定用于为标准单元（standard cell）行创建电源轨与地轨的金属层、轨宽与轨偏移。按下面的示例使用 create_pg_std_cell_conn_pattern 定义标准单元电源轨图案。

```
icc2_shell> create_pg_std_cell_conn_pattern rail_pat -layers {M1} \
   -rail_width {0.2 0.2}
```

也可以用 Task Assistant 定义标准单元电源轨图案。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 在 Task Assistant 中选择 **PG Planning > Ring/Hard Macro/Std Cell Pattern**。
3. 单击 Std cell 选项卡。
4. 在 Task Assistant 中输入图案名、层名、轨宽与偏移。
5. 单击 Apply 创建图案。

若不定义轨宽，该命令使用标准单元电源引脚与地引脚的宽度作为轨宽。

也可以按「创建电源与地环图案」一节所述，用参数定义标准单元电源轨的设置。下面的示例用参数定义标准单元电源轨图案的金属宽度，并用 set_pg_strategy 命令为参数赋值。

```
icc2_shell> create_pg_std_cell_conn_pattern rail_pat -layers {M1} \
   -rail_width {@wtop @wbottom} -parameters {wtop wbottom}

icc2_shell> set_pg_strategy rail_strat -core \
   -pattern {{name: rail_pat} {nets: VDD VSS}
             {parameters: {0.2 0.2}}}
```

10.8 创建通道与对齐电源带

Creating Channel and Alignment Power Straps

特殊图案为下列类型的电源地带定义结构：

- 放在通道（channel）中的电源带
- 为连接并对齐端子而插入的电源带与过孔
- 为对齐电源开关（power switch）而插入的电源带
- 为对齐物理专用单元（physical-only cell）与填充单元（filler cell）而插入的电源带

单条 create_pg_special_pattern 命令中只能指定一种图案类型。按下面的示例使用 create_pg_special_pattern 命令定义这些图案。

```
icc2_shell> create_pg_special_pattern channel_pattern \
   -insert_channel_straps {{layer: M6} {direction: vertical} \
      {width: 2} \
      {channel_between_objects: {macro placement_blockage voltage_area} \
```

```
}}
```

也可以用 Task Assistant 定义特殊图案。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 在 Task Assistant 中选择 **PG Planning > Special Pattern**。
3. 单击 Channel straps 选项卡。Switch alignment、Terminal alignment、Physical cell alignment 与 Max stdcell distance 选项卡也同样可用。
4. 填写表单信息，例如层名、方向、宽度、间距以及其他设置。
5. 单击 Apply 创建图案。

10.9 创建复杂复合图案

Creating Complex Composite Patterns

本节用示例说明如何使用 `create_pg_composite_pattern` 命令创建复杂的电源地图案。

- 创建中心对齐过孔
- 创建桥接带
- 创建过孔桥接带
- 创建锥形带
- 创建棋盘格过孔图案

10.9.1 创建中心对齐过孔

Creating Center-Aligned Vias

下面的示例在 M3 层上创建水平电源带，在 M8 层上创建垂直电源带。堆叠过孔连接水平带与垂直带，并且过孔被推向每一对 VDD/VSS 垂直电源带的中心，以节省电源带下方的布线资源。`create_pg_wire_pattern` 命令定义基本图案，`create_pg_composite_pattern` 命令定义水平与垂直电源地走线的参数。

本示例定义两条过孔主单元规则：`via_5x6_shift_right` 与 `via_5x6_shift_left`。对每个过孔，`-offset` 选项把过孔相对中心平移 0.8 微米。

`create_pg_composite_pattern` 命令的 `-via_rule` 选项规定在 M3 层与 M8 层的每个交点上放置哪一种过孔，如图 **??** 所示。本示例中，`-add_patterns` 选项实例化两个 `create_pg_wire_pattern` 定义：M8 层上的垂直电源带图案（`pattern_id: 1`），以及 M3 层上的水平电源带图案（`pattern_id: 2`）。`create_pg_composite_pattern` 还为两条电源地网引用四个线网标识：VSS（`net_id: 1` 与 `4`）以及 VDD（`net_id: 2` 与 `3`）。

`via_5x6_shift_right` 过孔插在第一图案的 `pattern_id: 1`、`net_id: 1`（M8 层、线网 VSS）与第二图案的 `pattern_id: 2`、`net_id: 2`（M3 层、线网 VSS）的交点处，即图中位置 (A)。`via_5x6_shift_right` 过孔也插在 `pattern_id: 1`、`net_id: 3`（M8 层、线网 VDD）与 `pattern_id: 2`、`net_id: 1`（M3 层、线网 VDD）处，即图中位置 (D)。其余语句以同样方式定义位置 (B) 与 (C) 处的过孔。

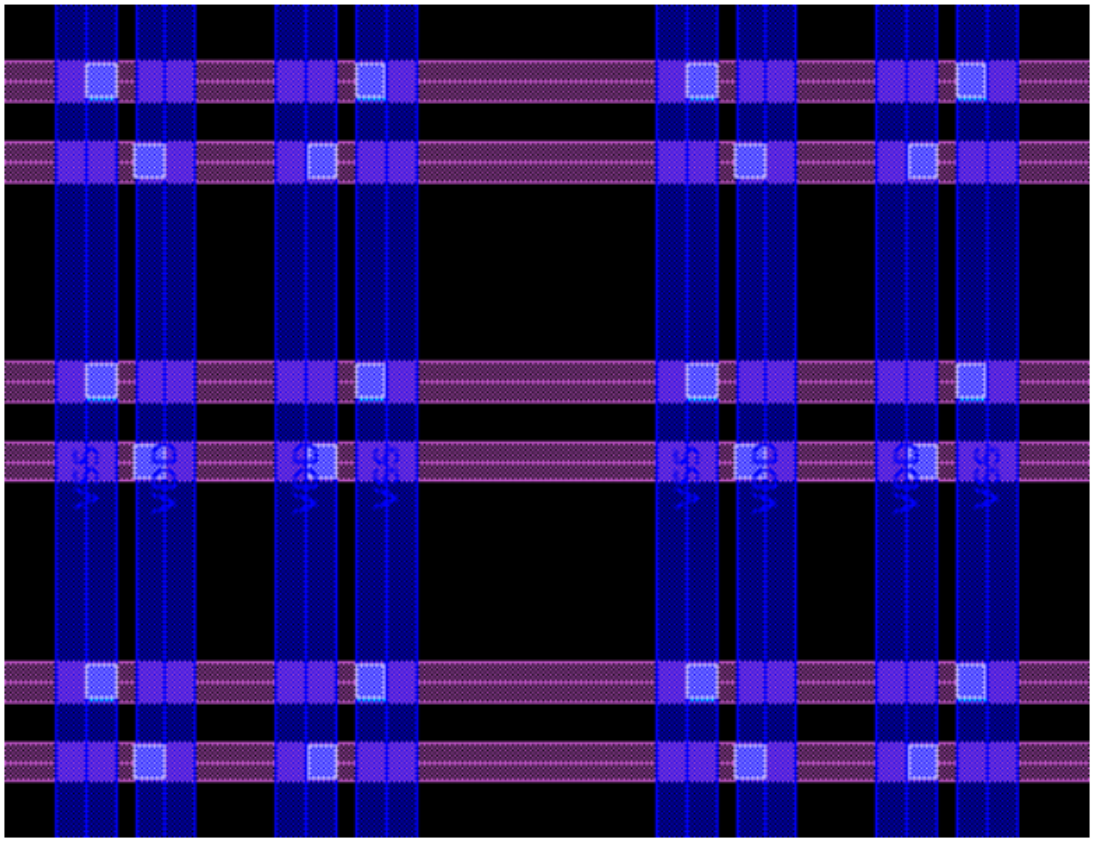


图 **10.2:** 带中心对齐过孔的电源带

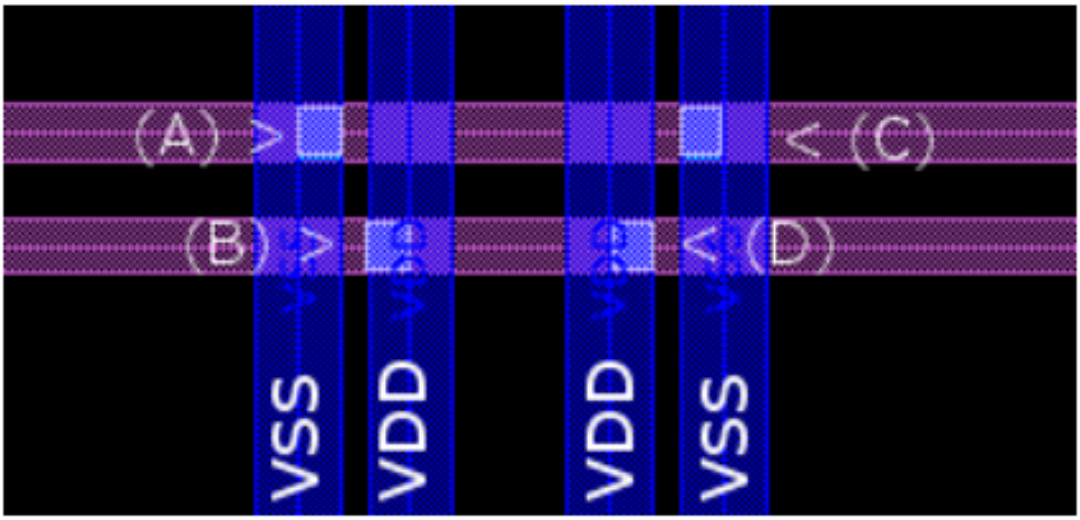


图 **10.3:** 标注后的中心对齐过孔

创建该电源规划的命令如下：

```
set_pg_via_master_rule via_5x6_shift_right \
   -via_array_dimension {5 6} -offset {0.8 0}

set_pg_via_master_rule via_5x6_shift_left \
   -via_array_dimension {5 6} -offset {-0.8 0}

create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

# For the vertical straps on M8 defined by pattern_id: 1
# - net_id: 1 is VSS
# - net_id: 2 is VDD
# - net_id: 3 is VDD
# - net_id: 4 is VSS
#
# For the horizontal straps on M3 defined by pattern_id: 2
# - net_id: 1 is VDD
# - net_id: 2 is VSS

create_pg_composite_pattern center_aligned_vias -nets {VDD VSS} \
   -add_patterns { \
      {{pattern: wire_base} {nets: VSS VDD VDD VSS} \
       {parameters: {M8 vertical 3 30 {1 4 1}}}{offset: 4}} \
      {{pattern: wire_base} {nets: VDD VSS} \
       {parameters: {M3 horizontal 2 15 2}}{offset: 4}} \
   } \
   -via_rule { \
      {{{pattern_id: 1}{net_id: 1}} \
       {{pattern_id: 2}{net_id: 2}} \
       {via_master: via_5x6_shift_right}} \
      {{{pattern_id: 1}{net_id: 2}} \
       {{pattern_id: 2}{net_id: 1}} \
       {via_master: via_5x6_shift_left}} \
      {{{pattern_id: 1}{net_id: 3}} \
       {{pattern_id: 2}{net_id: 1}} \
       {via_master: via_5x6_shift_right}} \
      {{{pattern_id: 1}{net_id: 4}} \
       {{pattern_id: 2}{net_id: 2}} \
       {via_master: via_5x6_shift_left}} \
      {{intersection: undefined}{via_master: NIL}} \
  }
set_pg_strategy core_patterns \
    -pattern {{name: center_aligned_vias} \
              {nets: VDD VSS } {offset: {5 5}}} -core

compile_pg -strategies core_patterns
```

10.9.2 创建桥接带

Creating Bridging Straps

下面的示例创建桥接带（bridging strap），用以连接平行的垂直电源带。该电源规划包含下列特征：

- 宽度分别为 2 微米、4 微米与 2 微米的三条 VDD 电源带，在 M6 层上组成一组垂直带
- 宽度分别为 2 微米、4 微米与 2 微米的三条 M6 层 VSS 电源带，同样在 M6 层上组成一组垂直带
- 每组三条垂直 VDD 带用 M3 层上宽 2 微米的电源带连在一起，节距 15 微米，长度 12 微米，并在 M6 与 M3 之间使用 5×1 堆叠过孔阵列
- 每组三条垂直 VSS 带用 M5 层上宽 2 微米的电源带连在一起，节距 15 微米，长度 12 微米，并在 M5 与 M6 之间使用 5×3 过孔阵列

用于连接这些电源带的桥接带与过孔结构如图 **??** 与图 **??** 所示。

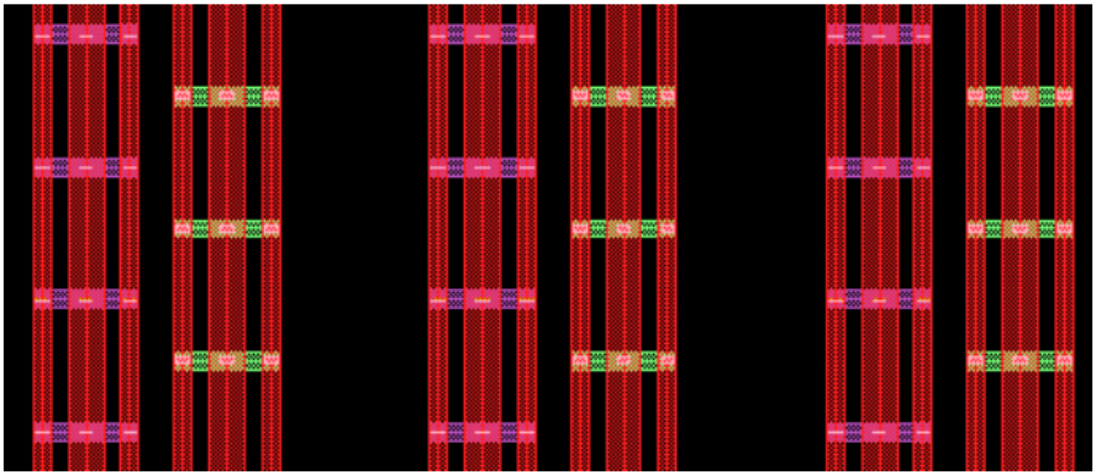


图 **10.4:** 桥接带

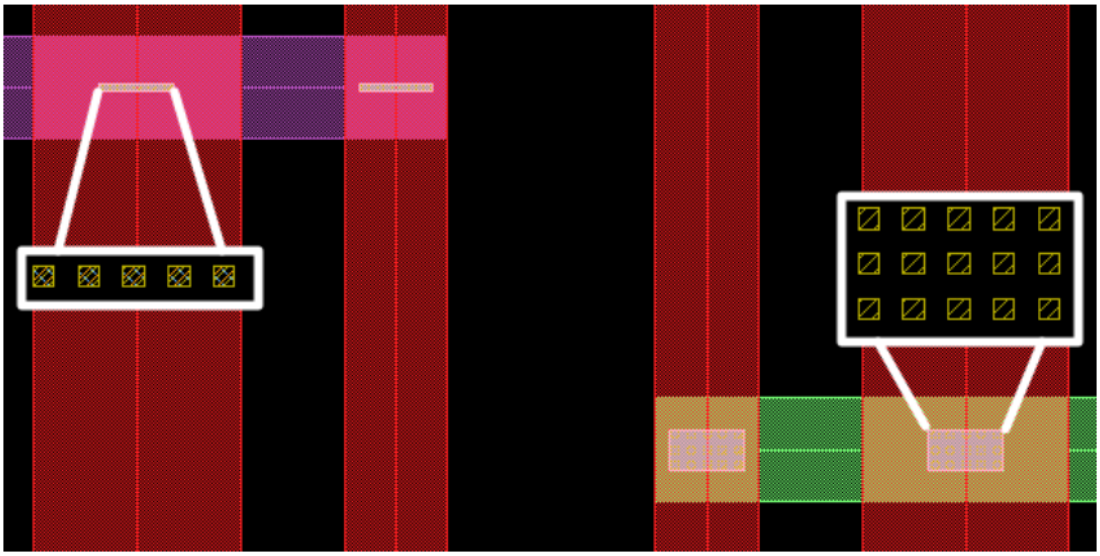


图 **10.5:** 5×1 与 5×3 过孔

set_pg_via_master_rule 命令定义 5×1 与 5×3 过孔阵列的过孔规则。create_pg_composite_pattern 命令定义一份使用两种不同走线图案的电源规划：名为 wire_base 的图案用于定义 VDD 与 VSS 垂直带，名为 segment_base 的图案用于定义连接 VDD 的 M3 层电源带以及连接 VSS 的 M5 层电源带。走线图案通过 {pattern: wire_base} 与 {pattern: segment_base} 语句引用。

create_pg_composite_pattern 的 -via_rule 选项定义在电源规划的不同金属交点处插入哪些过孔。语句 {{layers: M6} {layers: M5} {via_master: via65_5x3}} 在电源规划中 M6 层与 M5 层的任意交点处插入 via65_5x3 过孔主单元。

-via_rule 语句 {{intersection: undefined} {via_master: NIL}} 保证电源规划中只插入所指定的过孔。对于包含许多走线图案的设计，该语句保证不会在未定义的层对之间插入过孔；默认情况下，未定义的层对也会创建过孔。

实现该电源规划所用的命令如下：

```
set_pg_via_master_rule via63_5x1 \
   -contact_code {VIA34 VIA45 VIA56} \
   -via_array_dimension {5 1}

set_pg_via_master_rule via65_5x3 \
   -contact_code {VIA56} \
   -via_array_dimension {5 3}

create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_wire_pattern segment_base -layer @l -direction @d \
   -width @w -low_end_reference_point 0 \
   -high_end_reference_point @len -pitch @p \
   -parameters {l d w len p}
create_pg_composite_pattern bridging_straps -nets {VDD VSS} \
   -add_patterns { \
   {{pattern: wire_base} {nets: VDD VSS} \
    {parameters: {M7 horizontal 2 60 2}}{offset: 13}} \
   {{pattern: wire_base} {nets: VDD VDD VDD VSS VSS VSS} \
    {parameters: {M6 vertical {2 4 2 2 4 2} 45 {2 2 4 2 2}}}
    {offset: 4}} \
   {{pattern: segment_base} {nets: VDD} \
    {parameters: {M3 horizontal 2 12 {45 15}}} {offset: {3 2}}} \
   {{pattern: segment_base} {nets: VSS} \
    {parameters: {M5 horizontal 2 12 {45 15}}} {offset: {19 10}}} } \
   -via_rule { \
       {{layers: M7}{layers: M6} {via_master: default}} \
       {{layers: M6}{layers: M5} {via_master: via65_5x3}} \
       {{layers: M6}{layers: M3} {via_master: via63_5x1}} \
       {{intersection: undefined} {via_master: NIL}} \
  }

set_pg_strategy core_patterns \
    -pattern {{name: bridging_straps} \
                  {nets: VDD VSS } {offset: {5 5}}} -core
```

```
compile_pg -strategies core_patterns
```

10.9.3 创建过孔桥接带

Creating Via Bridging Straps

下面的示例创建一种复杂的、经过平移的过孔桥接结构，把 M8 层上的垂直 VDD 带连接到 M3 层上的水平 VDD 带。水平与垂直的 VSS 带使用标准堆叠过孔连接 M8 层与 M3 层。

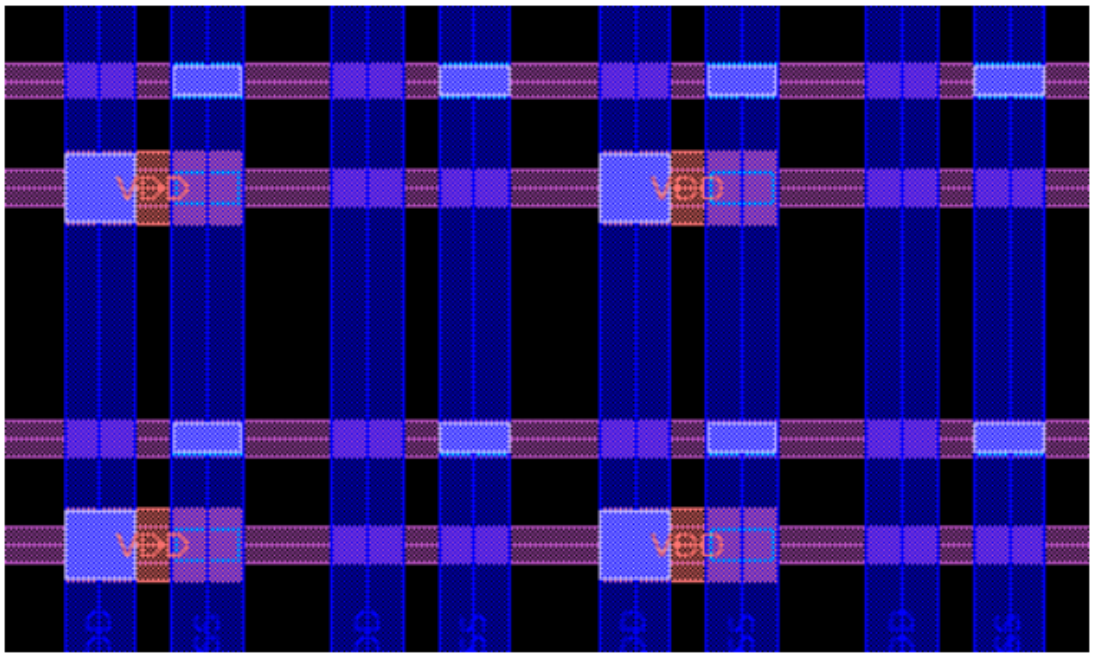


图 **10.6:** 复合桥接图案

对于 VDD 电源带，13×13 过孔阵列把 M8 层上的垂直带与 M7 层上的短水平金属段连接起来。在同一段 M7 上，较小的 6×12 堆叠过孔阵列把 M7 与 M3 层上的水平 VDD 带连接起来。

复合图案技术允许平移过孔位置，并打开额外的垂直布线轨道，以减少布线折拐并改善时序。`via73_12x6_shift_right` 过孔阵列通常插在 M7 层金属段与 M3 层水平带交点的中心。本示例中，用 `set_pg_via_master_rule` 命令的 `-offset {3 0}` 选项把过孔向右平移。该偏移如图 **??** 所示。M8 层 VDD 带下方的 M6 层与 M4 层垂直布线轨道不用于电源地，可以用于信号布线。

`set_pg_via_master_rule` 命令定义 12×6 堆叠过孔阵列的过孔规则。`create_pg_composite_pattern` 命令定义一份使用两种不同走线图案 `wire_base` 与 `segment_base` 的复合图案。`wire_base` 图案规定 VDD 与 VSS 在 M8 上的垂直带以及在 M3 上的水平带。`segment_base` 图案规定线网 VDD 在 M7 层上宽 10 微米的短水平段。走线图案通过 `{pattern: wire_base}` 与 `{pattern: segment_base}` 语句引用。

`-via_rule` 选项规定每个层交点处插入过孔的位置。考虑下面这条右移堆叠过孔 `via73_12x6_shift_right` 的定义：

```
{{{pattern_id: 2}{net_id: 1}}
 {{pattern_id: 3}{net_id: 1}}
 {via_master: via73_12x6_shift_right} {between_parallel: true}}
```

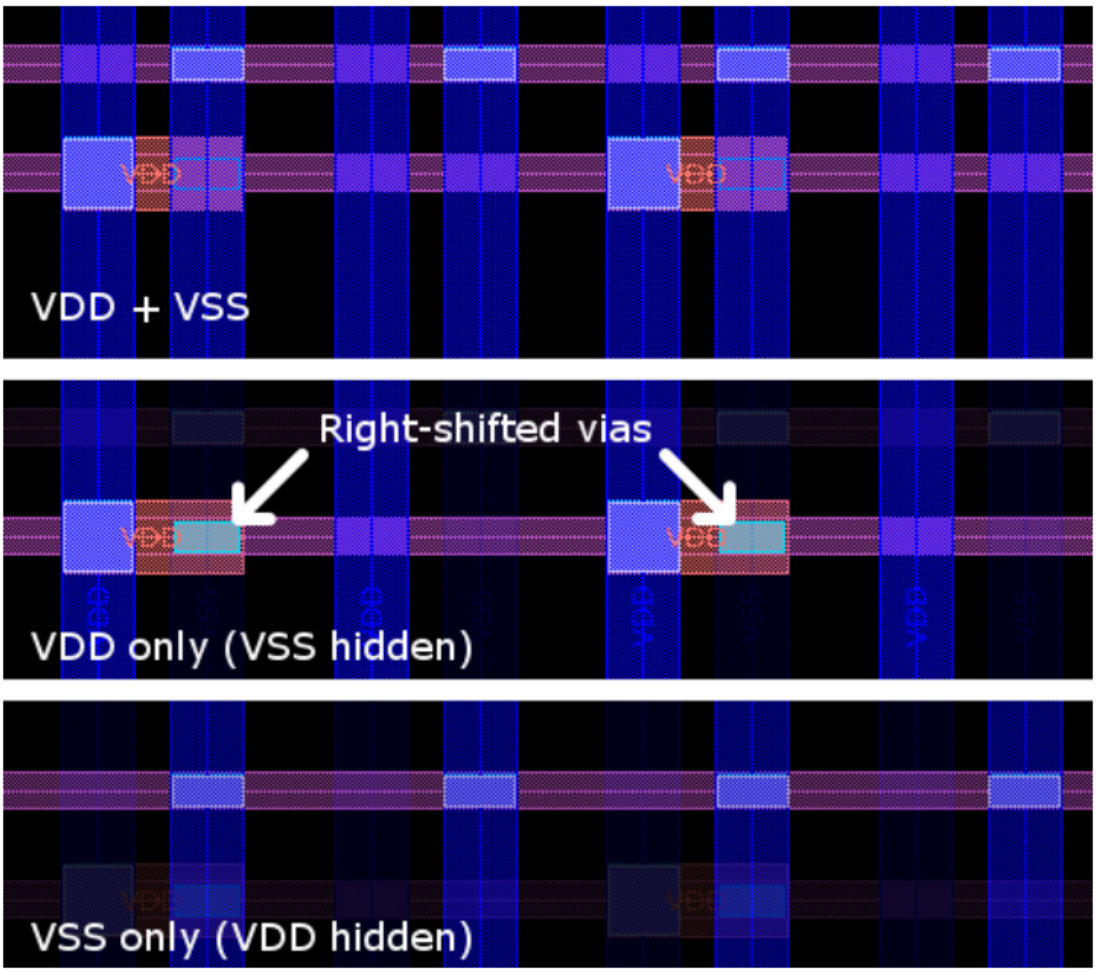


图 **10.7:** 复合桥接图案结构

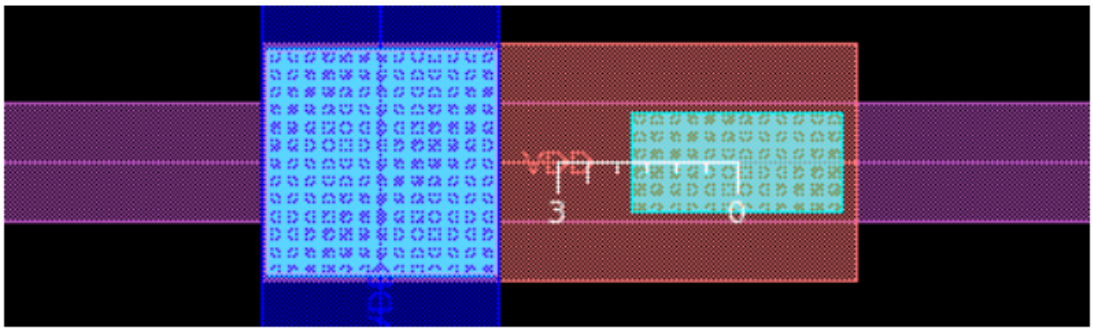


图 **10.8:** 相对中心的过孔偏移

在上述定义中，{{pattern_id: 2}{net_id: 1}} 指 -add_patterns 选项所指定的第二图案：{{pattern: wire_base} ... {parameters: {M3 horizontal ...}}}，以及 -nets 选项所指定的第一条线网 VDD。同样，{{pattern_id: 3}{net_id: 1}} 指第三图案定义：{{pattern: segment_base} ... {parameters: {M7 horizontal ...}}}，以及第一条线网 VDD。根据该定义，工具在 M3 带与 M7 带的交点处插入 via73_12x6_shift_right 堆叠过孔。语句 {between_parallel: true} 允许工具在平行电源带之间插入过孔。

创建该电源规划的命令，包括过孔主单元规则，如下：

```
set_pg_via_master_rule via73_12x6_shift_right \
   -contact_code {VIA34 VIA45 VIA56 VIA67} \
   -via_array_dimension {12 6} -offset {3 0}

create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_wire_pattern segment_base -layer @l -direction @d \
   -width @w -low_end_reference_point 0 \
   -high_end_reference_point @len -pitch @p \
   -parameters {l d w len p}

create_pg_composite_pattern via_bridging -nets {VDD VSS} \
   -add_patterns { \
    {{pattern: wire_base} {nets: VDD VSS} \
     {parameters: {M8 vertical 4 15 2}}{offset: 4}} \
    {{pattern: wire_base} {nets: VDD VSS} \
     {parameters: {M3 horizontal 2 20 4}}{offset: 4}} \
    {{pattern: segment_base} {nets: VDD} \
     {parameters: {M7 horizontal 4 10 {30 20}}} {offset: {2 4}}} \
    } \
   -via_rule { \
    {{{pattern_id: 1}{net_id: 1}}
     {{pattern_id: 2}{net_id: 1}} {via_master: NIL}} \
    {{{pattern_id: 1}{net_id: 2}}
     {{pattern_id: 2}{net_id: 2}} {via_master: default}} \
    {{{pattern_id: 1}{net_id: 1}}
     {{pattern_id: 3}{net_id: 1}} {via_master: default}} \
    {{{pattern_id: 2}{net_id: 1}}
     {{pattern_id: 3}{net_id: 1}}
     {via_master: via73_12x6_shift_right} {between_parallel: true}} \
     {{intersection: undefined} {via_master: NIL}} \
 }

set_pg_strategy core_patterns \
    -pattern {{name: via_bridging} \
              {nets: VDD VSS } {offset: {5 5}}} -core

compile_pg -strategies core_patterns
```

10.9.4 创建锥形带

Creating Tapering Straps

下面的示例创建一种在芯片中心收窄电源带的电源结构。垂直的 VDD 与 VSS 带创建在 M6 层上，水平的 VDD 与 VSS 带创建在 M7 层上。在芯片外围，水平 M7 带宽 10 微米，垂直 M6 带宽 8 微米。在芯片中心，M7 带收窄到 5 微米，M6 带收窄到 4 微米。绿色方框标注包围盒 `{{500 500} {1000 1000}}`。

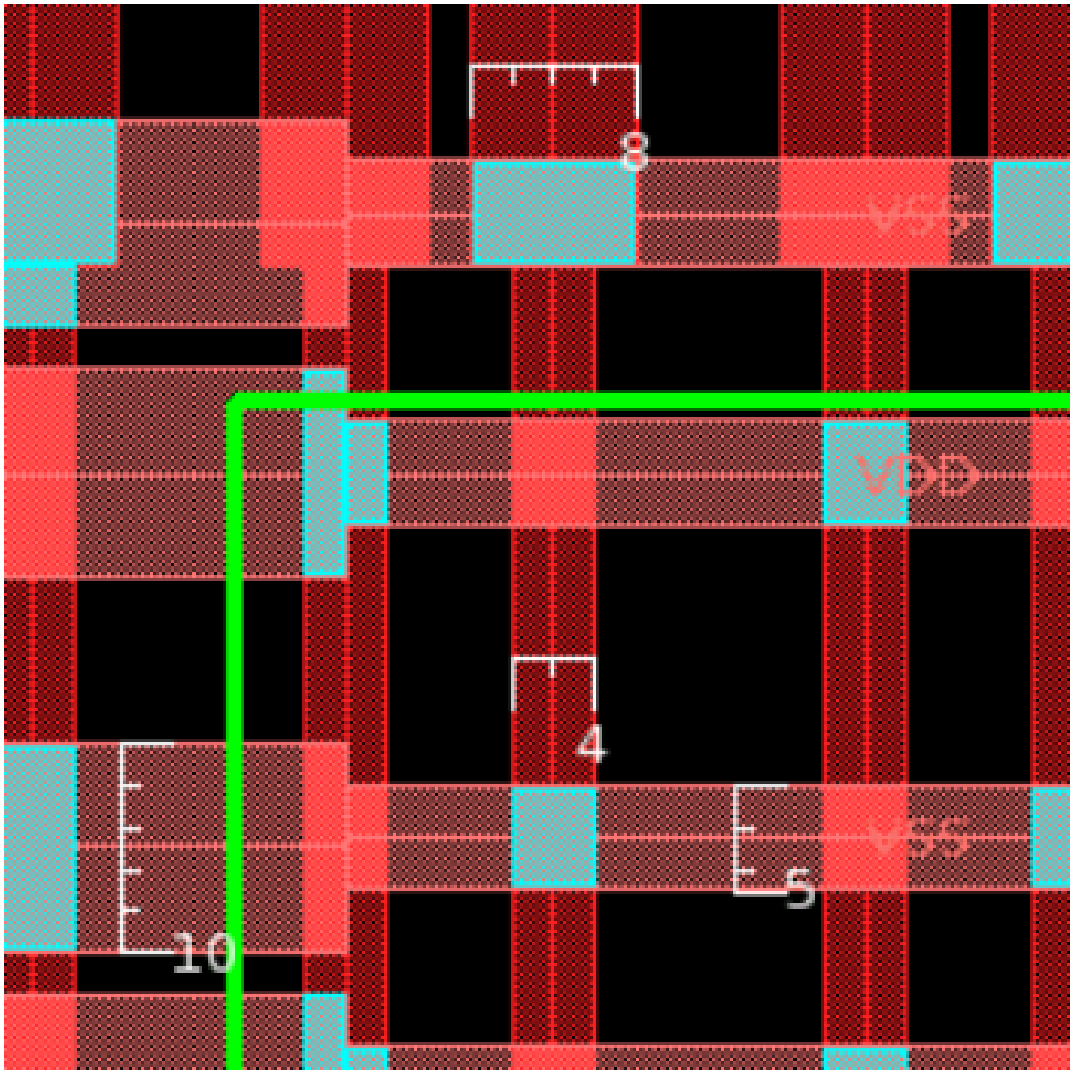


图 **10.9:** 锥形带

该电源规划首先用三条 `create_pg_wire_pattern` 命令创建三个走线图案定义。这些命令在布线边界的某一区域内定义图案的层、方向、宽度与间距。`first_wire` 走线图案用 `-extend_low boundary` 选项规定从布线边界的左侧与底侧开始的图案，并延伸到 `-high_end_reference_point @pt1` 参数所指定的点。`center_wire` 走线图案从 `-low_end_reference_point @pt1` 延伸到 `-high_end_reference_point @pt2`。`last_wire` 图案从 `-low_end_reference_point @pt2` 所指定的点开始，延伸到 `-extend_high boundary`，即布线边界的顶边或右边。

两条 `create_pg_composite_pattern` 命令用于定义图案之间的两种不同关系。`create_pg_composite_pattern tapering_base` 命令把前面的走线图案分别关联到布线边界的左、中、右区域，以及底、

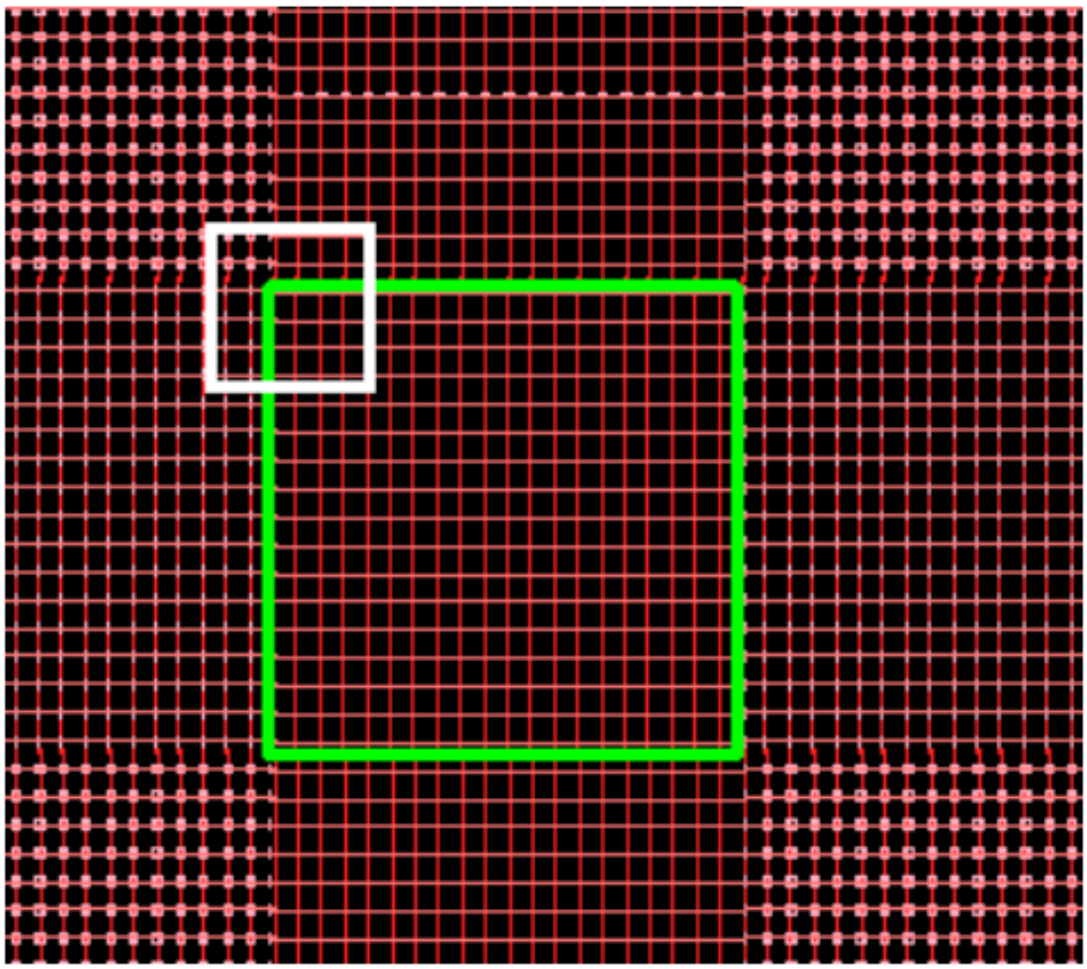


图 **10.10:** 锥形带缩小视图

中、顶区域。第二条命令 create_pg_composite_pattern tapering_straps 使用前面的 create_pg_composite_pattern tapering_base 定义，规定电源地走线的层、方向、侧边宽度、中心宽度、侧边间距与中心间距的实际取值，从而创建水平与垂直的电源地走线。

tapering_straps 复合图案使用另一个名为 tapering_base 的复合图案。复合图案可以分层定义，以便为要求较高的设计创建更复杂的图案。

set_pg_strategy 命令传入锥形带应当开始与结束的实际 x 坐标与 y 坐标，以及电源地带的信号名。这些取值通过 Tcl 变量指定。

实现该电源规划所用的命令如下。

```
create_pg_wire_pattern first_wire -layer @l -direction @d \
   -width @w -spacing @s -extend_low boundary \
   -high_end_reference_point @pt1 -parameters {l d w s pt1}

create_pg_wire_pattern center_wire -layer @l -direction @d \
   -width @w -spacing @s -low_end_reference_point @pt1 \
   -high_end_reference_point @pt2 -parameters {l d w s pt1 pt2}

create_pg_wire_pattern last_wire -layer @l -direction @d \
   -width @w -spacing @s -low_end_reference_point @pt2 \
   -extend_high boundary -parameters {l d w s pt2}

create_pg_composite_pattern tapering_base \
   -parameters {layer dir side_w center_w side_s center_s pt1 pt2} \
   -add_patterns { \
   {{pattern: first_wire} \
    {parameters: {@layer @dir @side_w @side_s @pt1}}} \
   {{pattern: center_wire} \
    {parameters: {@layer @dir @center_w @center_s @pt1 @pt2}}} \
   {{pattern: last_wire} \
    {parameters: {@layer @dir @side_w @side_s @pt2}}} \
}

create_pg_composite_pattern tapering_straps \
   -parameters {h_pt1 h_pt2 v_pt1 v_pt2} \
   -add_patterns { \
   {{pattern: tapering_base}
    {parameters: {M7 horizontal 10 5 2 7.5 @h_pt1 @h_pt2}} \
    {pitch: {0 30}}} \
   {{pattern: tapering_base}
    {parameters: {M6 vertical 8 4 2 6 @v_pt1 @v_pt2}} \
    {pitch: {25 0}}} \
}

set x_start 500
set x_end 1000
set y_start 500
set y_end 1000
set c_offset 5
set_pg_strategy core_patterns -core \
```

```
    -pattern [list {name: tapering_straps} \
                   {nets: VDD VSS } \
                   {parameters: $x_start $x_end $y_start $y_end} \
                   [list offset: [list 5 $c_offset]]]

compile_pg -strategies core_patterns
```

10.9.5 创建棋盘格过孔图案

Creating a Checkerboard Via Pattern

下面的示例为线网 VDD 在 M5 层上创建水平电源带，在 M4 层上创建垂直电源带。过孔插在交替的位置上，形成棋盘格图案。create_pg_wire_pattern 命令定义基本图案，create_pg_composite_pattern 命令定义水平与垂直电源地走线的参数。

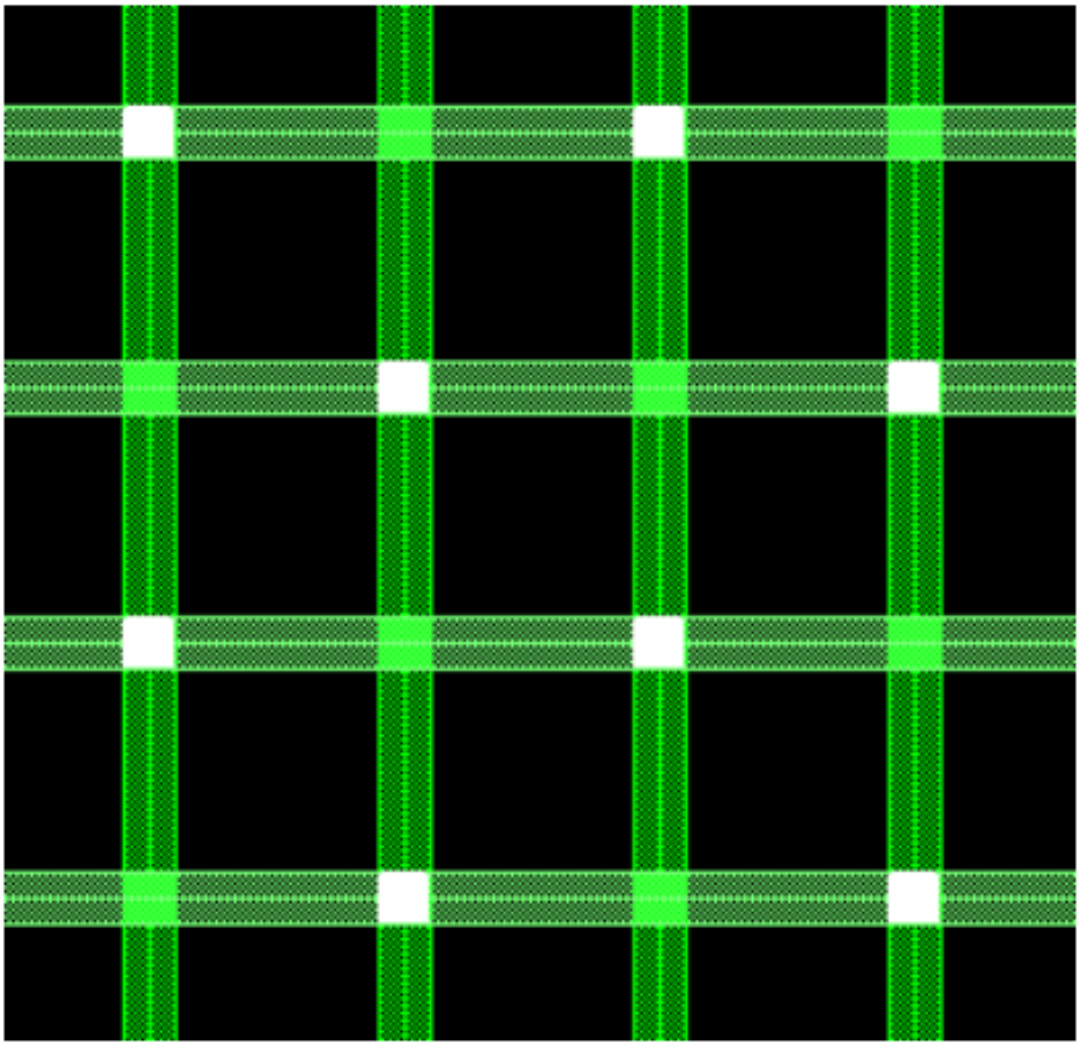


图 **10.11:** 棋盘格过孔

单条 create_pg_wire_pattern 命令创建名为 wire_base 的走线图案。create_pg_composite_pattern 命令把 wire_base 走线图案实例化两次：一次用于 M4 层上的垂直带，一次用于 M5 层上的水平带。每个图案定义一对线网 {VDD VDD}。线网对之间的节距为 20 微米；由于 create_pg_wire_pattern

命令的 -spacing interleaving 选项，两条线网在该节距内均匀分布。interleaving 由 create_pg_composite_pattern 命令中的语句 {parameters: {M4 vertical 2 20 interleaving}} 作为参数传入。

-via_rule 选项规定在 M4 层与 M5 层的每个交点上放置哪一种过孔。下面的图标注了该电源规划的 pattern_id 与 net_id 编号。本示例中，{pattern_id: 1} 指 -add_patterns 选项中的第一图案，即 M4 层上的垂直电源带。{pattern_id: 2} 指第二图案，即 M5 层上的水平电源带。{pattern_id: 1} {net_id: 1} 指第一图案规格中 nets: 关键字之后列出的第一条线网。

根据该规格，过孔放在 {pattern_id: 1} {net_id: 1} 与 {pattern_id: 2} {net_id: 1} 的交点处。过孔也放在 {pattern_id: 1} {net_id: 2} 与 {pattern_id: 2} {net_id: 2} 的交点处。在其他位置，{via_master: NIL} 规定那些交点不放置过孔。语句 {intersection: undefined} {via_master: NIL} 规定电源规划中其他任何地方都不应再插入过孔。

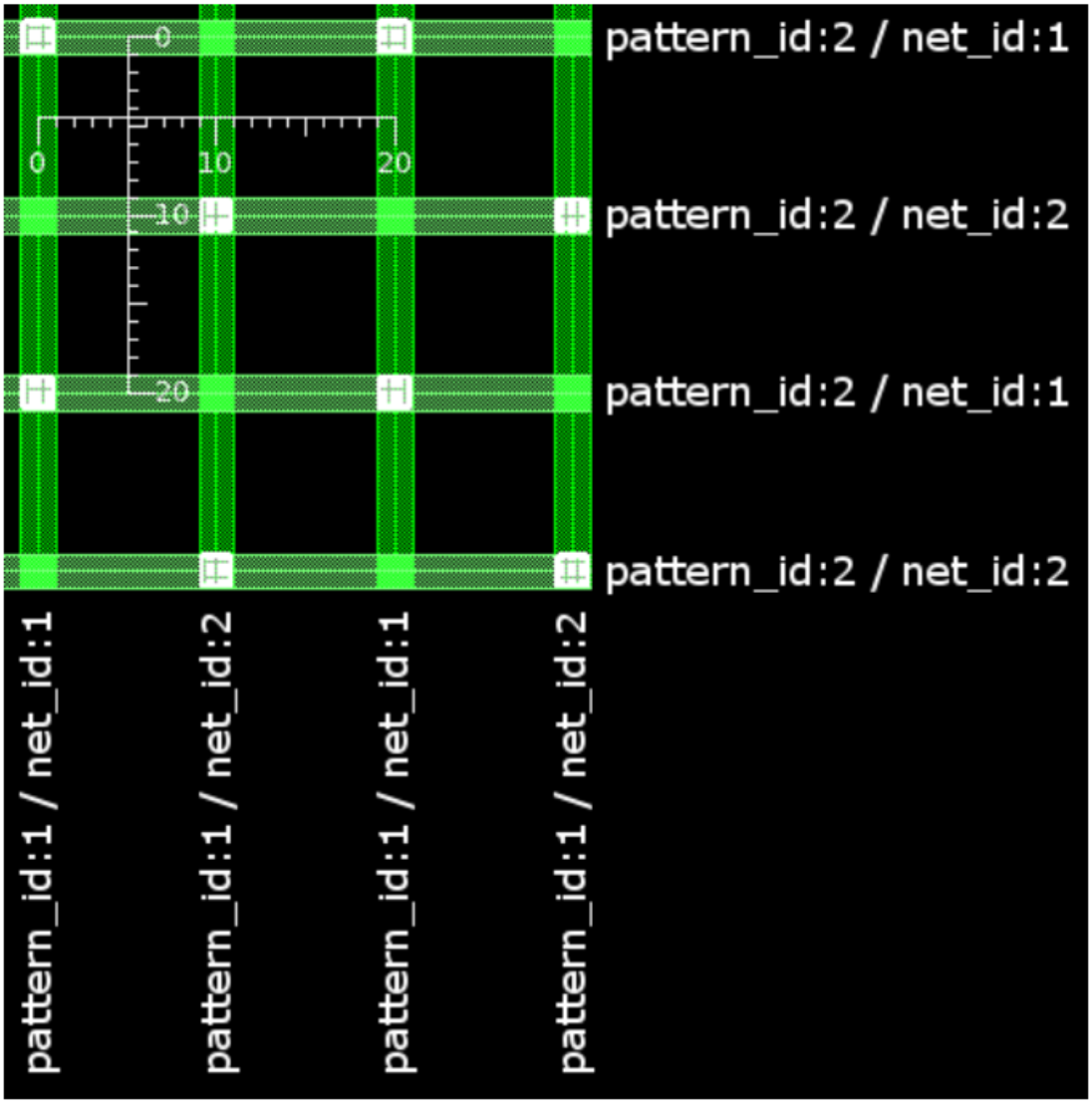


图 **10.12:** 标注后的棋盘格图案

创建该电源规划的命令如下：

```
create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}
```

```
create_pg_composite_pattern checker_board_vias -nets {VDD} \
   -add_patterns { \
      {{pattern: wire_base} {nets: VDD VDD} \
       {parameters: {M4 vertical 2 20 interleaving}} \
       {offset: 4}} \
      {{pattern: wire_base} {nets: VDD VDD} \
       {parameters: {M5 horizontal 2 20 interleaving}} \
       {offset: 4}}} \
   -via_rule { \
      {{{pattern_id: 1} {net_id: 1}} \
       {{pattern_id: 2} {net_id: 1}} {via_master: default}} \
      {{{pattern_id: 1} {net_id: 2}} \
       {{pattern_id: 2} {net_id: 2}} {via_master: default}} \
      {{{pattern_id: 1} {net_id: 2}} \
       {{pattern_id: 2} {net_id: 1}} {via_master: NIL}} \
      {{{pattern_id: 1} {net_id: 1}} \
       {{pattern_id: 2} {net_id: 2}} {via_master: NIL}} \
      {{intersection: undefined} {via_master: NIL}}}

set_pg_strategy core_patterns -core \
   -pattern {{name: checker_board_vias} {nets: VDD VSS } \
      {offset: {5 5}}}

compile_pg -strategies core_patterns
```

10.10 定义电源规划区域

Defining Power Plan Regions

在基于图案的电源网络布线流程中，电源规划区域定义设计中用于创建电源环与电源带、从而形成电源网络的区域。按下面的示例使用 `create_pg_region` 命令创建电源规划区域。

```
icc2_shell> create_pg_region region1 \
   -exclude_macros {macro1 macro2} -core -expand -35 \
   -macro_offset 35
{region1}
```

也可以用 Task Assistant 定义电源规划区域。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 在 Task Assistant 中选择 **PG Planning > PG Region**。
3. 在 Task Assistant 中输入区域名、区域面积、扩展参数以及其他信息。
4. 单击 Preview，显示所指定的 Tcl 命令。
5. 单击 Apply 创建电源规划区域。

要创建由多边形界定的电源规划区域，指定 `-polygon` 选项并后跟多边形坐标。下面的示例创建名为 `region_polygon` 的电源规划区域，其边界坐标为 (2000, 2000)、(4000, 2000)、(4000, 4000) 与 (2000, 4000)。

```
icc2_shell> create_pg_region region_polygon \
   -polygon {{2000 2000} {4000 2000} {4000 4000} {2000 4000}}
{region_polygon}
```

要创建排除指定宏单元的电源规划区域，使用 -exclude_macros 选项并指定要排除的宏单元列表。可以加上 -macro_offset 选项，在宏单元周围创建偏移。下面的示例中，工具在区域中做出一块凹进，以排除指定的宏单元。

```
icc2_shell> create_pg_region region_exclude -core \
   -exclude_macros {u_l/u_m/ahbram0/aram_1/x0_t0} -macro_offset 50
{region_exclude}
```

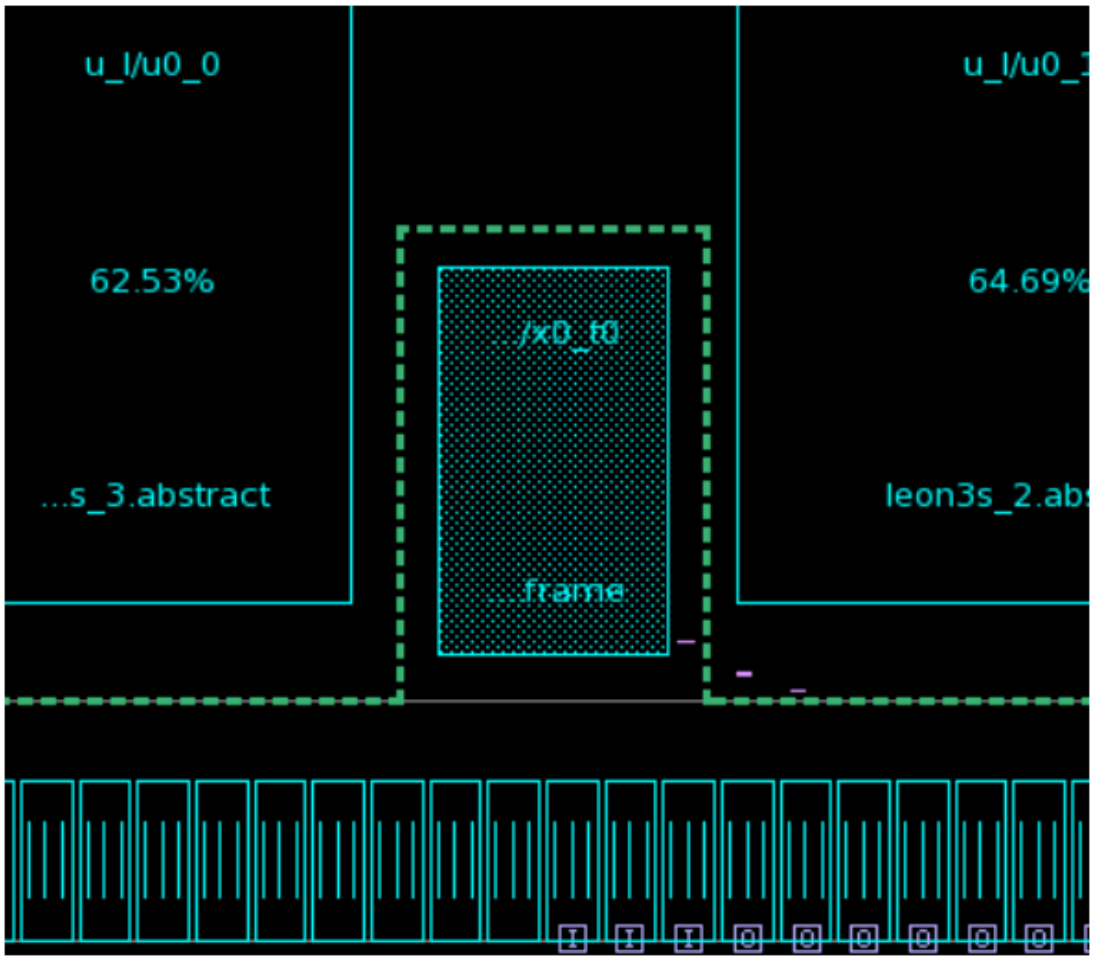


图 **10.13:** 排除宏单元的区域

要创建包围设计中宏单元的电源地区域，使用 -group_of_macros 选项。可以加上 -expand 选项来创建偏移。下面的示例中，工具在指定宏单元周围创建区域，并把区域扩展 50 微米。

```
icc2_shell> set macros {u_l/u_m/ahbram0/aram_0/x0_t0
   u_l/u_m/dsu0/x0/mem0_ram0_0_x0_x0_t0
   u_l/u_m/ahbram0/aram_2/x0_t0
   u_l/u_m/dsu0/x0/mem0_ram0_0_x1_x0_t0}

icc2_shell> create_pg_region region_surround -group_of_macros $macros \
```

```
   -expand 50
{region_surround}
```

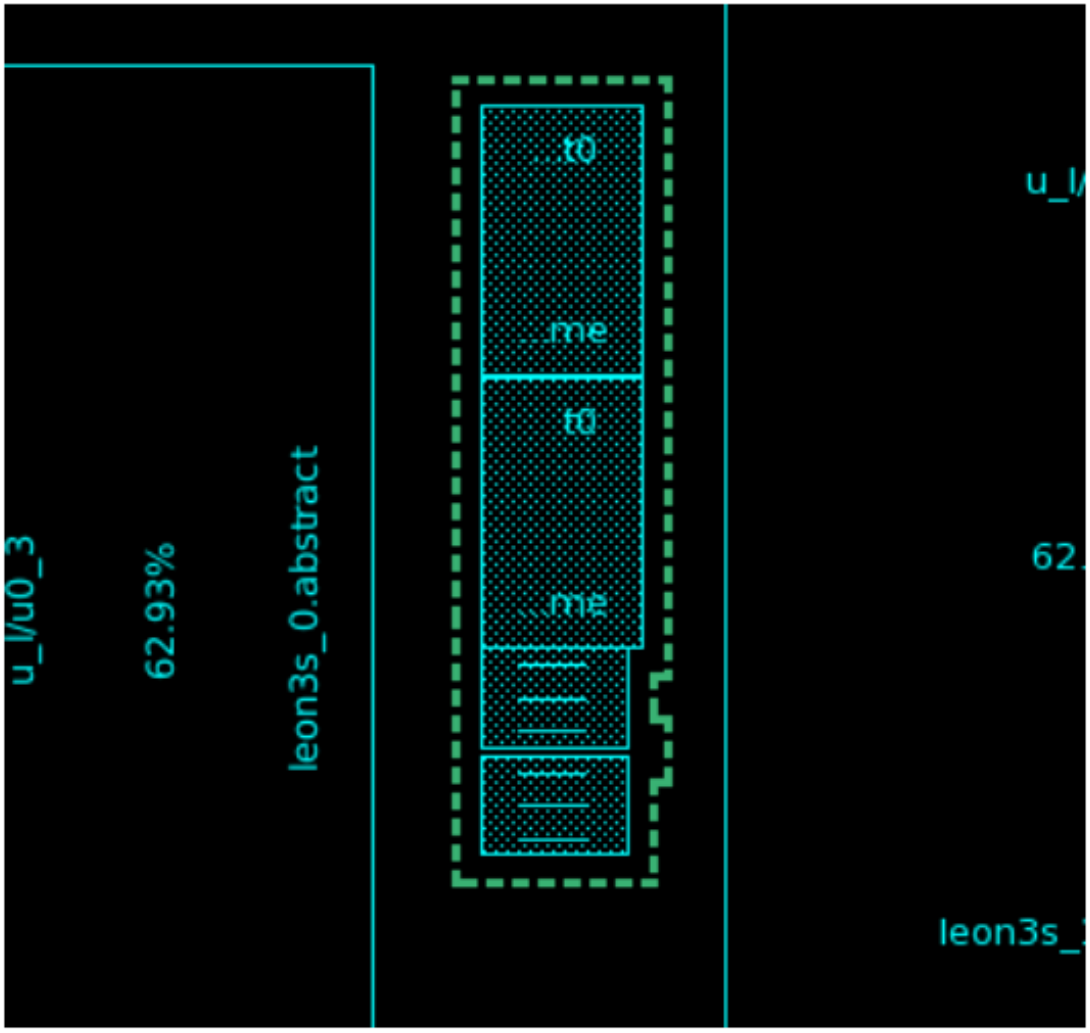


图 **10.14:** 包围一组宏单元的区域

若运行 `create_pg_region` 命令并指定已有的区域名，该命令会删除已有区域、复用该区域名，并创建新的区域定义。也可以用 `-update` 选项复制已有区域，并对区域副本进行修改。下一个示例复制前一示例中的 `region_surround` 电源规划区域，扩展该区域，并去掉折拐。

```
icc2_shell> create_pg_region region_copy -update region_surround \
   -expand 75 -remove_jog {expand: 50}
```

要列出设计中的全部电源规划区域，使用 `report_pg_regions` 命令。要只列出单个区域，在命令后指定区域名。下面的示例报告设计中当前已定义的电源规划区域。

```
icc2_shell> report_pg_regions
Region: region_surround
Points: {{2202.360 3092.270} {2202.360 3575.890} ... }
Region: region_copy
Points: {{2127.360 3017.270} {2127.360 4647.760} ... }
```

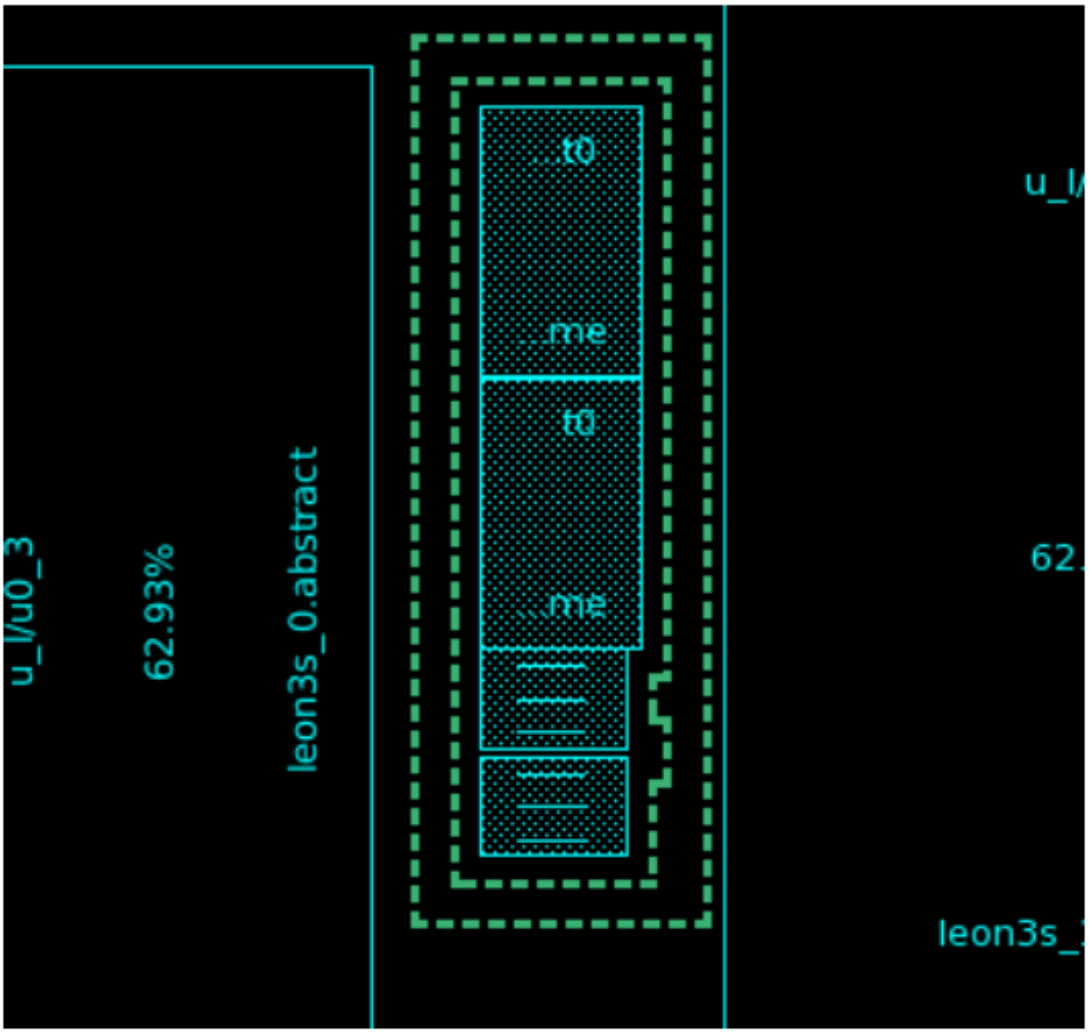


图 **10.15:** 复制区域并扩展

要删除电源规划区域，使用 remove_pg_regions 命令并后跟区域名。要删除全部区域，使用 remove_pg_regions -all 命令。下面的示例删除名为 region_copy 的电源规划区域。

```
icc2_shell> remove_pg_regions region_copy
PG region region_copy is removed.
```

10.11 设置电源规划策略

Setting the Power Plan Strategy

为设计定义环、网格、宏单元连接与标准单元电源轨图案之后，可以用 set_pg_strategy 命令把图案与电源规划区域或设计的其他区域关联起来。该命令还定义图案偏移、图案如何延伸到电源规划区域或区域边界之外，以及不创建图案的布线阻挡。

按下面的示例使用 set_pg_strategy 命令定义电源规划策略。

```
icc2_shell> set_pg_strategy ring_strat -core \
   -pattern {{name: ring_pattern} {nets: {VDD VSS}} \
             {offset: {3 3}} {parameters: {M7 10 2 M8 10 2 true}}} \
   -extension {{stop: design_boundary}}
```

也可以用 Task Assistant 定义标准单元电源轨图案。

1. 在 GUI 中选择 **Task > Task Assistant**。
2. 在 Task Assistant 中选择 **PG Planning > Strategy**。
3. 在 Task Assistant 中输入策略名、电源地网、图案名、布线区域以及其他设置。
4. 单击 Apply 应用该策略。

在前一示例中，set_pg_strategy 命令定义了一份策略，把 ring_pattern 图案与 ring_strat 策略关联起来。下面的示例定义一份策略，把「创建电源与地网格图案」中所定义的电源网格图案 mesh_pattern 与 mesh_strat 策略关联起来。-extension 选项把电源网格延伸到最外层环。

```
icc2_shell> set_pg_strategy mesh_strat -core \
   -extension {{stop: outermost_ring}} \
   -pattern {{pattern: mesh_pattern} {nets: {VDD VSS}}}
```

下面的示例定义一份策略，把「创建电源与地宏单元连接」中所定义的宏单元连接图案 macro_pattern 与 macro_strat 策略关联起来。

```
icc2_shell> set_pg_strategy macro_strat -core \
   -pattern {{pattern: hm_pattern} {nets: {VDD VSS}}}
```

下面的示例定义一份策略，把「创建电源与地标准单元电源轨」中所定义的标准单元连接图案 rail_pattern 与 rail_strat 策略关联起来。

```
icc2_shell> set_pg_strategy rail_strat -core \
   -pattern {{pattern: std_cell_rail} {nets: VDD VSS}}
```

下面各小节汇总 -pattern、-blockage 与 -extension 选项的规格关键字及其作用。

10.11.1 -pattern 选项的策略设置

Strategy Settings for the -pattern Option

-pattern 选项的参数定义所指定的图案如何在设计中创建。下面的列表说明可用参数。并非所有参数对所有图案类型都有效。

- {name: pattern_name} 参数指定本策略所使用的图案名。该名称是用 create_pg_*_pattern pattern_name 命令创建图案时所用的名称。
- {nets: {net1 net2 ...}} 参数指定策略的电源网名与地网名。也可以在 create_pg_*_pattern 命令中定义占位名，再用本参数设置实际线网名。

  要为某条线网指定空槽，用连字符（-）替换线网名。例如 {nets: {VDD - VSS -}}。
- {offset: {x_offset y_offset}} 参数指定创建第一个图案时所施加的 x 偏移与 y 偏移。分散引脚宏单元连接图案或标准单元电源轨图案不使用本参数。
- {offset_start: boundary | {x y}} 参数指定图案偏移的起点。分散引脚宏单元连接图案或标准单元电源轨图案不使用本参数。
- {parameters: value1 value2 ...} 参数指定与 create_pg_*_pattern 命令的 -parameters 选项所定义参数相对应的有序取值列表。
- {skip_sides: {side1 side2 ...}} 参数定义创建电源环时要省略的边。本参数只对环图案有效。
- {side_offset: {side: number} {offset: offset}} 参数定义环图案中每一指定边的偏移。本参数只对环图案有效。

10.11.2 -blockage 选项的策略设置

Strategy Settings for the -blockage Option

-blockage 选项的参数定义不应创建图案的区域。阻挡由三个参数定义：线网、层，以及区域规格（voltage_areas、macros、polygon、blocks 或 pg_regions）。可以在单条 set_pg_strategy 命令中提供多份阻挡规格。下面的列表说明可用参数。并非所有参数对所有图案类型都有效。

- {layers: {layer1 layer2 ...}} 参数指定要避免在阻挡上布线的层。
- {nets: {net1 net2 ...}} 参数指定要避免在阻挡上布线的电源网名与地网名。该线网列表是 -pattern 选项所指定线网的子集。
- 区域由 pg_regions、blocks、macros、pg_regions、polygon 或 voltage_areas 参数定义，后跟元素列表或点列表。

10.11.3 -extension 选项的策略设置

Strategy Settings for the -extension Option

-extension 选项的参数定义电源网格如何延伸到所指定的电源规划区域或设计区域之外。延伸规格使该电源结构能够与设计中的其他电源结构连接。延伸由方向、层、线网、边与停止参数定义。